

JavaScript 196 Lessons

*Complete Tutorial & Reference Guide
(Excluded: ## Document)*

Table of Contents (196 Lessons)

- [Lesson 001](#): JavaScript Where To
- [Lesson 002](#): JavaScript Output
- [Lesson 003](#): JavaScript Syntax
- [Lesson 004](#): JavaScript Statements
- [Lesson 005](#): JavaScript Comments
- [Lesson 006](#): JavaScript Variables
- [Lesson 007](#): JavaScript Let
- [Lesson 008](#): JavaScript Const
- [Lesson 009](#): JavaScript Data Types
- [Lesson 010](#): JavaScript Operators
- [Lesson 011](#): JavaScript Arithmetic
- [Lesson 012](#): JavaScript Assignment
- [Lesson 013](#): JavaScript Comparisons
- [Lesson 014](#): JavaScript Conditions
- [Lesson 015](#): JavaScript For Loop
- [Lesson 016](#): JavaScript While Loop
- [Lesson 017](#): JavaScript Strings
- [Lesson 018](#): JavaScript Numbers
- [Lesson 019](#): JavaScript Functions
- [Lesson 020](#): JavaScript Function Parameters
- [Lesson 021](#): JavaScript Function Return
- [Lesson 022](#): JavaScript Function Arguments
- [Lesson 023](#): JavaScript Function Expressions
- [Lesson 024](#): JavaScript Arrow Functions
- [Lesson 025](#): JavaScript Timers
- [Lesson 026](#): JavaScript Objects
- [Lesson 027](#): JavaScript Object Introduction
- [Lesson 028](#): JavaScript Object Properties
- [Lesson 029](#): JavaScript Object Methods
- [Lesson 030](#): JavaScript Object this
- [Lesson 031](#): JavaScript Object Display

- **Lesson 032:** JavaScript Object Constructors
- **Lesson 033:** JavaScript Scopes
- **Lesson 034:** JavaScript Code Blocks
- **Lesson 035:** JavaScript Hoisting
- **Lesson 036:** JavaScript var, let, const
- **Lesson 037:** JavaScript Strict Mode
- **Lesson 038:** JavaScript Dates
- **Lesson 039:** JavaScript Date Formats
- **Lesson 040:** JavaScript Date Methods
- **Lesson 041:** JavaScript Date Set Methods
- **Lesson 042:** JavaScript Date Reference
- **Lesson 043:** JavaScript Temporal
- **Lesson 044:** JavaScript Arrays
- **Lesson 045:** JavaScript Array Constructor
- **Lesson 046:** JavaScript Array Methods
- **Lesson 047:** JavaScript Array Search
- **Lesson 048:** JavaScript Array Sort
- **Lesson 049:** JavaScript Array Iteration
- **Lesson 050:** JavaScript Array Reference
- **Lesson 051:** JavaScript Array const
- **Lesson 052:** JavaScript Sets
- **Lesson 053:** JavaScript Set Methods
- **Lesson 054:** JavaScript Set Logic
- **Lesson 055:** JavaScript WeakSet
- **Lesson 056:** JavaScript Set Reference
- **Lesson 057:** JavaScript Maps
- **Lesson 058:** JavaScript Map Methods
- **Lesson 059:** JavaScript WeakMap
- **Lesson 060:** JavaScript Map Reference
- **Lesson 061:** JavaScript Looping
- **Lesson 062:** JavaScript Iterables
- **Lesson 063:** JavaScript Iterators
- **Lesson 064:** JavaScript Generators
- **Lesson 065:** JavaScript Math
- **Lesson 066:** JavaScript Math Reference
- **Lesson 067:** JavaScript Random
- **Lesson 068:** JavaScript RegExp
- **Lesson 069:** JavaScript RegExp Flags
- **Lesson 070:** JavaScript RegExp Characters
- **Lesson 071:** JavaScript RegExp Meta Characters
- **Lesson 072:** JavaScript RegExp Assertions
- **Lesson 073:** JavaScript RegExp Groups
- **Lesson 074:** JavaScript RegExp Quantifiers
- **Lesson 075:** JavaScript RegExp Patterns

- **Lesson 076:** JavaScript RegExp Objects
- **Lesson 077:** JavaScript RegExp Methods
- **Lesson 078:** JavaScript Data Types
- **Lesson 079:** JavaScript Primitive Types
- **Lesson 080:** JavaScript Datatypes Objects
- **Lesson 081:** JavaScript Symbol Datatype
- **Lesson 082:** JavaScript typeof
- **Lesson 083:** JavaScript undefined
- **Lesson 084:** JavaScript NaN
- **Lesson 085:** JavaScript toString()
- **Lesson 086:** JavaScript toLocaleString()
- **Lesson 087:** JavaScript Type Coercion
- **Lesson 088:** JavaScript Type Conversion
- **Lesson 089:** JavaScript Destructuring
- **Lesson 090:** JavaScript Errors
- **Lesson 091:** JavaScript Silent Errors
- **Lesson 092:** JavaScript Error Statements
- **Lesson 093:** JavaScript Error Reference
- **Lesson 094:** JavaScript Debugging
- **Lesson 095:** JavaScript Debugging Console
- **Lesson 096:** JavaScript Debugging Breakpoints
- **Lesson 097:** JavaScript Debugging Errors
- **Lesson 098:** JavaScript localStorage Counter
- **Lesson 099:** JavaScript Project - Event Listener
- **Lesson 100:** JavaScript Project - To-Do List
- **Lesson 101:** JavaScript Project - Modal Popup
- **Lesson 102:** JavaScript Project - Form Validation
- **Lesson 103:** JavaScript HTML DOM
- **Lesson 104:** JavaScript HTML DOM Methods
- **Lesson 105:** JavaScript HTML DOM Elements
- **Lesson 106:** JavaScript HTML DOM Content
- **Lesson 107:** JavaScript HTML DOM CSS
- **Lesson 108:** JavaScript Validation
- **Lesson 109:** JavaScript Events
- **Lesson 110:** JavaScript Mouse Events
- **Lesson 111:** JavaScript Keyboard Events
- **Lesson 112:** JavaScript Load Events
- **Lesson 113:** JavaScript Event Management
- **Lesson 114:** JavaScript HTML DOM Events
- **Lesson 115:** JavaScript Event Listener
- **Lesson 116:** JavaScript HTML-First
- **Lesson 117:** JavaScript Progressive Enhancement
- **Lesson 118:** JavaScript HTML Features
- **Lesson 119:** JavaScript CSS Features

- **Lesson 120:** JavaScript Function Definitions
- **Lesson 121:** JavaScript Callbacks
- **Lesson 122:** JavaScript this
- **Lesson 123:** JavaScript Function call()
- **Lesson 124:** JavaScript Function apply()
- **Lesson 125:** JavaScript Function bind()
- **Lesson 126:** JavaScript IIFE
- **Lesson 127:** JavaScript Closures
- **Lesson 128:** JavaScript Object Definitions
- **Lesson 129:** JavaScript Object Definition
- **Lesson 130:** JavaScript this Keyword
- **Lesson 131:** JavaScript Object Iterations
- **Lesson 132:** JavaScript Object Accessors
- **Lesson 133:** JavaScript Object Management
- **Lesson 134:** JavaScript Object Protection
- **Lesson 135:** JavaScript Object Prototypes
- **Lesson 136:** JavaScript Classes
- **Lesson 137:** JavaScript Class Inheritance
- **Lesson 138:** JavaScript Class Static
- **Lesson 139:** JavaScript JSON
- **Lesson 140:** JavaScript JSON Syntax
- **Lesson 141:** JavaScript JSON Data Types
- **Lesson 142:** JavaScript JSON parse()
- **Lesson 143:** JavaScript JSON stringify()
- **Lesson 144:** JavaScript JSON Server
- **Lesson 145:** JavaScript JSON HTML
- **Lesson 146:** JavaScript JSON vs XML
- **Lesson 147:** JavaScript Async
- **Lesson 148:** JavaScript Async Callbacks
- **Lesson 149:** JavaScript Promises
- **Lesson 150:** JavaScript Async/Await
- **Lesson 151:** JavaScript Async Parallel
- **Lesson 152:** JavaScript Event Loop
- **Lesson 153:** JavaScript Async Fetch
- **Lesson 154:** JavaScript Async Common Mistakes
- **Lesson 155:** JavaScript Async Debugging
- **Lesson 156:** JavaScript AbortController
- **Lesson 157:** JavaScript Web Workers
- **Lesson 158:** JavaScript Modules
- **Lesson 159:** JavaScript Module Export
- **Lesson 160:** JavaScript Module Import
- **Lesson 161:** JavaScript Module Namespace
- **Lesson 162:** JavaScript Dynamic Import
- **Lesson 163:** JavaScript Metaprogramming

- [Lesson 164](#): JavaScript Reflect
- [Lesson 165](#): JavaScript Proxy
- [Lesson 166](#): JavaScript Typed Arrays
- [Lesson 167](#): JavaScript Typed Array Methods
- [Lesson 168](#): JavaScript Typed Array Reference
- [Lesson 169](#): JavaScript ArrayBuffer
- [Lesson 170](#): JavaScript DataView
- [Lesson 171](#): JavaScript Atomics
- [Lesson 172](#): JavaScript DOM Navigation
- [Lesson 173](#): JavaScript DOM Nodes
- [Lesson 174](#): JavaScript HTMLCollection
- [Lesson 175](#): JavaScript NodeList
- [Lesson 176](#): JavaScript Window Object
- [Lesson 177](#): JavaScript Window Screen
- [Lesson 178](#): JavaScript Window Location
- [Lesson 179](#): JavaScript Window History
- [Lesson 180](#): JavaScript Window Navigator
- [Lesson 181](#): JavaScript Popup Boxes
- [Lesson 182](#): JavaScript Cookies
- [Lesson 183](#): JavaScript Web API Intro
- [Lesson 184](#): JavaScript API Fetch
- [Lesson 185](#): JavaScript Geolocation API
- [Lesson 186](#): JavaScript API History
- [Lesson 187](#): JavaScript Pointer Events API
- [Lesson 188](#): JavaScript Web Storage API
- [Lesson 189](#): JavaScript Validation API
- [Lesson 190](#): JavaScript API Web Workers
- [Lesson 191](#): JavaScript Graphics
- [Lesson 192](#): JavaScript Canvas Graphics
- [Lesson 193](#): JavaScript Plotly Graphics
- [Lesson 194](#): JavaScript Chart.js Graphics
- [Lesson 195](#): JavaScript Google Chart Graphics
- [Lesson 196](#): JavaScript D3.js Graphics

JavaScript Where To

Table of Contents

- [JavaScript Where To](#)
- [The script Tag](#)
- [Example 1](#)
- [JavaScript in Head or Body](#)
- [Example 2](#)
- [JavaScript in Body](#)
- [Example 3](#)
- [External JavaScript](#)
- [Example 4](#)
- [Reference](#)

The script Tag

In HTML, JavaScript code is inserted between `<script>` and `</script>` tags.

```
<!DOCTYPE html>
<html>
<body>

<h2>Demo JavaScript in Body</h2>

<p id="demo">A paragraph</p>

<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript in Head or Body

You can place any number of scripts in an HTML document.

Scripts can be placed in the `<body>`, or in the `<head>` section of an HTML page, or in both.

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
```

```
<body>

<h2>Demo JavaScript in Head</h2>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript in Body

In this example, a JavaScript function is placed in the `<body>` section of an HTML page.

The function is invoked when a button is clicked.


```
<!DOCTYPE html>
<html>
<body>

<h2>Demo JavaScript in Body</h2>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>

<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

External JavaScript

Scripts can also be placed in external files.

External scripts are useful when the same code is used in many different web pages.

```
<!DOCTYPE html>
<html>
<body>

<h2>Demo External JavaScript</h2>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>

<script src="myScript.js"></script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Where To](#)

JavaScript Output

Table of Contents

- [JavaScript Output](#)
- [JavaScript Display Possibilities](#)
- [Using innerHTML](#)
- [Example 1](#)
- [Using document.write\(\)](#)
- [Example 2](#)
- [Using window.alert\(\)](#)
- [Example 3](#)
- [Using console.log\(\)](#)
- [Example 4](#)
- [JavaScript Print](#)
- [Example 5](#)
- [Reference](#)

JavaScript Display Possibilities

JavaScript can "display" data in different ways:

- Writing into an HTML element, using `innerHTML`.
- Writing into the HTML output using `document.write()`.
- Writing into an alert box, using `window.alert()`.
- Writing into the browser console, using `console.log()`.

Using innerHTML

To access an HTML element, JavaScript uses the `document.getElementById(id)` method.

The `innerHTML` property defines the HTML content.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Output</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using innerHTML</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using document.write()

For testing purposes, it is convenient to use `document.write()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Output</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using document.write()</h4>
<script>
document.write(5 + 6);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using window.alert()

You can use an alert box to display data:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Output</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using window.alert()</h4>
<script>
window.alert(5 + 6);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Using console.log()

For debugging purposes, you can call the `console.log()` method in the browser to display data.

You will need to activate the browser console with F12, and select "Console" in the menu.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Output</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.log()</h4>
<script>
console.log(5 + 6);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript Print

JavaScript does not have any print object or print methods.

You cannot access output devices from JavaScript.

The only exception is that you can call the `window.print()` method in the browser to print the content of the current window.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Output</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Print</h4>
<button onclick="window.print()">Print this page</button>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Output](#)

JavaScript Syntax

Table of Contents

- [JavaScript Syntax](#)
- [JavaScript Values](#)
- [JavaScript Literals](#)
- [Example 1](#)
- [JavaScript Variables](#)
- [Example 2](#)
- [JavaScript Identifiers](#)
- [JavaScript Operators](#)
- [Example 3](#)
- [JavaScript Expressions](#)
- [Example 4](#)
- [JavaScript is Case Sensitive](#)
- [Example 5](#)
- [JavaScript and Camel Case](#)
- [Reference](#)

JavaScript Values

The JavaScript syntax defines two types of values:

- **Fixed values** are called **Literals**.
- **Variable values** are called **Variables**.

JavaScript Literals

The most important syntax rules for **literals** (fixed values) are:

Numbers are written with or without decimals:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Numbers</h4>
<p>Number can be written with or without decimals:</p>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
document.getElementById("demo1").innerHTML = 10.50;
document.getElementById("demo2").innerHTML = 1001;
</script>
```



```
</body>
</html>
```

Strings are text, written within double or single quotes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Strings</h4>
<p>Strings can be written with double or single quotes:</p>
<p id="demo3"></p>
<p id="demo4"></p>

<script>
document.getElementById("demo3").innerHTML = "John Doe";
document.getElementById("demo4").innerHTML = 'John Doe';
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Variables

Variables are containers for **storing data values**.

Variables must be **identified** with **unique names**.

The **let** and **const** keywords create variables:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Variables</h4>
<p id="demo"></p>

<script>
let x;
```

```
x = 6;
document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Identifiers

An **identifier** is the **name** given to a variable.

Rules for identifiers:

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter.
- Names can also begin with \$ and _.
- Names are case sensitive (y and Y are different variables).
- Reserved words (like JavaScript keywords) cannot be used as names.

JavaScript Operators

JavaScript **assignment operators** (=) assign values to variables:

JavaScript uses **arithmetic operators** (+ - * /) to **compute** values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Operators</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let a, b, c;
a = 5;
b = 2;
c = a + b;
document.getElementById("demo1").innerHTML = "Value of a + b = " + c;
document.getElementById("demo2").innerHTML = "Value of a * b = " + (a * b);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

JavaScript Expressions

An expression is a combination of values, variables, and operators, which computes to a value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Expressions</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
document.getElementById("demo1").innerHTML = (5 + 6) * 10;
let x = 5;
document.getElementById("demo2").innerHTML = x * 10;
document.getElementById("demo3").innerHTML = "John" + " " + "Doe";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript is Case Sensitive

JavaScript identifiers are **case sensitive**.

The variables `lastName` and `lastname`, are different variables:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript is Case Sensitive</h4>
<p id="demo1"></p>
<p id="demo2"></p>
```

```
<script>
let lastName = "Doe";
let lastname = "Smith";
document.getElementById("demo1").innerHTML = lastName;
document.getElementById("demo2").innerHTML = lastname;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

JavaScript and Camel Case

Historically, programmers have used different ways of joining multiple words into one variable name:

- **Hyphens:** first-name, last-name. Hyphens are **not allowed** in JavaScript. They are reserved for subtractions.
- **Underscore:** first_name, last_name.
- **Upper Camel Case (Pascal Case):** FirstName, LastName.
- **Lower Camel Case:** firstName, lastName.

JavaScript programmers tend to use **lower camel case**.

Reference

- [W3Schools JavaScript Syntax](#)

JavaScript Statements

Table of Contents

- [JavaScript Statements](#)
- [JavaScript Programs](#)
- [JavaScript Statements](#)
- [Example 1](#)
- [Semicolons](#)
- [Example 2](#)
- [JavaScript White Space](#)
- [JavaScript Line Length and Line Breaks](#)
- [Example 3](#)
- [JavaScript Code Blocks](#)
- [Example 4](#)
- [JavaScript Keywords](#)
- [Reference](#)

JavaScript Programs

A **computer program** is a list of "instructions" to be "executed" by a computer.

These programming instructions are called **statements**.

Most JavaScript programs contain many statements.

The statements are executed, one by one, in the same order as they are written.

In HTML, JavaScript programs are executed by the web browser.

JavaScript Statements

JavaScript statements are composed of: Values, Operators, Expressions, Keywords, and Comments.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Statements</h4>
<p>A <b>JavaScript program</b> is a list of <b>statements</b> to be executed by a
computer.</p>
<p id="demo"></p>

<script>
let x, y, z;
x = 5;
y = 6;
z = x + y;
```

```
document.getElementById("demo").innerHTML = "The value of z is " + z + ".";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Semicolons

Semicolons separate JavaScript statements.

Add a semicolon at the end of each executable statement:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Semicolons</h4>
<p>Multiple statements on one line are allowed when separated by semicolons:</p>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
let a, b, c;
a = 5;
b = 6;
c = a + b;
document.getElementById("demo1").innerHTML = "Value of a is " + a + ";";
document.getElementById("demo2").innerHTML = "Value of b is " + b + ";";
document.getElementById("demo3").innerHTML = "Value of c is " + c + ";";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript White Space

JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.

The following lines are equivalent:

```
let person = "Hege";  
let person="Hege";
```

A good practice is to put spaces around operators (= + - * /).

JavaScript Line Length and Line Breaks

For best readability, programmers often like to avoid code lines longer than 80 characters.

If a JavaScript statement does not fit on one line, the best place to break it is after an operator:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Statements</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>Line Length and Line Breaks</h4>  
<p>The best place to break a code line is after an operator:</p>  
<p id="demo"></p>  
  
<script>  
document.getElementById("demo").innerHTML =  
"Hello Dolly!";  
</script>  
  
</body>  
</html>
```

Example 3

Result [View Example](#)

JavaScript Code Blocks

JavaScript statements can be grouped together in code blocks, inside curly brackets {...}.

The purpose of code blocks is to define statements to be executed together.

One place you will find statements grouped together in blocks is in JavaScript functions:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Statements</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>JavaScript Code Blocks</h4>
```

```
<p>JavaScript code blocks are used to group statements together.</p>
<button type="button" onclick="myFunction()">Click Me!</button>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function myFunction() {
  document.getElementById("demo1").innerHTML = "Hello Dolly!";
  document.getElementById("demo2").innerHTML = "How are you?";
}
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript Keywords

JavaScript statements often start with a **keyword** to identify the JavaScript action to be performed.

Here is a list of some of the keywords you will learn about in this tutorial:

Keyword	Description
let	Declares a variable
const	Declares a constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
function	Declares a function
return	Exits a function
for	Marks a block of statements to be executed in a loop
while	Marks a block of statements to be executed in a loop

JavaScript keywords are reserved words. Reserved words cannot be used as names for variables.

Reference

- [W3Schools JavaScript Statements](#)

JavaScript Comments

Table of Contents

- [JavaScript Comments](#)
- [Single Line Comments](#)
- [Example 1](#)
- [Multi-line Comments](#)
- [Example 2](#)
- [Using Comments to Prevent Execution](#)
- [Example 3](#)
- [Reference](#)

Single Line Comments

Single line comments start with `//`.

Any text between `//` and the end of the line will be ignored by JavaScript (will not be executed).

Single-line comments before each code line to explain the code:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Comments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Single Line Comments</h4>
<p id="demo"></p>

<script>
// Change heading
document.getElementById("demo").innerHTML = "Your first name";
// Change paragraph
document.getElementById("demo").innerHTML = "Peter";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Multi-line Comments

Multi-line comments start with `/*` and end with `*/`.

Any text between `/*` and `*/` will be ignored by JavaScript.

This example uses a multi-line comment (a comment block) to explain the code:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Comments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Multi-line Comments</h4>
<p id="demo"></p>

<script>
/*
The code below will change
the heading with id = "myH"
and the paragraph with id = "myP"
*/
let x = 5;
let y = 6;
document.getElementById("demo").innerHTML = x + y;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using Comments to Prevent Execution

Using comments to prevent execution of code is suitable for code testing.

Adding `//` in front of a code line changes the code lines from an executable line to a comment.

This example uses `//` to prevent execution of one of the code lines, and uses a comment block to prevent execution of multiple lines:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Comments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Comments to Prevent Execution</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
//document.getElementById("demo1").innerHTML = "This line is commented out";
```

```
document.getElementById("demo1").innerHTML = "Hello Dolly!";

/*
document.getElementById("demo2").innerHTML = "This is block commented out";
document.getElementById("demo2").innerHTML = "Also block commented out";
*/
document.getElementById("demo2").innerHTML = "Only this runs!";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Comments](#)

JavaScript Variables

Table of Contents

- [JavaScript Variables](#)
- [Variables are Data Containers](#)
- [Example 1](#)
- [JavaScript Identifiers](#)
- [JavaScript Underscore \(_ \)](#)
- [Example 2](#)
- [JavaScript Dollar Sign \(\\$\)](#)
- [Example 3](#)
- [Declare a Variable Using let](#)
- [Example 4](#)
- [Declare a Variable Using const](#)
- [Example 5](#)
- [One Statement, Many Variables](#)
- [Example 6](#)
- [JavaScript Data Types](#)
- [Example 7](#)
- [The Assignment Operator](#)
- [Example 8](#)
- [Reference](#)

Variables are Data Containers

JavaScript variables are containers for data.

JavaScript variables can be **declared** in 4 ways: **let**, **const**, **var** (older), or automatically.

Variables are **labels** for data values and **containers** for storing data.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Variables are Data Containers</h4>
<p id="demo"></p>

<script>
let carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Identifiers

Variables are identified with **names** called **identifiers**.

Names can be short like x, y, z or descriptive like age, sum, carName.

Rules for constructing names (identifiers):

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter.
- Names can also begin with **\$** and **_**.
- Numbers are not allowed as the first character in names.
- Names are case sensitive (y and Y are different variables).
- Reserved words (like JavaScript keywords) cannot be used as names.

JavaScript Underscore (_)

JavaScript treats underscore as a letter. Identifiers containing **_** are valid variable names.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Underscore (_)</h4>
<p id="demo"></p>

<script>
let _firstName = "John";
let _lastName = "Doe";
let _age = 30;
document.getElementById("demo").innerHTML = _firstName + " " + _lastName + " is " +
_age;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Dollar Sign (\$)

JavaScript also treats a dollar sign as a letter. Identifiers containing **\$** are valid variable names.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Dollar Sign ($)</h4>
<p id="demo"></p>

<script>
let $ = "Hello World";
let $$$ = 2;
let $myMoney = 5;
document.getElementById("demo").innerHTML = $ + " " + ($ + $$$ + $myMoney);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Declare a Variable Using let

Creating a variable in JavaScript is called **declaring** a variable.

You declare a JavaScript variable with the **let** keyword.

After the declaration, the variable has no value (technically it is **undefined**).

To **assign** a value to the variable, use the equal sign:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Declare a Variable Using let</h4>
<p id="demo"></p>

<script>
let carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Declare a Variable Using const

Always use **const** if the value should not be changed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Declare a Variable Using const</h4>
<p id="demo"></p>

<script>
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo").innerHTML = "Total: " + total;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

One Statement, Many Variables

You can declare many variables in one statement.

Start the statement with **let** or **const** and separate the variables by **comma**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>One Statement, Many Variables</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>
```



```
<script>
let person = "John Doe", carName = "Volvo", price = 200;
document.getElementById("demo1").innerHTML = person;
document.getElementById("demo2").innerHTML = carName;
document.getElementById("demo3").innerHTML = price;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

JavaScript Data Types

JavaScript variables can hold many **data types**: numbers, strings, objects and more.

Strings are text written **inside quotes**.

Numbers are written **without quotes**.

If you put a number in quotes, it will be treated as a text string.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Data Types</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
let pi = 3.14;
let person = "John Doe";
let answer = 'Yes I am!';
document.getElementById("demo1").innerHTML = pi;
document.getElementById("demo2").innerHTML = person;
document.getElementById("demo3").innerHTML = answer;
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

The Assignment Operator

In JavaScript, the equal sign (=) is an **assignment** operator, not an **equal to** operator.

In JavaScript, `x = x + 5` makes perfect sense: it assigns the value of `x + 5` to `x`.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Assignment Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
x = x + 5;
document.getElementById("demo").innerHTML = "Value of x = x + 5 is " + x;
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Reference

- [W3Schools JavaScript Variables](#)

JavaScript Let

Table of Contents

- [JavaScript Let](#)
- [Block Scope](#)
- [Example 1](#)
- [Function Scope](#)
- [Example 2](#)
- [Global Scope](#)
- [Example 3](#)
- [Cannot be Redeclared](#)
- [Example 4](#)
- [Redeclaring Variables](#)
- [Example 5](#)
- [Let Hoisting](#)
- [Example 6](#)
- [Reference](#)

Block Scope

Before ES6 (2015), JavaScript did not have **Block Scope**.

ES6 introduced the **let** and **const** keywords which provided **Block Scope**.

Variables declared inside a `{ }` block cannot be accessed from outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Block Scope</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
{
  let x = 2;
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// x can NOT be used here (outside the block)
document.getElementById("demo2").innerHTML = "Outside block: x is not accessible";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Function Scope

Inside a function all variables declared with **var**, **let** or **const** have **Function Scope**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Scope</h4>
<p id="demo"></p>

<script>
function myFunction() {
  let carName = "Volvo";
  document.getElementById("demo").innerHTML = "Inside function: " + carName;
}
myFunction();
// carName is NOT accessible outside the function
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Global Scope

Variables declared with **var** always have **Global Scope**.

Variables declared with **var** inside a **{ }** block can be accessed from outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Global Scope (var)</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
```

```

{
  var x = 2;
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// x CAN be used here (var has global scope)
document.getElementById("demo2").innerHTML = "Outside block: x = " + x;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Cannot be Redeclared

Variables defined with **let** can not be redeclared.

You can not accidentally redeclare a variable declared with **let**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot be Redeclared</h4>
<p>let variables cannot be redeclared in the same scope.</p>
<p id="demo"></p>

<script>
let x = "John Doe";
document.getElementById("demo").innerHTML = x;
// let x = 0; // This would cause an error (commented out)
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Redeclaring Variables

Redeclaring a variable using the **var** keyword can impose problems.

Redeclaring a variable inside a block will also redeclare the variable outside the block.

Redeclaring a variable using the **let** keyword solves this problem - redeclaring inside a block will not redeclare the variable outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Redeclaring Variables (let)</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = 10;
// Here x is 10
{
  let x = 2;
  // Here x is 2
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// Here x is 10 (not affected by block)
document.getElementById("demo2").innerHTML = "Outside block: x = " + x;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Let Hoisting

Variables defined with **var** are **hoisted** to the top and can be initialized at any time. You can use the variable before it is declared.

Variables defined with **let** are also hoisted to the top of the block, but not initialized. Using a **let** variable before it is declared will result in a **ReferenceError**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Let</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Let Hoisting</h4>
<p id="demo"></p>
```

```
<script>
try {
  // Using let variable before declaring it causes ReferenceError
  carName = "Volvo";
  let carName;
} catch(err) {
  document.getElementById("demo").innerHTML = "Error: " + err.message;
}
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Let](#)

JavaScript Const

Table of Contents

- [JavaScript Const](#)
- [Cannot be Reassigned](#)
- [Example 1](#)
- [Must be Assigned](#)
- [Example 2](#)
- [Constant Arrays](#)
- [Example 3](#)
- [Constant Objects](#)
- [Example 4](#)
- [Block Scope](#)
- [Example 5](#)
- [Redeclaring](#)
- [Example 6](#)
- [Hoisting](#)
- [Example 7](#)
- [Reference](#)

Cannot be Reassigned

A variable defined with the **const** keyword cannot be reassigned:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot be Reassigned</h4>
<p id="demo"></p>

<script>
const PI = 3.141592653589793;
// PI = 3.14; // This will cause an error (commented out)
document.getElementById("demo").innerHTML = PI;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Must be Assigned

JavaScript **const** variables must be assigned a value when they are declared:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Must be Assigned</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Correct: const variables must be assigned at declaration
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo1").innerHTML = "Price1: " + price1 + ", Price2: " +
price2;
document.getElementById("demo2").innerHTML = "Total: " + total;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Constant Arrays

You can change the elements of a constant array, but you can NOT reassign the array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constant Arrays</h4>
<p id="demo"></p>

<script>
const cars = ["Saab", "Volvo", "BMW"];
// You can change an element:
cars[0] = "Toyota";
// You can add an element:
cars.push("Audi");
```

```
// But you can NOT reassign the array
// cars = ["Toyota", "Volvo", "BMW"]; // ERROR
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Constant Objects

You can change the properties of a constant object, but you can NOT reassign the object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constant Objects</h4>
<p id="demo"></p>

<script>
const car = {type:"Fiat", model:"500", color:"white"};
// You can change a property:
car.color = "red";
// You can add a property:
car.owner = "Johnson";
// But you can NOT reassign the object
// car = {type:"Volvo", model:"EX60", color:"red"}; // ERROR
document.getElementById("demo").innerHTML = "Car: " + car.type + " " + car.model + " (" + car.color + ")";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Block Scope

Declaring a variable with **const** is similar to **let** when it comes to **Block Scope**.

The x declared in the block is not the same as the x declared outside the block:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Block Scope</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const x = 10;
// Here x is 10
{
  const x = 2;
  // Here x is 2
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// Here x is 10 (not affected by block)
document.getElementById("demo2").innerHTML = "Outside block: x = " + x;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Redeclaring

Redeclaring a variable with **const**, in another scope, or in another block, is allowed. But redeclaring in the same scope is not allowed:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Redeclaring</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const x = 2;
// const x = 3; // Not allowed in same scope (commented out)
document.getElementById("demo1").innerHTML = "Outer x: " + x;
{

```

```

    const x = 3; // Allowed in another block scope
    document.getElementById("demo2").innerHTML = "Inner x: " + x;
  }
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

Hoisting

Variables defined with **const** are also hoisted to the top, but not initialized.

Using a **const** variable before it is declared will result in a **ReferenceError**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Const Hoisting</h4>
<p id="demo"></p>

<script>
try {
  // Using const variable before declaring it is not allowed (hoisting but not
  initialized)
  const x = 5;
  // Uncommenting the line below would cause an error:
  // console.log(y);
  // const y = 10;
  document.getElementById("demo").innerHTML = "const x: " + x + " - Variables defined
  with const are hoisted to the top but not initialized.";
} catch(err) {
  document.getElementById("demo").innerHTML = "Error: " + err.message;
}
</script>

</body>
</html>

```

Example 7

Result [View Example](#)

Reference

- [W3Schools JavaScript Const](#)

JavaScript Data Types

Table of Contents

- [JavaScript Data Types](#)
- [JavaScript has 8 Datatypes](#)
- [Example 1](#)
- [The typeof Operator](#)
- [Example 2](#)
- [JavaScript Strings](#)
- [Example 3](#)
- [JavaScript Numbers](#)
- [Example 4](#)
- [Exponential Notation](#)
- [Example 5](#)
- [JavaScript Booleans](#)
- [Example 6](#)
- [Datatype undefined](#)
- [Example 7](#)
- [Empty Values](#)
- [Example 8](#)
- [Reference](#)

JavaScript has 8 Datatypes

JavaScript has 8 datatypes:

1. **String**
2. **Number**
3. **Bigint**
4. **Boolean**
5. **Undefined**
6. **Null**
7. **Symbol**
8. **Object**

Objects can be: An object, an array, or a date.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript has 8 Datatypes</h4>
<p id="demo1"></p>
```

```

<p id="demo2"></p>
<p id="demo3"></p>
<p id="demo4"></p>
<p id="demo5"></p>

<script>
// Numbers:
let length = 16;
let weight = 7.5;

// Strings:
let color = "Yellow";
let lastName = "Johnson";

// Booleans:
let x = true;
let y = false;

// Object:
const person = {firstName:"John", lastName:"Doe"};

// Array object:
const cars = ["Saab", "Volvo", "BMW"];

// Date object:
const date = new Date("2022-03-25");

document.getElementById("demo1").innerHTML = "Number: " + length + ", " + weight;
document.getElementById("demo2").innerHTML = "String: " + color + ", " + lastName;
document.getElementById("demo3").innerHTML = "Boolean: " + x + ", " + y;
document.getElementById("demo4").innerHTML = "Object: " + person.firstName + " " +
person.lastName;
document.getElementById("demo5").innerHTML = "Array: " + cars;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

The typeof Operator

You can use the **typeof** operator to find the data type of a JavaScript variable:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

```

```

<h4>The typeof Operator</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "typeof 'John' is: " + typeof "John" + "<br>" +
  "typeof 3.14 is: " + typeof 3.14 + "<br>" +
  "typeof true is: " + typeof true + "<br>" +
  "typeof x is: " + typeof x + "<br>" +
  "typeof [1,2,3] is: " + typeof [1,2,3] + "<br>" +
  "typeof {name:'John'} is: " + typeof {name:"John"};
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

JavaScript Strings

A string (or a text string) is a series of characters like "John Doe".

Strings are written with quotes. You can use single or double quotes:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Strings</h4>
<p id="demo"></p>

<script>
let carName1 = "Volvo XC60";
let carName2 = 'Volvo XC60';
document.getElementById("demo").innerHTML =
  carName1 + "<br>" +
  carName2;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JavaScript Numbers

All JavaScript numbers are stored as decimal numbers. Floating point arithmetic is not always 100% accurate:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Numbers</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x1 = 34.00;
let x2 = 34;
let x3 = 3.14;

document.getElementById("demo1").innerHTML =
  "x1 = " + x1 + "<br>" +
  "x2 = " + x2 + "<br>" +
  "x3 = " + x3;

let y1 = 123e5;
let y2 = 123e-5;
document.getElementById("demo2").innerHTML =
  "123e5 = " + y1 + "<br>" +
  "123e-5 = " + y2;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Exponential Notation

Extra large or extra small numbers can be written with scientific (exponential) notation:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Exponential Notation</h4>
<p id="demo"></p>
```

```

<script>
let y1 = 123e5;
let y2 = 123e-5;
document.getElementById("demo").innerHTML =
  "123e5 = " + y1 + " (large number)" + "<br>" +
  "123e-5 = " + y2 + " (small number)";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

JavaScript Booleans

Booleans can only have two values: **true** or **false**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Booleans</h4>
<p id="demo"></p>

<script>
let x = 5;
let y = 5;
let z = 6;
document.getElementById("demo").innerHTML =
  "(x == y) = " + (x == y) + "<br>" +
  "(x == z) = " + (x == z);
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

Datatype undefined

A variable without a value has the value **undefined**. The type is also **undefined**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Datatype undefined</h4>
<p id="demo"></p>

<script>
let car;
document.getElementById("demo").innerHTML =
  "Value: " + car + "<br>" +
  "Type: " + typeof car;
</script>

</body>
</html>

```

Example 7

Result [View Example](#)

Empty Values

An empty value has nothing to do with **undefined**. An empty string has both a legal value and a type:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Empty Values</h4>
<p id="demo"></p>

<script>
let car = "";
document.getElementById("demo").innerHTML =
  "Value: '" + car + "'<br>" +
  "Type: " + typeof car;
</script>

</body>
</html>

```

Example 8

Result [View Example](#)

Reference

- [W3Schools JavaScript Data Types](#)

JavaScript Operators

Table of Contents

- [JavaScript Operators](#)
- [JavaScript Assignment](#)
- [Example 1](#)
- [JavaScript Addition](#)
- [Example 2](#)
- [JavaScript Multiplication](#)
- [Example 3](#)
- [Types of JavaScript Operators](#)
- [JavaScript Arithmetic Operators](#)
- [Example 4](#)
- [JavaScript String Addition](#)
- [Example 5](#)
- [Adding Strings and Numbers](#)
- [Example 6](#)
- [JavaScript Assignment Operators](#)
- [Example 7](#)
- [JavaScript Comparison Operators](#)
- [Example 8](#)
- [Reference](#)

JavaScript Assignment

The **Assignment Operator** (=) assigns a value to a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Assignment</h4>
<p id="demo"></p>

<script>
let x = 10;
document.getElementById("demo").innerHTML = "x = " + x;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Addition

The **Addition Operator** (+) adds numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Addition</h4>
<p id="demo"></p>

<script>
let x = 5;
let y = 2;
let z = x + y;
document.getElementById("demo").innerHTML = "x + y = " + z;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Multiplication

The **Multiplication Operator** (*) multiplies numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Multiplication</h4>
<p id="demo"></p>

<script>
let x = 5;
let y = 2;
let z = x * y;
document.getElementById("demo").innerHTML = "x * y = " + z;
</script>
```

```
</body>
</html>
```

Example 3

Result [View Example](#)

Types of JavaScript Operators

There are different types of JavaScript operators:

- **Arithmetic Operators:** `+`, `-`, `*`, `/`, `%`, `++`, `--`
- **Assignment Operators:** `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`
- **String Operators:** `+`, `+=`
- **Comparison Operators:** `==`, `===`, `!=`, `!==`, `>`, `<`, `>=`, `<=`, `?`
- **Logical Operators:** `&&`, `||`, `!`

JavaScript Arithmetic Operators

Arithmetic Operators are used to perform arithmetic on numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arithmetic Operators</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>
<p id="demo4"></p>

<script>
let a = 10;
let b = 3;
document.getElementById("demo1").innerHTML = "a + b = " + (a + b);
document.getElementById("demo2").innerHTML = "a - b = " + (a - b);
document.getElementById("demo3").innerHTML = "a * b = " + (a * b);
document.getElementById("demo4").innerHTML = "a / b = " + (a / b);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript String Addition

The **+** operator can also be used to add (concatenate) strings:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Addition (Concatenation)</h4>
<p id="demo"></p>

<script>
let text1 = "John";
let text2 = "Doe";
let text3 = text1 + " " + text2;
document.getElementById("demo").innerHTML = text3;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Adding Strings and Numbers

Adding two numbers will return the sum as a number.

Adding a number and a string will return the sum as a concatenated string:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Strings and Numbers</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
let x = 5 + 5;
let y = "5" + 5;
let z = "Hello" + 5;
document.getElementById("demo1").innerHTML = "5 + 5 = " + x + " (number)";
document.getElementById("demo2").innerHTML = "'5' + 5 = ' + y + " (string)";
```



```
document.getElementById("demo3").innerHTML = '"Hello" + 5 = ' + z + " (string)";
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

JavaScript Assignment Operators

Assignment operators assign values to JavaScript variables.

The **Addition Assignment Operator** (**+=**) adds a value to a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Assignment Operators</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
let x = 10;
let y = 10;
let z = 10;
x += 5;
y -= 3;
z *= 2;
document.getElementById("demo1").innerHTML = "x += 5 => " + x;
document.getElementById("demo2").innerHTML = "y -= 3 => " + y;
document.getElementById("demo3").innerHTML = "z *= 2 => " + z;
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

JavaScript Comparison Operators

Comparison operators are used to **compare two values** and always return **true** or **false**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Operators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Comparison Operators</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
let x = 5;
document.getElementById("demo1").innerHTML = "5 == '5' is " + (5 == "5");
document.getElementById("demo2").innerHTML = "5 === '5' is " + (5 === "5");
document.getElementById("demo3").innerHTML = "5 > 8 is " + (5 > 8);
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Reference

- [W3Schools JavaScript Operators](#)

JavaScript Arithmetic

Table of Contents

- [JavaScript Arithmetic](#)
- [Arithmetic Operators](#)
- [Example 1](#)
- [Adding](#)
- [Example 2](#)
- [Subtracting](#)
- [Example 3](#)
- [Multiplying](#)
- [Example 4](#)
- [Dividing](#)
- [Example 5](#)
- [Remainder](#)
- [Example 6](#)
- [Incrementing](#)
- [Example 7](#)
- [Decrementing](#)
- [Example 8](#)
- [Exponentiation](#)
- [Example 9](#)
- [Operator Precedence](#)
- [Example 10](#)
- [Reference](#)

Arithmetic Operators

Arithmetic operators perform arithmetic on numbers (literals or variables).

A typical arithmetic operation operates on two numbers.

The numbers (in an arithmetic operation) are called **operands**. The operation (to be performed between the two operands) is defined by an **operator**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arithmetic Operators Overview</h4>
<p id="demo"></p>

<script>
let x = 10;
```

```

let y = 3;
document.getElementById("demo").innerHTML =
  "x = " + x + ", y = " + y + "<br><br>" +
  "x + y = " + (x + y) + " (Addition)<br>" +
  "x - y = " + (x - y) + " (Subtraction)<br>" +
  "x * y = " + (x * y) + " (Multiplication)<br>" +
  "x / y = " + (x / y) + " (Division)<br>" +
  "x % y = " + (x % y) + " (Modulus)<br>" +
  "x ** y = " + (x ** y) + " (Exponentiation)";
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Adding

The **addition** operator (+) adds numbers:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding</h4>
<p id="demo"></p>

<script>
let a = 100;
let b = 50;
let x = a + b;
document.getElementById("demo").innerHTML = "a + b = " + x;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Subtracting

The **subtraction** operator (-) subtracts numbers:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Subtracting</h4>
<p id="demo"></p>

<script>
let a = 100;
let b = 50;
let x = a - b;
document.getElementById("demo").innerHTML = "a - b = " + x;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Multiplying

The **multiplication** operator (*) multiplies numbers:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Multiplying</h4>
<p id="demo"></p>

<script>
let a = 100;
let b = 50;
let x = a * b;
document.getElementById("demo").innerHTML = "a * b = " + x;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Dividing

The **division** operator (/) divides numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Dividing</h4>
<p id="demo"></p>

<script>
let a = 100;
let b = 50;
let x = a / b;
document.getElementById("demo").innerHTML = "a / b = " + x;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Remainder

The **modulus** operator (%) returns the division remainder:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Remainder (Modulus)</h4>
<p id="demo"></p>

<script>
let a = 17;
let b = 5;
let x = a % b;
document.getElementById("demo").innerHTML = "a % b = " + x + " (remainder of " + a + "
/ " + b + ")";
</script>
```

```
</body>
</html>
```

Example 6

Result [View Example](#)

Incrementing

The **increment** operator (**++**) increments numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Incrementing</h4>
<p id="demo"></p>

<script>
let x = 5;
x++;
let y = x;
document.getElementById("demo").innerHTML = "x = 5, x++ => y = " + y;
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

Decrementing

The **decrement** operator (**--**) decrements numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Decrementing</h4>
<p id="demo"></p>
```

```
<script>
let x = 5;
x--;
let y = x;
document.getElementById("demo").innerHTML = "x = 5, x-- => y = " + y;
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Exponentiation

The **exponentiation** operator (******) raises the first operand to the power of the second operand:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Exponentiation</h4>
<p id="demo"></p>

<script>
let x = 5;
let y = x ** 2;
document.getElementById("demo").innerHTML = "x = 5, x ** 2 = " + y;
</script>

</body>
</html>
```

Example 9

Result [View Example](#)

Operator Precedence

Operator precedence describes the order in which operations are performed in an arithmetic expression.

Multiplication (*****) and division (**/**) have higher **precedence** than addition (**+**) and subtraction (**-**).

Precedence can be changed by using parentheses:


```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arithmetic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Operator Precedence</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
document.getElementById("demo1").innerHTML = "100 + 50 * 3 = " + (100 + 50 * 3);
document.getElementById("demo2").innerHTML = "(100 + 50) * 3 = " + ((100 + 50) * 3);
document.getElementById("demo3").innerHTML = "100 + 50 - 3 * 2 = " + (100 + 50 - 3 *
2);
</script>

</body>
</html>
```

Example 10

Result [View Example](#)

Reference

- [W3Schools JavaScript Arithmetic](#)

JavaScript Assignment

Table of Contents

- [JavaScript Assignment](#)
- [The = Operator](#)
- [Example 1](#)
- [The += Operator](#)
- [Example 2](#)
- [The -= Operator](#)
- [Example 3](#)
- [The *= Operator](#)
- [Example 4](#)
- [The **= Operator](#)
- [Example 5](#)
- [The /= Operator](#)
- [Example 6](#)
- [The %= Operator](#)
- [Example 7](#)
- [String Assignment](#)
- [Example 8](#)
- [Reference](#)

JavaScript Assignment Operators

Assignment operators assign values to JavaScript variables.

Given that $x = 10$ and $y = 5$, the table below explains the assignment operators:

Operator	Example	Same As	Result
=	$x = y$	$x = y$	$x = 5$
+=	$x += y$	$x = x + y$	$x = 15$
-=	$x -= y$	$x = x - y$	$x = 5$
*=	$x *= y$	$x = x * y$	$x = 50$
/=	$x /= y$	$x = x / y$	$x = 2$
%=	$x \% = y$	$x = x \% y$	$x = 0$
**=	$x ** = y$	$x = x ** y$	$x = 100000$

The = Operator

The **Simple Assignment Operator** assigns a simple value to a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Assignment</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The = Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
document.getElementById("demo").innerHTML = "x = " + x;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The += Operator

The **Addition Assignment Operator** adds a value to a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Assignment</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The += Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
x += 5;
document.getElementById("demo").innerHTML = "x += 5 => " + x;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The -= Operator

The **Subtraction Assignment Operator** subtracts a value from a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Assignment</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The -= Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
x -= 5;
document.getElementById("demo").innerHTML = "x -= 5 => " + x;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The *= Operator

The **Multiplication Assignment Operator** multiplies a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Assignment</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The *= Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
x *= 5;
document.getElementById("demo").innerHTML = "x *= 5 => " + x;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The **= Operator

The **Exponentiation Assignment Operator** raises a variable to the power of the operand:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Assignment</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The **= Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
x **= 5;
document.getElementById("demo").innerHTML = "x **= 5 => " + x;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

The /= Operator

The **Division Assignment Operator** divides a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Assignment</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The /= Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
x /= 5;
document.getElementById("demo").innerHTML = "x /= 5 => " + x;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 6

Result [View Example](#)

The %= Operator

The **Remainder Assignment Operator** assigns a remainder to a variable:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Assignment</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>The %= Operator</h4>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let x = 10;
```

```
x %= 5;
```

```
document.getElementById("demo").innerHTML = "x %= 5 => " + x;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 7

Result [View Example](#)

String Assignment

Two assignment operators can assign values to strings:

The **Simple Assignment Operator** (=) assigns a simple value to a string.

The **Addition Assignment Operator** (+=) adds (concatenates) strings:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Assignment</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>String Assignment</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let text1 = "Hello";
let text2 = "World";
text2 += "!";
document.getElementById("demo1").innerHTML = "text1 = " + text1;
document.getElementById("demo2").innerHTML = "text2 += \"!\" => " + text2;
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Reference

- [W3Schools JavaScript Assignment](#)

JavaScript Comparisons

Table of Contents

- [JavaScript Comparisons](#)
- [Comparison Operators](#)
- [Example 1](#)
- [JavaScript String Comparison](#)
- [Example 2](#)
- [Comparing Different Types](#)
- [Example 3](#)
- [Reference](#)

Comparison Operators

Comparison operators are used to **compare two values** and always return **true** or **false**.

Given that `x = 5`, the table below explains the comparison operators:

Operator	Description	Comparing	Returns
<code>==</code>	equal to	<code>x == 8</code>	<code>false</code>
<code>==</code>	equal to	<code>x == 5</code>	<code>true</code>
<code>==</code>	equal to	<code>x == "5"</code>	<code>true</code>
<code>===</code>	equal value and type	<code>x === 5</code>	<code>true</code>
<code>===</code>	equal value and type	<code>x === "5"</code>	<code>false</code>
<code>!=</code>	not equal	<code>x != 8</code>	<code>true</code>
<code>!==</code>	not equal value or type	<code>x !== 5</code>	<code>false</code>
<code>!==</code>	not equal value or type	<code>x !== "5"</code>	<code>true</code>
<code>></code>	greater than	<code>x > 8</code>	<code>false</code>
<code><</code>	less than	<code>x < 8</code>	<code>true</code>
<code>>=</code>	greater than or equal to	<code>x >= 8</code>	<code>false</code>
<code><=</code>	less than or equal to	<code>x <= 8</code>	<code>true</code>


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Comparisons</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Comparison Operators</h4>
<p id="demo"></p>

<script>
let x = 5;
document.getElementById("demo").innerHTML =
  "x = " + x + "<br><br>" +
  "x == 8 : " + (x == 8) + "<br>" +
  "x == 5 : " + (x == 5) + "<br>" +
  "x == '5' : " + (x == "5") + "<br>" +
  "x === 5 : " + (x === 5) + "<br>" +
  "x === '5' : " + (x === "5") + "<br>" +
  "x != 8 : " + (x != 8) + "<br>" +
  "x > 8 : " + (x > 8) + "<br>" +
  "x < 8 : " + (x < 8) + "<br>" +
  "x >= 8 : " + (x >= 8) + "<br>" +
  "x <= 8 : " + (x <= 8);
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

JavaScript String Comparison

All the comparison operators can also be used on strings. Note that strings are compared alphabetically:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Comparisons</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Comparison</h4>
<p id="demo"></p>

<script>
let text1 = "A";
let text2 = "B";
let result = text1 < text2;
document.getElementById("demo").innerHTML =

```

```

    "A < B : " + result + "<br><br>" +
    "\"Apple\" < \"Banana\" : " + ("Apple" < "Banana") + "<br>" +
    "\"John\" > \"Jane\" : " + ("John" > "Jane");
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Comparing Different Types

Comparing data of different types may give unexpected results.

When comparing a string with a number, JavaScript will convert the string to a number when doing the comparison.

An empty string converts to 0. A non-numeric string converts to **NaN** which is always **false**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Comparisons</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Comparing Different Types</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
    "2 < 12 : " + (2 < 12) + "<br>" +
    "\"2\" < \"12\" : " + ("2" < "12") + "<br>" +
    "\"2\" < 12 : " + ("2" < 12) + "<br>" +
    "\"\" == 0 : " + ("\" == 0) + "<br>" +
    "\"\" === 0 : " + ("\" === 0);
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Comparisons](#)

JavaScript Conditions

Table of Contents

- [JavaScript Conditions](#)
- [The if Statement](#)
- [Example 1](#)
- [The else Statement](#)
- [Example 2](#)
- [The else if Statement](#)
- [Example 3](#)
- [Reference](#)

The if Statement

Use the **if** statement to specify a block of JavaScript code to be executed if a condition is **true**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Conditions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The if Statement</h4>
<p id="demo"></p>

<script>
let hour = 12;
if (hour < 18) {
  document.getElementById("demo").innerHTML = "Good day!";
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The else Statement

Use the **else** statement to specify a block of code to be executed if a condition is **false**.

If the hour is less than 18, create a "Good day" greeting, otherwise "Good evening":

```
<!DOCTYPE html>
<html>
```

```

<body>

<h2>JavaScript Conditions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The else Statement</h4>
<p id="demo"></p>

<script>
let hour = 20;
if (hour < 18) {
    document.getElementById("demo").innerHTML = "Good day!";
} else {
    document.getElementById("demo").innerHTML = "Good evening!";
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

The else if Statement

Use the **else if** statement to specify a new condition if the first condition is **false**.

If time is less than 10:00, create a "Good morning" greeting, if not but time is less than 20:00, create a "Good day" greeting, otherwise a "Good evening":

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Conditions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The else if Statement</h4>
<p id="demo"></p>

<script>
let time = 14;
if (time < 10) {
    document.getElementById("demo").innerHTML = "Good morning!";
} else if (time < 20) {
    document.getElementById("demo").innerHTML = "Good day!";
} else {
    document.getElementById("demo").innerHTML = "Good evening!";
}
</script>

```

```
</body>  
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Conditions](#)

JavaScript For Loop

Table of Contents

- [JavaScript For Loop](#)
- [The For Loop](#)
- [Example 1](#)
- [How to use exp 1](#)
- [Example 2](#)
- [How to use exp 2](#)
- [Example 3](#)
- [How to use exp 3](#)
- [Example 4](#)
- [Loop Scope](#)
- [Example 5](#)
- [Reference](#)

The For Loop

The **for** statement creates a loop with 3 optional expressions:

exp 1 is executed (one time) before the execution of the code block.

exp 2 defines the condition for executing the code block.

exp 3 is executed (every time) after the code block has been executed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The For Loop</h4>
<p id="demo"></p>

<script>
let text = "";
for (let i = 0; i < 5; i++) {
  text += "Number: " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

How to use exp 1

exp 1 is used to initialize the variable(s) used in the loop (let i = 0).

exp 1 is optional. You can omit exp 1 if the value is set before the loop starts:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How to use exp 1</h4>
<p id="demo"></p>

<script>
let i = 0;
let text = "";
for (; i < 5; i++) {
    text += "Number: " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

How to use exp 2

exp 2 is used to evaluate the condition of the initial variable. If exp 2 returns **false**, the loop will end.

If you omit exp 2, you must provide a **break** inside the loop. Otherwise the loop will never end:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How to use exp 2</h4>
<p id="demo"></p>

<script>
let text = "";
```



```

let i = 0;
for ( ; ; i++) {
  if (i > 4) break;
  text += "Number: " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

How to use exp 3

exp 3 increments the value of the initial variable (i++).

exp 3 is optional. It can be omitted if you increment the value inside the loop:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How to use exp 3</h4>
<p id="demo"></p>

<script>
let text = "";
let i = 0;
for ( ; i < 5; ) {
  text += "Number: " + i + "<br>";
  i++;
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Loop Scope

Using **var** in a loop: the variable declared in the loop redeclares the variable outside the loop.

Using **let** in a loop: the variable declared in the loop does not redeclare the variable outside the loop:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loop Scope (let)</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let text = "";
for (let i = 0; i < 5; i++) {
    text += i + " ";
}
document.getElementById("demo1").innerHTML = "Loop values: " + text;
// i is NOT accessible here (let has block scope)
document.getElementById("demo2").innerHTML = "Using let: i is not accessible outside
the loop";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript For Loop](#)

JavaScript While Loop

Table of Contents

- [JavaScript While Loop](#)
- [The While Loop](#)
- [Example 1](#)
- [The Do While Loop](#)
- [Example 2](#)
- [Comparing For and While](#)
- [Example 3](#)
- [Reference](#)

The While Loop

The **while** loop loops through a block of code as long as a specified condition is **true**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript While Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The While Loop</h4>
<p id="demo"></p>

<script>
let text = "";
let i = 0;
while (i < 5) {
  text += "Number: " + i + "<br>";
  i++;
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Do While Loop

The **do while** loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, and then it will repeat the loop as long as the condition is true.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript While Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Do While Loop</h4>
<p id="demo"></p>

<script>
let text = "";
let i = 0;
do {
  text += "Number: " + i + "<br>";
  i++;
}
while (i < 5);
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Comparing For and While

If you have read the previous chapter about the for loop, you will discover that a while loop is much the same as a for loop, with statement 1 and statement 3 omitted.

The loop in this example uses a **for** loop to collect the car names from the cars array:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript While Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Comparing For and While</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const cars = ["BMW", "Volvo", "Saab", "Ford"];

// Using for loop
let text1 = "";

```

```
for (let i = 0; i < cars.length; i++) {  
  text1 += cars[i] + "<br>";  
}  
document.getElementById("demo1").innerHTML = "For loop:<br>" + text1;  
  
// Using while loop  
let text2 = "";  
let j = 0;  
while (j < cars.length) {  
  text2 += cars[j] + "<br>";  
  j++;  
}  
document.getElementById("demo2").innerHTML = "While loop:<br>" + text2;  
</script>  
  
</body>  
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript While Loop](#)

JavaScript Strings

Table of Contents

- [JavaScript Strings](#)
- [Using Quotes](#)
- [Example 1](#)
- [Quotes Inside Quotes](#)
- [Example 2](#)
- [String Length](#)
- [Example 3](#)
- [Escape Characters](#)
- [Example 4](#)
- [Breaking Long Lines](#)
- [Example 5](#)
- [Template Strings](#)
- [Example 6](#)
- [JavaScript Strings as Objects](#)
- [Example 7](#)
- [Reference](#)

Using Quotes

A JavaScript string is zero or more characters written inside quotes.

You can use single or double quotes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strings</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Quotes</h4>
<p id="demo"></p>

<script>
let carName1 = "Volvo XC60";
let carName2 = 'Volvo XC60';
document.getElementById("demo").innerHTML =
  carName1 + "<br>" + carName2;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Quotes Inside Quotes

You can use quotes inside a string, as long as they don't match the surrounding quotes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strings</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Quotes Inside Quotes</h4>
<p id="demo"></p>

<script>
let answer1 = "It's alright";
let answer2 = "He is called 'Johnny'";
let answer3 = 'He is called "Johnny"';
document.getElementById("demo").innerHTML =
  answer1 + "<br>" + answer2 + "<br>" + answer3;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

String Length

The **length** property returns the length of a string:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strings</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Length</h4>
<p id="demo"></p>

<script>
let text = "ABCDEFGHJKLMNOPQRSTUVWXYZ";
document.getElementById("demo").innerHTML = "Length: " + text.length;
</script>

</body>
```

```
</html>
```

Example 3

Result [View Example](#)

Escape Characters

The backslash escape character `\` turns special characters into string characters:

Code	Result
<code>\'</code>	<code>'</code>
<code>\"</code>	<code>"</code>
<code>\\</code>	<code>\</code>
<code>\n</code>	new line

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strings</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Escape Characters</h4>
<p id="demo"></p>

<script>
let text = "We are the so-called \"Vikings\" from the north.";
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Breaking Long Lines

For best readability, programmers often like to avoid code lines longer than 80 characters. You can break up a code line within a text string with a single backslash:


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strings</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Breaking Long Lines</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello \
Dolly!";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Template Strings

Template strings use backticks (` ``) instead of quotes to define a string.

With template strings you can use both single and double quotes inside a string:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strings</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Template Strings</h4>
<p id="demo"></p>

<script>
let text = `He's often called "Johnny"`;
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

JavaScript Strings as Objects

Normally, JavaScript strings are primitive values, created from literals.

But strings can also be defined as objects with the keyword **new**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strings</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Strings as Objects</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = "John";
let y = new String("John");
document.getElementById("demo1").innerHTML = typeof x + "<br>" + typeof y;
document.getElementById("demo2").innerHTML = "x === y is " + (x === y);
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

Reference

- [W3Schools JavaScript Strings](#)

JavaScript Numbers

Table of Contents

- [JavaScript Numbers](#)
- [Number Types](#)
- [Example 1](#)
- [Integer Precision](#)
- [Example 2](#)
- [Adding Numbers and Strings](#)
- [Example 3](#)
- [Numeric Strings](#)
- [Example 4](#)
- [NaN - Not a Number](#)
- [Example 5](#)
- [Infinity](#)
- [Example 6](#)
- [Hexadecimal](#)
- [Example 7](#)
- [JavaScript Numbers as Objects](#)
- [Example 8](#)
- [Reference](#)

Number Types

JavaScript has only one type of number. Numbers can be written with or without decimals:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Number Types</h4>
<p id="demo"></p>

<script>
let x = 3.14;    // A number with decimals
let y = 3;      // A number without decimals
let z = 123e5;  // Exponential notation
document.getElementById("demo").innerHTML =
  x + "<br>" + y + "<br>" + z;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Integer Precision

Integers (numbers without a period or exponent notation) are accurate up to 15 digits:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Integer Precision</h4>
<p id="demo"></p>

<script>
let x = 999999999999999; // 15 digits
let y = 9999999999999999; // 16 digits
document.getElementById("demo").innerHTML =
  "15 digits: " + x + "<br>" +
  "16 digits: " + y;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Adding Numbers and Strings

When adding numbers and strings, JavaScript treats numbers as strings when a string is present:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Numbers and Strings</h4>
<p id="demo"></p>

<script>
let x = 10;
let y = 20;
let z = "The result is: " + x + y;
```

```
document.getElementById("demo").innerHTML = z;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Numeric Strings

JavaScript strings can have numeric content. JavaScript will try to convert strings to numbers in some operations:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numeric Strings</h4>
<p id="demo"></p>

<script>
let x = "100";
let y = "10";
let z = x / y;
document.getElementById("demo").innerHTML = "100 / 10 = " + z;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

NaN - Not a Number

NaN is a JavaScript reserved word indicating that a number is not a legal number:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>NaN - Not a Number</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = 100 / "Apple";
document.getElementById("demo1").innerHTML = "100 / 'Apple' = " + x;
document.getElementById("demo2").innerHTML = "isNaN(100 / 'Apple'): " + isNaN(x);
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Infinity

Infinity (or **-Infinity**) is the value JavaScript will return if you calculate a number outside the largest possible number:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Infinity</h4>
<p id="demo"></p>

<script>
let x = 2 / 0;
let y = -2 / 0;
document.getElementById("demo").innerHTML =
  "2 / 0 = " + x + "<br>" +
  "-2 / 0 = " + y + "<br>" +
  "typeof Infinity: " + typeof x;
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

Hexadecimal

JavaScript interprets numeric constants as hexadecimal if they are preceded by **0x**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hexadecimal</h4>
<p id="demo"></p>

<script>
let x = 0xFF;
document.getElementById("demo").innerHTML = "0xFF = " + x;
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

JavaScript Numbers as Objects

Normally, JavaScript numbers are primitive values created from literals.

But numbers can also be defined as objects with the keyword **new**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numbers as Objects</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = 123;
let y = new Number(123);
document.getElementById("demo1").innerHTML = typeof x + "<br>" + typeof y;
document.getElementById("demo2").innerHTML = "x === y is " + (x === y);
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Reference

- [W3Schools JavaScript Numbers](#)

JavaScript Functions

Table of Contents

- [JavaScript Functions](#)
- [Functions are Code Blocks](#)
- [Example 1](#)
- [Functions Run When You Call Them](#)
- [Example 2](#)
- [JavaScript Function Syntax](#)
- [Example 3](#)
- [A Function Can Be Used Many Times](#)
- [Example 4](#)
- [Local Variables](#)
- [Example 5](#)
- [Functions Used as Variables](#)
- [Example 6](#)
- [Reference](#)

Functions are Code Blocks

A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions are Code Blocks</h4>
<p id="demo"></p>

<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Hello World!";
}
myFunction();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Functions Run When You Call Them

Functions run when you call them. You can call the function by its name:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions Run When You Call Them</h4>
<p id="demo"></p>

<script>
function myFunction(a, b) {
  return a * b;
}
let result = myFunction(4, 3);
document.getElementById("demo").innerHTML = "4 * 3 = " + result;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Function Syntax

A JavaScript function is defined with the **function** keyword, followed by a **name**, followed by parentheses **()**.

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The parentheses may include parameter names separated by commas:

(parameter1, parameter2, ...)

The code to be executed, by the function, is placed inside curly brackets: **{ }**

```
function name(parameter1, parameter2, parameter3) {
  // code to be executed
}
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>JavaScript Function Syntax</h4>
<p id="demo"></p>

<script>
// Function is called, the return value will end up in x
let x = myFunction(4, 3);
document.getElementById("demo").innerHTML = x;

function myFunction(a, b) {
  // Function returns the product of a and b
  return a * b;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

A Function Can Be Used Many Times

Functions can be reused to perform the same task with different values:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Function Can Be Used Many Times</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
function toCelsius(fahrenheit) {
  return (5/9) * (fahrenheit - 32);
}

document.getElementById("demo1").innerHTML = "32°F = " + toCelsius(32) + "°C";
document.getElementById("demo2").innerHTML = "68°F = " + toCelsius(68) + "°C";
document.getElementById("demo3").innerHTML = "100°F = " + toCelsius(100) + "°C";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Local Variables

Variables declared within a JavaScript function become **LOCAL** to the function.

Local variables can only be accessed from within the function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Local Variables</h4>
<p id="demo"></p>

<script>
function myFunction() {
  let carName = "Volvo";
  document.getElementById("demo").innerHTML = "Inside function: " + carName;
}
myFunction();
// carName is NOT accessible outside the function
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Functions Used as Variables

Functions can be used the same way as variables, in all types of formulas, assignments, and calculations:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions Used as Variables</h4>
<p id="demo"></p>

<script>
function myFunction(a, b) {
```

```
    return a * b;
}
// Function result used directly as a variable
document.getElementById("demo").innerHTML = "5 * 3 = " + myFunction(5, 3);
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Functions](#)

JavaScript Function Parameters

Table of Contents

- [JavaScript Function Parameters](#)
- [Parameters \(Function Input\)](#)
- [Example 1](#)
- [Functions with One Parameter](#)
- [Example 2](#)
- [Functions with Multiple Parameters](#)
- [Example 3](#)
- [Incorrect Parameters](#)
- [Example 4](#)
- [Default Parameter Values](#)
- [Example 5](#)
- [Reference](#)

Parameters (Function Input)

Function **parameters** are the names listed in the function definition.

Function **arguments** are the real values passed to (and received by) the function.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Parameters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Parameters (Function Input)</h4>
<p id="demo"></p>

<script>
function myFunction(firstName, lastName) {
    return "Hello " + firstName + " " + lastName;
}
document.getElementById("demo").innerHTML = myFunction("John", "Doe");
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Functions with One Parameter

A function can have a single parameter:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Parameters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions with One Parameter</h4>
<p id="demo"></p>

<script>
function square(x) {
    return x * x;
}
document.getElementById("demo").innerHTML = "Square of 5: " + square(5);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Functions with Multiple Parameters

A function can have multiple parameters separated by commas:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Parameters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions with Multiple Parameters</h4>
<p id="demo"></p>

<script>
function sum(a, b, c) {
    return a + b + c;
}
document.getElementById("demo").innerHTML = "Sum of 5, 10, 15 = " + sum(5, 10, 15);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Incorrect Parameters

If a function is called with missing arguments (fewer than declared), the missing values are set to **undefined**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Parameters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Incorrect Parameters</h4>
<p id="demo"></p>

<script>
function multiply(a, b) {
    return a * b;
}
// Only one argument passed, b will be undefined
document.getElementById("demo").innerHTML = "multiply(5) = " + multiply(5) + " (NaN
because b is undefined)";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Default Parameter Values

ES6 allows function parameters to have default values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Parameters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Default Parameter Values</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function myFunction(x, y = 10) {
```



```
    return x + y;
}
document.getElementById("demo1").innerHTML = "myFunction(5) = " + myFunction(5);
document.getElementById("demo2").innerHTML = "myFunction(5, 3) = " + myFunction(5, 3);
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Function Parameters](#)

JavaScript Function Return

Table of Contents

- [JavaScript Function Return](#)
- [The return Statement](#)
- [Example 1](#)
- [Returning a Calculated Value](#)
- [Example 2](#)
- [Using Return Values in Expressions](#)
- [Example 3](#)
- [Return Statements Stop Execution](#)
- [Example 4](#)
- [Functions Without return](#)
- [Example 5](#)
- [Returning Values Early](#)
- [Example 6](#)
- [Reference](#)

The return Statement

When JavaScript reaches a **return** statement, the function will stop executing.

If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The return Statement</h4>
<p id="demo"></p>

<script>
function myFunction(a, b) {
  return a * b;
}
document.getElementById("demo").innerHTML = myFunction(4, 3);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Returning a Calculated Value

A function can return a calculated value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Returning a Calculated Value</h4>
<p id="demo"></p>

<script>
function calculateArea(width, height) {
    return width * height;
}
let area = calculateArea(5, 10);
document.getElementById("demo").innerHTML = "Area: " + area;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using Return Values in Expressions

The return value of a function can be used directly in expressions:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Return Values in Expressions</h4>
<p id="demo"></p>

<script>
function add(x, y) {
    return x + y;
}
// Using return value directly in expression
let result = add(5, 10) * 2;
document.getElementById("demo").innerHTML = "(5 + 10) * 2 = " + result;
</script>
```

```
</body>
</html>
```

Example 3

Result [View Example](#)

Return Statements Stop Execution

After the **return** statement is executed, the rest of the code in the function does NOT execute:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Return Statements Stop Execution</h4>
<p id="demo"></p>

<script>
function myFunction() {
  return "This is returned";
  // Code after return will NOT execute
  let x = 5; // This line is ignored
}
document.getElementById("demo").innerHTML = myFunction();
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Functions Without return

A function without a **return** statement will return **undefined**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions Without return</h4>
```

```

<p id="demo"></p>

<script>
function myFunction(a, b) {
  a + b;
  // No return statement
}
let result = myFunction(5, 3);
document.getElementById("demo").innerHTML = "Return value: " + result;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Returning Values Early

You can use **return** to exit a function early when a condition is met:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Returning Values Early</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function checkAge(age) {
  if (age < 18) {
    return "Too young!";
  }
  return "Access granted";
}
document.getElementById("demo1").innerHTML = "Age 15: " + checkAge(15);
document.getElementById("demo2").innerHTML = "Age 21: " + checkAge(21);
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Function Return](#)

JavaScript Function Arguments

Table of Contents

- [JavaScript Function Arguments](#)
- [Parameters vs. Arguments](#)
- [Example 1](#)
- [The Arguments Object](#)
- [Example 2](#)
- [The Order of Arguments Matters](#)
- [Example 3](#)
- [Arguments Can Be Variables](#)
- [Example 4](#)
- [Missing Arguments](#)
- [Example 5](#)
- [Default Parameters](#)
- [Example 6](#)
- [Function Rest Parameter](#)
- [Example 7](#)
- [Reference](#)

Parameters vs. Arguments

Function **parameters** are the names listed in the function definition.

Function **arguments** are the real values passed to (and received by) the function.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Parameters vs. Arguments</h4>
<p id="demo"></p>

<script>
function myFunction(firstName, lastName) {
  return "Hello " + firstName + " " + lastName;
}
// "John" and "Doe" are arguments
document.getElementById("demo").innerHTML = myFunction("John", "Doe");
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Arguments Object

JavaScript functions have a built-in object called the **arguments** object.

The **arguments** object contains an array of the arguments used when the function was called:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Arguments Object</h4>
<p id="demo"></p>

<script>
function findMax() {
  let max = -Infinity;
  for (let i = 0; i < arguments.length; i++) {
    if (arguments[i] > max) {
      max = arguments[i];
    }
  }
  return max;
}
document.getElementById("demo").innerHTML = "Max of (4, 7, 2, 9, 5): " + findMax(4, 7, 2, 9, 5);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The Order of Arguments Matters

The order of arguments passed to a function must match the order of parameters defined:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```



```

<h4>The Order of Arguments Matters</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function carInfo(brand, model, year) {
    return brand + " " + model + " (" + year + ")";
}
// Correct order
document.getElementById("demo1").innerHTML = "Correct: " + carInfo("Toyota",
"Corolla", 2020);
// Wrong order - produces different result
document.getElementById("demo2").innerHTML = "Wrong: " + carInfo(2020, "Toyota",
"Corolla");
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Arguments Can Be Variables

Arguments can be variables, not just literal values:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arguments Can Be Variables</h4>
<p id="demo"></p>

<script>
function add(x, y) {
    return x + y;
}
let a = 10;
let b = 25;
document.getElementById("demo").innerHTML = a + " + " + b + " = " + add(a, b);
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Missing Arguments

If a function is called with missing arguments (fewer than declared), the missing values are set to **undefined**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Missing Arguments</h4>
<p id="demo"></p>

<script>
function multiply(a, b) {
  return a * b;
}
document.getElementById("demo").innerHTML = "multiply(5) = " + multiply(5) + " (NaN
because b is undefined)";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Default Parameters

ES6 allows function parameters to have default values, used when arguments are missing:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Default Parameters</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function myFunction(x, y = 10) {
  return x + y;
}
document.getElementById("demo1").innerHTML = "myFunction(5) = " + myFunction(5);
```

```
document.getElementById("demo2").innerHTML = "myFunction(5, 3) = " + myFunction(5, 3);
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Function Rest Parameter

The rest parameter (`...`) allows a function to treat an indefinite number of arguments as an array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Rest Parameter</h4>
<p id="demo"></p>

<script>
function sum(...args) {
  let total = 0;
  for (let arg of args) {
    total += arg;
  }
  return total;
}
document.getElementById("demo").innerHTML = "Sum of (1, 2, 3, 4, 5): " + sum(1, 2, 3, 4, 5);
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

Reference

- [W3Schools JavaScript Function Arguments](#)

JavaScript Function Expressions

Table of Contents

- [JavaScript Function Expressions](#)
- [What is a Function Expression?](#)
- [Example 1](#)
- [Anonymous Functions](#)
- [Example 2](#)
- [Functions Stored in Variables](#)
- [Example 3](#)
- [Function Declarations vs Function Expressions](#)
- [Example 4](#)
- [Hoisting](#)
- [Example 5](#)
- [Reference](#)

What is a Function Expression?

A JavaScript function can also be defined using an **expression**.

A function expression can be stored in a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is a Function Expression?</h4>
<p id="demo"></p>

<script>
const x = function (a, b) { return a * b };
document.getElementById("demo").innerHTML = "Result: " + x(4, 3);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Anonymous Functions

A function expression can be stored in a variable without a function name, called an **anonymous function**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Anonymous Functions</h4>
<p id="demo"></p>

<script>
const myFunction = function() {
  return "Hello from anonymous function!";
};
document.getElementById("demo").innerHTML = myFunction();
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Functions Stored in Variables

A function expression can be stored in a variable using **var**, **let**, or **const**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions Stored in Variables</h4>
<p id="demo"></p>

<script>
const square = function(x) {
  return x * x;
};
document.getElementById("demo").innerHTML = "Square of 7: " + square(7);
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Function Declarations vs Function Expressions

A **function declaration** defines a named function. A **function expression** defines a function inside an expression:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Declaration vs Expression</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Function Declaration
function sumDeclaration(a, b) {
    return a + b;
}

// Function Expression
const sumExpression = function(a, b) {
    return a + b;
};

document.getElementById("demo1").innerHTML = "Declaration: " + sumDeclaration(5, 3);
document.getElementById("demo2").innerHTML = "Expression: " + sumExpression(5, 3);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Hoisting

Function declarations are hoisted (can be called before they are defined). Function expressions are **NOT** hoisted:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hoisting</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Function declaration is hoisted - can be called before definition
document.getElementById("demo1").innerHTML = "Declaration hoisted: " +
myDeclaration(5);

function myDeclaration(x) {
  return x * 2;
}

// Function expression is NOT hoisted - this would cause an error:
try {
  let result = myExpression(5);
  document.getElementById("demo2").innerHTML = "Expression: " + result;
} catch(err) {
  document.getElementById("demo2").innerHTML = "Expression error: " + err.message;
}

const myExpression = function(x) {
  return x * 2;
};
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Function Expressions](#)

JavaScript Arrow Functions

Table of Contents

- [JavaScript Arrow Functions](#)
- [Arrow Function Syntax](#)
- [Example 1](#)
- [Shorter Syntax](#)
- [Example 2](#)
- [Arrow Functions with One Parameter](#)
- [Example 3](#)
- [Arrow Functions Return Value by Default](#)
- [Example 4](#)
- [Arrow Functions with No Parameters](#)
- [Example 5](#)
- [Arrow Functions Are Not Declarations](#)
- [Example 6](#)
- [Reference](#)

Arrow Function Syntax

Arrow functions were introduced in ES6 (2015).

Arrow functions allow us to write shorter function syntax:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrow Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arrow Function Syntax</h4>
<p id="demo"></p>

<script>
// Traditional function expression
const hello1 = function() {
  return "Hello World!";
};

// Arrow function
const hello2 = () => {
  return "Hello World!";
};

document.getElementById("demo").innerHTML = hello2();
</script>

</body>
```



```
</html>
```

Example 1

Result [View Example](#)

Shorter Syntax

If the function has only one statement, and the statement returns a value, you can remove the brackets and the **return** keyword:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrow Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Shorter Syntax</h4>
<p id="demo"></p>

<script>
// Traditional function
const add1 = function(a, b) {
    return a + b;
};

// Arrow function (shorter)
const add2 = (a, b) => a + b;

document.getElementById("demo").innerHTML = "5 + 3 = " + add2(5, 3);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Arrow Functions with One Parameter

With one parameter, parentheses are optional:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrow Functions</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arrow Functions with One Parameter</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// With parentheses
const square1 = (x) => x * x;

// Without parentheses (one parameter only)
const square2 = x => x * x;

document.getElementById("demo1").innerHTML = "With (): " + square1(5);
document.getElementById("demo2").innerHTML = "Without (): " + square2(5);
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Arrow Functions Return Value by Default

Arrow functions return value by default when using the shorthand syntax without curly brackets:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrow Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arrow Functions Return Value by Default</h4>
<p id="demo"></p>

<script>
const multiply = (a, b) => a * b;
document.getElementById("demo").innerHTML = "6 * 7 = " + multiply(6, 7);
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Arrow Functions with No Parameters

Even with no parameters, we still need empty parentheses:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrow Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arrow Functions with No Parameters</h4>
<p id="demo"></p>

<script>
const greeting = () => "Hello World!";
document.getElementById("demo").innerHTML = greeting();
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Arrow Functions Are Not Declarations

Arrow functions are expressions, NOT declarations. They cannot be hoisted:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrow Functions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arrow Functions Are Not Declarations</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Function declaration can be called before definition
document.getElementById("demo1").innerHTML = "Declaration: " + myFunction(5);

function myFunction(x) {
  return x * 2;
}

// Arrow function is an expression - must be defined before use
const myArrow = (x) => x * 2;
```

```
document.getElementById("demo2").innerHTML = "Arrow: " + myArrow(5);  
</script>  
  
</body>  
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Arrow Functions](#)

JavaScript Timers

Table of Contents

- [JavaScript Timers](#)
- [The setTimeout\(\) Function](#)
- [Example 1](#)
- [Using an Anonymous Function](#)
- [Example 2](#)
- [Canceling a Timeout](#)
- [Example 3](#)
- [The setInterval\(\) Function](#)
- [Example 4](#)
- [Canceling an Interval](#)
- [Example 5](#)
- [A Countdown Example](#)
- [Example 6](#)
- [Reference](#)

The setTimeout() Function

The `setTimeout()` function runs a function after a specified number of milliseconds:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Timers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The setTimeout() Function</h4>
<p>Wait 2 seconds and a message will appear.</p>
<p id="demo"></p>

<script>
setTimeout(function() {
  document.getElementById("demo").innerHTML = "Hello after 2 seconds!";
}, 2000);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using an Anonymous Function

You can pass an anonymous function directly to `setTimeout()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Timers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using an Anonymous Function</h4>
<button onclick="startTimer()">Start Timer</button>
<p id="demo"></p>

<script>
function startTimer() {
  setTimeout(function() {
    document.getElementById("demo").innerHTML = "Timer finished!";
  }, 3000);
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Canceling a Timeout

The `clearTimeout()` function cancels a timeout set with `setTimeout()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Timers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Canceling a Timeout</h4>
<button onclick="startTimer()">Start Timer</button>
<button onclick="cancelTimer()">Cancel Timer</button>
<p id="demo"></p>

<script>
let timeoutId;

function startTimer() {
  timeoutId = setTimeout(function() {
    document.getElementById("demo").innerHTML = "Timer finished!";
  }, 3000);
  document.getElementById("demo").innerHTML = "Timer started...";
}
```

```

}

function cancelTimer() {
  clearTimeout(timeoutId);
  document.getElementById("demo").innerHTML = "Timer canceled!";
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The setInterval() Function

The `setInterval()` function repeats a function at a specified interval:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Timers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The setInterval() Function</h4>
<p id="demo"></p>

<script>
let counter = 0;
setInterval(function() {
  counter++;
  document.getElementById("demo").innerHTML = "Counter: " + counter;
}, 1000);
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Canceling an Interval

The `clearInterval()` function stops an interval set with `setInterval()`:

```

<!DOCTYPE html>

```

```

<html>
<body>

<h2>JavaScript Timers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Canceling an Interval</h4>
<p id="demo"></p>
<button onclick="stopTimer()">Stop</button>

<script>
let counter = 0;
let intervalId = setInterval(function() {
    counter++;
    document.getElementById("demo").innerHTML = "Counter: " + counter;
}, 1000);

function stopTimer() {
    clearInterval(intervalId);
    document.getElementById("demo").innerHTML += " (stopped)";
}
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

A Countdown Example

A countdown timer using `setInterval()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Timers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Countdown Example</h4>
<p id="demo"></p>
<button onclick="startCountdown()">Start Countdown</button>

<script>
function startCountdown() {
    let count = 10;
    document.getElementById("demo").innerHTML = count;

    let intervalId = setInterval(function() {
        count--;

```



```
document.getElementById("demo").innerHTML = count;
if (count === 0) {
  clearInterval(intervalId);
  document.getElementById("demo").innerHTML = "Blast off!";
}
}, 1000);
}
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Timers](#)

JavaScript Objects

Table of Contents

- [JavaScript Objects](#)
- [Object Literals](#)
- [Example 1](#)
- [Accessing Properties](#)
- [Example 2](#)
- [Object Methods](#)
- [Example 3](#)
- [The this Keyword](#)
- [Example 4](#)
- [Objects are Mutable](#)
- [Example 5](#)
- [Reference](#)

Object Literals

A JavaScript object is a collection of properties. Properties are defined as name:value pairs.

Objects can be created using **object literals**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Literals</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  eyeColor: "blue"
};

document.getElementById("demo").innerHTML =
  person.firstName + " " + person.lastName + ", age " + person.age + ", " +
  person.eyeColor + " eyes";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Accessing Properties

You can access object properties in two ways:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Accessing Properties</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Dot notation
document.getElementById("demo1").innerHTML =
  "Dot notation: " + person.firstName + " " + person.lastName;

// Bracket notation
document.getElementById("demo2").innerHTML =
  "Bracket notation: " + person["firstName"] + " " + person["lastName"];

// Using a variable
const key = "age";
document.getElementById("demo3").innerHTML =
  "Using variable: " + person[key];
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Object Methods

Objects can also have methods. A method is a function stored as a property:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Methods</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML =
  "Full name: " + person.fullName();
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The this Keyword

In an object method, **this** refers to the object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The this Keyword</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  fullName: function() {

```

```

        return this.firstName + " " + this.lastName;
    },
    getAge: function() {
        return this.age;
    }
};

document.getElementById("demo").innerHTML =
    person.fullName() + " is " + person.getAge() + " years old.";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Objects are Mutable

Objects are mutable: they are accessed by reference, not by value:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Objects are Mutable</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };

// Copy by reference
const employee = person;
employee.firstName = "Jane";

document.getElementById("demo1").innerHTML =
    "person.firstName: " + person.firstName;

document.getElementById("demo2").innerHTML =
    "employee.firstName: " + employee.firstName + "<br>" +
    "(Both changed because they reference the same object)";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Objects](#)

JavaScript Object Introduction

Table of Contents

- [JavaScript Object Introduction](#)
- [What Are Objects?](#)
- [Example 1](#)
- [Object Properties](#)
- [Example 2](#)
- [Adding and Deleting Properties](#)
- [Example 3](#)
- [Nested Objects](#)
- [Example 4](#)
- [Iterating Over Properties](#)
- [Example 5](#)
- [Reference](#)

What Are Objects?

JavaScript objects are containers for named values called properties.

Objects are variables too. But objects can contain many values.

This code assigns many values (firstName, lastName, age) to a variable named person:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Introduction</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What Are Objects?</h4>
<p id="demo"></p>

<script>
// Create an object
const car = {
  brand: "Toyota",
  model: "Corolla",
  year: 2020,
  color: "blue"
};

// Display object properties
document.getElementById("demo").innerHTML =
  "Car: " + car.brand + " " + car.model + " (" + car.year + ")";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Object Properties

Properties are the most important part of JavaScript objects. Properties can be accessed, added, changed, and removed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Introduction</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Properties</h4>
<p id="demo"></p>

<script>
const student = {
  name: "Alice",
  grade: "A",
  subject: "Mathematics"
};

// Access properties using different methods
let text = "Student: " + student.name + "<br>";
text += "Grade: " + student["grade"] + "<br>";
text += "Subject: " + student["subject"] + "<br><br>";

// List all properties
text += "All properties: " + Object.keys(student).join(", ") + "<br>";
text += "Total properties: " + Object.keys(student).length;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Adding and Deleting Properties

You can add new properties to an object and delete existing ones:


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Introduction</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding and Deleting Properties</h4>
<p id="demo"></p>

<script>
const book = {
  title: "JavaScript Guide",
  author: "John Doe",
  pages: 500
};

let text = "Original properties:<br>";
for (let key in book) {
  text += key + ": " + book[key] + "<br>";
}

// Add a new property
book.publisher = "Tech Books Inc.";
book.year = 2024;

text += "<br>After adding properties:<br>";
for (let key in book) {
  text += key + ": " + book[key] + "<br>";
}

// Delete a property
delete book.pages;

text += "<br>After deleting 'pages':<br>";
for (let key in book) {
  text += key + ": " + book[key] + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Nested Objects

Property values can be other objects, creating nested objects:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Introduction</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Nested Objects</h4>
<p id="demo"></p>

<script>
const user = {
  id: 1001,
  name: "John Doe",
  contact: {
    email: "john@example.com",
    phone: "555-1234",
    address: {
      street: "123 Main St",
      city: "New York",
      zip: "10001"
    }
  }
};

// Access nested properties
document.getElementById("demo").innerHTML =
  "Name: " + user.name + "<br>" +
  "Email: " + user.contact.email + "<br>" +
  "Phone: " + user.contact.phone + "<br>" +
  "City: " + user.contact.address.city + "<br>" +
  "ZIP: " + user.contact.address.zip;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Iterating Over Properties

You can loop through all properties of an object using `for...in`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Introduction</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

```

```
<h4>Iterating Over Properties</h4>
<p id="demo"></p>

<script>
const fruit = {
  name: "Apple",
  color: "Red",
  taste: "Sweet",
  origin: "China",
  price: 1.25
};

let text = "<ul>";
for (let key in fruit) {
  text += "<li>" + key + ": " + fruit[key] + "</li>";
}
text += "</ul>";

// Count properties
text += "Total properties: " + Object.keys(fruit).length;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Introduction](#)

JavaScript Object Properties

Table of Contents

- [JavaScript Object Properties](#)
- [Accessing Properties](#)
- [Example 1](#)
- [Adding New Properties](#)
- [Example 2](#)
- [Deleting Properties](#)
- [Example 3](#)
- [Nested Properties](#)
- [Example 4](#)
- [Using Variables as Property Names](#)
- [Example 5](#)
- [Reference](#)

Accessing Properties

JavaScript object properties can be accessed in three ways:

- `object.property` (dot notation)
- `object["property"]` (bracket notation)
- `object[expression]` (using a variable)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Accessing Properties</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  "home town": "New York"
};

let text = "Dot notation: " + person.firstName + "<br>";
text += "Bracket notation: " + person["lastName"] + "<br>";
text += "Property with space: " + person["home town"] + "<br>";

document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Adding New Properties

You can add new properties to an existing object by simply giving it a value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding New Properties</h4>
<p id="demo"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };

// Add new properties
person.age = 30;
person["eyeColor"] = "blue";
person["favorite color"] = "green";

let text = "Person properties:<br>";
for (let key in person) {
  text += key + ": " + person[key] + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Deleting Properties

The **delete** keyword deletes a property from an object:

```
<!DOCTYPE html>
```

```

<html>
<body>

<h2>JavaScript Object Properties</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Deleting Properties</h4>
<p id="demo"></p>

<script>
const car = {
  brand: "Toyota",
  model: "Corolla",
  year: 2020,
  color: "blue",
  price: 25000
};

let text = "Before delete:<br>";
for (let key in car) {
  text += key + ": " + car[key] + "<br>";
}

// Delete the price property
delete car.price;
// Delete using bracket notation
delete car["color"];

text += "<br>After delete (price, color):<br>";
for (let key in car) {
  text += key + ": " + car[key] + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Nested Properties

Properties can contain other objects, creating nested structures:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>

```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Nested Properties</h4>
<p id="demo"></p>

<script>
const company = {
  name: "Tech Corp",
  address: {
    street: "456 Oak Ave",
    city: "San Francisco",
    state: "CA"
  },
  employees: {
    manager: { name: "Alice", salary: 80000 },
    developer: { name: "Bob", salary: 65000 }
  }
};

// Access nested properties
let text = "Company: " + company.name + "<br>";
text += "City: " + company.address.city + "<br>";
text += "Manager: " + company.employees.manager.name + "<br>";
text += "Developer salary: $" + company.employees.developer.salary;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Using Variables as Property Names

You can use variables to access object properties dynamically:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Variables as Property Names</h4>
<p id="demo"></p>

<script>
const person = {
  name: "John",

```

```
    age: 30,  
    city: "New York",  
    country: "USA"  
  };  
  
  const keys = ["name", "age", "city", "country"];  
  
  let text = "Dynamic property access:<br><br>";  
  for (let i = 0; i < keys.length; i++) {  
    const key = keys[i];  
    text += key + ": " + person[key] + "<br>";  
  }  
  
  document.getElementById("demo").innerHTML = text;  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Properties](#)

JavaScript Object Methods

Table of Contents

- [JavaScript Object Methods](#)
- [What are Object Methods?](#)
- [Example 1](#)
- [The this Keyword](#)
- [Example 2](#)
- [Accessing Object Methods](#)
- [Example 3](#)
- [Adding a Method to an Object](#)
- [Example 4](#)
- [Using Methods for Calculations](#)
- [Example 5](#)
- [Reference](#)

What are Object Methods?

Methods are actions that can be performed on objects. A method is a function stored as a property of an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What are Object Methods?</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML =
  "Method result: " + person.fullName();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The this Keyword

In object methods, **this** refers to the owner object. Without **this**, a function cannot access object properties:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The this Keyword</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  introduce: function() {
    return "Hi, I'm " + this.firstName + " " + this.lastName;
  },
  getAge: function() {
    return "I am " + this.age + " years old.";
  }
};

document.getElementById("demo1").innerHTML = person.introduce();
document.getElementById("demo2").innerHTML = person.getAge();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Accessing Object Methods

Object methods can be accessed with or without parentheses. Without parentheses, you get the function definition:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript Object Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Accessing Object Methods</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

// Access method WITH parentheses
document.getElementById("demo1").innerHTML =
  "With parentheses: " + person.fullName();

// Access method WITHOUT parentheses (returns function)
document.getElementById("demo2").innerHTML =
  "Without parentheses: " + person.fullName;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Adding a Method to an Object

You can add methods to an existing object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding a Method to an Object</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe"
};

```

```
// Add a method
person.fullName = function() {
    return this.firstName + " " + this.lastName;
};

// Add another method
person.greet = function(name) {
    return "Hello " + name + ", I'm " + this.firstName;
};

document.getElementById("demo").innerHTML =
    person.fullName() + "<br>" +
    person.greet("Jane");
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Using Methods for Calculations

Object methods can perform calculations using object properties:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Methods for Calculations</h4>
<p id="demo"></p>

<script>
const rectangle = {
    width: 10,
    height: 5,
    area: function() {
        return this.width * this.height;
    },
    perimeter: function() {
        return 2 * (this.width + this.height);
    },
    isSquare: function() {
        return this.width === this.height;
    }
};

document.getElementById("demo").innerHTML =
```

```
"Width: " + rectangle.width + "<br>" +  
"Height: " + rectangle.height + "<br>" +  
"Area: " + rectangle.area() + "<br>" +  
"Perimeter: " + rectangle.perimeter() + "<br>" +  
"Is square? " + rectangle.isSquare();  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Methods](#)

JavaScript Object this

Table of Contents

- [JavaScript Object this](#)
- [this in Objects](#)
- [Example 1](#)
- [this in an Object Method](#)
- [Example 2](#)
- [Why Use this?](#)
- [Example 3](#)
- [this Alone](#)
- [Example 4](#)
- [this in a Function](#)
- [Example 5](#)
- [Reference](#)

this in Objects

In JavaScript, the **this** keyword refers to an **object**.

Which object depends on how **this** is being invoked (used or called).

The **this** keyword refers to different objects depending on how it is used:

- In an object method, **this** refers to the **object**
- Alone, **this** refers to the **global object**
- In a function, **this** refers to the **global object**
- In a function (strict mode), **this** is **undefined**
- In an event, **this** refers to the **element** that received the event

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in Objects</h4>
<p id="demo"></p>

<script>
// Create an object
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Using 'this' would refer to the global object here
```

```
// But when used in a method, 'this' refers to the object

document.getElementById("demo").innerHTML =
  "person.firstName: " + person.firstName + "<br>" +
  "person.lastName: " + person.lastName;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

this in an Object Method

When used in an object method, **this** refers to the object itself:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in an Object Method</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  },
  birthYear: function() {
    const year = new Date().getFullYear();
    return year - this.age;
  }
};

document.getElementById("demo").innerHTML =
  "Full name: " + person.fullName() + "<br>" +
  "Born in: " + person.birthYear();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Why Use this?

The **this** keyword allows you to reuse methods across different objects without rewriting the code:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Why Use this?</h4>
<p id="demo"></p>

<script>
// A method that uses 'this' can work with any object
function introduce() {
  return "Hi, I'm " + this.firstName + " " + this.lastName;
}

const person1 = {
  firstName: "John",
  lastName: "Doe",
  introduce: introduce
};

const person2 = {
  firstName: "Jane",
  lastName: "Smith",
  introduce: introduce
};

document.getElementById("demo").innerHTML =
  person1.introduce() + "<br>" +
  person2.introduce();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

this Alone

When used alone, **this** refers to the global object (window in a browser):

```
<!DOCTYPE html>
```



```

<html>
<body>

<h2>JavaScript Object this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this Alone</h4>
<p id="demo"></p>

<script>
// In a browser, 'this' alone refers to the window object
let text = "this (global): " + this + "<br>";
text += "this === window: " + (this === window) + "<br>";
text += "this.document === document: " + (this.document === document);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

this in a Function

In a regular function, **this** refers to the global object (window in a browser):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in a Function</h4>
<p id="demo"></p>

<script>
function myFunction() {
  return this;
}

document.getElementById("demo").innerHTML =
  "Function returns: " + myFunction() + "<br>" +
  "It's the window object: " + (myFunction() === window);
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object this](#)

JavaScript Object Display

Table of Contents

- [JavaScript Object Display](#)
- [Displaying Object Properties](#)
- [Example 1](#)
- [Using a For .. In Loop](#)
- [Example 2](#)
- [Using Object.values\(\)](#)
- [Example 3](#)
- [Using Object.entries\(\)](#)
- [Example 4](#)
- [Using JSON.stringify\(\)](#)
- [Example 5](#)
- [Reference](#)

Displaying Object Properties

Displaying a JavaScript object directly will output `[object Object]`. You must access properties individually:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Display</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Displaying Object Properties</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = {
  name: "John",
  age: 30,
  city: "New York"
};

// Direct display shows [object Object]
document.getElementById("demo1").innerHTML =
  "Direct display: " + person;

// Display individual properties
document.getElementById("demo2").innerHTML =
  "Properties: " + person.name + ", " + person.age + ", " + person.city;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using a For .. In Loop

The **for...in** loop iterates over all properties of an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Display</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using a For .. In Loop</h4>
<p id="demo"></p>

<script>
const person = {
  name: "John",
  age: 30,
  city: "New York",
  country: "USA"
};

let text = "";
for (let key in person) {
  text += key + ": " + person[key] + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using Object.values()

Object.values() returns an array of the property values of an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Display</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Object.values()</h4>
<p id="demo"></p>

<script>
const person = {
  name: "John",
  age: 30,
  city: "New York"
};

// Get all values as an array
const values = Object.values(person);
document.getElementById("demo").innerHTML =
  "Object.values(): " + values;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Using Object.entries()

Object.entries() returns an array of the key/value pairs as arrays:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Display</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Object.entries()</h4>
<p id="demo"></p>

<script>
const person = {
  name: "John",
  age: 30,
  city: "New York"
};

// Get entries as [key, value] arrays
const entries = Object.entries(person);
let text = "";
for (let entry of entries) {
  text += entry[0] + ": " + entry[1] + "<br>";
}

```

```
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Using JSON.stringify()

JSON.stringify() converts a JavaScript object into a JSON string for display:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Display</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using JSON.stringify()</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = {
  name: "John",
  age: 30,
  city: "New York",
  hobbies: ["reading", "music", "swimming"]
};

// Simple stringify
document.getElementById("demo1").innerHTML =
  "JSON stringify: " + JSON.stringify(person);

// Pretty print with indentation (2 spaces)
document.getElementById("demo2").innerHTML =
  "Pretty print:<br>" + JSON.stringify(person, null, 2);
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Display](#)

JavaScript Object Constructors

Table of Contents

- [JavaScript Object Constructors](#)
- [Object Constructor Functions](#)
- [Example 1](#)
- [Property Default Values](#)
- [Example 2](#)
- [Adding a Property to an Object](#)
- [Example 3](#)
- [Constructor Function Methods](#)
- [Example 4](#)
- [Built-in JavaScript Constructors](#)
- [Example 5](#)
- [Reference](#)

Object Constructor Functions

To create an object type, use an **object constructor function**. It is considered good practice to name constructor functions with an upper-case first letter:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Constructor Functions</h4>
<p id="demo"></p>

<script>
// Constructor function - named with uppercase first letter
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

// Create objects using the constructor
const myFather = new Person("John", "Doe", 50, "blue");
const myMother = new Person("Jane", "Doe", 48, "green");

document.getElementById("demo").innerHTML =
  "Father: " + myFather.firstName + " " + myFather.lastName + ", " + myFather.age +
  "<br>" +
  "Mother: " + myMother.firstName + " " + myMother.lastName + ", " + myMother.age;
</script>
```



```
</body>
</html>
```

Example 1

Result [View Example](#)

Property Default Values

Constructor functions can have default property values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Property Default Values</h4>
<p id="demo"></p>

<script>
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
  this.nationality = "American"; // Default value
}

const myFather = new Person("John", "Doe", 50, "blue");
const mySister = new Person("Emily", "Doe", 25, "blue");

document.getElementById("demo").innerHTML =
  "Father: " + myFather.nationality + "<br>" +
  "Sister: " + mySister.nationality + "<br><br>" +
  "Both share the default nationality property";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Adding a Property to an Object

You can add a new property to an existing object. It only applies to that object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding a Property to an Object</h4>
<p id="demo"></p>

<script>
function Person(first, last, age) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
}

const myFather = new Person("John", "Doe", 50);
const myMother = new Person("Jane", "Doe", 48);

// Add a property to myFather only (not the constructor)
myFather.nationality = "American";

document.getElementById("demo").innerHTML =
  "Father nationality: " + myFather.nationality + "<br>" +
  "Mother nationality: " + myMother.nationality + " (undefined)<br><br>" +
  "Property added to myFather only";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Constructor Function Methods

You can define methods inside a constructor function:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constructor Function Methods</h4>
<p id="demo"></p>

<script>

```

```
function Person(first, last, age) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.fullName = function() {
    return this.firstName + " " + this.lastName;
  };
  this.getAge = function() {
    return this.age;
  };
}

const myFather = new Person("John", "Doe", 50);
const myMother = new Person("Jane", "Doe", 48);

document.getElementById("demo").innerHTML =
  "Father: " + myFather.fullName() + " (" + myFather.getAge() + ")<br>" +
  "Mother: " + myMother.fullName() + " (" + myMother.getAge() + ")";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Built-in JavaScript Constructors

JavaScript has built-in constructors for native objects:

Constructor	Example
<code>Object()</code>	<code>new Object()</code>
<code>String()</code>	<code>new String("Hello")</code>
<code>Number()</code>	<code>new Number(42)</code>
<code>Boolean()</code>	<code>new Boolean(true)</code>
<code>Array()</code>	<code>new Array(1, 2, 3)</code>
<code>Date()</code>	<code>new Date()</code>

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript Object Constructors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Built-in JavaScript Constructors</h4>
<p id="demo"></p>

<script>
let text = "Built-in constructors:<br><br>";

const obj = new Object();
text += "new Object(): " + obj + "<br>";

const str = new String("Hello");
text += "new String('Hello'): " + str + "<br>";

const num = new Number(42);
text += "new Number(42): " + num + "<br>";

const bool = new Boolean(true);
text += "new Boolean(true): " + bool + "<br>";

const arr = new Array(1, 2, 3);
text += "new Array(1, 2, 3): " + arr + "<br>";

const date = new Date();
text += "new Date(): " + date;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Constructors](#)

JavaScript Scopes

Table of Contents

- [JavaScript Scopes](#)
- [Global Scope](#)
- [Example 1](#)
- [Function Scope](#)
- [Example 2](#)
- [Block Scope](#)
- [Example 3](#)
- [Automatically Global](#)
- [Example 4](#)
- [The Lifetime of JavaScript Variables](#)
- [Example 5](#)
- [Reference](#)

Global Scope

JavaScript has **function scope**: each function creates a new scope.

Global scope means that variables declared outside any function are accessible from anywhere in the program:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Global Scope</h4>
<p id="demo"></p>

<script>
// Global variable - accessible everywhere
let carName = "Volvo";

function myFunction() {
  // Can access global variable inside a function
  return "Inside function: " + carName;
}

document.getElementById("demo").innerHTML =
  "Outside function: " + carName + "<br>" +
  myFunction();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Function Scope

Function scope means that variables declared inside a function are only accessible within that function. They cannot be accessed outside:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Scope</h4>
<p id="demo"></p>

<script>
function myFunction() {
  // Local variable - only accessible inside this function
  let carName = "Volvo";
  return "Inside function: " + carName;
}

document.getElementById("demo").innerHTML =
  myFunction() + "<br><br>" +
  "Trying to access carName outside would cause an error";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Block Scope

Block scope means that variables declared with **let** and **const** inside a block `{ }` are only accessible within that block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Block Scope</h4>
<p id="demo"></p>

<script>
{
  // Block scope - only accessible inside this block
  let x = 2;
  const y = 3;
  var z = 5; // var is NOT block scoped
}

// x and y are NOT accessible here
// z IS accessible here because var is function scoped

document.getElementById("demo").innerHTML =
  "z (var - accessible): " + z + "<br>" +
  "x (let - NOT accessible outside block)" + "<br>" +
  "y (const - NOT accessible outside block)";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Automatically Global

If you assign a value to a variable that has not been declared, it will automatically become a **global** variable:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Automatically Global</h4>
<p id="demo"></p>

<script>
function myFunction() {
  // This becomes GLOBAL automatically (not recommended!)
  carName = "Volvo";
}

// Call the function first
myFunction();

// Now carName is global and accessible here

```

```

document.getElementById("demo").innerHTML =
  "After running myFunction():<br>" +
  "carName = " + carName + "<br><br>" +
  "Warning: This variable is now global!<br>" +
  "Always declare variables with let/const/var";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

The Lifetime of JavaScript Variables

The lifetime of a JavaScript variable starts when it is declared. Global variables live until the page is closed. Local variables are deleted when the function completes:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Lifetime of JavaScript Variables</h4>
<p id="demo"></p>

<script>
// Global variable - lives until page is closed
let globalVar = "I live as long as the page";

function counterExample() {
  // Local variable - created each time function runs
  // Deleted when function completes
  let localVar = "I only live inside the function";
  return localVar;
}

// Call the function to see local variable
let result = counterExample();

let text = "Global variable: " + globalVar + "<br>";
text += "Local variable (from function): " + result + "<br><br>";
text += "Global variables: live until page is closed<br>";
text += "Local variables: deleted after function completes<br>";
text += "Each function call creates new local variables";

document.getElementById("demo").innerHTML = text;
</script>

```



```
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Scopes](#)

JavaScript Code Blocks

Table of Contents

- [JavaScript Code Blocks](#)
- [Code Blocks and Statements](#)
- [Example 1](#)
- [Defining Scope](#)
- [Example 2](#)
- [Standalone Blocks](#)
- [Example 3](#)
- [Encapsulation](#)
- [Example 4](#)
- [Organized Code](#)
- [Example 5](#)
- [Reference](#)

Code Blocks and Statements

JavaScript statements are grouped together in **code blocks**. A code block is defined by curly braces `{}`.

Statements inside the same block execute together:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Code Blocks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Code Blocks and Statements</h4>
<p id="demo"></p>

<script>
// A code block groups statements together
{
  let x = 5;
  let y = 10;
  let result = x + y;
  document.getElementById("demo").innerHTML =
    "x = " + x + "<br>" +
    "y = " + y + "<br>" +
    "x + y = " + result;
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Defining Scope

Code blocks define scope for variables declared with **let** and **const**. Variables declared inside a block are only accessible within that block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Code Blocks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Defining Scope</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Outside block scope
let globalResult = 0;

{
  // Inside block scope
  let blockResult = 42;
  globalResult = blockResult * 2;
  document.getElementById("demo1").innerHTML =
    "Inside block - blockResult: " + blockResult;
}

// blockResult is NOT accessible here
// globalResult IS accessible
document.getElementById("demo2").innerHTML =
  "Outside block - globalResult: " + globalResult + "<br>" +
  "blockResult is not accessible outside the block";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Standalone Blocks

Blocks can be used on their own, without functions or control flow. They are useful for organizing temporary code:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Code Blocks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Standalone Blocks</h4>
<p id="demo"></p>

<script>
// Standalone block - no function or if statement needed
{
  let name = "Alice";
  let score = 95;

  let grade = score >= 90 ? "A" : "B";

  document.getElementById("demo").innerHTML =
    "Student: " + name + "<br>" +
    "Score: " + score + "<br>" +
    "Grade: " + grade;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Encapsulation

Code blocks provide **encapsulation** - temporary variables are kept inside the block and don't pollute the global scope:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Code Blocks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Encapsulation</h4>
<p id="demo"></p>

<script>
let message = ""; // Global variable

{

```

```

    // Temporary variables - only used inside the block
    let temp1 = 15;
    let temp2 = 25;
    let sum = temp1 + temp2;

    message = "Sum of temporary values: " + sum;
}

// temp1, temp2, and sum are NOT accessible here
// They are encapsulated inside the block
document.getElementById("demo").innerHTML =
    message + "<br><br>" +
    "Temporary variables were kept inside the block";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Organized Code

Code blocks help organize code into logical sections, making it easier to read and maintain:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Code Blocks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Organized Code</h4>
<p id="demo"></p>

<script>
// Section 1: Calculate area
{
    const radius = 5;
    let area = Math.PI * radius * radius;
    document.getElementById("demo").innerHTML +=
        "Circle area: " + area.toFixed(2) + "<br>";
}

// Section 2: Calculate perimeter
{
    const width = 10;
    const height = 6;
    let perimeter = 2 * (width + height);
    document.getElementById("demo").innerHTML +=
        "Rectangle perimeter: " + perimeter + "<br>";
}

```

```
}

// Section 3: Display summary
{
  let summary = "Code organized into logical sections";
  document.getElementById("demo").innerHTML += "<br>" + summary;
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Code Blocks](#)

JavaScript Hoisting

Table of Contents

- [JavaScript Hoisting](#)
- [JavaScript Declarations are Hoisted](#)
- [Example 1](#)
- [The let and const Keywords](#)
- [Example 2](#)
- [JavaScript Initializations are Not Hoisted](#)
- [Example 3](#)
- [Declare Your Variables At the Top](#)
- [Example 4](#)
- [Reference](#)

JavaScript Declarations are Hoisted

In JavaScript, a variable can be declared **after** it has been used. This is called **hoisting**.

Hoisting is JavaScript's default behavior of moving all **declarations** to the top of the current scope (to the top of the current script or the current function).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Hoisting</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Declarations are Hoisted</h4>
<p id="demo"></p>

<script>
// Using the variable BEFORE it is declared
// This works because var declarations are hoisted
x = 5;
document.getElementById("demo").innerHTML =
  "x is " + x + " (used before declaration!)";

var x; // Declare x - this is hoisted to the top
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The let and const Keywords

Variables defined with **let** and **const** are NOT hoisted like **var**. Using them before declaration causes an error.

This is because of the **Temporal Dead Zone (TDZ)** - variables declared with **let/const** are hoisted to the top of their block, but they are NOT initialized until their declaration statement is executed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Hoisting</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The let and const Keywords</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// let and const are NOT hoisted like var
try {
  let carName = "Volvo";
  document.getElementById("demo1").innerHTML =
    "let carName: " + carName + " (declared before use - OK)";
} catch(err) {
  document.getElementById("demo1").innerHTML = "Error: " + err.message;
}

// Using let BEFORE declaration would cause an error
try {
  document.getElementById("demo2").innerHTML =
    "Attempting to use a let variable before declaration:";
  // This would throw: Cannot access 'x' before initialization
  console.log(city);
  let city = "New York";
} catch(err) {
  document.getElementById("demo2").innerHTML =
    "Error caught: " + err.message + "<br>" +
    "let/const are NOT hoisted like var";
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Initializations are Not Hoisted

While **declarations** are hoisted, **initializations** (assignments) are not hoisted. The variable exists but has the value **undefined**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Hoisting</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Initializations are Not Hoisted</h4>
<p id="demo"></p>

<script>
// The declaration of y is hoisted, but the assignment is not
// So y is undefined when we try to use it

function showHoisting() {
  document.getElementById("demo").innerHTML =
    "y is " + x + " and " + y + "<br><br>" +
    "x is 5 (declaration and initialization at top)<br>" +
    "y is undefined (only declaration hoisted, not value)";
}

var x = 5; // Declaration AND initialization
var y = 7; // Declaration is hoisted, initialization is not
showHoisting();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Declare Your Variables At the Top

Because hoisting can cause unexpected results, it is good practice to always declare your variables at the top of the scope. This makes the code easier to read and understand:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Hoisting</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Declare Your Variables At the Top</h4>
<p id="demo"></p>
```

```
<script>
// Good practice: declare all variables at the top
var firstName = "John";
var lastName = "Doe";
var age = 30;
var city = "New York";

// Then use them below
document.getElementById("demo").innerHTML =
  "Good practice - declarations at top:<br><br>" +
  firstName + " " + lastName + "<br>" +
  "Age: " + age + "<br>" +
  "City: " + city + "<br><br>" +
  "This makes code easier to read and understand";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Hoisting](#)

JavaScript var, let, const

Table of Contents

- [JavaScript var, let, const](#)
- [Scope Rules](#)
- [Example 1](#)
- [Reassignment vs. Redeclaration](#)
- [Example 2](#)
- [Hoisting and the Temporal Dead Zone \(TDZ\)](#)
- [Example 3](#)
- [Best Practice](#)
- [Example 4](#)
- [Reference](#)

Difference Between var, let and const

The table below summarizes the differences:

	`var`	`let`	`const`
Scope	Function	Block	Block
Redeclaration	Yes	No	No
Reassignment	Yes	Yes	No
Hoisted	Yes	No (TDZ)	No (TDZ)

Scope Rules

var is function-scoped, while **let** and **const** are block-scoped:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Scope Rules</h4>
<p id="demo"></p>

<script>
{
  var varX = 1;      // Function scoped (accessible outside block)
```

```

    let letX = 2;      // Block scoped
    const constX = 3;  // Block scoped
}

document.getElementById("demo").innerHTML =
    "var (function scope): " + varX + " - accessible<br>" +
    "let (block scope): not accessible outside<br>" +
    "const (block scope): not accessible outside";
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Reassignment vs. Redeclaration

var can be redeclared and reassigned. **let** can be reassigned but NOT redeclared. **const** can be neither:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reassignment vs. Redeclaration</h4>
<p id="demo"></p>

<script>
// var - can be redeclared and reassigned
var name = "John";
var name = "Jane";    // Redeclaration OK
name = "Bob";         // Reassignment OK

// let - can be reassigned but NOT redeclared
let age = 30;
age = 31;             // Reassignment OK
// let age = 32;      // Redeclaration ERROR

// const - cannot be reassigned or redeclared
const city = "New York";
// city = "London";   // Reassignment ERROR
// const city = "LA";  // Redeclaration ERROR

document.getElementById("demo").innerHTML =
    "var name: " + name + " (redeclared + reassigned)<br>" +
    "let age: " + age + " (reassigned)<br>" +
    "const city: " + city + " (cannot change)";
</script>

```

```
</body>
</html>
```

Example 2

Result [View Example](#)

Hoisting and the Temporal Dead Zone (TDZ)

var is hoisted and initialized with **undefined**. **let** and **const** are hoisted but NOT initialized (Temporal Dead Zone):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hoisting and the Temporal Dead Zone (TDZ)</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// var is hoisted and initialized with undefined
document.getElementById("demo1").innerHTML =
    "var x: " + x + " (hoisted, initialized as undefined)";
var x = 5;

// let is NOT initialized before declaration (TDZ)
try {
    console.log(y); // Would throw ReferenceError
    let y = 10;
} catch(err) {
    document.getElementById("demo2").innerHTML =
        "let y: Error - " + err.message + "<br>" +
        "let is in Temporal Dead Zone before declaration";
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Best Practice

The best practice is to always use **const** unless you know the value will change. Use **let** if you plan to reassign. Avoid **var**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Best Practice</h4>
<p id="demo"></p>

<script>
// Best practice: use const by default
const PI = 3.14159;
const MAX_USERS = 100;

// Use let when reassignment is needed
let counter = 0;
counter = counter + 1;
counter++;

// Avoid var in modern JavaScript
// var oldStyle = "not recommended";

let text = "Best practices:<br><br>";
text += "1. Use <b>const</b> by default:<br>";
text += "    const PI = " + PI + "<br>";
text += "    const MAX_USERS = " + MAX_USERS + "<br><br>";
text += "2. Use <b>let</b> when reassigning:<br>";
text += "    let counter = " + counter + "<br><br>";
text += "3. Avoid <b>var</b> in modern code";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript var, let, const](#)

JavaScript Strict Mode

Table of Contents

- [JavaScript Strict Mode](#)
- [Declaring Strict Mode](#)
- [Example 1](#)
- [Why Strict Mode?](#)
- [Example 2](#)
- [Not Allowed in Strict Mode](#)
- [Example 3](#)
- [Watch Out!](#)
- [Example 4](#)
- [Reference](#)

The "use strict" Directive

The **"use strict"** directive was introduced in ECMAScript 5. It is **not a statement**, but a literal expression, ignored by earlier versions of JavaScript.

The purpose of **"use strict"** is to indicate that the code should be executed in **strict mode**. With strict mode, you cannot, for example, use undeclared variables.

Declaring Strict Mode

Strict mode is declared by adding **"use strict";** to the beginning of a script or a function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strict Mode</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Declaring Strict Mode</h4>
<p id="demo"></p>

<script>
// Declare strict mode at the top of the script
"use strict";

// This works fine in strict mode (declared variable)
let carName = "Volvo";
document.getElementById("demo").innerHTML =
  "carName: " + carName + "<br>" +
  "Strict mode is active at script level";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Why Strict Mode?

Strict mode helps you write cleaner code by catching common coding mistakes and "unsafe" actions:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strict Mode</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Why Strict Mode?</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Without strict mode - undeclared variable is silently created
function nonStrict() {
    undeclaredVar = 42; // No error (bad!)
}
nonStrict();
document.getElementById("demo1").innerHTML =
    "Non-strict: undeclaredVar = " + undeclaredVar + " (silently created)";

// With strict mode - undeclared variable causes an error
function strictFunction() {
    "use strict";
    try {
        undeclaredVar2 = 99; // Throws ReferenceError in strict mode
    } catch(err) {
        document.getElementById("demo2").innerHTML =
            "Strict mode error: " + err.message;
    }
}
strictFunction();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Not Allowed in Strict Mode

Strict mode prevents several unsafe actions:


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strict Mode</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Not Allowed in Strict Mode</h4>
<p id="demo"></p>

<script>
"use strict";

let text = "<b>Not allowed in strict mode:</b><br><br>";
text += "1. Using undeclared variables (ReferenceError)<br>";
text += "2. Deleting variables (SyntaxError)<br>";
text += "3. Duplicating parameters (SyntaxError)<br>";
text += "4. Using reserved words as names<br>";
text += "5. Octal numeric literals<br>";
text += "6. Writing to read-only properties<br><br>";

// Example: writing to read-only property
try {
  const obj = {};
  Object.defineProperty(obj, "readonly", {value: 10, writable: false});
  obj.readonly = 20; // TypeError in strict mode
  text += "Read-only property write: no error (non-strict behavior)";
} catch(err) {
  text += "Read-only property error: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Watch Out!

Strict mode only applies to the code where it is declared. If you declare it inside a function, it only applies to that function:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Strict Mode</h2>

```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Watch Out!</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Strict mode is NOT active here (global scope without directive)

// Function with strict mode - only affects this function
function strictFunction() {
  "use strict";
  try {
    undeclaredVar = 100; // Error in strict mode
  } catch(err) {
    document.getElementById("demo1").innerHTML =
      "Inside strict function: " + err.message;
  }
}

// Function WITHOUT strict mode - affected by global non-strict
function nonStrictFunction() {
  undeclaredVar2 = 200; // No error (non-strict)
  document.getElementById("demo2").innerHTML =
    "Outside strict function (global): undeclaredVar2 = " + undeclaredVar2 + "<br>" +
    "Strict mode only applies where it is declared";
}

strictFunction();
nonStrictFunction();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Strict Mode](#)

JavaScript Dates

Table of Contents

- [JavaScript Dates](#)
- [Creating Date Objects](#)
- [Example 1](#)
- [new Date\(year, month, ...\)](#)
- [Example 2](#)
- [JavaScript Stores Dates as Milliseconds](#)
- [Example 3](#)
- [Displaying Dates](#)
- [Example 4](#)
- [Date Methods](#)
- [Example 5](#)
- [Reference](#)

Creating Date Objects

Date objects are created with the `new Date()` constructor. There are 4 ways to create a date:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dates</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating Date Objects</h4>
<p id="demo"></p>

<script>
// new Date() - current date and time
const currentDate = new Date();

// new Date(date string)
const dateString = new Date("2024-02-14");

// new Date(year, month, day)
const specificDate = new Date(2024, 1, 14);

document.getElementById("demo").innerHTML =
  "new Date(): " + currentDate + "<br>" +
  "new Date('2024-02-14')": " + dateString + "<br>" +
  "new Date(2024, 1, 14): " + specificDate;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

new Date(year, month, ...)

`new Date(year, month, day, hours, minutes, seconds, ms)` creates a date with the specified components. JavaScript counts months from 0 (January) to 11 (December):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dates</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>new Date(year, month, ...)</h4>
<p id="demo"></p>

<script>
// Month is 0-indexed: 0=Jan, 1=Feb, ..., 11=Dec
const date1 = new Date(2024, 0, 1);      // Jan 1, 2024
const date2 = new Date(2024, 5, 15, 12);  // Jun 15, 2024 at 12:00
const date3 = new Date(2024, 11, 31, 23, 59, 59); // Dec 31, 2024 at 23:59:59

// Using 2 numbers: year + month
const date4 = new Date(2024, 6);

document.getElementById("demo").innerHTML =
  "new Date(2024, 0, 1): " + date1 + "<br>" +
  "new Date(2024, 5, 15, 12): " + date2 + "<br>" +
  "new Date(2024, 11, 31, 23, 59, 59): " + date3 + "<br>" +
  "new Date(2024, 6): " + date4;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Stores Dates as Milliseconds

JavaScript stores dates as the number of milliseconds since January 1, 1970, 00:00:00 UTC:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dates</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Stores Dates as Milliseconds</h4>
<p id="demo"></p>

<script>
// Zero time is January 1, 1970
const zeroTime = new Date(0);

// One day in milliseconds: 24 * 60 * 60 * 1000
const oneDay = 24 * 60 * 60 * 1000;
const oneDayAfter = new Date(oneDay);

// Current time in milliseconds
const nowMs = Date.now();

document.getElementById("demo").innerHTML =
  "new Date(0): " + zeroTime + " (Jan 1, 1970)<br>" +
  "new Date(86400000): " + oneDayAfter + " (Jan 2, 1970)<br>" +
  "Date.now(): " + nowMs + " ms<br><br>" +
  "Dates are stored as milliseconds since Jan 1, 1970";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Displaying Dates

Dates can be displayed in different formats using various methods:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dates</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Displaying Dates</h4>
<p id="demo"></p>

<script>
const d = new Date(2024, 5, 15, 12, 30, 45);

document.getElementById("demo").innerHTML =
  "toString(): " + d.toString() + "<br>" +
  "toDateString(): " + d.toDateString() + "<br>" +
  "toISOString(): " + d.toISOString() + "<br>" +
  "toUTCString(): " + d.toUTCString() + "<br>" +

```

```

    "toLocaleDateString(): " + d.toLocaleDateString() + "<br>" +
    "toLocaleTimeString(): " + d.toLocaleTimeString();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Date Methods

Common date getter methods:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dates</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date Methods</h4>
<p id="demo"></p>

<script>
const d = new Date(2024, 5, 15, 12, 30, 45);

document.getElementById("demo").innerHTML =
  "getFullYear(): " + d.getFullYear() + "<br>" +
  "getMonth(): " + d.getMonth() + " (0-11)<br>" +
  "getDate(): " + d.getDate() + "<br>" +
  "getDay(): " + d.getDay() + " (0=Sunday)<br>" +
  "getHours(): " + d.getHours() + "<br>" +
  "getMinutes(): " + d.getMinutes() + "<br>" +
  "getSeconds(): " + d.getSeconds() + "<br>" +
  "getMilliseconds(): " + d.getMilliseconds() + "<br>" +
  "getTime(): " + d.getTime();
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Dates](#)

JavaScript Date Formats

Table of Contents

- [JavaScript Date Formats](#)
- [JavaScript ISO Dates](#)
- [Example 1](#)
- [ISO Dates \(Date-Time\)](#)
- [Example 2](#)
- [JavaScript Short Dates](#)
- [Example 3](#)
- [JavaScript Long Dates](#)
- [Example 4](#)
- [Date Input - Parsing Dates](#)
- [Example 5](#)
- [Reference](#)

JavaScript ISO Dates

ISO 8601 is the international standard for the representation of dates and times. The ISO 8601 syntax (YYYY-MM-DD) is also the preferred JavaScript date format:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Formats</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript ISO Dates</h4>
<p id="demo"></p>

<script>
// Complete date (year-month-day)
const completeDate = new Date("2024-03-15");

// Year and month (yyyy-mm)
const yearMonth = new Date("2024-03");

// Only year (yyyy)
const onlyYear = new Date("2024");

document.getElementById("demo").innerHTML =
  "new Date('2024-03-15'): " + completeDate + "<br>" +
  "new Date('2024-03'): " + yearMonth + "<br>" +
  "new Date('2024'): " + onlyYear;
</script>

</body>
</html>
```


Example 1

Result [View Example](#)

ISO Dates (Date-Time)

ISO dates can be written with added hours, minutes, and seconds (YYYY-MM-DDTHH:MM:SSZ):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Formats</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>ISO Dates (Date-Time)</h4>
<p id="demo"></p>

<script>
// Date and time with T separator
const dateTime = new Date("2024-03-15T10:30:00");

// Date and time with UTC (Z = Zulu time / UTC)
const utcTime = new Date("2024-03-15T10:30:00Z");

// Date and time with timezone offset
const timezoneTime = new Date("2024-03-15T10:30:00+07:00");

document.getElementById("demo").innerHTML =
  "With T separator: " + dateTime + "<br>" +
  "With Z (UTC): " + utcTime + "<br>" +
  "With +07:00 offset: " + timezoneTime;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Short Dates

Short dates are written with the "MM/DD/YYYY" syntax. Be careful with month/day order:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript Date Formats</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Short Dates</h4>
<p id="demo"></p>

<script>
// Short date format: MM/DD/YYYY
const shortDate1 = new Date("03/15/2024");    // March 15, 2024

// Short date with dashes
const shortDate2 = new Date("2024-03-15");

// Short date or long date
const shortDate3 = new Date("March 15, 2024");

document.getElementById("demo").innerHTML =
  "MM/DD/YYYY: " + shortDate1 + "<br>" +
  "YYYY-MM-DD: " + shortDate2 + "<br>" +
  "Month name: " + shortDate3;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JavaScript Long Dates

Long dates are most often written with a "MMM DD YYYY" syntax. Month and day can be in any order:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Formats</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Long Dates</h4>
<p id="demo"></p>

<script>
// Long date formats
const longDate1 = new Date("Mar 15 2024");      // Month name short
const longDate2 = new Date("March 15, 2024");   // Full month name
const longDate3 = new Date("15 Mar 2024");      // Day first

// With time
const longDate4 = new Date("March 15, 2024 10:30:00");

```

```

document.getElementById("demo").innerHTML =
  "Mar 15 2024: " + longDate1 + "<br>" +
  "March 15, 2024: " + longDate2 + "<br>" +
  "15 Mar 2024: " + longDate3 + "<br>" +
  "March 15, 2024 10:30:00: " + longDate4;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Date Input - Parsing Dates

If you have a valid date string, you can use the `Date.parse()` method to convert it to milliseconds and back to a Date object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Formats</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date Input - Parsing Dates</h4>
<p id="demo"></p>

<script>
// Date.parse() converts a date string to milliseconds
let ms = Date.parse("March 15, 2024");
const parsedDate = new Date(ms);

// Parse ISO format
let ms2 = Date.parse("2024-03-15T10:30:00");
const parsedDate2 = new Date(ms2);

document.getElementById("demo").innerHTML =
  "Date.parse('March 15, 2024'): " + ms + " ms<br>" +
  "new Date(ms): " + parsedDate + "<br><br>" +
  "Date.parse('2024-03-15T10:30:00'): " + ms2 + " ms<br>" +
  "new Date(ms2): " + parsedDate2;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Date Formats](#)

JavaScript Date Methods

Table of Contents

- [JavaScript Date Methods](#)
- [Get the Current Time](#)
- [Example 1](#)
- [Date Get Methods](#)
- [Example 2](#)
- [The Date.now\(\) Method](#)
- [Example 3](#)
- [UTC Date Get Methods](#)
- [Example 4](#)
- [The getTimezoneOffset\(\) Method](#)
- [Example 5](#)
- [Reference](#)

Get the Current Time

The `new Date()` constructor creates a date object with the current date and time:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Get the Current Time</h4>
<p id="demo"></p>

<script>
const d = new Date();
document.getElementById("demo").innerHTML = "Current date and time: " + d;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Date Get Methods

Date get methods return parts of a date. The most common methods:

- `getFullYear()` - year (4 digits)
- `getMonth()` - month (0-11)

- `getDate()` - day of month (1-31)
- `getDay()` - day of week (0=Sunday)
- `getHours()` - hours (0-23)
- `getMinutes()` - minutes (0-59)
- `getSeconds()` - seconds (0-59)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date Get Methods</h4>
<p id="demo"></p>

<script>
const d = new Date(2024, 5, 15, 14, 30, 45);

document.getElementById("demo").innerHTML =
  "getFullYear(): " + d.getFullYear() + "<br>" +
  "getMonth(): " + d.getMonth() + " (June = 5)<br>" +
  "getDate(): " + d.getDate() + "<br>" +
  "getDay(): " + d.getDay() + " (Saturday = 6)<br>" +
  "getHours(): " + d.getHours() + "<br>" +
  "getMinutes(): " + d.getMinutes() + "<br>" +
  "getSeconds(): " + d.getSeconds();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The Date.now() Method

`Date.now()` returns the number of milliseconds since January 1, 1970:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Date.now() Method</h4>
<p id="demo"></p>

<script>
```

```
// Date.now() returns milliseconds since Jan 1, 1970
const ms = Date.now();
const date = new Date(ms);

// Also getTime() returns the same
const d = new Date();
const time = d.getTime();

document.getElementById("demo").innerHTML =
  "Date.now(): " + ms + " ms<br>" +
  "new Date(Date.now()): " + date + "<br><br>" +
  "d.getTime(): " + time + " ms<br>" +
  "Date.now() === d.getTime(): " + (ms === time);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

UTC Date Get Methods

UTC methods return date parts according to UTC (Coordinated Universal Time):

- `getUTCFullYear()` - year in UTC
- `getUTCMonth()` - month in UTC
- `getUTCDate()` - day in UTC
- `getUTCDay()` - weekday in UTC
- `getUTCHours()` - hours in UTC

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>UTC Date Get Methods</h4>
<p id="demo"></p>

<script>
const d = new Date();

document.getElementById("demo").innerHTML =
  "Local vs UTC:<br><br>" +
  "getHours(): " + d.getHours() + " (local)<br>" +
  "getUTCHours(): " + d.getUTCHours() + " (UTC)<br><br>" +
  "getFullYear(): " + d.getFullYear() + " (local)<br>" +
  "getUTCFullYear(): " + d.getUTCFullYear() + " (UTC)<br><br>" +
```

```

    "getMonth(): " + d.getMonth() + " (local)<br>" +
    "getUTCMonth(): " + d.getUTCMonth() + " (UTC)<br><br>" +
    "UTC methods use the universal time standard";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

The getTimezoneOffset() Method

`getTimezoneOffset()` returns the difference (in minutes) between local time and UTC time:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The getTimezoneOffset() Method</h4>
<p id="demo"></p>

<script>
const d = new Date();

// Returns the timezone offset in minutes
const offset = d.getTimezoneOffset();

// Convert to hours
const offsetHours = offset / 60;

document.getElementById("demo").innerHTML =
    "getTimezoneOffset(): " + offset + " minutes<br>" +
    "Offset in hours: " + offsetHours + " hours<br><br>" +
    "This is the difference between your local time and UTC. " +
    "For example, Bangkok (UTC+7) has an offset of -420 minutes.";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Date Methods](#)

JavaScript Date Set Methods

Table of Contents

- [JavaScript Date Set Methods](#)
- [The setFullYear\(\) Method](#)
- [Example 1](#)
- [The setMonth\(\) and setDate\(\) Methods](#)
- [Example 2](#)
- [The setHours\(\) and setMinutes\(\) Methods](#)
- [Example 3](#)
- [Compare Dates](#)
- [Example 4](#)
- [Reference](#)

Set Date Methods

Set methods allow you to set date values (years, months, days, hours, minutes, seconds, milliseconds) for a Date Object.

The setFullYear() Method

`setFullYear()` sets the year of a date object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The setFullYear() Method</h4>
<p id="demo"></p>

<script>
const d = new Date();
d.setFullYear(2020);

document.getElementById("demo").innerHTML =
  "After setFullYear(2020):<br>" + d;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The setMonth() and setDate() Methods

setMonth() sets the month (0-11). **setDate()** sets the day of month (1-31):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The setMonth() and setDate() Methods</h4>
<p id="demo"></p>

<script>
// Set to March (month 2)
const d1 = new Date();
d1.setMonth(2);

// Set to day 15
const d2 = new Date();
d2.setDate(15);

document.getElementById("demo").innerHTML =
  "setMonth(2) - March: " + d1 + "<br>" +
  "setDate(15): " + d2;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The setHours() and setMinutes() Methods

setHours() sets the hours (0-23). **setMinutes()** sets the minutes (0-59):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The setHours() and setMinutes() Methods</h4>
<p id="demo"></p>

<script>
// Set hours to 14 (2 PM)
```

```

const d1 = new Date();
d1.setHours(14);

// Set minutes to 45
const d2 = new Date();
d2.setMinutes(45);

document.getElementById("demo").innerHTML =
  "setHours(14) - 2 PM: " + d1 + "<br>" +
  "setMinutes(45): " + d2;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Compare Dates

Dates can easily be compared using comparison operators (<, >, ==, !=):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Compare Dates</h4>
<p id="demo"></p>

<script>
let text = "";

const today = new Date();
const someday = new Date();
someday.setFullYear(2100);

if (someday > today) {
  text = "Today is before 2100.";
} else {
  text = "Today is after 2100.";
}

// Also compare with getTime()
const d1 = new Date(2024, 0, 1);
const d2 = new Date(2024, 0, 15);
text += "<br><br>d1 (Jan 1) < d2 (Jan 15): " + (d1.getTime() < d2.getTime());

document.getElementById("demo").innerHTML = text;

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Date Set Methods](#)

JavaScript Date Reference

Table of Contents

- [JavaScript Date Reference](#)
- [Date Get Methods](#)
- [Example 1](#)
- [Date Set Methods](#)
- [Example 2](#)
- [Date Display Methods](#)
- [Example 3](#)
- [Reference](#)

Date Get Methods

Reference table for all Date get methods:

Method	Description
<code>getFullYear()</code>	Year (4 digits)
<code>getMonth()</code>	Month (0-11)
<code>getDate()</code>	Day of month (1-31)
<code>getDay()</code>	Day of week (0=Sunday)
<code>getHours()</code>	Hours (0-23)
<code>getMinutes()</code>	Minutes (0-59)
<code>getSeconds()</code>	Seconds (0-59)
<code>getMilliseconds()</code>	Milliseconds (0-999)
<code>getTime()</code>	Milliseconds since Jan 1, 1970
<code>Date.now()</code>	Same as <code>getTime()</code> but static
<code>getTimezoneOffset()</code>	Minutes between local and UTC

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript Date Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date Get Methods</h4>
<p id="demo"></p>

<script>
const d = new Date(2024, 5, 15, 12, 30, 45, 123);

document.getElementById("demo").innerHTML =
  "getFullYear(): " + d.getFullYear() + "<br>" +
  "getMonth(): " + d.getMonth() + "<br>" +
  "getDate(): " + d.getDate() + "<br>" +
  "getDay(): " + d.getDay() + "<br>" +
  "getHours(): " + d.getHours() + "<br>" +
  "getMinutes(): " + d.getMinutes() + "<br>" +
  "getSeconds(): " + d.getSeconds() + "<br>" +
  "getMilliseconds(): " + d.getMilliseconds() + "<br>" +
  "getTime(): " + d.getTime();
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Date Set Methods

Reference table for all Date set methods:

Method	Description
<code>setFullYear()</code>	Set year
<code>setMonth()</code>	Set month (0-11)
<code>setDate()</code>	Set day of month (1-31)
<code>setHours()</code>	Set hours (0-23)
<code>setMinutes()</code>	Set minutes (0-59)
<code>setSeconds()</code>	Set seconds (0-59)
<code>setMilliseconds()</code>	Set milliseconds (0-999)

setTime()

Set time (milliseconds)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date Set Methods</h4>
<p id="demo"></p>

<script>
const d = new Date();
d.setFullYear(2024);
d.setMonth(5);
d.setDate(15);
d.setHours(12);
d.setMinutes(30);
d.setSeconds(45);

document.getElementById("demo").innerHTML = "After using set methods:<br>" + d;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Date Display Methods

Reference table for display methods:

Method	Description
toString()	Standard string format
toDateString()	Date only, no time
toISOString()	ISO 8601 format
toUTCString()	UTC string format
toLocaleDateString()	Local date format

toLocaleTimeString()

Local time format

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Date Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date Display Methods</h4>
<p id="demo"></p>

<script>
const d = new Date(2024, 5, 15, 12, 30);

document.getElementById("demo").innerHTML =
  "toString(): " + d.toString() + "<br>" +
  "toDateString(): " + d.toDateString() + "<br>" +
  "toISOString(): " + d.toISOString() + "<br>" +
  "toUTCString(): " + d.toUTCString() + "<br>" +
  "toLocaleDateString(): " + d.toLocaleDateString() + "<br>" +
  "toLocaleTimeString(): " + d.toLocaleTimeString();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Date Reference](#)

JavaScript Temporal

Table of Contents

- [JavaScript Temporal](#)
- [What is JavaScript Temporal?](#)
- [Example 1](#)
- [Temporal.Instant](#)
- [Example 2](#)
- [Temporal.PlainDate](#)
- [Example 3](#)
- [Temporal.Now](#)
- [Example 4](#)
- [Temporal.Arithmetic](#)
- [Example 5](#)
- [Reference](#)

What is JavaScript Temporal?

Temporal is a new JavaScript API for date and time. It solves the problems of the old **Date** object.

Temporal provides separate objects for different use cases:

Object	Description
<code>Temporal.Instant</code>	A point in time (UTC)
<code>Temporal.PlainDate</code>	A date without time
<code>Temporal.PlainTime</code>	A time without date
<code>Temporal.PlainDateTime</code>	Date and time without timezone
<code>Temporal.Duration</code>	A duration of time
<code>Temporal.ZonedDateTime</code>	Date and time with timezone

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Temporal</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is JavaScript Temporal?</h4>
<p id="demo"></p>
```

```

<script>
let text = "Temporal vs Date:<br><br>";
text += "<b>Temporal provides:</b><br>";
text += "1. Separate objects for different date/time needs<br>";
text += "2. Immutable objects (safer)<br>";
text += "3. Better timezone support<br>";
text += "4. Simple arithmetic operations<br>";
text += "5. Clearer API names<br><br>";

// Check browser support
if (typeof Temporal !== "undefined") {
  text += "✓ Temporal is supported in this browser";
} else {
  text += "✗ Temporal needs a polyfill in this browser";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Temporal.Instant

A **Temporal.Instant** represents a single point in time, always in UTC:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Temporal</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Temporal.Instant</h4>
<p id="demo"></p>

<script>
// Get the current instant (if available)
if (typeof Temporal !== "undefined") {
  const now = Temporal.Now.instant();
  document.getElementById("demo").innerHTML =
    "Temporal.Now.instant(): " + now.toString() + "<br>" +
    "Epoch milliseconds: " + now.epochMilliseconds + "<br>" +
    "Epoch seconds: " + now.epochSeconds;
} else {
  document.getElementById("demo").innerHTML =
    "Temporal is not supported in this browser.<br>" +

```

```
        "Use Date.now() instead: " + Date.now() + " ms";
    }
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Temporal.PlainDate

A **Temporal.PlainDate** represents a date without time or timezone:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Temporal</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Temporal.PlainDate</h4>
<p id="demo"></p>

<script>
if (typeof Temporal !== "undefined") {
    // Create a plain date
    const date = Temporal.PlainDate.from("2024-05-15");

    document.getElementById("demo").innerHTML =
        "Temporal.PlainDate.from('2024-05-15'): " + date.toString() + "<br>" +
        "Year: " + date.year + "<br>" +
        "Month: " + date.month + "<br>" +
        "Day: " + date.day + "<br>" +
        "Day of week: " + date.dayOfWeek;
} else {
    document.getElementById("demo").innerHTML =
        "Temporal is not supported in this browser.";
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Temporal.Now

Temporal.Now provides easy access to the current date and time in different forms:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Temporal</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Temporal.Now</h4>
<p id="demo"></p>

<script>
if (typeof Temporal !== "undefined") {
  document.getElementById("demo").innerHTML =
    "Temporal.Now.instant(): " + Temporal.Now.instant().toString() + "<br>" +
    "Temporal.Now.zonedDateTimeISO(): " + Temporal.Now.zonedDateTimeISO().toString() +
    "<br>" +
    "Temporal.Now.plainDateTimeISO(): " + Temporal.Now.plainDateTimeISO().toString() +
    "<br>" +
    "Temporal.Now.plainDateISO(): " + Temporal.Now.plainDateISO().toString() + "<br>"
  +
    "Temporal.Now.plainTimeISO(): " + Temporal.Now.plainTimeISO().toString();
} else {
  document.getElementById("demo").innerHTML =
    "Temporal is not supported in this browser.<br>" +
    "Equivalent Date.now(): " + new Date().toLocaleString();
}
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Temporal Arithmetic

Temporal makes date arithmetic simple and intuitive:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Temporal</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Temporal Arithmetic</h4>
<p id="demo"></p>

<script>
```

```

if (typeof Temporal !== "undefined") {
  const date = Temporal.PlainDate.from("2024-05-15");

  // Add and subtract durations
  const plusWeek = date.add({ days: 7 });
  const minusMonth = date.subtract({ months: 1 });

  // Compare dates
  const date2 = Temporal.PlainDate.from("2024-06-01");
  const isBefore = Temporal.PlainDate.compare(date, date2);

  document.getElementById("demo").innerHTML =
    "Date: " + date.toString() + "<br>" +
    "Add 7 days: " + plusWeek.toString() + "<br>" +
    "Subtract 1 month: " + minusMonth.toString() + "<br><br>" +
    "Compare with 2024-06-01: " + isBefore + " (-1 = before, 0 = equal, 1 = after)";
} else {
  document.getElementById("demo").innerHTML =
    "Temporal is not supported in this browser.";
}
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Temporal](#)

JavaScript Arrays

Table of Contents

- [JavaScript Arrays](#)
- [Creating an Array](#)
- [Example 1](#)
- [Accessing and Changing Array Elements](#)
- [Example 2](#)
- [Converting an Array to a String](#)
- [Example 3](#)
- [The length Property](#)
- [Example 4](#)
- [Adding Array Elements](#)
- [Example 5](#)
- [Reference](#)

Creating an Array

An array is a special variable that can hold more than one value. There are two ways to create an array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating an Array</h4>
<p id="demo"></p>

<script>
// Array literal (preferred)
const cars = ["Volvo", "BMW", "Toyota"];

// Using the new keyword (not recommended)
const fruits = new Array("Apple", "Banana", "Orange");

document.getElementById("demo").innerHTML =
  "Cars: " + cars + "<br>" +
  "Fruits: " + fruits + "<br><br>" +
  "Using the array literal is the recommended way";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Accessing and Changing Array Elements

Array elements are accessed by their index number (starting from 0) and can be changed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Accessing and Changing Array Elements</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const cars = ["Volvo", "BMW", "Toyota"];

// Access elements by index (0-based)
const firstCar = cars[0];
const lastCar = cars[2];

// Change an element
cars[1] = "Honda";

document.getElementById("demo1").innerHTML =
  "First car: " + firstCar + "<br>" +
  "Last car: " + lastCar;

document.getElementById("demo2").innerHTML =
  "After cars[1] = 'Honda': " + cars;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Converting an Array to a String

The `toString()` method converts an array to a comma-separated string:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>
```



```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Converting an Array to a String</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

// toString() joins with commas
const fruitString = fruits.toString();

document.getElementById("demo").innerHTML =
  "Array: " + fruits + "<br>" +
  "toString(): " + fruitString + "<br>" +
  "Type: " + typeof fruitString;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The length Property

The **length** property returns the number of array elements. It is always one more than the highest array index:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The length Property</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

document.getElementById("demo1").innerHTML =
  "Array: " + fruits + "<br>" +
  "length: " + fruits.length + "<br><br>" +
  "First element: " + fruits[0] + "<br>" +
  "Last element: " + fruits[fruits.length - 1];
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Adding Array Elements

You can add new elements to an array using `push()` or the array's length property:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Array Elements</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange"];

// push() adds to the end
fruits.push("Mango");

// Using length adds to the end
fruits[fruits.length] = "Apple";

// Adding at a higher index creates empty slots
fruits[5] = "Grape";

let text = "After adding elements: " + fruits + "<br><br>";

for (let i = 0; i < fruits.length; i++) {
  if (fruits[i] === undefined) {
    text += "Index " + i + ": (empty)" + "<br>";
  } else {
    text += "Index " + i + ": " + fruits[i] + "<br>";
  }
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Arrays](#)

JavaScript Array Constructor

Table of Contents

- [JavaScript Array Constructor](#)
- [new Array\(\) vs Array\(\)](#)
- [Example 1](#)
- [A Common Error](#)
- [Example 2](#)
- [Array Literal \(Preferred\)](#)
- [Example 3](#)
- [Reference](#)

new Array() vs Array()

There is no difference between `new Array()` and `Array()`. Both create a new array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Constructor</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>new Array() vs Array()</h4>
<p id="demo"></p>

<script>
// new Array() - creates a new array
const arr1 = new Array("Apple", "Banana", "Orange");

// Array() - same result, no 'new' needed
const arr2 = Array("Apple", "Banana", "Orange");

document.getElementById("demo").innerHTML =
  "new Array(...): " + arr1 + "<br>" +
  "Array(...): " + arr2 + "<br><br>" +
  "Both give the same result!";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

A Common Error

A common error is using `new Array(number)` with a single numeric argument. It creates an array with that many undefined elements, NOT an array with one element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Constructor</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Common Error</h4>
<p id="demo"></p>

<script>
// WARNING: new Array(5) creates an array with 5 empty slots!
const arr = new Array(5);

let text = "new Array(5):<br>";
text += "Length: " + arr.length + "<br>";
text += "Elements: " + arr + "<br><br>";

text += "This creates 5 EMPTY slots, not an array with value 5!<br><br>";

// To create an array with one element use brackets
const correct = [5];
text += "Correct way [5]: length = " + correct.length + ", value = " + correct[0];

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Array Literal (Preferred)

The array literal `[]` is the preferred way to create an array. It is simpler and safer:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Constructor</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Array Literal (Preferred)</h4>
<p id="demo"></p>
```

```
<script>
// Preferred: array literal
const fruits = ["Apple", "Banana", "Orange"];

// Using new Array() with explicit elements (works but not preferred)
const cars = new Array("Volvo", "BMW", "Toyota");

let text = "Array literal: " + fruits + "<br>";
text += "new Array(): " + cars + "<br><br>";
text += "<b>Why use the array literal?</b><br>";
text += "1. Simpler to read and write<br>";
text += "2. Avoids the new Array(number) error<br>";
text += "3. Better performance<br>";
text += "4. Consistent behavior";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Array Constructor](#)

JavaScript Array Methods

Table of Contents

- [JavaScript Array Methods](#)
- [Basic Array Methods](#)
- [Example 1](#)
- [Popping and Pushing](#)
- [Example 2](#)
- [Shifting Elements](#)
- [Example 3](#)
- [Merging Arrays](#)
- [Example 4](#)
- [Splicing and Slicing](#)
- [Example 5](#)
- [Reference](#)

Basic Array Methods

The **length** property returns the number of elements. **toString()** converts the array to a string. **join()** joins elements with a separator. **at()** accesses elements by index:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Basic Array Methods</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

let text = "length: " + fruits.length + "<br>";
text += "toString(): " + fruits.toString() + "<br>";
text += "join(' - '): " + fruits.join(" - ") + "<br>";
text += "at(0): " + fruits.at(0) + "<br>";
text += "at(-1): " + fruits.at(-1) + " (last element)<br>";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Popping and Pushing

`pop()` removes the last element and returns it. `push()` adds a new element to the end:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Popping and Pushing</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple"];

// pop() removes the last element
let popped = fruits.pop();
let text = "Popped: " + popped + "<br>";
text += "After pop(): " + fruits + "<br><br>";

// push() adds to the end
let newLength = fruits.push("Kiwi");
text += "Pushed 'Kiwi', new length: " + newLength + "<br>";
text += "After push(): " + fruits;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Shifting Elements

`shift()` removes the first element. `unshift()` adds a new element to the beginning:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Shifting Elements</h4>
```



```

<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple"];

// shift() removes the first element
let shifted = fruits.shift();
let text = "Shifted: " + shifted + "<br>";
text += "After shift(): " + fruits + "<br><br>";

// unshift() adds to the beginning
let newLength = fruits.unshift("Lemon");
text += "Unshifted 'Lemon', new length: " + newLength + "<br>";
text += "After unshift(): " + fruits;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Merging Arrays

concat() merges two or more arrays:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Merging Arrays</h4>
<p id="demo"></p>

<script>
const arr1 = ["Cecilie", "Lone"];
const arr2 = ["Emil", "Tobias", "Linus"];
const arr3 = ["Robin", "Morgan"];

// Merge two arrays
const merged2 = arr1.concat(arr2);

// Merge three arrays
const merged3 = arr1.concat(arr2, arr3);

// Merge array with values
const mergedValues = arr1.concat("Peter");

```

```

document.getElementById("demo").innerHTML =
  "Merged 2 arrays: " + merged2 + "<br>" +
  "Merged 3 arrays: " + merged3 + "<br>" +
  "Merged with values: " + mergedValues;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Splicing and Slicing

`splice()` adds/removes elements in the middle. `slice()` extracts a section without modifying the original:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Splicing and Slicing</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

// splice(position, count, items...) - adds/removes in middle
fruits.splice(2, 0, "Lemon", "Kiwi");
let text = "After splice(2, 0, 'Lemon', 'Kiwi'): " + fruits + "<br><br>";

// splice to remove elements
fruits.splice(2, 2);
text += "After splice(2, 2): " + fruits + "<br><br>";

// slice(start, end) - extracts without modifying
const citrus = fruits.slice(1, 3);
text += "slice(1, 3): " + citrus + "<br>";
text += "Original: " + fruits + " (unchanged)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Array Methods](#)

JavaScript Array Search

Table of Contents

- [JavaScript Array Search](#)
- [indexOf\(\) and lastIndexOf\(\)](#)
- [Example 1](#)
- [includes\(\)](#)
- [Example 2](#)
- [find\(\)](#)
- [Example 3](#)
- [findIndex\(\)](#)
- [Example 4](#)
- [findLast\(\) and findLastIndex\(\)](#)
- [Example 5](#)
- [Reference](#)

indexOf() and lastIndexOf()

`indexOf()` searches an array for an element value and returns its position. `lastIndexOf()` returns the last index of the element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Search</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>indexOf() and lastIndexOf()</h4>
<p id="demo"></p>

<script>
const fruits = ["Apple", "Orange", "Apple", "Mango"];

// indexOf() - first occurrence
let index = fruits.indexOf("Apple");

// lastIndexOf() - last occurrence
let lastIndex = fruits.lastIndexOf("Apple");

document.getElementById("demo").innerHTML =
  "Array: " + fruits + "<br><br>" +
  "indexOf('Apple'): " + index + "<br>" +
  "lastIndexOf('Apple'): " + lastIndex + "<br><br>" +
  "indexOf('Banana'): " + fruits.indexOf("Banana") + " (-1 = not found)";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

includes()

`includes()` checks if an element is present in the array. It returns true or false:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Search</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>includes()</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

document.getElementById("demo").innerHTML =
  "Array: " + fruits + "<br><br>" +
  "includes('Apple'): " + fruits.includes("Apple") + "<br>" +
  "includes('Grape'): " + fruits.includes("Grape") + "<br><br>" +
  "includes() returns true if the element is in the array";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

find()

`find()` returns the value of the first element that passes a test function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Search</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>find()</h4>
<p id="demo"></p>
```

```

<script>
const numbers = [4, 9, 16, 25, 29];

// Find the first element > 18
let first = numbers.find(function(value) {
    return value > 18;
});

// Find the first element > 10 and < 20
let first2 = numbers.find(function(value) {
    return value > 10 && value < 20;
});

document.getElementById("demo").innerHTML =
    "Array: " + numbers + "<br><br>" +
    "First > 18: " + first + "<br>" +
    "First > 10 and < 20: " + first2;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

findIndex()

findIndex() returns the index of the first element that passes a test function:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Search</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>findIndex()</h4>
<p id="demo"></p>

<script>
const numbers = [4, 9, 16, 25, 29];

// Find the index of the first element > 18
let index = numbers.findIndex(function(value) {
    return value > 18;
});

document.getElementById("demo").innerHTML =
    "Array: " + numbers + "<br><br>" +
    "findIndex(v > 18): " + index + " (element is " + numbers[index] + ")";
</script>

```

```
</body>
</html>
```

Example 4

Result [View Example](#)

findLast() and findLastIndex()

`findLast()` returns the value of the LAST element that passes a test. `findLastIndex()` returns its index:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Search</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>findLast() and findLastIndex(</h4>
<p id="demo"></p>

<script>
const numbers = [4, 9, 16, 25, 29];

// findLast() - last element > 10
let last = numbers.findLast(function(value) {
  return value > 10;
});

// findLastIndex() - index of last element > 10
let lastIndex = numbers.findLastIndex(function(value) {
  return value > 10;
});

document.getElementById("demo").innerHTML =
  "Array: " + numbers + "<br><br>" +
  "findLast(v > 10): " + last + "<br>" +
  "findLastIndex(v > 10): " + lastIndex;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Array Search](#)

JavaScript Array Sort

Table of Contents

- [JavaScript Array Sort](#)
- [Alphabetic Sort](#)
- [Example 1](#)
- [Reverse Sort](#)
- [Example 2](#)
- [Numeric Sort](#)
- [Example 3](#)
- [The Compare Function](#)
- [Example 4](#)
- [Find Min or Max with sort\(\)](#)
- [Example 5](#)
- [Reference](#)

Alphabetic Sort

The `sort()` method sorts an array alphabetically:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Sort</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Alphabetic Sort</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

// Sort alphabetically
fruits.sort();

document.getElementById("demo").innerHTML =
  "Original: Banana, Orange, Apple, Mango<br>" +
  "After sort(): " + fruits;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Reverse Sort

The `reverse()` method reverses the elements in an array. You can reverse after sorting for descending order:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Sort</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reverse Sort</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

// Sort first, then reverse
fruits.sort();
fruits.reverse();

document.getElementById("demo").innerHTML =
  "After sort() and reverse(): " + fruits;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Numeric Sort

The `sort()` method sorts numbers as strings, so **25** comes before **100**. Use a compare function for numeric sort:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Sort</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numeric Sort</h4>
<p id="demo"></p>

<script>
const points = [40, 100, 1, 5, 25, 10];

// Without compare function: sorts as strings (WRONG for numbers)
```

```

const wrong = points.slice();
wrong.sort();

// With compare function: sorts as numbers
const right = points.slice();
right.sort(function(a, b) { return a - b; });

document.getElementById("demo").innerHTML =
  "Original: " + points + "<br><br>" +
  "Without compare function: " + wrong + " (wrong for numbers)<br>" +
  "With compare function: " + right + " (correct)";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The Compare Function

The compare function takes two arguments **a** and **b**. If the result is:

- **Negative:** **a** is sorted before **b**
- **Positive:** **b** is sorted before **a**
- **Zero:** no change

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Sort</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Compare Function</h4>
<p id="demo"></p>

<script>
const points = [40, 100, 1, 5, 25, 10];

// Ascending: a - b
const ascending = points.slice();
ascending.sort(function(a, b) { return a - b; });

// Descending: b - a
const descending = points.slice();
descending.sort(function(a, b) { return b - a; });

document.getElementById("demo").innerHTML =
  "Original: " + points + "<br><br>" +
  "Ascending (a - b): " + ascending + "<br>" +

```

```
"Descending (b - a): " + descending;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Find Min or Max with sort()

You can find the lowest or highest value in an array by sorting and accessing the first/last element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Sort</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Find Min or Max with sort()</h4>
<p id="demo"></p>

<script>
const points = [40, 100, 1, 5, 25, 10];

// Sort ascending to find min and max
const sorted = points.slice();
sorted.sort(function(a, b) { return a - b; });

const min = sorted[0];
const max = sorted[sorted.length - 1];

document.getElementById("demo").innerHTML =
  "Array: " + points + "<br><br>" +
  "Sorted ascending: " + sorted + "<br>" +
  "Minimum value: " + min + "<br>" +
  "Maximum value: " + max;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Array Sort](#)

JavaScript Array Iteration

Table of Contents

- [JavaScript Array Iteration](#)
- [Looping Array Values](#)
- [Example 1](#)
- [forEach\(\)](#)
- [Example 2](#)
- [map\(\)](#)
- [Example 3](#)
- [filter\(\) and reduce\(\)](#)
- [Example 4](#)
- [every\(\) and some\(\)](#)
- [Example 5](#)
- [Reference](#)

Looping Array Values

The `for...of` loop iterates over array values. The `for...in` loop iterates over array indexes (not recommended):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Iteration</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Looping Array Values</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

// for...of loop (recommended) - iterates values
let text1 = "for...of: ";
for (let fruit of fruits) {
  text1 += fruit + " ";
}
document.getElementById("demo1").innerHTML = text1;

// for...in loop (not recommended) - iterates indexes
let text2 = "for...in: ";
for (let index in fruits) {
  text2 += index + " ";
}
document.getElementById("demo2").innerHTML = text2;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

forEach()

The **forEach()** method calls a function once for each array element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Iteration</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>forEach()</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5];
let text = "";

numbers.forEach(function(value, index) {
  text += "Index " + index + ": " + value + "<br>";
});

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

map()

The **map()** method creates a new array by performing a function on each element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Iteration</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>map()</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5];

// Create a new array with each value multiplied by 2
const doubled = numbers.map(function(value) {
  return value * 2;
});

// Create a new array with string labels
const labels = numbers.map(function(value) {
  return "Number: " + value;
});

document.getElementById("demo").innerHTML =
  "Original: " + numbers + "<br>" +
  "map(x * 2): " + doubled + "<br>" +
  "map(labels): " + labels;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

filter() and reduce()

filter() creates a new array with elements that pass a test. **reduce()** reduces the array to a single value:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Iteration</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>filter() and reduce()</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// filter - keep values > 5
const filtered = numbers.filter(function(value) {
  return value > 5;
});

```



```
// reduce - sum all values
const sum = numbers.reduce(function(total, value) {
  return total + value;
}, 0);

// reduce - find max value
const max = numbers.reduce(function(acc, value) {
  return Math.max(acc, value);
});

document.getElementById("demo").innerHTML =
  "Original: " + numbers + "<br><br>" +
  "filter(v > 5): " + filtered + "<br>" +
  "reduce(sum): " + sum + "<br>" +
  "reduce(max): " + max;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

every() and some()

every() returns true if ALL elements pass a test. **some()** returns true if ANY element passes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Iteration</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>every() and some()</h4>
<p id="demo"></p>

<script>
const numbers = [2, 4, 6, 8, 10];

// every() - all values > 1?
const allGreater = numbers.every(function(value) {
  return value > 1;
});

// every() - all even?
const allEven = numbers.every(function(value) {
  return value % 2 === 0;
});
```

```
// some() - any value > 6?
const anyGreater = numbers.some(function(value) {
  return value > 6;
});

// some() - any value > 10?
const anyOver10 = numbers.some(function(value) {
  return value > 10;
});

document.getElementById("demo").innerHTML =
  "Array: " + numbers + "<br><br>" +
  "every(v > 1): " + allGreater + "<br>" +
  "every(v even): " + allEven + "<br>" +
  "some(v > 6): " + anyGreater + "<br>" +
  "some(v > 10): " + anyOver10;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Array Iteration](#)

JavaScript Array Reference

Table of Contents

- [JavaScript Array Reference](#)
- [Array Properties](#)
- [Example 1](#)
- [Basic Methods](#)
- [Example 2](#)
- [Search Methods](#)
- [Example 3](#)
- [Iteration Methods](#)
- [Example 4](#)
- [Add/Remove Methods](#)
- [Example 5](#)
- [Reference](#)

Array Properties

The complete **Array** object reference:

Property	Description
length	Number of elements in the array
constructor	The array constructor function

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Array Properties</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

document.getElementById("demo").innerHTML =
  "Array: " + fruits + "<br>" +
  "length: " + fruits.length + "<br>" +
  "constructor: " + fruits.constructor;
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Basic Methods

Basic array methods for converting and joining:

Method	Description
<code>toString()</code>	Convert array to comma-separated string
<code>join(sep)</code>	Join elements with a separator
<code>at(index)</code>	Get element at index (supports negative)
<code>fill(value)</code>	Fill all elements with a value

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Basic Methods</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple"];

// Create a copy for fill demonstration
const filled = new Array(3);
filled.fill("Kiwi");

document.getElementById("demo").innerHTML =
  "toString(): " + fruits.toString() + "<br>" +
  "join(' - '): " + fruits.join(" - ") + "<br>" +
  "at(-1): " + fruits.at(-1) + "<br>" +
  "fill('Kiwi'): " + filled;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Search Methods

Array search methods reference:

Method	Description
<code>indexOf(item)</code>	First index of item (-1 if not found)
<code>lastIndexOf(item)</code>	Last index of item
<code>includes(item)</code>	True if item exists
<code>find(fn)</code>	First value passing test
<code>findIndex(fn)</code>	First index passing test

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Search Methods</h4>
<p id="demo"></p>

<script>
const numbers = [10, 20, 30, 20, 40];

document.getElementById("demo").innerHTML =
  "Array: " + numbers + "<br><br>" +
  "indexOf(20): " + numbers.indexOf(20) + "<br>" +
  "lastIndexOf(20): " + numbers.lastIndexOf(20) + "<br>" +
  "includes(30): " + numbers.includes(30) + "<br>" +
  "find(v > 25): " + numbers.find(v => v > 25) + "<br>" +
  "findIndex(v > 25): " + numbers.findIndex(v => v > 25);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Iteration Methods

Array iteration methods reference:

Method	Description
<code>forEach(fn)</code>	Call function for each element
<code>map(fn)</code>	Create new array from results
<code>filter(fn)</code>	Keep elements passing test
<code>reduce(fn)</code>	Reduce to a single value
<code>every(fn)</code>	True if all pass
<code>some(fn)</code>	True if any pass

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iteration Methods</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5];

let text = "Array: " + numbers + "<br><br>";

// forEach
let sum = 0;
numbers.forEach(v => sum += v);
text += "forEach (sum): " + sum + "<br>";

// map
text += "map(v * 10): " + numbers.map(v => v * 10) + "<br>";

// filter
text += "filter(v % 2 == 0): " + numbers.filter(v => v % 2 === 0) + "<br>";

// reduce
text += "reduce(sum): " + numbers.reduce((acc, v) => acc + v, 0) + "<br>";

// every/some
text += "every(v > 0): " + numbers.every(v => v > 0) + "<br>";
```

```

text += "some(v > 4): " + numbers.some(v => v > 4);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Add/Remove Methods

Methods for adding and removing elements:

Method	Description
<code>push(item)</code>	Add to the end
<code>pop()</code>	Remove from the end
<code>shift()</code>	Remove from the beginning
<code>unshift(item)</code>	Add to the beginning
<code>splice(start, count)</code>	Add/remove in the middle
<code>concat(arr)</code>	Merge arrays

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add/Remove Methods</h4>
<p id="demo"></p>

<script>
const arr = ["A", "B"];

let text = "Start: " + arr + "<br><br>";

arr.push("C");
text += "push('C'): " + arr + "<br>";

```

```
arr.unshift("Z");
text += "unshift('Z'): " + arr + "<br>";

arr.pop();
text += "pop(): " + arr + "<br>";

arr.shift();
text += "shift(): " + arr + "<br>";

arr.splice(1, 0, "X");
text += "splice(1,0,'X'): " + arr + "<br>";

text += "concat(['Y']): " + arr.concat(["Y"]);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Array Reference](#)

JavaScript Array const

Table of Contents

- [JavaScript Array const](#)
- [Cannot be Reassigned](#)
- [Example 1](#)
- [Elements Can be Reassigned](#)
- [Example 2](#)
- [Const Block Scope](#)
- [Example 3](#)
- [Redeclaring Arrays](#)
- [Example 4](#)
- [Reference](#)

Cannot be Reassigned

Arrays declared with **const** cannot be reassigned to a different array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot be Reassigned</h4>
<p id="demo"></p>

<script>
const cars = ["Volvo", "BMW", "Toyota"];

// This works - modifying elements is allowed
cars[0] = "Honda";

// But this would cause an error (reassignment)
// cars = ["Audi", "Ford"]; // TypeError: Assignment to constant variable

document.getElementById("demo").innerHTML =
  "const cars: " + cars + "<br><br>" +
  "You can modify elements, but NOT reassign the array";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Elements Can be Reassigned

While the array itself cannot be reassigned, its elements CAN be changed. This is because `const` only prevents reassignment of the variable, not modification of the array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Elements Can be Reassigned</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange"];

// Element reassignment is allowed
fruits[0] = "Mango";
fruits[1] = "Apple";

// push() and pop() are also allowed
fruits.push("Kiwi");
fruits.pop();

// Const only protects the variable, not the array contents
document.getElementById("demo").innerHTML =
  "fruits after changes: " + fruits;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Const Block Scope

Arrays declared with `const` have block scope. They are not accessible outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Const Block Scope</h4>
<p id="demo"></p>
```

```

<script>
{
  const cars = ["Volvo", "BMW"];
  document.getElementById("demo").innerHTML +=
    "Inside block: " + cars + "<br>";
}

// cars is NOT accessible here
document.getElementById("demo").innerHTML +=
  "Outside block: cars is not accessible (block scope)";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Redeclaring Arrays

A **const** array cannot be redeclared in the same scope. It can be redeclared in a different block scope:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Redeclaring Arrays</h4>
<p id="demo"></p>

<script>
const cars = ["Volvo", "BMW"];

// This is OK - different block scope
{
  const cars = ["Toyota", "Honda"];
  document.getElementById("demo").innerHTML +=
    "Inside block: " + cars + "<br>";
}

// This is OK - different block scope again
{
  const cars = ["Audi", "Ford"];
  document.getElementById("demo").innerHTML +=
    "Second block: " + cars + "<br>";
}

// cars still refers to the original array

```

```
document.getElementById("demo").innerHTML +=  
  "Outer scope: " + cars;  
</script>  
  
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Array const](#)

JavaScript Sets

Table of Contents

- [JavaScript Sets](#)
- [How to Create a Set](#)
- [Example 1](#)
- [The add\(\) Method](#)
- [Example 2](#)
- [Listing the Elements](#)
- [Example 3](#)
- [Sets are Objects](#)
- [Example 4](#)
- [Reference](#)

How to Create a Set

A JavaScript Set is a collection of unique values. Each value can only occur once in a Set.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Sets</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How to Create a Set</h4>
<p id="demo"></p>

<script>
// Create a Set with an array
const letters = new Set(["a", "b", "c"]);

// Create an empty Set and add values later
const numbers = new Set();

document.getElementById("demo").innerHTML =
  "Set from array: " + [...letters] + "<br>" +
  "Set size: " + letters.size + "<br>" +
  "Empty Set size: " + numbers.size;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The add() Method

Use `add()` to add new elements to a Set. Duplicate values are automatically ignored:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Sets</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The add() Method</h4>
<p id="demo"></p>

<script>
const fruits = new Set();

// Add values with add()
fruits.add("Apple");
fruits.add("Banana");
fruits.add("Orange");

// Duplicate values are ignored
fruits.add("Apple"); // Already exists - ignored!

document.getElementById("demo").innerHTML =
  "Set: " + [...fruits] + "<br>" +
  "Size: " + fruits.size + "<br><br>" +
  "Duplicates are automatically removed";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Listing the Elements

You can list all Set elements using `forEach()` or the `values()` method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Sets</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Listing the Elements</h4>
<p id="demo"></p>
```

```

<script>
const letters = new Set(["a", "b", "c"]);

let text = "Using forEach():<br>";
letters.forEach(function(value) {
    text += value + "<br>";
});

// values() returns an iterator of all values
const values = letters.values();
text += "<br>Using values() iterator:<br>";
for (let v of values) {
    text += v + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Sets are Objects

typeof a Set is an object. A Set can be created with **new Set()**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Sets</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sets are Objects</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b", "c"]);

let text = "typeof Set: " + typeof letters + "<br>";
text += "Instance of Set: " + (letters instanceof Set) + "<br><br>";

// Other Useful Set properties
text += "<b>Other Set methods:</b><br>";
text += "has('b'): " + letters.has("b") + "<br>";
text += "delete('c'): " + letters.delete("c") + "<br>";
text += "After delete: " + [...letters] + "<br>";
text += "entries: " + [...letters.entries()] + "<br>";

```

```
document.getElementById("demo").innerHTML = text;  
</script>
```

```
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Sets](#)

JavaScript Set Methods

Table of Contents

- [JavaScript Set Methods](#)
- [The add\(\) Method](#)
- [Example 1](#)
- [The has\(\) Method](#)
- [Example 2](#)
- [The values\(\) and keys\(\) Methods](#)
- [Example 3](#)
- [The entries\(\) Method](#)
- [Example 4](#)
- [Reference](#)

Set Methods & Properties

Set has the following methods and properties:

Method/Property	Description
<code>new Set()</code>	Create a new Set
<code>add(value)</code>	Add a new element
<code>delete(value)</code>	Delete an element
<code>has(value)</code>	Check if element exists
<code>clear()</code>	Remove all elements
<code>forEach(fn)</code>	Call function for each element
<code>values()</code>	Returns iterator of values
<code>keys()</code>	Same as <code>values()</code> for Set
<code>entries()</code>	Returns [value, value] pairs
<code>size</code>	Number of elements

The add() Method

`add()` adds a new element to the Set. Chain multiple `add()` calls for cleaner code:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The add() Method</h4>
<p id="demo"></p>

<script>
const letters = new Set();

// Chained add() calls
letters.add("a").add("b").add("c").add("d");

document.getElementById("demo").innerHTML =
  "Set: " + [...letters] + "<br>" +
  "size: " + letters.size;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

The has() Method

has() returns true if the Set contains a specific value:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The has() Method</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b", "c"]);

let text = "Set: " + [...letters] + "<br><br>";
text += "has('a'): " + letters.has("a") + "<br>";
text += "has('x'): " + letters.has("x") + "<br><br>";

// delete() removes an element
letters.delete("b");

```

```

text += "After delete('b'): " + [...letters] + "<br>";
text += "has('b') after delete: " + letters.has("b");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

The values() and keys() Methods

For a Set, **values()** and **keys()** are the same - both return an iterator of all values:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The values() and keys() Methods</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b", "c"]);

// values() returns iterator
let text = "values(): ";
for (let v of letters.values()) {
  text += v + " ";
}
text += "<br>";

// keys() returns same as values() for Set
text += "keys(): ";
for (let k of letters.keys()) {
  text += k + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The entries() Method

`entries()` returns an iterator with [value, value] pairs:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The entries() Method</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b", "c"]);

let text = "entries():<br>";
for (let entry of letters.entries()) {
  text += "[" + entry[0] + ", " + entry[1] + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Set Methods](#)

JavaScript Set Logic

Table of Contents

- [JavaScript Set Logic](#)
- [The union\(\) Method](#)
- [Example 1](#)
- [The intersection\(\) Method](#)
- [Example 2](#)
- [The difference\(\) Method](#)
- [Example 3](#)
- [The symmetricDifference\(\) Method](#)
- [Example 4](#)
- [isSubsetOf\(\), isSupersetOf\(\), isDisjointFrom\(\)](#)
- [Example 5](#)
- [Reference](#)

The union() Method

`union()` creates a new Set containing all elements from both Sets:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Logic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The union() Method</h4>
<p id="demo"></p>

<script>
const setA = new Set(["a", "b", "c"]);
const setB = new Set(["c", "d", "e"]);

let text = "Set A: " + [...setA] + "<br>";
text += "Set B: " + [...setB] + "<br><br>";

if (setA.union) {
  const result = setA.union(setB);
  text += "union(): " + [...result] + "<br>";
  text += "All elements from both sets (no duplicates)";
} else {
  text += "union() is not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

The intersection() Method

intersection() creates a new Set with elements in BOTH Sets:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Logic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The intersection() Method</h4>
<p id="demo"></p>

<script>
const setA = new Set(["a", "b", "c"]);
const setB = new Set(["c", "d", "e"]);

let text = "Set A: " + [...setA] + "<br>";
text += "Set B: " + [...setB] + "<br><br>";

if (setA.intersection) {
  const result = setA.intersection(setB);
  text += "intersection(): " + [...result] + "<br>";
  text += "Only elements present in BOTH sets";
} else {
  text += "intersection() is not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The difference() Method

difference() creates a new Set with elements in A but NOT in B:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Logic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The difference() Method</h4>
<p id="demo"></p>

<script>
const setA = new Set(["a", "b", "c"]);
const setB = new Set(["c", "d", "e"]);

let text = "Set A: " + [...setA] + "<br>";
text += "Set B: " + [...setB] + "<br><br>";

if (setA.difference) {
  const result = setA.difference(setB);
  text += "difference(): " + [...result] + "<br>";
  text += "Elements in A but NOT in B";
} else {
  text += "difference() is not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The symmetricDifference() Method

symmetricDifference() creates a new Set with elements in either Set, but NOT in both:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Logic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The symmetricDifference() Method</h4>
<p id="demo"></p>

<script>
const setA = new Set(["a", "b", "c"]);

```

```

const setB = new Set(["c", "d", "e"]);

let text = "Set A: " + [...setA] + "<br>";
text += "Set B: " + [...setB] + "<br><br>";

if (setA.symmetricDifference) {
  const result = setA.symmetricDifference(setB);
  text += "symmetricDifference(): " + [...result] + "<br>";
  text += "Elements in either set but NOT in both";
} else {
  text += "symmetricDifference() is not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

isSubsetOf(), isSupersetOf(), isDisjointFrom()

These methods check the relationships between two Sets:

- **isSubsetOf()** - true if every element of A is in B
- **isSupersetOf()** - true if every element of B is in A
- **isDisjointFrom()** - true if A and B have no common elements

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Logic</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>isSubsetOf(), isSupersetOf(), isDisjointFrom()</h4>
<p id="demo"></p>

<script>
const setA = new Set(["a", "b"]);
const setB = new Set(["a", "b", "c", "d"]);
const setC = new Set(["x", "y"]);

let text = "Set A: " + [...setA] + "<br>";
text += "Set B: " + [...setB] + "<br>";
text += "Set C: " + [...setC] + "<br><br>";

if (setA.isSubsetOf) {
  text += "A.isSubsetOf(B): " + setA.isSubsetOf(setB) + "<br>";
}

```



```
text += "B.isSupersetOf(A): " + setB.isSupersetOf(setA) + "<br>";
text += "A.isDisjointFrom(C): " + setA.isDisjointFrom(setC) + "<br><br>";
text += "A is subset of B ✓<br>";
text += "B is superset of A ✓<br>";
text += "A and C are disjoint ✓";
} else {
text += "These methods are not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Set Logic](#)

JavaScript WeakSet

Table of Contents

- [JavaScript WeakSet](#)
- [Creating a WeakSet](#)
- [Example 1](#)
- [WeakSet Methods](#)
- [Example 2](#)
- [WeakSet vs Set](#)
- [Example 3](#)
- [Weak References](#)
- [Example 4](#)
- [Reference](#)

Creating a WeakSet

A **WeakSet** is a collection of objects. Unlike a Set, a WeakSet can only contain **objects** (not primitive values like strings or numbers):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakSet</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating a WeakSet</h4>
<p id="demo"></p>

<script>
// Create a WeakSet with objects
const obj1 = { name: "John" };
const obj2 = { name: "Jane" };
const obj3 = { name: "Bob" };

const weakSet = new WeakSet([obj1, obj2, obj3]);

// Add another object
const obj4 = { name: "Alice" };
weakSet.add(obj4);

document.getElementById("demo").innerHTML =
  "WeakSet created with objects<br>" +
  "has(obj1): " + weakSet.has(obj1) + "<br>" +
  "has(obj4): " + weakSet.has(obj4) + "<br><br>" +
  "Note: WeakSet can only hold objects,<br>" +
  "not strings, numbers, or other primitives";
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

WeakSet Methods

A WeakSet supports only three methods: `add()`, `delete()`, and `has()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakSet</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>WeakSet Methods</h4>
<p id="demo"></p>

<script>
const weakSet = new WeakSet();

const obj1 = { id: 1 };
const obj2 = { id: 2 };

// add() - add an object
weakSet.add(obj1);
weakSet.add(obj2);

let text = "After adding 2 objects:<br>";
text += "has(obj1): " + weakSet.has(obj1) + "<br>";
text += "has(obj2): " + weakSet.has(obj2) + "<br><br>";

// delete() - remove an object
weakSet.delete(obj1);
text += "After delete(obj1):<br>";
text += "has(obj1): " + weakSet.has(obj1) + "<br><br>";

text += "<b>Methods available:</b><br>";
text += "add(obj) - add an object<br>";
text += "delete(obj) - remove an object<br>";
text += "has(obj) - check if object exists<br><br>";

text += "WeakSet has NO size property and NO iteration methods";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

WeakSet vs Set

The main differences between WeakSet and Set:

Feature	Set	WeakSet
Values	Any type (objects + primitives)	Objects only
size property	Yes	No
Iteration	Yes (forEach, values, etc.)	No
Weak references	No	Yes
Garbage collection	Not automatic	Automatic

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakSet</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>WeakSet vs Set</h4>
<p id="demo"></p>

<script>
// Set can hold any type
const set = new Set(["string", 42, true, {obj: true}]);

// WeakSet can only hold objects
const obj = {name: "John"};
const weakSet = new WeakSet([obj]);

let text = "<b>Set can hold:</b><br>";
text += "- Strings: ✓<br>";
text += "- Numbers: ✓<br>";
text += "- Booleans: ✓<br>";
text += "- Objects: ✓<br><br>";

text += "<b>WeakSet can only hold:</b><br>";
text += "- Objects: ✓ (only types allowed)<br><br>";
```

```

text += "<b>Set has:</b> size, forEach(), values(), keys()<br>";
text += "<b>WeakSet has:</b> add(), delete(), has() only<br><br>";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Weak References

WeakSet holds objects by **weak references**. When an object is no longer referenced elsewhere, it can be garbage collected:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakSet</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Weak References</h4>
<p id="demo"></p>

<script>
const weakSet = new WeakSet();

let john = { name: "John" };
weakSet.add(john);

let text = "Object added to WeakSet<br><br>";

// Display object
text += "Object in WeakSet: " + (weakSet.has(john)) + "<br><br>";

// Remove all references to john
john = null;

text += "After setting john = null:<br>";
text += "The object can now be garbage collected<br>";
text += "It will be automatically removed from the WeakSet<br><br>";
text += "This is why WeakSet is useful for<br>";
text += "managing memory efficiently";

document.getElementById("demo").innerHTML = text;
</script>

```

```
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript WeakSet](#)

JavaScript Set Reference

Table of Contents

- [JavaScript Set Reference](#)
- [Set Properties](#)
- [Example 1](#)
- [Set Add/Delete Methods](#)
- [Example 2](#)
- [Set Iteration Methods](#)
- [Example 3](#)
- [Set Logic Methods](#)
- [Example 4](#)
- [Reference](#)

Set Properties

The complete **Set** object reference:

Property	Description
size	Number of elements in the Set
constructor	The Set constructor function

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Set Properties</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b", "c", "d"]);

document.getElementById("demo").innerHTML =
  "Set: " + [...letters] + "<br>" +
  "size: " + letters.size + "<br>" +
  "constructor: " + letters.constructor;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Set Add/Delete Methods

Methods for adding and removing elements:

Method	Description
<code>add(value)</code>	Add a new element
<code>delete(value)</code>	Remove an element
<code>has(value)</code>	Check if element exists
<code>clear()</code>	Remove all elements

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Set Add/Delete Methods</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b"]);

let text = "Start: " + [...letters] + "<br><br>";

letters.add("c");
text += "add('c'): " + [...letters] + "<br>";
text += "has('c'): " + letters.has("c") + "<br><br>";

letters.delete("a");
text += "delete('a'): " + [...letters] + "<br><br>";

letters.clear();
text += "clear(): " + [...letters] + " (empty)<br>";
text += "size: " + letters.size;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```


Example 2

Result [View Example](#)

Set Iteration Methods

Methods for iterating over Set elements:

Method	Description
<code>forEach(fn)</code>	Call function for each element
<code>values()</code>	Iterator of all values
<code>keys()</code>	Same as <code>values()</code> for Set
<code>entries()</code>	Iterator of [value, value] pairs

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Set Iteration Methods</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b", "c"]);

let text = "<b>forEach():</b> ";
letters.forEach(v => text += v + " ");
text += "<br>";

text += "<b>values():</b> ";
for (let v of letters.values()) text += v + " ";
text += "<br>";

text += "<b>keys():</b> ";
for (let k of letters.keys()) text += k + " ";
text += "<br>";

text += "<b>entries():</b> ";
for (let e of letters.entries()) text += "[" + e[0] + "," + e[1] + "] ";

document.getElementById("demo").innerHTML = text;
</script>

</body>
```

```
</html>
```

Example 3

Result [View Example](#)

Set Logic Methods

Set logic methods (newer Set methods):

Method	Description
<code>union(set)</code>	All elements from both Sets
<code>intersection(set)</code>	Elements in both Sets
<code>difference(set)</code>	Elements in this but not other
<code>symmetricDifference(set)</code>	Elements in either, not both
<code>isSubsetOf(set)</code>	Every element in this also in other
<code>isSupersetOf(set)</code>	Every element in other also in this
<code>isDisjointFrom(set)</code>	No common elements

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Set Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Set Logic Methods</h4>
<p id="demo"></p>

<script>
const setA = new Set(["a", "b", "c"]);
const setB = new Set(["c", "d"]);

let text = "Set A: " + [...setA] + "<br>";
text += "Set B: " + [...setB] + "<br><br>";

if (setA.union) {
  text += "A.union(B): " + [...setA.union(setB)] + "<br>";
  text += "A.intersection(B): " + [...setA.intersection(setB)] + "<br>";
}
```

```
text += "A.difference(B): " + [...setA.difference(setB)] + "<br>";
text += "A.symmetricDifference(B): " + [...setA.symmetricDifference(setB)] + "<br>";
text += "A.isSubsetOf(B): " + setA.isSubsetOf(setB) + "<br>";
text += "A.isDisjointFrom(new Set(['x'])): " + setA.isDisjointFrom(new Set(["x"]));
} else {
  text += "Logic methods are not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Set Reference](#)

JavaScript Maps

Table of Contents

- [JavaScript Maps](#)
- [How to Create a Map](#)
- [Example 1](#)
- [Adding and Changing Map Values](#)
- [Example 2](#)
- [The get\(\) Method](#)
- [Example 3](#)
- [Maps are Objects](#)
- [Example 4](#)
- [JavaScript Objects vs Maps](#)
- [Example 5](#)
- [Reference](#)

How to Create a Map

A Map holds key-value pairs where the keys can be any datatype. To create a Map, use `new Map()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Maps</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How to Create a Map</h4>
<p id="demo"></p>

<script>
// Create a Map with key-value pairs
const fruits = new Map([
  ["apple", 500],
  ["banana", 300],
  ["orange", 200]
]);

// Create an empty Map
const emptyMap = new Map();

document.getElementById("demo").innerHTML =
  "Map from array: " + [...fruits] + "<br>" +
  "Map size: " + fruits.size + "<br><br>" +
  "Keys can be any datatype:<br>" +
  "fruits.get('apple'): " + fruits.get("apple");
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Adding and Changing Map Values

Use `set()` to add or change Map values. The `get()` method gets the value of a key:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Maps</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding and Changing Map Values</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500]
]);

// Add new entries with set()
fruits.set("banana", 300);
fruits.set("orange", 200);

// Change an existing value
fruits.set("apple", 600);

// set() can be chained
fruits.set("mango", 150).set("grape", 100);

document.getElementById("demo").innerHTML =
  "apple: " + fruits.get("apple") + " (changed to 600)<br>" +
  "banana: " + fruits.get("banana") + "<br>" +
  "orange: " + fruits.get("orange") + "<br>" +
  "mango: " + fruits.get("mango") + "<br>" +
  "grape: " + fruits.get("grape") + "<br><br>" +
  "Map size: " + fruits.size;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The get() Method

`get()` returns the value of a specified key. It returns `undefined` if the key doesn't exist:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Maps</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The get() Method</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300]
]);

document.getElementById("demo").innerHTML =
  "get('apple'): " + fruits.get("apple") + "<br>" +
  "get('banana'): " + fruits.get("banana") + "<br>" +
  "get('orange'): " + fruits.get("orange") + " (undefined - not in Map)<br><br>" +
  "has('apple'): " + fruits.has("apple") + "<br>" +
  "size: " + fruits.size;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Maps are Objects

`typeof` a Map is an object. A Map can use objects as keys:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Maps</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Maps are Objects</h4>
<p id="demo"></p>

<script>
// Using objects as keys
const person1 = { name: "John" };

```

```

const person2 = { name: "Jane" };

const persons = new Map();
persons.set(person1, "Engineer");
persons.set(person2, "Doctor");

let text = "typeof Map: " + typeof persons + "<br>";
text += "Instance of Map: " + (persons instanceof Map) + "<br><br>";

text += "person1's job: " + persons.get(person1) + "<br>";
text += "person2's job: " + persons.get(person2) + "<br><br>";

text += "<b>Other Map methods:</b><br>";
text += "delete(key): removes entry<br>";
text += "clear(): removes all entries<br>";
text += "keys(): iterator of keys<br>";
text += "values(): iterator of values<br>";
text += "entries(): iterator of [key, value] pairs";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

JavaScript Objects vs Maps

Feature	Object	Map
Iterable	Not directly	Yes
Size property	No	Yes (size)
Key types	Strings only	Any type
Key order	Not guaranteed	Insertion order
Default values	Has default keys	No default keys

```

<!DOCTYPE html>
<html>
<body>

```

```

<h2>JavaScript Maps</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Objects vs Maps</h4>
<p id="demo"></p>

<script>
// Object
const obj = { name: "John", age: 30 };

// Map
const map = new Map([
  ["name", "John"],
  ["age", 30]
]);

let text = "<b>Object:</b><br>";
text += "size: " + Object.keys(obj).length + " (via keys())<br>";
text += "Keys: strings only<br>";
text += "Iterable: no<br><br>";

text += "<b>Map:</b><br>";
text += "size: " + map.size + "<br>";
text += "Keys: any type<br>";
text += "Iterable: yes<br><br>";

text += "<b>When to use Map?</b><br>";
text += "1. Need keys other than strings<br>";
text += "2. Need size property<br>";
text += "3. Need iteration<br>";
text += "4. Need insertion order";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Maps](#)

JavaScript Map Methods

Table of Contents

- [JavaScript Map Methods](#)
- [Map.get\(\) and Map.set\(\)](#)
- [Example 1](#)
- [Map.delete\(\) and Map.clear\(\)](#)
- [Example 2](#)
- [Map.has\(\) and Map.size](#)
- [Example 3](#)
- [Map.forEach\(\) and Map.entries\(\)](#)
- [Example 4](#)
- [Map.keys\(\) and Map.values\(\)](#)
- [Example 5](#)
- [Reference](#)

Map Methods

Map has the following methods and properties:

Method/Property	Description
<code>new Map()</code>	Create a new Map
<code>set(key, value)</code>	Set the value for a key
<code>get(key)</code>	Get the value for a key
<code>delete(key)</code>	Remove an entry
<code>has(key)</code>	Check if a key exists
<code>clear()</code>	Remove all entries
<code>forEach(fn)</code>	Call function for each entry
<code>keys()</code>	Iterator of keys
<code>values()</code>	Iterator of values
<code>entries()</code>	Iterator of [key, value] pairs
<code>size</code>	Number of entries

Map.get() and Map.set()

set() adds or updates a key-value pair. **get()** retrieves the value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map.get() and Map.set()</h4>
<p id="demo"></p>

<script>
const fruits = new Map();

// set() adds new entries
fruits.set("apple", 500);
fruits.set("banana", 300);

// set() can update an existing entry
fruits.set("apple", 600);

document.getElementById("demo").innerHTML =
  "get('apple'): " + fruits.get("apple") + "<br>" +
  "get('banana'): " + fruits.get("banana") + "<br><br>" +
  "set() both adds AND updates entries";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Map.delete() and Map.clear()

delete() removes one entry. **clear()** removes all entries:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map.delete() and Map.clear()</h4>
<p id="demo"></p>
```

```

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300],
  ["orange", 200]
]);

// delete() removes one entry
fruits.delete("banana");

let text = "After delete('banana'): " + [...fruits] + "<br><br>";

// clear() removes all entries
fruits.clear();
text += "After clear(): size = " + fruits.size + " (empty)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Map.has() and Map.size

has() returns true if a key exists. **size** returns the number of entries:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map.has() and Map.size</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300]
]);

document.getElementById("demo").innerHTML =
  "Map: " + [...fruits] + "<br>" +
  "size: " + fruits.size + "<br><br>" +
  "has('apple'): " + fruits.has("apple") + "<br>" +
  "has('orange'): " + fruits.has("orange") + "<br><br>" +
  "has() checks if a key exists in the Map";

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 3

Result [View Example](#)

Map.forEach() and Map.entries()

forEach() calls a function for each entry. **entries()** returns an iterator of [key, value] pairs:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map.forEach() and Map.entries()</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300]
]);

let text = "forEach():<br>";
fruits.forEach(function(value, key) {
  text += key + " = " + value + "<br>";
});

text += "<br>entries():<br>";
for (let entry of fruits.entries()) {
  text += "[" + entry[0] + ", " + entry[1] + "]<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Map.keys() and Map.values()

`keys()` returns an iterator of all keys. `values()` returns an iterator of all values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map.keys() and Map.values()</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300],
  ["orange", 200]
]);

let text = "keys(): ";
for (let key of fruits.keys()) {
  text += key + " ";
}
text += "<br><br>";

text += "values(): ";
for (let value of fruits.values()) {
  text += value + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Map Methods](#)

JavaScript WeakMap

Table of Contents

- [JavaScript WeakMap](#)
- [The WeakMap Object](#)
- [Example 1](#)
- [Keys Must Be Objects](#)
- [Example 2](#)
- [Weak References](#)
- [Example 3](#)
- [Not Iterable and Limited Methods](#)
- [Example 4](#)
- [WeakMap Secret Data](#)
- [Example 5](#)
- [Reference](#)

The WeakMap Object

A **WeakMap** is a collection of key/value pairs where the keys must be **objects** (not primitives). Values can be any type:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakMap</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The WeakMap Object</h4>
<p id="demo"></p>

<script>
// WeakMap keys must be objects
const obj1 = { name: "John" };
const obj2 = { name: "Jane" };

const weakMap = new WeakMap();
weakMap.set(obj1, "Engineer");
weakMap.set(obj2, "Doctor");

document.getElementById("demo").innerHTML =
  "WeakMap size (no size property)<br>" +
  "get(obj1): " + weakMap.get(obj1) + "<br>" +
  "get(obj2): " + weakMap.get(obj2) + "<br><br>" +
  "WeakMap keys must be objects";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Keys Must Be Objects

If you try to use a primitive value (string, number, etc.) as a key in a WeakMap, it causes an error:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakMap</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Keys Must Be Objects</h4>
<p id="demo"></p>

<script>
const weakMap = new WeakMap();

// Keys must be objects
const obj = { id: 1 };
weakMap.set(obj, "Valid");

let text = "set(obj, 'Valid'): OK (object key)<br><br>";

// This would cause an error: Invalid value used as weak map key
try {
  weakMap.set("name", "John"); // TypeError!
  text += "set('name', 'John'): OK?";
} catch(err) {
  text += "set('name', 'John'): ERROR - " + err.message + "<br><br>";
  text += "WeakMap keys MUST be objects";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Weak References

WeakMap holds keys by **weak references**. When a key object is no longer referenced elsewhere, it can be garbage collected:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakMap</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Weak References</h4>
<p id="demo"></p>

<script>
const weakMap = new WeakMap();

let person = { name: "John" };
weakMap.set(person, "Engineer");

let text = "Person object added to WeakMap<br>";
text += "get(person): " + weakMap.get(person) + "<br><br>";

// Remove all references to the object
person = null;

text += "After person = null:<br>";
text += "The object can be garbage collected<br>";
text += "It will be automatically removed from WeakMap<br><br>";
text += "This automatic cleanup is the key benefit<br>";
text += "of WeakMap for memory management";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Not Iterable and Limited Methods

A WeakMap is **not iterable**. It only has four methods: `set()`, `get()`, `delete()`, and `has()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakMap</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Not Iterable and Limited Methods</h4>
<p id="demo"></p>

```



```

<script>
const weakMap = new WeakMap();
const obj = { id: 1 };

// The 4 available methods
weakMap.set(obj, "value");
let text = "After set(obj, 'value'):<br>";
text += "get(obj): " + weakMap.get(obj) + "<br>";
text += "has(obj): " + weakMap.has(obj) + "<br><br>";

weakMap.delete(obj);
text += "After delete(obj):<br>";
text += "has(obj): " + weakMap.has(obj) + "<br><br>";

text += "<b>WeakMap limitations:</b><br>";
text += "1. NOT iterable (no forEach, keys, values)<br>";
text += "2. NO size property<br>";
text += "3. Only 4 methods: set(), get(), delete(), has()";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

WeakMap Secret Data

WeakMap can store private data associated with objects without exposing it:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript WeakMap</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>WeakMap Secret Data</h4>
<p id="demo"></p>

<script>
// Store secret data in a WeakMap
const secrets = new WeakMap();

function Person(name, age, secret) {
  this.name = name;
  this.age = age;
  // Store secret data in WeakMap (not directly on object)

```

```

    secrets.set(this, secret);
}

const person1 = new Person("John", 30, "secret code: 1234");
const person2 = new Person("Jane", 25, "secret password: abc");

let text = "Person 1: " + person1.name + ", " + person1.age + "<br>";
text += "Person 2: " + person2.name + ", " + person2.age + "<br><br>";

// Access secret data via WeakMap
text += "Secret 1 (via WeakMap): " + secrets.get(person1) + "<br>";
text += "Secret 2 (via WeakMap): " + secrets.get(person2) + "<br><br>";

// Secrets are NOT visible directly on the object
text += "person1.secret: " + person1.secret + " (undefined - private!)<br>";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript WeakMap](#)

JavaScript Map Reference

Table of Contents

- [JavaScript Map Reference](#)
- [Map Properties](#)
- [Example 1](#)
- [Map Add/Remove Methods](#)
- [Example 2](#)
- [Map Iteration Methods](#)
- [Example 3](#)
- [Reference](#)

Map Properties

The complete **Map** object reference:

Property	Description
size	Number of entries in the Map
constructor	The Map constructor function

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map Properties</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300],
  ["orange", 200]
]);

document.getElementById("demo").innerHTML =
  "Map: " + [...fruits] + "<br>" +
  "size: " + fruits.size + "<br>" +
  "constructor: " + fruits.constructor;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Map Add/Remove Methods

Methods for adding and removing entries:

Method	Description
<code>set(key, value)</code>	Add or update an entry
<code>get(key)</code>	Get the value of a key
<code>delete(key)</code>	Remove an entry
<code>has(key)</code>	Check if a key exists
<code>clear()</code>	Remove all entries

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map Add/Remove Methods</h4>
<p id="demo"></p>

<script>
const fruits = new Map();

// set() adds entries
fruits.set("apple", 500);
fruits.set("banana", 300);

// get() retrieves values
let text = "get('apple'): " + fruits.get("apple") + "<br>";
text += "has('banana'): " + fruits.has("banana") + "<br><br>";

// delete() removes one entry
fruits.delete("banana");
text += "After delete('banana'): " + [...fruits] + "<br><br>";

// clear() removes all
fruits.clear();
```

```

text += "After clear(): size = " + fruits.size;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Map Iteration Methods

Methods for iterating over Map entries:

Method	Description
<code>forEach(fn)</code>	Call function for each entry
<code>keys()</code>	Iterator of all keys
<code>values()</code>	Iterator of all values
<code>entries()</code>	Iterator of [key, value] pairs

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Map Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Map Iteration Methods</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300]
]);

let text = "<b>forEach():</b><br>";
fruits.forEach((value, key) => text += key + " = " + value + "<br>");
text += "<br>";

text += "<b>keys():</b> ";
for (let k of fruits.keys()) text += k + " ";

```

```
text += "<br><br>";

text += "<b>values():</b> ";
for (let v of fruits.values()) text += v + " ";
text += "<br><br>";

text += "<b>entries():</b> ";
for (let e of fruits.entries()) text += "[" + e[0] + "," + e[1] + "] ";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Map Reference](#)

JavaScript Looping

Table of Contents

- [JavaScript Looping](#)
- [The for Loop](#)
- [Example 1](#)
- [The while Loop](#)
- [Example 2](#)
- [The do...while Loop](#)
- [Example 3](#)
- [The for...in Loop](#)
- [Example 4](#)
- [The for...of Loop](#)
- [Example 5](#)
- [Reference](#)

The for Loop

The **for** loop repeats a block of code a specific number of times:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Looping</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The for Loop</h4>
<p id="demo"></p>

<script>
let text = "";
for (let i = 0; i < 5; i++) {
  text += "Iteration: " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The while Loop

The **while** loop repeats while a condition is true:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Looping</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The while Loop</h4>
<p id="demo"></p>

<script>
let i = 0;
let text = "";
while (i < 5) {
  text += "Number: " + i + "<br>";
  i++;
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

The do...while Loop

The **do...while** loop runs at least once before checking the condition:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Looping</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The do...while Loop</h4>
<p id="demo"></p>

<script>
let i = 0;
let text = "";
do {
  text += "Number: " + i + "<br>";
  i++;
} while (i < 5);
document.getElementById("demo").innerHTML = text;
</script>

```



```
</body>
</html>
```

Example 3

Result [View Example](#)

The for...in Loop

The **for...in** loop iterates over the keys/properties of an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Looping</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The for...in Loop</h4>
<p id="demo"></p>

<script>
const person = { name: "John", age: 30, city: "New York" };
let text = "";

for (let key in person) {
  text += key + ": " + person[key] + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The for...of Loop

The **for...of** loop iterates over the values of an iterable (arrays, strings, Maps, Sets):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Looping</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>The for...of Loop</h4>
<p id="demo"></p>

<script>
const fruits = ["Apple", "Banana", "Orange"];
let text = "";

for (let fruit of fruits) {
  text += fruit + "<br>";
}

// Can also iterate over strings
text += "<br>String characters:<br>";
for (let ch of "JS") {
  text += ch + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Looping](#)

JavaScript Iterables

Table of Contents

- [JavaScript Iterables](#)
- [Iterating Over a String](#)
- [Example 1](#)
- [Iterating Over an Array](#)
- [Example 2](#)
- [Iterating Over a Set](#)
- [Example 3](#)
- [Iterating Over a Map](#)
- [Example 4](#)
- [Home Made Iterable](#)
- [Example 5](#)
- [Reference](#)

An Iterable is an Iterable Object

An iterable is an object that can be iterated over with `for...of`. Iterables include: strings, arrays, Sets, and Maps.

Iterating Over a String

Strings are iterable, so you can loop over each character:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iterating Over a String</h4>
<p id="demo"></p>

<script>
const word = "Hello";
let text = "";

for (let char of word) {
  text += char + " ";
}

document.getElementById("demo").innerHTML =
  "String: " + word + "<br>" +
  "Characters: " + text;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Iterating Over an Array

Arrays are iterable, so you can loop over each element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iterating Over an Array</h4>
<p id="demo"></p>

<script>
const fruits = ["Apple", "Banana", "Orange"];
let text = "";

for (let fruit of fruits) {
  text += fruit + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Iterating Over a Set

Sets are iterable, so you can loop over each value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Iterating Over a Set</h4>
<p id="demo"></p>

<script>
const letters = new Set(["a", "b", "c"]);
let text = "";

for (let letter of letters) {
  text += letter + " ";
}

document.getElementById("demo").innerHTML =
  "Set: a, b, c<br>" +
  "Iterated: " + text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Iterating Over a Map

Maps are iterable, so you can loop over each entry:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iterating Over a Map</h4>
<p id="demo"></p>

<script>
const fruits = new Map([
  ["apple", 500],
  ["banana", 300]
]);
let text = "";

for (let entry of fruits) {
  text += entry[0] + " = " + entry[1] + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

```

```
</body>
</html>
```

Example 4

Result [View Example](#)

Home Made Iterable

You can create your own iterable by implementing the `Symbol.iterator` method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Home Made Iterable</h4>
<p id="demo"></p>

<script>
// Create a home made iterable
function myIterable() {
  let i = 0;
  return {
    [Symbol.iterator]: function() {
      return {
        next: function() {
          i++;
          return i <= 5 ? {value: i, done: false} : {done: true};
        }
      };
    }
  };
};

let text = "";
for (let num of myIterable()) {
  text += num + " ";
}

document.getElementById("demo").innerHTML =
  "Home made iterable: " + text + "<br><br>" +
  "Custom iterable using Symbol.iterator and next()";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Iterables](#)

JavaScript Iterators

Table of Contents

- [JavaScript Iterators](#)
- [The Iterator Object](#)
- [Example 1](#)
- [The next\(\) Method](#)
- [Example 2](#)
- [Iterator.from\(\) Method](#)
- [Example 3](#)
- [Iterator Helper Methods](#)
- [Example 4](#)
- [Reference](#)

The Iterator Object

An iterator is an object that defines a sequence of values and a termination condition. It implements the `next()` method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Iterator Object</h4>
<p id="demo"></p>

<script>
// An iterator is an object with a next() method
// It defines a sequence of values

// Arrays are iterables (have Symbol.iterator)
const arr = [10, 20, 30];
const iterator = arr[Symbol.iterator]();

let text = "Array iterator for [10, 20, 30]:<br><br>";
text += "typeof arr[Symbol.iterator]: " + typeof arr[Symbol.iterator] + "<br>";
text += "Iterator has next(): " + (typeof iterator.next === "function");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The next() Method

The `next()` method returns an object with `value` and `done` properties. When `done` is true, iteration is complete:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The next() Method</h4>
<p id="demo"></p>

<script>
const fruits = ["Apple", "Banana", "Orange"];
const iterator = fruits[Symbol.iterator]();

let text = "Iterating with next():<br><br>";
let result = iterator.next();
while (!result.done) {
  text += "next(): value = " + result.value + ", done = " + result.done + "<br>";
  result = iterator.next();
}
text += "next(): value = " + result.value + ", done = " + result.done + " (done!)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Iterator.from() Method

`Iterator.from()` creates an iterator from an iterable object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iterator.from() Method</h4>
```

```

<p id="demo"></p>

<script>
let text = "";

if (Iterator.from) {
  // Create an iterator from an array
  const iterator = Iterator.from([1, 2, 3]);

  text += "Iterator.from([1, 2, 3]):<br>";
  let result = iterator.next();
  while (!result.done) {
    text += "value = " + result.value + "<br>";
    result = iterator.next();
  }
} else {
  text += "Iterator.from() is not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Iterator Helper Methods

Newer iterators support helper methods like `map()`, `filter()`, and `take()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iterator Helper Methods</h4>
<p id="demo"></p>

<script>
let text = "";

if (Iterator.from) {
  const iterator = Iterator.from([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);

  // Using helper methods
  const doubled = iterator.map(v => v * 2);

```

```
text += "Iterator helpers:<br><br>";
text += "map(v * 2): ";
let result = doubled.next();
while (!result.done) {
  text += result.value + " ";
  result = doubled.next();
}
text += "<br><br>";
}

text += "<b>Available helper methods:</b><br>";
text += "map(), filter(), take(), drop()<br>";
text += "every(), some(), find(), reduce()<br>";
text += "forEach(), flatMap()";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Iterators](#)

JavaScript Generators

Table of Contents

- [JavaScript Generators](#)
- [Generator Functions](#)
- [Example 1](#)
- [The yield Keyword](#)
- [Example 2](#)
- [Generator Object Methods](#)
- [Example 3](#)
- [Custom Iterators with Generators](#)
- [Example 4](#)
- [Reference](#)

Generator Functions

A generator function is a special function that can pause and resume execution. It is defined with **function***:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Generator Functions</h4>
<p id="demo"></p>

<script>
// Generator function defined with function*
function* myGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

// Calling a generator returns a generator object
const gen = myGenerator();

let text = "Generator function created<br><br>";

// Iterate over values
for (let value of gen) {
  text += "Yielded: " + value + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

The yield Keyword

The **yield** keyword pauses the generator and returns a value. The generator resumes when **next()** is called:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The yield Keyword</h4>
<p id="demo"></p>

<script>
function* fruitGenerator() {
  yield "Apple";
  yield "Banana";
  yield "Orange";
}

const gen = fruitGenerator();

let text = "Using yield with next():<br><br>";
text += gen.next().value + "<br>";
text += gen.next().value + "<br>";
text += gen.next().value + "<br>";
text += "done: " + gen.next().done;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Generator Object Methods

Generator objects have methods like **next()** and **return()**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Generator Object Methods</h4>
<p id="demo"></p>

<script>
function* counter() {
  let i = 0;
  while (true) {
    yield i++;
  }
}

const gen = counter();

let text = "Counter generator:<br><br>";
text += "next(): " + gen.next().value + "<br>";
text += "next(): " + gen.next().value + "<br>";
text += "next(): " + gen.next().value + "<br><br>";

// Other generator methods
text += "typeof gen.next: " + typeof gen.next + "<br>";
text += "typeof gen.return: " + typeof gen.return + "<br>";

// return() stops the generator
gen.return("stopped");
text += "After return(): " + gen.next().done + " (done)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Custom Iterators with Generators

Generators make custom iterators easy to create:

```

<!DOCTYPE html>
<html>
<body>

```

```
<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Custom Iterators with Generators</h4>
<p id="demo"></p>

<script>
// Create a custom iterable using a generator
const myIterable = {
  *[Symbol.iterator]() {
    yield "One";
    yield "Two";
    yield "Three";
  }
};

let text = "Custom iterable with generator:<br><br>";
for (let value of myIterable) {
  text += value + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Generators](#)

JavaScript Math

Table of Contents

- [JavaScript Math](#)
- [Math Properties \(Constants\)](#)
- [Example 1](#)
- [Number to Integer](#)
- [Example 2](#)
- [Math.pow\(\) and Math.sqrt\(\)](#)
- [Example 3](#)
- [Math.min\(\) and Math.max\(\)](#)
- [Example 4](#)
- [Math.random\(\)](#)
- [Example 5](#)
- [Reference](#)

Math Properties (Constants)

The **Math** object provides mathematical constants as properties:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Math Properties (Constants)</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.E: " + Math.E + "<br>" +
  "Math.PI: " + Math.PI + "<br>" +
  "Math.SQRT2: " + Math.SQRT2 + "<br>" +
  "Math.SQRT1_2: " + Math.SQRT1_2 + "<br>" +
  "Math.LN2: " + Math.LN2 + "<br>" +
  "Math.LN10: " + Math.LN10 + "<br>" +
  "Math.LOG2E: " + Math.LOG2E + "<br>" +
  "Math.LOG10E: " + Math.LOG10E;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Number to Integer

Methods to convert numbers to integers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Number to Integer</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.round(4.6): " + Math.round(4.6) + " (nearest)<br>" +
  "Math.ceil(4.2): " + Math.ceil(4.2) + " (up)<br>" +
  "Math.floor(4.8): " + Math.floor(4.8) + " (down)<br>" +
  "Math.trunc(4.7): " + Math.trunc(4.7) + " (remove decimals)<br>" +
  "Math.sign(-5): " + Math.sign(-5) + " (-1, 0, or 1)<br>" +
  "Math.abs(-7.5): " + Math.abs(-7.5) + " (absolute value)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Math.pow() and Math.sqrt()

Power and square root functions:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Math.pow() and Math.sqrt()</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.pow(2, 3): " + Math.pow(2, 3) + " (2³)<br>" +
  "Math.pow(5, 2): " + Math.pow(5, 2) + " (5²)<br>" +
  "Math.pow(3, 4): " + Math.pow(3, 4) + " (3⁴)<br><br>" +
  "Math.sqrt(16): " + Math.sqrt(16) + "<br>" +
  "Math.sqrt(81): " + Math.sqrt(81) + "<br>" +
```

```
"Math.sqrt(2): " + Math.sqrt(2).toFixed(4);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Math.min() and Math.max()

Find the lowest and highest values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Math.min() and Math.max()</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.min(5, 10, 3, 8): " + Math.min(5, 10, 3, 8) + "<br>" +
  "Math.max(5, 10, 3, 8): " + Math.max(5, 10, 3, 8) + "<br><br>" +
  "Math.min(-5, 0, -10): " + Math.min(-5, 0, -10) + "<br>" +
  "Math.max(-5, 0, -10): " + Math.max(-5, 0, -10);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Math.random()

Math.random() returns a random number between 0 (inclusive) and 1 (exclusive):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>Math.random()</h4>
<p id="demo"></p>

<script>
let text = "Random numbers:<br><br>";

for (let i = 0; i < 3; i++) {
  text += Math.random() + "<br>";
}

text += "<br>Random integer 1-10: " +
  Math.floor(Math.random() * 10) + 1;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Math](#)

JavaScript Math Reference

Table of Contents

- [JavaScript Math Reference](#)
- [Math Properties](#)
- [Example 1](#)
- [Math Rounding Methods](#)
- [Example 2](#)
- [Math Power and Root Methods](#)
- [Example 3](#)
- [Math Trigonometry Methods](#)
- [Example 4](#)
- [Math Utility Methods](#)
- [Example 5](#)
- [Reference](#)

Math Properties

The complete **Math** object reference - properties (constants):

Property	Description
Math.E	Euler's number (~2.718)
Math.PI	Pi (~3.14159)
Math.SQRT2	Square root of 2 (~1.414)
Math.SQRT1_2	Square root of 1/2 (~0.707)
Math.LN2	Natural log of 2 (~0.693)
Math.LN10	Natural log of 10 (~2.303)
Math.LOG2E	Log base 2 of E (~1.443)
Math.LOG10E	Log base 10 of E (~0.434)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math Reference</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Math Properties</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.E: " + Math.E + "<br>" +
  "Math.PI: " + Math.PI + "<br>" +
  "Math.SQRT2: " + Math.SQRT2 + "<br>" +
  "Math.SQRT1_2: " + Math.SQRT1_2 + "<br>" +
  "Math.LN2: " + Math.LN2 + "<br>" +
  "Math.LN10: " + Math.LN10 + "<br>" +
  "Math.LOG2E: " + Math.LOG2E + "<br>" +
  "Math.LOG10E: " + Math.LOG10E;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Math Rounding Methods

Reference for rounding methods:

Method	Description
<code>Math.round(x)</code>	Rounds to nearest integer
<code>Math.ceil(x)</code>	Rounds up to nearest integer
<code>Math.floor(x)</code>	Rounds down to nearest integer
<code>Math.trunc(x)</code>	Removes decimal part (no rounding)
<code>Math.sign(x)</code>	Returns -1, 0, or 1

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Math Rounding Methods</h4>

```

```

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.round(4.6): " + Math.round(4.6) + "<br>" +
  "Math.ceil(4.2): " + Math.ceil(4.2) + "<br>" +
  "Math.floor(4.9): " + Math.floor(4.9) + "<br>" +
  "Math.trunc(4.7): " + Math.trunc(4.7) + "<br>" +
  "Math.sign(-5): " + Math.sign(-5) + "<br>" +
  "Math.sign(0): " + Math.sign(0) + "<br>" +
  "Math.sign(8): " + Math.sign(8);
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Math Power and Root Methods

Reference for power and root methods:

Method	Description
<code>Math.pow(x, y)</code>	x raised to the power y
<code>Math.sqrt(x)</code>	Square root of x
<code>Math.cbrt(x)</code>	Cube root of x
<code>Math.exp(x)</code>	e raised to the power x
<code>Math.log(x)</code>	Natural log of x
<code>Math.log2(x)</code>	Log base 2 of x
<code>Math.log10(x)</code>	Log base 10 of x

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

```

```

<h4>Math Power and Root Methods</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.pow(2, 3): " + Math.pow(2, 3) + "<br>" +
  "Math.sqrt(16): " + Math.sqrt(16) + "<br>" +
  "Math.cbrt(8): " + Math.cbrt(8) + "<br>" +
  "Math.exp(1): " + Math.exp(1).toFixed(4) + "<br>" +
  "Math.log(10): " + Math.log(10).toFixed(4) + "<br>" +
  "Math.log2(8): " + Math.log2(8) + "<br>" +
  "Math.log10(1000): " + Math.log10(1000);
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Math Trigonometry Methods

Reference for trigonometry methods:

Method	Description
<code>Math.sin(x)</code>	Sine of x (radians)
<code>Math.cos(x)</code>	Cosine of x (radians)
<code>Math.tan(x)</code>	Tangent of x (radians)
<code>Math.asin(x)</code>	Arcsine of x
<code>Math.acos(x)</code>	Arccosine of x
<code>Math.atan(x)</code>	Arctangent of x

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Math Trigonometry Methods</h4>

```

```

<p id="demo"></p>

<script>
// Angle in radians: 90 degrees = PI/2
const angle = Math.PI / 2;

document.getElementById("demo").innerHTML =
  "sin(PI/2): " + Math.sin(angle).toFixed(2) + "<br>" +
  "cos(0): " + Math.cos(0).toFixed(2) + "<br>" +
  "tan(PI/4): " + Math.tan(Math.PI / 4).toFixed(2) + "<br><br>" +
  "asin(1): " + Math.asin(1).toFixed(2) + " rad<br>" +
  "acos(1): " + Math.acos(1).toFixed(2) + " rad<br>" +
  "atan(1): " + Math.atan(1).toFixed(2) + " rad";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Math Utility Methods

Reference for utility methods:

Method	Description
<code>Math.abs(x)</code>	Absolute value of x
<code>Math.min(a, b, ...)</code>	Lowest value
<code>Math.max(a, b, ...)</code>	Highest value
<code>Math.random()</code>	Random number 0-1
<code>Math.hypot(a, b)</code>	Square root of (a ² + b ²)
<code>Math.fround(x)</code>	Nearest 32-bit float

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

```



```
<h4>Math Utility Methods</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Math.abs(-7.5): " + Math.abs(-7.5) + "<br>" +
  "Math.min(3, 8, 1): " + Math.min(3, 8, 1) + "<br>" +
  "Math.max(3, 8, 1): " + Math.max(3, 8, 1) + "<br>" +
  "Math.random(): " + Math.random().toFixed(4) + "<br>" +
  "Math.hypot(3, 4): " + Math.hypot(3, 4) + " (3-4-5 triangle)<br>" +
  "Math.fround(1.5): " + Math.fround(1.5);
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Math Reference](#)

JavaScript Random

Table of Contents

- [JavaScript Random](#)
- [Math.random\(\)](#)
- [Example 1](#)
- [JavaScript Random Integers](#)
- [Example 2](#)
- [Random Integers in a Range](#)
- [Example 3](#)
- [A Proper Random Function](#)
- [Example 4](#)
- [Reference](#)

Math.random()

Math.random() returns a random number between 0 (inclusive) and 1 (exclusive):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Random</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Math.random()</h4>
<p id="demo"></p>

<script>
let text = "Random numbers between 0 and 1:<br><br>";

for (let i = 0; i < 5; i++) {
  text += Math.random() + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Random Integers

Math.random() used with **Math.floor()** generates random integers:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Random</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Random Integers</h4>
<p id="demo"></p>

<script>
let text = "Random integers:<br><br>";

// 0 to 9
text += "Math.floor(Math.random() * 10): ";
for (let i = 0; i < 3; i++) {
    text += Math.floor(Math.random() * 10) + " ";
}
text += "<br>";

// 1 to 10
text += "Math.floor(Math.random() * 10) + 1: ";
for (let i = 0; i < 3; i++) {
    text += Math.floor(Math.random() * 10) + 1 + " ";
}
text += "<br>";

// 0 to 100
text += "Math.floor(Math.random() * 100): ";
for (let i = 0; i < 3; i++) {
    text += Math.floor(Math.random() * 100) + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Random Integers in a Range

You can generate random integers in a specific range (min to max):

```

<!DOCTYPE html>
<html>
<body>

```

```

<h2>JavaScript Random</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Random Integers in a Range</h4>
<p id="demo"></p>

<script>
// Random integer between min (inclusive) and max (inclusive)
function getRndInteger(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

let text = "Random integers between 1 and 6 (dice):<br>";
for (let i = 0; i < 6; i++) {
  text += getRndInteger(1, 6) + " ";
}
text += "<br><br>";

text += "Random integers between 10 and 20:<br>";
for (let i = 0; i < 6; i++) {
  text += getRndInteger(10, 20) + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

A Proper Random Function

A reusable function for generating random integers between any two values:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Random</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Proper Random Function</h4>
<p id="demo"></p>

<button onclick="rollDice()">Roll Dice</button>
<p id="diceResult"></p>

<script>
// Proper random function: min (inclusive) to max (inclusive)

```

```
function getRndInteger(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}

function rollDice() {
    const dice1 = getRndInteger(1, 6);
    const dice2 = getRndInteger(1, 6);
    document.getElementById("diceResult").innerHTML =
        "Dice 1: " + dice1 + " | Dice 2: " + dice2 + " | Total: " + (dice1 + dice2);
}

document.getElementById("demo").innerHTML =
    "This function returns a random integer:<br>" +
    "from min (inclusive) to max (inclusive):<br><br>" +
    "function getRndInteger(min, max) {<br>" +
    "    &nbsp;   return Math.floor(Math.random() * (max - min + 1)) + min;<br>" +
    " "}";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Random](#)

JavaScript RegExp

Table of Contents

- [JavaScript RegExp](#)
- [Regular Expression Syntax](#)
- [Example 1](#)
- [Using String Methods](#)
- [Example 2](#)
- [JavaScript Regex Flags](#)
- [Example 3](#)
- [RegExp Metacharacters](#)
- [Example 4](#)
- [Reference](#)

Regular Expression Syntax

A regular expression is a sequence of characters that forms a search pattern. It is written between slashes `/pattern/`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Regular Expression Syntax</h4>
<p id="demo"></p>

<script>
const text = "The rain in Spain falls mainly in the plain";

// Search for "ain" using a regular expression
const pattern = /ain/;

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "Pattern: /ain/<br>" +
  "match() result: " + text.match(pattern);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using String Methods

Regular expressions are commonly used with string methods: `match()`, `replace()`, and `search()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using String Methods</h4>
<p id="demo"></p>

<script>
const text = "Visit W3Schools";

// match() - returns matches
const matchResult = text.match(/W3Schools/);

// replace() - replaces matches
const replaceResult = text.replace(/Visit/, "Welcome to");

// search() - returns position
const searchResult = text.search(/W3Schools/);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/W3Schools/): " + matchResult + "<br>" +
  "replace(/Visit/, 'Welcome to'): " + replaceResult + "<br>" +
  "search(/W3Schools/): " + searchResult + " (position)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Regex Flags

Flags modify how the pattern is searched:

Flag	Description
<code>/g</code>	Global - matches all occurrences
<code>/i</code>	Insensitive - case insensitive

/m

Multiline - ^ and \$ match line breaks

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Regex Flags</h4>
<p id="demo"></p>

<script>
const text = "The rain in Spain";

// Without /g - only first match
const noG = text.match(/ain/);

// With /g - all matches
const withG = text.match(/ain/g);

// With /i - case insensitive
const withI = text.match(/THE/i);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/ain/): " + noG + " (first only)<br>" +
  "match(/ain/g): " + withG + " (all occurrences)<br>" +
  "match(/THE/i): " + withI + " (case insensitive)";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

RegExp Metacharacters

Metacharacters have special meanings in regular expressions:

Metacharacter	Description
\d	Find a digit
\s	Find a whitespace

`\w`

Find a word character

`.`

Find a single character

`\b`

Find at word boundary

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp Metacharacters</h4>
<p id="demo"></p>

<script>
const text = "Give 100% and 25% effort";

// \d finds digits
const digits = text.match(/\d/g);

// \w finds word characters
const words = text.match(/\w+/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\d/g) digits: " + digits + "<br>" +
  "match(/\w+/g) words: " + words;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp](#)

JavaScript RegExp Flags

Table of Contents

- [JavaScript RegExp Flags](#)
- [RegExp /g Flag \(Global\)](#)
- [Example 1](#)
- [RegExp /i Flag \(Insensitive\)](#)
- [Example 2](#)
- [RegExp /m Flag \(Multiline\)](#)
- [Example 3](#)
- [RegExp /y Flag \(Sticky\)](#)
- [Example 4](#)
- [RegExp /u Flag \(Unicode\)](#)
- [Example 5](#)
- [Reference](#)

RegExp Modifier Flags

Flags modify how a regular expression searches for patterns. The syntax is `/pattern/flags`:

Flag	Description
<code>/g</code>	Global - matches all occurrences
<code>/i</code>	Insensitive - case insensitive
<code>/m</code>	Multiline - <code>^</code> and <code>\$</code> match line breaks
<code>/s</code>	DotAll - <code>.</code> matches newlines too
<code>/y</code>	Sticky - matches only from <code>lastIndex</code>
<code>/u</code>	Unicode - proper Unicode handling

RegExp /g Flag (Global)

The `/g` flag searches for **all** occurrences, not just the first:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>RegExp /g Flag (Global)</h4>
<p id="demo"></p>

<script>
const text = "apple banana apple cherry apple";

// Without /g - only first match
const first = text.match(/apple/);

// With /g - all matches
const all = text.match(/apple/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "without /g: " + first + "<br>" +
  "with /g: " + all + " (all occurrences)";
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

RegExp /i Flag (Insensitive)

The **/i** flag makes the search **case insensitive**:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /i Flag (Insensitive)</h4>
<p id="demo"></p>

<script>
const text = "JavaScript is FUN!";

// Case sensitive
const sensitive = text.match(/fun/);

// Case insensitive
const insensitive = text.match(/fun/i);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/fun/): " + sensitive + " (no match)<br>" +

```

```
"match(/fun/i): " + insensitive + " (matched)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

RegExp /m Flag (Multiline)

The `/m` flag makes `^` and `$` match the beginning/end of each line:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /m Flag (Multiline)</h4>
<p id="demo"></p>

<script>
const text = "line1\nline2\nline3";

// Without /m - ^ only matches start of string
const withoutM = text.match(/^line/g);

// With /m - ^ matches start of each line
const withM = text.match(/^line/gm);

document.getElementById("demo").innerHTML =
  "Text:<br>" + text + "<br><br>" +
  "match(/^line/g): " + withoutM + "<br>" +
  "match(/^line/gm): " + withM + " (each line)";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

RegExp /y Flag (Sticky)

The `/y` flag makes the search **sticky** - it only matches starting at `lastIndex`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /y Flag (Sticky)</h4>
<p id="demo"></p>

<script>
const text = "abcabc";
const sticky = /abc/y;

// First match at position 0
const first = sticky.exec(text);

// After lastIndex = 3, matches "abc" again at position 3
const second = sticky.exec(text);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "sticky = /abc/y<br>" +
  "First exec: " + first[0] + " at index " + first.index + "<br>" +
  "lastIndex after: " + sticky.lastIndex + "<br>" +
  "Second exec: " + second[0] + " at index " + second.index;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

RegExp /u Flag (Unicode)

The `/u` flag enables proper Unicode handling:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /u Flag (Unicode)</h4>
<p id="demo"></p>

<script>
const text = "Café 100";

```

```
// \d finds ASCII digits only
const asciiDigits = text.match(/\d/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\d/g): " + asciiDigits + "<br><br>" +
  "<b>Other flag properties:</b><br>" +
  "global: " + /a/g.global + "<br>" +
  "ignoreCase: " + /a/i.ignoreCase + "<br>" +
  "multiline: " + /a/m.multiline + "<br>" +
  "sticky: " + /a/y.sticky + "<br>" +
  "unicode: " + /a/u.unicode;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Flags](#)

JavaScript RegExp Characters

Table of Contents

- [JavaScript RegExp Characters](#)
- [Character Class \[HW\]](#)
- [Example 1](#)
- [Character Range \[A-Z\]](#)
- [Example 2](#)
- [Character Class \[1234\]](#)
- [Example 3](#)
- [Character Range \[1-4\]](#)
- [Example 4](#)
- [Reference](#)

RegExp Character Classes

Character classes match any one of the characters inside square brackets [...].

Character Class [HW]

[HW] matches any single character in the set H or W:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Characters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Character Class [HW]</h4>
<p id="demo"></p>

<script>
const text = "HELLO world";
const matches = text.match(/[HW]/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/[HW]/g): " + matches + "<br>" +
  "Matches any H or W character";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Character Range [A-Z]

[A-Z] matches any uppercase letter from A to Z:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Characters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Character Range [A-Z]</h4>
<p id="demo"></p>

<script>
const text = "Hello World 123";
const matches = text.match(/[A-Z]/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/[A-Z]/g): " + matches + "<br>" +
  "Matches any uppercase letter";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Character Class [1234]

[1234] matches any single digit in the set 1, 2, 3, or 4:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Characters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Character Class [1234]</h4>
<p id="demo"></p>

<script>
const text = "123456789";
const matches = text.match(/[1234]/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
```



```
"match(/[1234]/g): " + matches + "<br>" +  
"Matches digits 1, 2, 3, or 4 only";  
</script>  
  
</body>  
</html>
```

Example 3

Result [View Example](#)

Character Range [1-4]

[1-4] matches any digit from 1 to 4 (same as [1234] but using a range):

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript RegExp Characters</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>Character Range [1-4]</h4>  
<p id="demo"></p>  
  
<script>  
const text = "123456789";  
const matches = text.match(/[1-4]/g);  
  
document.getElementById("demo").innerHTML =  
  "Text: " + text + "<br><br>" +  
  "match(/[1-4]/g): " + matches + "<br>" +  
  "Range syntax = same as [1234]";  
</script>  
  
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Characters](#)

JavaScript RegExp Meta Characters

Table of Contents

- [JavaScript RegExp Meta Characters](#)
- [RegExp \d \(digits\) Metacharacter](#)
- [Example 1](#)
- [RegExp \D Metacharacter](#)
- [Example 2](#)
- [RegExp \w \(word\) Metacharacter](#)
- [Example 3](#)
- [The \s \(space\) Metacharacter](#)
- [Example 4](#)
- [Reference](#)

Regular Expression Metacharacters

Metacharacters are characters with special meanings in regular expressions:

Metacharacter	Description
<code>\d</code>	Find a digit (0-9)
<code>\D</code>	Find a non-digit
<code>\w</code>	Find a word character (letter, digit, underscore)
<code>\W</code>	Find a non-word character
<code>\s</code>	Find a whitespace character
<code>\S</code>	Find a non-whitespace character
<code>\xhh</code>	Find a character by hexadecimal value
<code>\uhhhh</code>	Find a Unicode character

RegExp \d (digits) Metacharacter

`\d` matches any digit character (0-9):

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript RegExp Meta Characters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp \d (digits) Metacharacter</h4>
<p id="demo"></p>

<script>
const text = "Phone: 123-456-7890";
const matches = text.match(/\d/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\d/g): " + matches + "<br>" +
  "Finds all digits (0-9)";
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

RegExp \D Metacharacter

\D matches any character that is NOT a digit:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Meta Characters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp \D Metacharacter</h4>
<p id="demo"></p>

<script>
const text = "Phone: 123-456";
const matches = text.match(/\D/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\D/g): " + matches.join("") + "<br>" +
  "Finds all non-digit characters";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

RegExp \w (word) Metacharacter

\w matches any word character (letters, digits, and underscore). \W is the opposite:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Meta Characters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp \w (word) Metacharacter</h4>
<p id="demo"></p>

<script>
const text = "Hello_world 123!";
const words = text.match(/\w+/g);
const nonWords = text.match(/\W/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\w+/g): " + words + "<br>" +
  "match(/\W/g): " + nonWords;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The \s (space) Metacharacter

\s matches any whitespace character (spaces, tabs, newlines):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Meta Characters</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The \s (space) Metacharacter</h4>
<p id="demo"></p>

<script>
```

```
const text = "Hello World!";
const spaces = text.match(/\s/g);
const noSpaces = text.match(/\S/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\s/g): " + spaces + " (" + spaces.length + " spaces)<br>" +
  "match(/\S/g): " + noSpaces.join("");
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Meta Characters](#)

JavaScript RegExp Assertions

Table of Contents

- [JavaScript RegExp Assertions](#)
- [RegExp ^ Metacharacter](#)
- [Example 1](#)
- [RegExp \\$ Metacharacter](#)
- [Example 2](#)
- [The \b Metacharacter](#)
- [Example 3](#)
- [RegExp Lookahead x\(?:y\)](#)
- [Example 4](#)
- [Negative Lookahead x\(?:!y\)](#)
- [Example 5](#)
- [Reference](#)

RegExp Assertions

Assertions are conditions that must be true at a position in the string:

Assertion	Description
<code>^</code>	Matches the beginning of the string
<code>\$</code>	Matches the end of the string
<code>\b</code>	Matches at a word boundary
<code>x(?:y)</code>	Lookahead: x only if followed by y
<code>x(?:!y)</code>	Negative lookahead: x only if NOT followed by y
<code>(?<=y)x</code>	Lookbehind: x only if preceded by y
<code>(?<!y)x</code>	Negative lookbehind: x only if NOT preceded by y

RegExp ^ Metacharacter

`^` asserts the position at the start of the string:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript RegExp Assertions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp ^ Metacharacter</h4>
<p id="demo"></p>

<script>
const text = "Hello World, Hello Again";
const matches = text.match(/^Hello/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/^Hello/g): " + matches + "<br>" +
  "^ matches 'Hello' only at the start";
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

RegExp \$ Metacharacter

\$ asserts the position at the end of the string:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Assertions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp $ Metacharacter</h4>
<p id="demo"></p>

<script>
const text = "Hello World, Hello";
const matches = text.match(/Hello$/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/Hello$/g): " + matches + "<br>" +
  "$ matches 'Hello' only at the end";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

The \b Metacharacter

\b asserts a word boundary - where a word starts or ends:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Assertions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The \b Metacharacter</h4>
<p id="demo"></p>

<script>
const text = "car cart scar";
const matches = text.match(/\bcar\b/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\bcar\b/g): " + matches + "<br>" +
  "Finds 'car' as a whole word only";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

RegExp Lookahead x(?=y)

Lookahead matches **x** only if it is followed by **y**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Assertions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp Lookahead x(?=y)</h4>
<p id="demo"></p>

<script>
const text = "apple banana applepie";
```



```
const matches = text.match(/apple(=pie)/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/apple(=pie)/g): " + matches + "<br>" +
  "Matches 'apple' only when followed by 'pie'";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Negative Lookahead x(?!y)

Negative lookahead matches **x** only if it is NOT followed by **y**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Assertions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Negative Lookahead x(?!y)</h4>
<p id="demo"></p>

<script>
const text = "apple banana applepie";
const matches = text.match(/apple(?!pie)/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/apple(?!pie)/g): " + matches + "<br>" +
  "Matches 'apple' only when NOT followed by 'pie'";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Assertions](#)

JavaScript RegExp Groups

Table of Contents

- [JavaScript RegExp Groups](#)
- [RegExp Capturing Groups \(x\)](#)
- [Example 1](#)
- [The Result Array](#)
- [Example 2](#)
- [Named Capturing Groups \(?<n>\)](#)
- [Example 3](#)
- [Non-Capturing Groups \(?:...\)](#)
- [Example 4](#)
- [Reference](#)

RegExp Groups

Groups allow you to combine multiple patterns as a single unit:

Syntax	Description
(x)	Capturing group - remembers the match
(?<name>x)	Named capturing group
(?:x)	Non-capturing group
x(?=y)	Lookahead
x(?!y)	Negative lookahead
(?<=y)x	Lookbehind
(?<!=y)x	Negative lookbehind

RegExp Capturing Groups (x)

Capturing groups (**. . .**) group patterns and remember the match:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Groups</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>RegExp Capturing Groups (x)</h4>
<p id="demo"></p>

<script>
const text = "John Smith, Jane Doe";
const matches = text.match(/(\w+) (\w+)/);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/(\\w+) (\\w+)/):<br>" +
  "Full match: " + matches[0] + "<br>" +
  "Group 1: " + matches[1] + "<br>" +
  "Group 2: " + matches[2];
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

The Result Array

When using `match()`, capturing groups appear in the result array:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Groups</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Result Array</h4>
<p id="demo"></p>

<script>
const text = "Date: 2024-05-15";
const matches = text.match(/(\d{4})-(\d{2})-(\d{2})/);

let text2 = "Text: " + text + "<br><br>";
text2 += "match(/(\\d{4})-(\\d{2})-(\\d{2})/):<br><br>";
text2 += "matches[0] (full): " + matches[0] + "<br>";
text2 += "matches[1] (year): " + matches[1] + "<br>";
text2 += "matches[2] (month): " + matches[2] + "<br>";
text2 += "matches[3] (day): " + matches[3] + "<br><br>";
text2 += "index: " + matches.index + "<br>";
text2 += "input: " + matches.input;

document.getElementById("demo").innerHTML = text2;
</script>

```

```
</body>
</html>
```

Example 2

Result [View Example](#)

Named Capturing Groups (?<n>)

Named groups use (**?<name>...**) syntax. You can access matches by name:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Groups</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Named Capturing Groups (?<n>)</h4>
<p id="demo"></p>

<script>
const text = "Date: 2024-05-15";
const matches = text.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "Named groups:<br>" +
  "matches.groups.year: " + matches.groups.year + "<br>" +
  "matches.groups.month: " + matches.groups.month + "<br>" +
  "matches.groups.day: " + matches.groups.day;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Non-Capturing Groups (?:...)

Non-capturing groups (**?:...**) group patterns without remembering the match:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript RegExp Groups</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Non-Capturing Groups (?:...)</h4>
<p id="demo"></p>

<script>
const text = "apple orange apple orange";
const matches = text.match(/(?:apple|orange)/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/(?:apple|orange)/g): " + matches + "<br><br>" +
  "Non-capturing groups don't save to result array:<br>" +
  "matches.length: " + matches.length;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Groups](#)

JavaScript RegExp Quantifiers

Table of Contents

- [JavaScript RegExp Quantifiers](#)
- [RegExp + Quantifier](#)
- [Example 1](#)
- [RegExp * Quantifier](#)
- [Example 2](#)
- [RegExp ? Quantifier](#)
- [Example 3](#)
- [RegExp {n} Quantifier](#)
- [Example 4](#)
- [Reference](#)

RegExp Quantifiers

Quantifiers specify how many times a pattern should match:

Quantifier	Description
+	Match 1 or more times
*	Match 0 or more times
?	Match 0 or 1 times
{n}	Match exactly n times
{n,m}	Match between n and m times
{n,}	Match n or more times

RegExp + Quantifier

+ matches 1 or more occurrences of the preceding character/pattern:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Quantifiers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp + Quantifier</h4>
```

```

<p id="demo"></p>

<script>
const text = "hello world hello!!";
const matches = text.match(/o+/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/o+/g): " + matches + "<br>" +
  "'+' matches one or more 'o' characters";
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

RegExp * Quantifier

* matches 0 or more occurrences of the preceding pattern:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Quantifiers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp * Quantifier</h4>
<p id="demo"></p>

<script>
const text = "Hello World";
const matches = text.match(/lo*/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/lo*/g): " + matches + "<br>" +
  "'*' matches zero or more 'o' characters after 'l'";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

RegExp ? Quantifier

? matches 0 or 1 occurrence of the preceding character:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Quantifiers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp ? Quantifier</h4>
<p id="demo"></p>

<script>
const text = "color colour";
const matches = text.match(/colou?r/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/colou?r/g): " + matches + "<br>" +
  "'?' makes 'u' optional (0 or 1)";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

RegExp {n} Quantifier

{n} matches exactly n occurrences:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Quantifiers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp {n} Quantifier</h4>
<p id="demo"></p>

<script>
const phone = "555-123-4567";
const matches = phone.match(/\d{3}/g);
const match4 = phone.match(/\d{4}/g);

document.getElementById("demo").innerHTML =
```



```
"Text: " + phone + "<br><br>" +  
"match(/\\d{3}/g): " + matches + " (exactly 3 digits)<br>" +  
"match(/\\d{4}/g): " + match4 + " (exactly 4 digits);  
</script>  
  
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Quantifiers](#)

JavaScript RegExp Patterns

Table of Contents

- [JavaScript RegExp Patterns](#)
- [Full RegExp Flag Reference](#)
- [Example 1](#)
- [Full Character Classes Reference](#)
- [Example 2](#)
- [Full Metacharacter Reference](#)
- [Example 3](#)
- [Full RegExp Assertions Reference](#)
- [Example 4](#)
- [RegExp Quantifiers](#)
- [Example 5](#)
- [Reference](#)

Full RegExp Flag Reference

All RegExp flags:

Flag	Description
<code>/g</code>	Global - matches all occurrences
<code>/i</code>	Insensitive - case insensitive
<code>/m</code>	Multiline - <code>^</code> and <code>\$</code> match line breaks
<code>/s</code>	DotAll - <code>.</code> matches newlines
<code>/y</code>	Sticky - matches from <code>lastIndex</code>
<code>/u</code>	Unicode - proper Unicode
<code>/v</code>	Unicode sets

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Patterns</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Full RegExp Flag Reference</h4>
```

```

<p id="demo"></p>

<script>
const text = "JavaScript JAVASCRIPT javascript";

let output = "Text: " + text + "<br><br>";
output += "flags: " + /javascript/gi.flags + "<br>";
output += "match(/javascript/gi): " + text.match(/javascript/gi) + "<br><br>";

output += "<b>All flags:</b><br>";
output += "/g global: " + /a/g.flags + "<br>";
output += "/i insensitive<br>/m multiline<br>";
output += "/s dotAll<br>/y sticky<br>/u unicode<br>/v unicodeSets";

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Full Character Classes Reference

Character classes match characters in brackets:

Class	Description
[abc]	Any character in the set
[^abc]	Any character NOT in the set
[0-9]	Any digit from 0 to 9
[A-Z]	Any uppercase letter
[a-z]	Any lowercase letter
[A-Za-z]	Any letter

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Patterns</h2>

```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Full Character Classes Reference</h4>
<p id="demo"></p>

<script>
const text = "Hello World 123!";

let output = "Text: " + text + "<br><br>";
output += "[A-Z] uppercase: " + text.match(/[A-Z]/g) + "<br>";
output += "[a-z] lowercase: " + text.match(/[a-z]/g) + "<br>";
output += "[0-9] digits: " + text.match(/[0-9]/g) + "<br>";
output += "[^a-z] not lowercase: " + text.match(/^[^a-z]/g);

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Full Metacharacter Reference

Metacharacters with special meanings:

Meta	Description
<code>\d</code>	Digit (0-9)
<code>\D</code>	Non-digit
<code>\w</code>	Word character
<code>\W</code>	Non-word character
<code>\s</code>	Whitespace
<code>\S</code>	Non-whitespace
<code>.</code>	Any character (except newline)
<code>\b</code>	Word boundary

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Patterns</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Full Metacharacter Reference</h4>
<p id="demo"></p>

<script>
const text = "Test 123";

let output = "Text: " + text + "<br><br>";
output += "\\d digits: " + text.match(/\d/g) + "<br>";
output += "\\D non-digits: " + text.match(/\D/g) + "<br>";
output += "\\w+ words: " + text.match(/\w+/g) + "<br>";
output += "\\s spaces: " + text.match(/\s/g).length + "<br>";
output += ". any chars: " + text.match(/./g).length;

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Full RegExp Assertions Reference

Assertions check positions:

Assertion	Description
[^]	Start of string
^{\$}	End of string
^{\b}	Word boundary
^{x(?=y)}	Positive lookahead
^{x(?:!y)}	Negative lookahead
^{(?<=y)x}	Positive lookbehind

(?<!y)x

Negative lookbehind

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Patterns</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Full RegExp Assertions Reference</h4>
<p id="demo"></p>

<script>
const text = "Hello Hello";

let output = "Text: " + text + "<br><br>";
output += "^Hello start: " + text.match(/^Hello/) + "<br>";
output += "Hello$ end: " + text.match(/Hello$/ ) + "<br>";
output += "Hello(?= Hello) lookahead: " + text.match(/Hello(?= Hello)/) + "<br>";
output += "(?<=Hello )Hello lookbehind: " + text.match(/(?<=Hello )Hello/);

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

RexExp Quantifiers

Quantifiers specify repetition:

Quantifier	Description
+	1 or more
*	0 or more
?	0 or 1
{n}	Exactly n
{n,m}	Between n and m

{n,}

n or more

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Patterns</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp Quantifiers</h4>
<p id="demo"></p>

<script>
const text = "100 1000 10000";

let output = "Text: " + text + "<br><br>";
output += "\\d+ 1+ digits: " + text.match(/\d+/g) + "<br>";
output += "1\\d? optional: " + text.match(/1\d?/g) + "<br>";
output += "\\d{4} exactly 4: " + text.match(/\d{4}/g) + "<br>";
output += "\\d{3,} 3+ digits: " + text.match(/\d{3,}/g);

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Patterns](#)

JavaScript RegExp Objects

Table of Contents

- [JavaScript RegExp Objects](#)
- [The RegExp Object](#)
- [Example 1](#)
- [Using test\(\)](#)
- [Example 2](#)
- [Using exec\(\)](#)
- [Example 3](#)
- [The RegExp.escape\(\) Method](#)
- [Example 4](#)
- [Reference](#)

The RegExp Object

The **RegExp** object is used to search for patterns in strings. It can be created with literal syntax `/pattern/` or `new RegExp()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The RegExp Object</h4>
<p id="demo"></p>

<script>
// Literal syntax
const pattern1 = /hello/;

// Constructor syntax
const pattern2 = new RegExp("hello");

let text = "Literal pattern: " + pattern1 + "<br>";
text += "Constructor pattern: " + pattern2 + "<br><br>";
text += "Both are RegExp objects: " + (pattern1 instanceof RegExp) + "<br>";
text += "with global flag: " + /hello/g.global;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using test()

The `test()` method searches a string for a pattern match. It returns `true` or `false`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using test()</h4>
<p id="demo"></p>

<script>
const text = "The rain in Spain";

// test() returns boolean
const result1 = /ain/.test(text);
const result2 = /xyz/.test(text);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "test(/ain/): " + result1 + " (found)<br>" +
  "test(/xyz/): " + result2 + " (not found)<br><br>" +
  "test() returns true or false";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using exec()

The `exec()` method searches for a match and returns the match details (or `null`):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using exec()</h4>
<p id="demo"></p>
```

```

<script>
const text = "The rain in Spain";

// exec() returns match details
const result = /ain/.exec(text);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "exec(/ain/):<br>" +
  "match[0]: " + result[0] + "<br>" +
  "index: " + result.index + "<br>" +
  "input: " + result.input;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The RegExp.escape() Method

RegExp.escape() escapes special characters in a string so it can be used as a literal pattern:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The RegExp.escape() Method</h4>
<p id="demo"></p>

<script>
let text = "";

if (RegExp.escape) {
  // Escape special characters
  const escaped = RegExp.escape("a.b*c");
  text += "RegExp.escape('a.b*c'): " + escaped + "<br><br>";

  // Use escaped pattern to match literal dots and stars
  const result = "a.b*c".match(new RegExp(escaped));
  text += "Matching literal 'a.b*c': " + result[0];
} else {
  text += "RegExp.escape() is not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

```

```
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Objects](#)

JavaScript RegExp Methods

Table of Contents

- [JavaScript RegExp Methods](#)
- [RegExp Object Methods](#)
- [Example 1](#)
- [String Methods with RegExp](#)
- [Example 2](#)
- [String.replace\(\) and split\(\)](#)
- [Example 3](#)
- [Pattern-Based Validation](#)
- [Example 4](#)
- [Reference](#)

Complete RegExp Reference

The complete RegExp reference includes:

RegExp Methods:

Method	Description
<code>test()</code>	Tests for a match - returns true/false
<code>exec()</code>	Returns match details or null

String Methods using RegExp:

Method	Description
<code>match(pattern)</code>	Returns array of matches
<code>matchAll(pattern)</code>	Returns iterator of matches
<code>replace(pattern, repl)</code>	Replaces matches
<code>search(pattern)</code>	Returns position of match
<code>split(pattern)</code>	Splits by pattern

RegExp Object Methods

`test()` and `exec()` are the two RegExp object methods:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp Object Methods</h4>
<p id="demo"></p>

<script>
const text = "Hello World";

// test() - returns boolean
const testResult = /World/.test(text);

// exec() - returns match object
const execResult = /World/.exec(text);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "test(/World/): " + testResult + "<br>" +
  "exec(/World/)[0]: " + execResult[0] + "<br>" +
  "exec() index: " + execResult.index;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

String Methods with RegExp

String methods that work with RegExp patterns:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Methods with RegExp</h4>
<p id="demo"></p>

<script>
const text = "JavaScript is awesome, JavaScript!";

let output = "Text: " + text + "<br><br>";

```

```

output += "match(/JavaScript/g): " + text.match(/JavaScript/g) + "<br>";
output += "search(/awesome/): " + text.search(/awesome/) + "<br>";

// matchAll returns iterator
const iterator = text.matchAll(/JavaScript/g);
output += "matchAll: ";
for (let m of iterator) output += m[0] + " ";

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

String.replace() and split()

replace() replaces matches. **split()** splits strings by a pattern:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String.replace() and split()</h4>
<p id="demo"></p>

<script>
const text = "Hello 123 World";

// replace() with RegExp
const replaced = text.replace(/\d+/, "NUMBERS");

// split() with RegExp
const parts = text.split(/\s+/);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "replace(/\\d+/, 'NUMBERS'): " + replaced + "<br>" +
  "split(/\\s+/): " + parts;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Pattern-Based Validation

Combine RegExp methods for practical form validation:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Pattern-Based Validation</h4>
<p id="demo"></p>

<script>
// Email validation pattern
const emailPattern = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/;

// Phone validation pattern
const phonePattern = /^\\d{3}-\\d{3}-\\d{4}$/;

let output = "Email: john@example.com → ";
output += emailPattern.test("john@example.com") ? "✓ Valid" : "✗ Invalid";
output += "<br>";

output += "Email: invalid → ";
output += emailPattern.test("invalid") ? "✓ Valid" : "✗ Invalid";
output += "<br><br>";

output += "Phone: 555-123-4567 → ";
output += phonePattern.test("555-123-4567") ? "✓ Valid" : "✗ Invalid";
output += "<br>";

output += "Phone: 5551234567 → ";
output += phonePattern.test("5551234567") ? "✓ Valid" : "✗ Invalid";

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript RegExp Methods](#)

JavaScript Data Types

Table of Contents

- [JavaScript Data Types](#)
- [The Concept of Data Types](#)
- [Example 1](#)
- [JavaScript Types are Dynamic](#)
- [Example 2](#)
- [Built-In Object Types](#)
- [Example 3](#)
- [The typeof Operator](#)
- [Example 4](#)
- [Reference](#)

The Concept of Data Types

JavaScript has dynamic types. Data types are the classification of data which tells the compiler how to use the data:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Concept of Data Types</h4>
<p id="demo"></p>

<script>
// Different data types
let name = "John";           // String
let age = 30;                // Number
let isStudent = true;       // Boolean
let score = 99.5;           // Number (float)
let colors = ["red", "blue"]; // Array (object)
let person = {name: "John"}; // Object

let text = "name: " + name + " (" + typeof name + "<br>";
text += "age: " + age + " (" + typeof age + "<br>";
text += "isStudent: " + isStudent + " (" + typeof isStudent + "<br>";
text += "score: " + score + " (" + typeof score + "<br>";
text += "colors: array (" + typeof colors + "<br>";
text += "person: object (" + typeof person + ")";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Types are Dynamic

A variable can change its type when assigned a new value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Types are Dynamic</h4>
<p id="demo"></p>

<script>
let x = 5;           // x is a Number
let type1 = typeof x;

x = "Hello";         // x is now a String
let type2 = typeof x;

x = true;            // x is now a Boolean
let type3 = typeof x;

x = {name: "John"};  // x is now an Object
let type4 = typeof x;

document.getElementById("demo").innerHTML =
  "let x = 5 → " + type1 + "<br>" +
  "x = 'Hello' → " + type2 + "<br>" +
  "x = true → " + type3 + "<br>" +
  "x = {name: 'John'} → " + type4 + "<br><br>" +
  "JavaScript types are dynamic - variables can change type";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Built-In Object Types

JavaScript has several built-in object types:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Built-In Object Types</h4>
<p id="demo"></p>

<script>
let text = "Built-in object types:<br><br>";

const obj = new Object();
text += "new Object() → " + obj.constructor.name + "<br>";

const arr = new Array(1, 2);
text += "new Array(1, 2) → " + arr.constructor.name + "<br>";

const date = new Date();
text += "new Date() → " + date.constructor.name + "<br>";

const str = new String("hi");
text += "new String('hi') → " + str.constructor.name + "<br>";

const num = new Number(42);
text += "new Number(42) → " + num.constructor.name + "<br>";

const bool = new Boolean(true);
text += "new Boolean(true) → " + bool.constructor.name;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The typeof Operator

typeof returns the type of a variable:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

```

```
<h4>The typeof Operator</h4>
<p id="demo"></p>

<script>
let text = "";

text += "typeof 'Hello': " + typeof "Hello" + "<br>";
text += "typeof 42: " + typeof 42 + "<br>";
text += "typeof 3.14: " + typeof 3.14 + "<br>";
text += "typeof true: " + typeof true + "<br>";
text += "typeof [1,2]: " + typeof [1,2] + " (arrays are objects)<br>";
text += "typeof {}: " + typeof {} + "<br>";
text += "typeof null: " + typeof null + " (null is object)<br>";
text += "typeof undefined: " + typeof undefined + "<br>";
text += "typeof function(): " + typeof function(){} + "<br>";
text += "typeof NaN: " + typeof NaN + " (NaN is number)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Data Types](#)

JavaScript Primitive Types

Table of Contents

- [JavaScript Primitive Types](#)
- [JavaScript Strings](#)
- [Example 1](#)
- [JavaScript Numbers](#)
- [Example 2](#)
- [JavaScript BigInt](#)
- [Example 3](#)
- [JavaScript Booleans](#)
- [Example 4](#)
- [Undefined, Empty Values, and null](#)
- [Example 5](#)
- [Reference](#)

JavaScript Strings

A string is a sequence of characters. Strings are written inside quotes. You can use single or double quotes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Primitive Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Strings</h4>
<p id="demo"></p>

<script>
// Strings can be written in single or double quotes
const name1 = "John Doe";
const name2 = 'John Doe';

// Numbers in quotes are still strings
const num1 = "25";
const num2 = "25.5";

let text = "name1: " + name1 + "<br>";
text += "name2: " + name2 + "<br>";
text += "typeof '25': " + typeof num1 + "<br>";
text += "typeof '25.5': " + typeof num2 + "<br><br>";

text += "Strings can be concatenated:<br>";
text += name1 + " is " + 25 + " years old";

document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Numbers

Numbers can be written with or without decimals. They can also use exponential notation:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Primitive Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Numbers</h4>
<p id="demo"></p>

<script>
// Numbers can be integers or floats
const x1 = 34.00;
const x2 = 34;
const x3 = 3.14;

// Exponential notation
const y1 = 123e5;      // 12300000
const y2 = 123e-5;     // 0.00123

let text = "x1 = 34.00: " + x1 + " (" + typeof x1 + ")<br>";
text += "x2 = 34: " + x2 + " (" + typeof x2 + ")<br>";
text += "x3 = 3.14: " + x3 + "<br><br>";
text += "123e5 = " + y1 + "<br>";
text += "123e-5 = " + y2 + "<br><br>";

text += "Integer (32-bit): " + (x2 | 0) + "<br>";
text += "Float (64-bit): " + x3;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript BigInt

BigInt is for numbers too large for normal Number. Created by appending **n** to the end:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Primitive Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript BigInt</h4>
<p id="demo"></p>

<script>
// BigInt is for very large integers
const big1 = 123456789012345678901234567890n;
const big2 = BigInt("123456789012345678901234567890");

// Normal number (limited precision)
const normal = 123456789012345678901234567890;

let text = "Normal number: " + normal + "<br>";
text += "(loses precision for large values)<br><br>";
text += "BigInt: " + big1 + "<br>";
text += "big2: " + big2 + "<br><br>";
text += "typeof BigInt: " + typeof big1 + "<br><br>";

text += "BigInt supports arithmetic:<br>";
text += "big1 + 1n: " + (big1 + 1n);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

JavaScript Booleans

Booleans can only have two values: **true** or **false**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Primitive Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>JavaScript Booleans</h4>
<p id="demo"></p>

<script>
// Booleans are true or false
const isTrue = true;
const isFalse = false;

// Booleans from comparisons
const isGreater = 5 > 2;
const isEqual = 5 === 2;

let text = "true: " + isTrue + " (" + typeof isTrue + ")<br>";
text += "false: " + isFalse + " (" + typeof isFalse + ")<br><br>";
text += "5 > 2: " + isGreater + "<br>";
text += "5 === 2: " + isEqual;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Undefined, Empty Values, and null

undefined is a variable without a value. **null** is an empty value. **""** is an empty string:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Primitive Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Undefined, Empty Values, and null</h4>
<p id="demo"></p>

<script>
// Undefined - variable without a value
let car;    // undefined

// null - explicitly empty value
let person = null;

// Empty string
let text = "";

```



```
let output = "let car; → " + car + " (" + typeof car + ")<br>";
output += "let person = null; → " + person + " (" + typeof person + ")<br>";
output += "let text = ''; → '" + text + "' (" + typeof text + ")<br><br>";

// Setting to undefined
let x = "Hello";
x = undefined;
output += "x = 'Hello'; x = undefined → " + x + " (" + typeof x + ")";

document.getElementById("demo").innerHTML = output;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Primitive Types](#)

JavaScript Datatypes Objects

Table of Contents

- [JavaScript Datatypes Objects](#)
- [Primitive vs Object Types](#)
- [Example 1](#)
- [Object Methods](#)
- [Example 2](#)
- [Reference](#)

JavaScript Datatypes Objects

In JavaScript, objects are the most important data types. Everything that is not a primitive (string, number, boolean, bigint, undefined, symbol) is an object.

Primitive vs Object Types

Primitive values vs object types - sometimes primitives behave like objects when using methods:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Datatypes Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Primitive vs Object Types</h4>
<p id="demo"></p>

<script>
// Primitive types
const str = "Hello";
const num = 42;
const bool = true;

// Object types
const arr = [1, 2, 3];
const obj = {name: "John"};
const date = new Date();

let text = "<b>Primitive types:</b><br>";
text += "typeof 'Hello': " + typeof str + "<br>";
text += "typeof 42: " + typeof num + "<br>";
text += "typeof true: " + typeof bool + "<br><br>";

text += "<b>Object types:</b><br>";
text += "typeof [1,2,3]: " + typeof arr + "<br>";
text += "typeof {name}: " + typeof obj + "<br>";
text += "typeof Date: " + typeof date + "<br><br>";
```

```

text += "Primitives can temporarily use methods:<br>";
text += "'Hello'.toUpperCase(): " + str.toUpperCase() + "<br>";
text += "(42).toString(): " + num.toString();

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Object Methods

Objects and other complex data types can contain methods. Common built-in objects include:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Datatypes Objects</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Methods</h4>
<p id="demo"></p>

<script>
// Complex data types
const car = {
  brand: "Toyota",
  model: "Corolla",
  start: function() {
    return this.brand + " " + this.model + " started!";
  }
};

let text = "Array methods:<br>";
const fruits = ["Apple", "Banana"];
fruits.push("Orange");
text += "fruits.push('Orange'): " + fruits + "<br>";
text += "fruits.length: " + fruits.length + "<br><br>";

text += "Date methods:<br>";
text += "new Date().getFullYear(): " + new Date().getFullYear() + "<br><br>";

text += "Custom object method:<br>";
text += "car.start(): " + car.start();

document.getElementById("demo").innerHTML = text;
</script>

```

```
</body>  
</html>
```

Example 2

Result [View Example](#)

Reference

- [W3Schools JavaScript Datatypes Objects](#)

JavaScript Symbol Datatype

Table of Contents

- [JavaScript Symbol Datatype](#)
- [Creating Symbols](#)
- [Example 1](#)
- [Using Symbols as Object Keys](#)
- [Example 2](#)
- [Symbols are Not Included in for...in](#)
- [Example 3](#)
- [Global Symbols](#)
- [Example 4](#)
- [Reference](#)

Creating Symbols

A Symbol is a unique and immutable primitive value. Two symbols with the same description are still different:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Symbol Datatype</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating Symbols</h4>
<p id="demo"></p>

<script>
// Create symbols
const sym1 = Symbol();
const sym2 = Symbol();
const sym3 = Symbol("id");

// Symbols with same description are still different
const sym4 = Symbol("id");

let text = "sym1: " + sym1.toString() + "<br>";
text += "sym2: " + sym2.toString() + "<br>";
text += "sym3: " + sym3.toString() + "<br><br>";

text += "sym1 === sym2: " + (sym1 === sym2) + "<br>";
text += "sym3 === sym4: " + (sym3 === sym4) + " (same description, different symbols)<br><br>";
text += "typeof Symbol: " + typeof sym1;

document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Using Symbols as Object Keys

Symbols can be used as unique object keys - they are hidden identifiers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Symbol Datatype</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Symbols as Object Keys</h4>
<p id="demo"></p>

<script>
// Create symbol for secret key
const secretKey = Symbol("secret");

// Use symbol as object key
const person = {
  name: "John",
  age: 30,
  [secretKey]: "Hidden value"
};

let text = "person.name: " + person.name + "<br>";
text += "person.age: " + person.age + "<br><br>";

// Access using the symbol
text += "person[secretKey]: " + person[secretKey] + "<br><br>";

// Object.keys() does NOT show symbols
text += "Object.keys(person): " + Object.keys(person) + "<br>";
text += "Symbol is hidden from normal enumeration";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Symbols are Not Included in for...in

Symbols are not included in `for...in` loops or `JSON.stringify()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Symbol Datatype</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Symbols are Not Included in for...in</h4>
<p id="demo"></p>

<script>
const secretKey = Symbol("secret");

const person = {
  name: "John",
  age: 30,
  [secretKey]: "Hidden"
};

// for...in does NOT show symbol keys
let text = "for...in loop:<br>";
for (let key in person) {
  text += key + ": " + person[key] + "<br>";
}

text += "<br>";

// JSON.stringify does NOT show symbols
text += "JSON.stringify(person):<br>";
text += JSON.stringify(person);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Global Symbols

Global symbols are shared across the entire program using `Symbol.for()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Symbol Datatype</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Global Symbols</h4>
<p id="demo"></p>

<script>
// Regular symbols - always different
const s1 = Symbol("id");
const s2 = Symbol("id");

// Global symbols - same key returns same symbol
const g1 = Symbol.for("id");
const g2 = Symbol.for("id");

let text = "Regular symbols:<br>";
text += "s1 === s2: " + (s1 === s2) + "<br><br>";

text += "Global symbols (Symbol.for):<br>";
text += "g1 === g2: " + (g1 === g2) + "<br><br>";

text += "Symbol.keyFor(g1): " + Symbol.keyFor(g1) + "<br>";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Symbol Datatype](#)

JavaScript typeof

Table of Contents

- [JavaScript typeof](#)
- [The typeof Operator](#)
- [Example 1](#)
- [How to Recognize an Array](#)
- [Example 2](#)
- [The instanceof Operator](#)
- [Example 3](#)
- [Difference Between Undefined and Null](#)
- [Example 4](#)
- [Reference](#)

The typeof Operator

The **typeof** operator returns the type of a variable or value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript typeof</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The typeof Operator</h4>
<p id="demo"></p>

<script>
// Primitive data types
let text = "";

text += "typeof 'John': " + typeof "John" + "<br>";
text += "typeof 3.14: " + typeof 3.14 + "<br>";
text += "typeof true: " + typeof true + "<br>";
text += "typeof 123n: " + typeof 123n + "<br>";
text += "typeof undefined: " + typeof undefined + "<br>";
text += "typeof Symbol(): " + typeof Symbol() + "<br><br>";

// Complex data types
text += "typeof [1,2,3]: " + typeof [1,2,3] + "<br>";
text += "typeof {name:'x'}: " + typeof {name:"x"} + "<br>";
text += "typeof new Date(): " + typeof new Date() + "<br>";
text += "typeof function(){}: " + typeof function(){} + "<br>";
text += "typeof null: " + typeof null;

document.getElementById("demo").innerHTML = text;
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

How to Recognize an Array

`typeof` returns "object" for arrays. Use `Array.isArray()` to identify arrays:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript typeof</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How to Recognize an Array</h4>
<p id="demo"></p>

<script>
const fruits = ["Apple", "Banana", "Orange"];

let text = "typeof fruits: " + typeof fruits + "<br>";
text += "Array.isArray(fruits): " + Array.isArray(fruits) + "<br><br>";

// Check with constructor
text += "fruits.constructor === Array: " + (fruits.constructor === Array);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The instanceof Operator

`instanceof` checks if an object is an instance of a specific constructor:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript typeof</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>The instanceof Operator</h4>
<p id="demo"></p>

<script>
const arr = [1, 2, 3];
const date = new Date();
const person = {name: "John"};

let text = "arr instanceof Array: " + (arr instanceof Array) + "<br>";
text += "date instanceof Date: " + (date instanceof Date) + "<br>";
text += "person instanceof Object: " + (person instanceof Object) + "<br><br>";

text += "typeof arr: " + typeof arr + " (object)<br>";
text += "instanceof is more specific: " + (arr instanceof Array);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Difference Between Undefined and Null

undefined and **null** are different: **undefined** is a type, **null** is an object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript typeof</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Difference Between Undefined and Null</h4>
<p id="demo"></p>

<script>
// undefined - variable not assigned
let car;

// null - explicitly declared empty
let person = null;

// empty - empty string
let fruit = "";

let text = "let car; → typeof: " + typeof car + "<br>";
text += "let person = null; → typeof: " + typeof person + "<br>";

```

```
text += "let fruit = ''; → typeof: " + typeof fruit + "<br><br>";

text += "car === undefined: " + (car === undefined) + "<br>";
text += "person === null: " + (person === null) + "<br>";
text += "undefined == null: " + (undefined == null) + " (loose equal)<br>";
text += "undefined === null: " + (undefined === null) + " (strict equal)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript typeof](#)

JavaScript undefined

Table of Contents

- [JavaScript undefined](#)
- [Undefined Variables](#)
- [Example 1](#)
- [The Type of undefined](#)
- [Example 2](#)
- [Empty Values](#)
- [Example 3](#)
- [Objects and undefined](#)
- [Example 4](#)
- [Functions and undefined](#)
- [Example 5](#)
- [You Can Empty an Object](#)
- [Example 6](#)
- [Reference](#)

Undefined Variables

In JavaScript, a variable without a value has the value **undefined**. The type is also **undefined**.

A variable declared without a value will automatically have the value **undefined**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript undefined</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Undefined Variables</h4>
<p id="demo"></p>

<script>
let car;

// car is declared but has no value
document.getElementById("demo").innerHTML = car;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Type of undefined

You can use `typeof` to check if a variable is `undefined`.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript undefined</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Type of undefined</h4>
<p id="demo"></p>

<script>
let car;

// typeof returns "undefined"
document.getElementById("demo").innerHTML = typeof car;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Empty Values

An empty string has a value and a type. It is **not** the same as `undefined`.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript undefined</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Empty Values</h4>
<p id="demo"></p>

<script>
let text = "";

// Empty string has a value and a type
document.getElementById("demo").innerHTML =
  text + "<br>" + typeof text;
</script>

</body>
```

```
</html>
```

Example 3

Result [View Example](#)

Objects and undefined

Accessing a non-existing object property returns **undefined**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript undefined</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Objects and undefined</h4>
<p id="demo"></p>

<script>
const person = {firstName: "John", lastName: "Doe"};

// person.age does not exist → undefined
document.getElementById("demo").innerHTML = person.age;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Functions and undefined

A function without a return value returns **undefined**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript undefined</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions and undefined</h4>
<p id="demo"></p>

<script>
```

```
function myFunction() {
  let x = 5;
}

// No return statement → undefined
document.getElementById("demo").innerHTML = myFunction();
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

You Can Empty an Object

Objects can be emptied by setting the value to **undefined**. Objects can also be emptied by setting the value to **null**.

Note: **undefined** is a JavaScript value. It means that a variable has been declared, but no value has been assigned.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript undefined</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>You Can Empty an Object</h4>
<p id="demo"></p>

<script>
let person = {firstName: "John", lastName: "Doe"};

let text = "Before: " + person.firstName + " " + person.lastName + "<br>";

// Empty the object by setting it to undefined
person = undefined;

text += "After (undefined): " + person;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript undefined](#)

JavaScript NaN

Table of Contents

- [JavaScript NaN](#)
- [Invalid Number Operations](#)
- [Example 1](#)
- [NaN is a Number](#)
- [Example 2](#)
- [Numeric Strings](#)
- [Example 3](#)
- [Non-Numeric Strings](#)
- [Example 4](#)
- [Using isNaN\(\)](#)
- [Example 5](#)
- [NaN is Not Equal to Itself](#)
- [Example 6](#)
- [NaN in Math](#)
- [Example 7](#)
- [Reference](#)

Invalid Number Operations

In JavaScript, **NaN** is short for **Not a Number**. **NaN** is a JavaScript number that is not a legal number.

You get **NaN** when JavaScript cannot calculate a number.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NaN</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Invalid Number Operations</h4>
<p id="demo"></p>

<script>
let x = 100 / "Apple";

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

NaN is a Number

The type of **NaN** is **number**. This may look strange, but **NaN** belongs to the JavaScript number type.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NaN</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>NaN is a Number</h4>
<p id="demo"></p>

<script>
let x = NaN;

document.getElementById("demo").innerHTML = typeof x;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Numeric Strings

JavaScript tries to convert numeric strings to numbers in arithmetic operations.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NaN</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numeric Strings</h4>
<p id="demo"></p>

<script>
let x = 100 / "10";

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

The result is **10**, because **"10"** is converted to the number **10**.

Example 3

Result [View Example](#)

Non-Numeric Strings

A non-numeric string cannot be converted to a number.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NaN</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Non-Numeric Strings</h4>
<p id="demo"></p>

<script>
let x = 100 / "Apple";

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

The result is **NaN**, because **"Apple"** cannot be converted to a number.

Example 4

Result [View Example](#)

Using isNaN()

You can use the JavaScript function **isNaN()** to find out if a value is not a number.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NaN</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using isNaN()</h4>
<p id="demo"></p>
```

```
<script>
let x = 100 / "Apple";

document.getElementById("demo").innerHTML = isNaN(x);
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

NaN is Not Equal to Itself

NaN is the only JavaScript value that is not equal to itself.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NaN</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>NaN is Not Equal to Itself</h4>
<p id="demo"></p>

<script>
let x = NaN;

document.getElementById("demo").innerHTML = x == x;
</script>

</body>
</html>
```

To test for **NaN**, use **isNaN()**.

Example 6

Result [View Example](#)

NaN in Math

If you use **NaN** in a mathematical operation, the result will also be **NaN**.

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript NaN</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>NaN in Math</h4>
<p id="demo"></p>

<script>
let x = NaN;
let y = 5;

document.getElementById("demo").innerHTML = x + y;
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

Note

NaN means **Not a Number**. But the type of **NaN** is **number**. Use **isNaN()** to check if a value is **NaN**.

Reference

- [W3Schools JavaScript NaN](#)

JavaScript toString()

Table of Contents

- [JavaScript toString\(\)](#)
- [JavaScript Array toString\(\)](#)
- [Example 1](#)
- [JavaScript Date toString\(\)](#)
- [Example 2](#)
- [JavaScript Number toString\(\)](#)
- [Example 3](#)
- [Example 4](#)
- [JavaScript Function toString\(\)](#)
- [JavaScript Object toString\(\)](#)
- [Example 5](#)
- [Reference](#)

The JavaScript `toString()` method converts a variable (or a value) to a string. It is a built-in method for many data types, including numbers, arrays, dates, and objects.

The method is useful for:

- Converting data to a readable format for display
- Ensuring type compatibility when a string is required
- Customizing objects other user interfaces
- Customizing objects for debugging

JavaScript Array toString()

When used on an array, `toString()` returns the array elements as a comma separated string.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Array toString()</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];

let myList = fruits.toString();

document.getElementById("demo").innerHTML = myList;
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

JavaScript Date toString()

When used on a date, `toString()` returns a human-readable date and time string.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toString()/h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date toString()/h4>
<p id="demo"></p>

<script>
const d = new Date();

let text = d.toString();

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Number toString()

When used on a number, `toString()` returns the number as a string.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toString()/h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Number toString()/h4>
<p id="demo"></p>
```



```
<script>
let x = 123;

let text = x.toString();

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Convert a number to a string, using base 2 (binary):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Number toString() with Base 2</h4>
<p id="demo"></p>

<script>
let x = 123;

let text = x.toString(2);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript Function toString()

When used on a function, `toString()` returns the source code of the function as a string.

JavaScript Object toString()

When used on an object, `toString()` an object returns "[object Object]". To provide a meaningful string representation, it can be overridden in the object definition:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object toString()</h4>
<p id="demo"></p>

<script>
let person = {
  firstname: "John",
  lastname: "Doe",
}

let text = person.toString();

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript toString\(\)](#)

JavaScript toLocaleString()

Table of Contents

- [JavaScript toLocaleString\(\)](#)
- [The toLocaleString\(\) Method](#)
- [Example 1](#)
- [Using Locales](#)
- [Example 2](#)
- [Formatting Currency](#)
- [Example 3](#)
- [Formatting Percentages](#)
- [Example 4](#)
- [Controlling Decimal Digits](#)
- [Example 5](#)
- [Formatting Dates](#)
- [Example 6](#)
- [Date Formatting Options](#)
- [Example 7](#)
- [A Clever Use: Readable File Sizes](#)
- [Example 8](#)
- [Arrays and toLocaleString\(\)](#)
- [Example 9](#)
- [Reference](#)

The `toLocaleString()` method converts a value to a string, using local formatting rules. The `toLocaleString()` method is especially useful for formatting numbers, dates, arrays, currencies, and percentages for different countries.

The toLocaleString() Method

The `toLocaleString()` method is available for many JavaScript datatypes / objects:

- Numbers
- Dates
- Arrays
- BigInts

In this example it is used on a Number (`num`):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The toLocaleString() Method</h4>
```

```
<p id="demo"></p>

<script>
let num = 1234567.89;

let text = num.toLocaleString();

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using Locales

You can specify a language and country code to format a value for a specific locale.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Locales</h4>
<p id="demo"></p>

<script>
let num = 1234567.89;

let us = num.toLocaleString("en-US");
let de = num.toLocaleString("de-DE");
let no = num.toLocaleString("no-NO");

document.getElementById("demo").innerHTML =
  "en-US: " + us + "<br>" +
  "de-DE: " + de + "<br>" +
  "no-NO: " + no;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Formatting Currency

With options, `toLocaleString()` can format numbers as currency.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Formatting Currency</h4>
<p id="demo"></p>

<script>
let price = 1299.95;

let dollars = price.toLocaleString("en-US",
{style:"currency", currency:"USD"});

let euros = price.toLocaleString("de-DE",
{style:"currency", currency:"EUR"});

let kroner = price.toLocaleString("no-NO",
{style:"currency", currency:"NOK"});

document.getElementById("demo").innerHTML =
  dollars + "<br>" + euros + "<br>" + kroner;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Formatting Percentages

The `style: "percent"` option formats a number as a percentage.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Formatting Percentages</h4>
<p id="demo"></p>
```

```
<script>
let score = 0.875;

let result = score.toLocaleString("en-US", {style:"percent"});

document.getElementById("demo").innerHTML = result;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Controlling Decimal Digits

You can control the number of decimal digits with `minimumFractionDigits` and `maximumFractionDigits`.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Controlling Decimal Digits</h4>
<p id="demo"></p>

<script>
let num = 3.14159;

let text = num.toLocaleString("en-US", {
  minimumFractionDigits: 2,
  maximumFractionDigits: 2
});

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Formatting Dates

Dates can also be formatted with `toLocaleString()`.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Formatting Dates</h4>
<p id="demo"></p>

<script>
let date = new Date();

let text = date.toLocaleString("en-US");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

Date Formatting Options

Many options can be used to control how dates are displayed.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Date Formatting Options</h4>
<p id="demo"></p>

<script>
let date = new Date();

let text = date.toLocaleString("en-US", {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric"
});

document.getElementById("demo").innerHTML = text;
</script>

```

```
</body>
</html>
```

Example 7

Result [View Example](#)

A Clever Use: Readable File Sizes

A clever use of `toLocaleString()` is to make file sizes easier to read.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Clever Use: Readable File Sizes</h4>
<p id="demo"></p>

<script>
function fileSize(bytes) {
  if (bytes < 1024) return bytes + " bytes";
  if (bytes < 1024 * 1024) return (bytes / 1024).toLocaleString("en-US",
{maximumFractionDigits: 1}) + " KB";
  return (bytes / 1024 / 1024).toLocaleString("en-US", {maximumFractionDigits: 1}) + "
MB";
}

let size = 1536000;
let text = fileSize(size);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Arrays and toLocaleString()

For arrays, the `toLocaleString()` methods converts each array element.

```
<!DOCTYPE html>
```



```
<html>
<body>

<h2>JavaScript toLocaleString()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arrays and toLocaleString()</h4>
<p id="demo"></p>

<script>
const dates = [
  new Date("2026-01-01"),
  new Date("2026-12-24")
];

let text = dates.toLocaleString("en-US");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 9

Result [View Example](#)

Reference

- [W3Schools JavaScript toLocaleString\(\)](#)

JavaScript Type Coercion

Table of Contents

- [JavaScript Type Coercion](#)
- [Type Coercion](#)
- [Example 1](#)
- [String Coercion \(+\)](#)
- [Example 2](#)
- [Numeric Coercion](#)
- [Example 3](#)
- [Boolean Coercion](#)
- [Example 4](#)
- [Loose Equality \(==\)](#)
- [Example 5](#)
- [Best Practices to Avoid Bugs](#)
- [Example 6](#)
- [Reference](#)

Type Coercion

Type coercion is the **automatic conversion** of values from one data type to another.

Type coercion happens when you perform an operation on different data types, and the JavaScript engine tries to make them "fit" together.

JavaScript coerces types differently per operator, so type coercion can hide bugs. The program continues according to the coercion rules, but these may differ from what the programmer wanted to achieve.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Type Coercion</h4>
<p id="demo"></p>

<script>
let result1 = ('5' + '2'); // = 52
let result2 = ('5' - '2'); // = 3

document.getElementById("demo").innerHTML =
  "('5' + '2') = " + result1 + "<br>" +
  "('5' - '2') = " + result2;
</script>

</body>
</html>
```

Coercion is **implicit** (handled automatically by the engine), while **type conversion** is explicit (you manually use functions like `Number()` or `String()`).

Because JavaScript is **weakly typed**, it doesn't throw errors when types don't match; it just tries to coerce them.

Example 1

Result [View Example](#)

String Coercion (+)

If any part of a `+` operation is a string, JavaScript converts everything to strings and concatenates them.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Coercion (+)</h4>
<p id="demo"></p>

<script>
let x = "5" + 2 // x = "52"

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Numeric Coercion

Other arithmetic operators (`-`, `*`, `/`, `%`) and the unary plus (`+x`) force values into numbers.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numeric Coercion</h4>
```

```

<p id="demo"></p>

<script>
let x = "5" - 2 // x = 3

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Boolean Coercion

Values are coerced to booleans in logical contexts like **if** statements or using the double-NOT operator (**!!**).

- **Falsy values:** `0`, `"`, `null`, `undefined`, `NaN`, and `false`
- **Truthy values:** Everything else (including empty objects `{}` and arrays `[]`)

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Boolean Coercion</h4>
<p id="demo"></p>

<script>
let text = "";

text += "Boolean(0) = " + Boolean(0) + "<br>";
text += "Boolean(\"\") = " + Boolean("") + "<br>";
text += "Boolean(null) = " + Boolean(null) + "<br>";
text += "Boolean(undefined) = " + Boolean(undefined) + "<br>";
text += "Boolean(NaN) = " + Boolean(NaN) + "<br>";
text += "Boolean(false) = " + Boolean(false) + "<br>";
text += "Boolean({}) = " + Boolean({}) + "<br>";
text += "Boolean([]) = " + Boolean([]) + "<br>";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Loose Equality (==)

This operator performs coercion to find a "common type" before comparing.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loose Equality (==)</h4>
<p id="demo"></p>

<script>
let x = (5 == "5") // x = true

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Best Practices to Avoid Bugs

- **Use `===` (Strict Equality):** This checks both value and type without coercion, preventing weird results.
- **Be Explicit:** Instead of relying on automatic behavior, use MDN's recommended conversion methods like `Number()` or `String()` to make your intent clear.
- **Watch for NaN:** If a string cannot be converted to a valid number, like `"abc" - 1`, the result is `NaN` (Not-a-Number).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Best Practices to Avoid Bugs</h4>
<p id="demo"></p>

<script>
let text = "";
```

```
// Use === (Strict Equality)
text += "5 == \"5\" : " + (5 == "5") + "<br>";
text += "5 === \"5\" : " + (5 === "5") + "<br><br>";

// Be Explicit
text += "Number(\"5\") + 2 : " + (Number("5") + 2) + "<br><br>";

// Watch for NaN
let result = "abc" - 1;
text += "\"abc\" - 1 : " + result;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Type Coercion](#)

JavaScript Type Conversion

Table of Contents

- [JavaScript Type Conversion](#)
- [Converting Strings to Numbers](#)
- [Example 1](#)
- [The Unary + Operator](#)
- [Example 2](#)
- [Converting Numbers to Strings](#)
- [Example 3](#)
- [Converting Dates](#)
- [Example 4](#)
- [Converting Booleans](#)
- [Example 5](#)
- [Automatic Type Conversion](#)
- [Example 6](#)
- [Reference](#)

Converting Strings to Numbers

The global method `Number()` converts a variable (or a value) into a number.

- A numeric string (like `"3.14"`) converts to a number (like `3.14`).
- An empty string (like `" "`) converts to `0`.
- A non numeric string (like `"John"`) converts to `NaN` (Not a Number).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Conversion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Converting Strings to Numbers</h4>
<p id="demo"></p>

<script>
let text = "";

text += "Number(\"3.14\") = " + Number("3.14") + "<br>";
text += "Number(\" \" ) = " + Number(" ") + "<br>";
text += "Number(\" \") = " + Number("") + "<br>";
text += "Number(\"99 88\") = " + Number("99 88") + "<br>";
text += "Number(\"John\") = " + Number("John");

document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

The Unary + Operator

The unary `+` operator can be used to convert a variable to a number.

If the variable cannot be converted, it will still become a number, but with the value **NaN** (Not a Number).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Conversion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Unary + Operator</h4>
<p id="demo"></p>

<script>
let text = "";

let y = "5";      // y is a string
let x = + y;      // x is a number

text += "typeof y (\`5\`): " + typeof y + "<br>";
text += "typeof x (+ y): " + typeof x + "<br><br>";

let z = "John";
let w = + z;

text += "\`John\` to number: " + w;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Converting Numbers to Strings

The global method **String()** can convert numbers to strings.

It can be used on any type of numbers, literals, variables, or expressions.

The Number method `toString()` does the same.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Conversion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Converting Numbers to Strings</h4>
<p id="demo"></p>

<script>
let text = "";

let x = 123;

text += "String(x): " + String(x) + "<br>";
text += "String(123): " + String(123) + "<br>";
text += "String(100 + 23): " + String(100 + 23) + "<br><br>";

text += "x.toString(): " + x.toString() + "<br>";
text += "(123).toString(): " + (123).toString() + "<br>";
text += "(100 + 23).toString(): " + (100 + 23).toString();

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Converting Dates

The global method `Number()` can be used to convert dates to numbers.

The date method `getTime()` does the same.

The global method `String()` can convert dates to strings. The `Date` method `toString()` does the same.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Conversion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Converting Dates</h4>
<p id="demo"></p>

<script>
let text = "";

let d = new Date();

text += "Number(d): " + Number(d) + "<br>";
text += "d.getTime(): " + d.getTime() + "<br><br>";

text += "String(d): " + String(d) + "<br>";
text += "d.toString(): " + d.toString();

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Converting Booleans

The global method `Number()` can also convert booleans to numbers:

- `Number(false)` returns `0`
- `Number(true)` returns `1`

The global method `String()` can convert booleans to strings. The Boolean method `toString()` does the same.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Conversion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Converting Booleans</h4>
<p id="demo"></p>

<script>
let text = "";

text += "Number(false) = " + Number(false) + "<br>";
text += "Number(true) = " + Number(true) + "<br><br>";

text += "String(false) = \"" + String(false) + "\"<br>";
text += "String(true) = \"" + String(true) + "\"<br><br>";

```

```

text += "false.toString() = \"" + false.toString() + "\"<br>";
text += "true.toString() = \"" + true.toString() + "\"";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Automatic Type Conversion

When JavaScript tries to operate on a "wrong" data type, it will try to convert the value to a "right" type.

The result is not always what you expect.

JavaScript automatically calls the variable's `toString()` function when you try to "output" an object or a variable.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Conversion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Automatic Type Conversion</h4>
<p id="demo"></p>

<script>
let text = "";

text += "(5 + null) = " + (5 + null) + "<br>";
text += "(\\"5\" + null) = \"" + ("5" + null) + "\"<br>";
text += "(\\"5\" + 2) = \"" + ("5" + 2) + "\"<br>";
text += "(\\"5\" - 2) = " + ("5" - 2) + "<br>";
text += "(\\"5\" * \\"2\") = " + ("5" * "2") + "<br><br>";

// Automatic string conversion
document.getElementById("demo").innerHTML = text;

// Object to string
text += "Automatic toString() on object: " + {firstName:"John", lastName:"Doe"};

document.getElementById("demo").innerHTML = text;
</script>

</body>

```

```
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Type Conversion](#)

JavaScript Destructuring

Table of Contents

- [JavaScript Destructuring](#)
- [Destructuring Assignment Syntax](#)
- [Example 1](#)
- [Object Destructuring](#)
- [Example 2](#)
- [String Destructuring](#)
- [Example 3](#)
- [Array Destructuring](#)
- [Example 4](#)
- [Destructuring Maps](#)
- [Example 5](#)
- [Swapping JavaScript Variables](#)
- [Example 6](#)
- [Reference](#)

Destructuring Assignment Syntax

The **destructuring assignment** syntax is a JavaScript expression that makes it possible to unpack values from arrays, or properties from objects, into distinct variables.

Destructuring makes it easy to extract only what is needed from data.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Destructuring</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Destructuring Assignment Syntax</h4>
<p id="demo"></p>

<script>
// Create an Array
const vehicles = ["mustang", "f-150", "expedition"];

// Old way
const car = vehicles[0];
const truck = vehicles[1];
const suv = vehicles[2];

// New way - Destructuring
const [car2, truck2, suv2] = vehicles;

document.getElementById("demo").innerHTML =
  "Deconstructed: " + car2 + ", " + truck2 + ", " + suv2;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Object Destructuring

Object destructuring allows you to extract properties from objects and assign them to variables. You can also assign **default values** and use **property aliases**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Destructuring</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Destructuring</h4>
<p id="demo"></p>

<script>
// Create an Object
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50
};

// Destructuring
let {firstName, lastName} = person;

// Default values
const {firstName: fName, country = "US"} = person;

// Property alias
const {lastName: lName} = person;

document.getElementById("demo").innerHTML =
  firstName + " " + lastName + "<br>" +
  "Alias: " + fName + " " + lName + "<br>" +
  "Country (default): " + country;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

String Destructuring

You can use destructuring to unpack characters from a string into variables.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Destructuring</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Destructuring</h4>
<p id="demo"></p>

<script>
// Create a String
let name = "W3Schools";

// Destructuring
let [a1, a2, a3, a4, a5] = name;

document.getElementById("demo").innerHTML =
  a1 + " " + a2 + " " + a3 + " " + a4 + " " + a5;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Array Destructuring

Array destructuring allows unpacking array values into separate variables. You can **skip values** using commas, use **position values**, and capture remaining elements with the **rest property** (...).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Destructuring</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Array Destructuring</h4>
<p id="demo"></p>
```

```

<script>
const numbers = [10, 20, 30, 40, 50, 60, 70];

// Basic array destructuring
const [x, y] = numbers;

// Skipping array values
const [a, , b] = numbers;

// Array position values
const {[0]:p1, [1]:p2} = numbers;

// The rest property
const [first, second, ...rest] = numbers;

document.getElementById("demo").innerHTML =
  "x = " + x + ", y = " + y + "<br>" +
  "Skipped: a = " + a + ", b = " + b + "<br>" +
  "Position: p1 = " + p1 + ", p2 = " + p2 + "<br>" +
  "Rest: first = " + first + ", second = " + second + ", rest = [" + rest + "];
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Destructuring Maps

Destructuring can also be used with Maps to extract key-value pairs.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Destructuring</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Destructuring Maps</h4>
<p id="demo"></p>

<script>
// Create a Map
const fruits = new Map([
  ["apples", 500],
  ["bananas", 300],
  ["oranges", 200]
]);

let text = "";

```



```
// Destructuring Maps
for (const [key, value] of fruits) {
  text += key + " is " + value + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Swapping JavaScript Variables

Destructuring makes it easy to swap the values of two variables without needing a temporary third variable.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Destructuring</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Swapping JavaScript Variables</h4>
<p id="demo"></p>

<script>
let firstName = "John";
let lastName = "Doe";

document.getElementById("demo").innerHTML =
  "Before: " + firstName + " " + lastName + "<br>";

// Swapping variables with destructuring
[firstName, lastName] = [lastName, firstName];

document.getElementById("demo").innerHTML +=
  "After: " + firstName + " " + lastName;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Destructuring](#)

JavaScript Errors

Table of Contents

- [JavaScript Errors](#)
- [How to Handle JavaScript Errors](#)
- [Example 1](#)
- [Reference Errors](#)
- [Example 2](#)
- [JavaScript Type Errors](#)
- [Example 3](#)
- [JavaScript Range Errors](#)
- [Example 4](#)
- [JavaScript URI Errors](#)
- [Example 5](#)
- [JavaScript Syntax Errors](#)
- [Example 6](#)
- [Reference](#)

How to Handle JavaScript Errors

The **try** statement lets you test a block of code for errors. The **catch** statement lets you handle the error. The **throw** statement lets you create custom errors. The **finally** statement lets you execute code after try and catch, regardless of the result.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Try Catch</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  adddler("Welcome guest!");
}
catch(err) {
  text += "Error: " + err.message + "<br><br>";
}
finally {
  text += "The 'finally' block always runs.";
}

document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>  
</html>
```

Example 1

Result [View Example](#)

Reference Errors

A **ReferenceError** is thrown if you use (reference) a variable that has not been declared.

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Errors</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>Reference Error</h4>  
<p id="demo"></p>  
  
<script>  
let text = "";  
  
try {  
  x = y + 1;    // y cannot be referenced (used)  
}  
catch(err) {  
  text += "Error Name: " + err.name + "<br>";  
  text += "Error Message: " + err.message;  
}  
  
document.getElementById("demo").innerHTML = text;  
</script>  
  
</body>  
</html>
```

Example 2

Result [View Example](#)

JavaScript Type Errors

A **TypeError** is thrown if you use a value that is outside the range of expected types.

```
<!DOCTYPE html>
```

```

<html>
<body>

<h2>JavaScript Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Type Error</h4>
<p id="demo"></p>

<script>
let text = "";
let num = 1;

try {
  num.toUpperCase();    // You cannot convert a number to upper case
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JavaScript Range Errors

A **RangeError** is thrown if you use a number that is outside the range of legal values.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Range Error</h4>
<p id="demo"></p>

<script>
let text = "";
let num = 1;

try {
  num.toPrecision(500);    // A number cannot have 500 significant digits
}

```

```

catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

JavaScript URI Errors

A **URIError** is thrown if you use illegal characters in a URI function.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>URI Error</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  decodeURI("%%%"); // You cannot URI decode percent signs
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

JavaScript Syntax Errors

A **SyntaxError** is thrown if you try to evaluate code with a syntax error. Note: Syntax errors are **not catchable** in the same scope — they are thrown when the code is parsed, before execution.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Syntax Error (with eval)</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  eval("alert('Hello')"); // Missing ' will produce an error
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Errors](#)

JavaScript Silent Errors

Table of Contents

- [JavaScript Silent Errors](#)
- [Silent Errors](#)
- [Example 1](#)
- [Type Coercion](#)
- [Example 2](#)
- [String Coercion \(+\)](#)
- [Example 3](#)
- [Numeric Coercion](#)
- [Example 4](#)
- [Loose Equality \(==\)](#)
- [Example 5](#)
- [Practices to Avoid Bugs](#)
- [Example 6](#)
- [Reference](#)

Silent Errors

Silent errors happen when JavaScript fails silently instead of throwing an error. Unlike runtime errors that stop execution, silent errors let the program continue but produce unexpected results.

Silent errors often occur due to type coercion, where JavaScript automatically converts between types without warning.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Silent Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Silent Errors</h4>
<p id="demo"></p>

<script>
let text = "";

// These do NOT throw errors but produce unexpected results
let result1 = 5 + null;           // silent: null becomes 0
let result2 = "5" - 2;           // silent: string "5" becomes number 5
let result3 = "5" + 2;           // silent: number 2 becomes string "2"

text += "5 + null = " + result1 + "<br>";
text += "\"5\" - 2 = " + result2 + "<br>";
text += "\"5\" + 2 = \"" + result3 + "\"<br><br>";
text += "No errors were thrown!";
```



```
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Type Coercion

Type coercion is the automatic conversion of values from one data type to another. It happens silently when you perform operations on different data types, and JavaScript tries to make them "fit" together.

Because JavaScript is **weakly typed**, it doesn't throw errors when types don't match — it just coerces them. This can hide bugs.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Silent Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Type Coercion</h4>
<p id="demo"></p>

<script>
let result1 = ('5' + '2'); // = 52 (concatenation)
let result2 = ('5' - '2'); // = 3  (numeric subtraction)

document.getElementById("demo").innerHTML =
  "('5' + '2') = " + result1 + "<br>" +
  "('5' - '2') = " + result2;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

String Coercion (+)

If any part of a **+** operation is a string, JavaScript converts everything to strings and concatenates them. This is a common source of silent errors.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Silent Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Coercion (+)</h4>
<p id="demo"></p>

<script>
let x = "5" + 2; // x = "52" (number converted to string)

document.getElementById("demo").innerHTML =
  "\"5\" + 2 = \"" + x + "\"";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Numeric Coercion

Other arithmetic operators (-, *, /, %) and the unary plus (+x) force values into numbers. If a value cannot be converted, the result is **NaN**.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Silent Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numeric Coercion</h4>
<p id="demo"></p>

<script>
let text = "";

let x = "5" - 2;      // x = 3 (string converted to number)
let y = "abc" - 1;    // y = NaN (cannot convert)

text += "\"5\" - 2 = " + x + "<br>";
text += "\"abc\" - 1 = " + y + "<br>";
text += "No errors thrown, just NaN.";

document.getElementById("demo").innerHTML = text;
</script>

```

```
</body>
</html>
```

Example 4

Result [View Example](#)

Loose Equality (==)

The `==` operator performs coercion to find a "common type" before comparing. This can lead to unexpected results — a classic source of silent bugs.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Silent Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loose Equality (==)</h4>
<p id="demo"></p>

<script>
let text = "";

text += "5 == \"5\" : " + (5 == "5") + "<br>";
text += "0 == false : " + (0 == false) + "<br>";
text += "\"\" == false : " + ("\"\" == false) + "<br>";
text += "null == undefined : " + (null == undefined);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Practices to Avoid Bugs

To avoid silent errors caused by type coercion:

- **Use `===` (Strict Equality):** This checks both value and type without coercion.
- **Be Explicit:** Use `Number()`, `String()`, or `Boolean()` to make your intent clear.
- **Watch for NaN:** If a conversion fails, check for `NaN` before using the result.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Silent Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Practices to Avoid Bugs</h4>
<p id="demo"></p>

<script>
let text = "";

// Use === (Strict Equality)
text += "5 == \"5\" : " + (5 == "5") + " (loose)<br>";
text += "5 === \"5\" : " + (5 === "5") + " (strict)<br><br>";

// Be Explicit
text += "Number(\"5\") + 2 : " + (Number("5") + 2) + "<br><br>";

// Watch for NaN
let result = "abc" - 1;
if (isNaN(result)) {
    text += "\"abc\" - 1 is NaN, handle it gracefully.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Silent Errors](#)

JavaScript Error Statements

Table of Contents

- [JavaScript Error Statements](#)
- [The try and catch Blocks](#)
- [Example 1](#)
- [The Error Object](#)
- [Example 2](#)
- [The finally Block](#)
- [Example 3](#)
- [The throw Statement](#)
- [Example 4](#)
- [Input Validation](#)
- [Example 5](#)
- [Error Object Properties](#)
- [Example 6](#)
- [Reference](#)

The try and catch Blocks

The **try** statement defines a code block to run (to try). The **catch** statement defines a code block to handle any error. If an error occurs in the **try** block, the **catch** block is executed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Try and Catch</h4>
<p id="demo"></p>

<script>
try {
    adddler("Welcome guest!");
}
catch(err) {
    document.getElementById("demo").innerHTML = err.message;
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Error Object

When an error occurs, JavaScript creates an **Error object** with two properties: **name** and **message**. The **name** property returns the error type (e.g., ReferenceError, TypeError). The **message** property returns a description of the error.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Error Object</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  adddler("Welcome guest!");
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The finally Block

The **finally** statement lets you execute code after **try** and **catch**, regardless of the result. The **finally** block will always run, whether an error occurred or not.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>The finally Block</h4>
<p id="demo"></p>

<script>
let text = "";

try {
    adddler("Welcome guest!");
}
catch(err) {
    text += "Error caught: " + err.message + "<br><br>";
}
finally {
    text += "The 'finally' block always runs.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The throw Statement

The **throw** statement allows you to create a custom error. You can throw a string, number, boolean, or an object. When you use **throw** together with **try** and **catch**, you can control program flow and generate custom error messages.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The throw Statement</h4>
<p id="demo"></p>

<script>
let text = "";

try {
    throw "Too big";    // throw a text
}
catch(err) {
    text += "Error: " + err;
}

```

```

}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Input Validation

The **throw** statement is commonly used for input validation. If the input is invalid, you can throw an error message and catch it to display to the user.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Input Validation</h4>
<p>Please input a number between 5 and 10:</p>

<input id="demo" type="text">
<button type="button" onclick="myFunction()">Test Input</button>
<p id="message"></p>

<script>
function myFunction() {
    const message = document.getElementById("message");
    message.innerHTML = "";
    let x = document.getElementById("demo").value;

    try {
        if(x.trim() == "") throw "empty";
        if(isNaN(x)) throw "not a number";
        x = Number(x);
        if(x < 5) throw "too low";
        if(x > 10) throw "too high";
    }
    catch(err) {
        message.innerHTML = "Input is " + err;
    }
}
</script>

</body>
</html>

```


Example 5

Result [View Example](#)

Error Object Properties

The Error object provides several properties to help debug and identify problems:

- **name:** The type of error (e.g., "ReferenceError", "TypeError")
- **message:** A human-readable description of the error

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Error Object Properties</h4>
<p id="demo"></p>

<script>
let text = "";

// ReferenceError
try {
  x = y + 1;
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message + "<br><br>";
}

// TypeError
try {
  let num = 1;
  num.toUpperCase();
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Error Statements](#)

JavaScript Error Reference

Table of Contents

- [JavaScript Error Reference](#)
- [The Error Object](#)
- [Example 1](#)
- [Error Object Properties & Methods](#)
- [Example 2](#)
- [EvalError](#)
- [Example 3](#)
- [RangeError](#)
- [Example 4](#)
- [ReferenceError](#)
- [Example 5](#)
- [TypeError](#)
- [Example 6](#)
- [Reference](#)

The Error Object

The **Error Object** provides error information when an error occurs. JavaScript has a built-in error object that provides error information when an error occurs. The error object provides two useful properties: **name** and **message**.

You can also create a new Error object with the **new Error()** constructor.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Error Object</h4>
<p id="demo"></p>

<script>
// Create a new Error object
const err = new Error("Something went wrong!");

document.getElementById("demo").innerHTML =
  "Error Name: " + err.name + "<br>" +
  "Error Message: " + err.message;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Error Object Properties & Methods

The Error object has the following important **properties**:

- **name**: Returns or sets the error name
- **message**: Returns or sets an error message (a string)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Error Object Properties & Methods</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  adddler("Welcome guest!");
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message + "<br><br>";
  text += "typeof err: " + typeof err;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

EvalError

An **EvalError** indicates an error regarding the global **eval()** function. These errors are no longer thrown by JavaScript, but the **EvalError** object remains for compatibility.

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript Error Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>EvalError</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  throw new EvalError("Hello", "someFile.js", 10);
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

RangeError

A **RangeError** is thrown when a value is not in the set or range of allowed values.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RangeError</h4>
<p id="demo"></p>

<script>
let text = "";
let num = 1;

try {
  num.toPrecision(500); // A number cannot have 500 significant digits
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

```

```
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

ReferenceError

A **ReferenceError** is thrown when a variable that doesn't exist (hasn't been declared) is referenced.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>ReferenceError</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  x = y + 1;    // y is not defined
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

TypeError

A **TypeError** is thrown when an operation cannot be performed, typically when a value is not of the expected type.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>TypeError</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  let num = 1;
  num.toUpperCase();    // Numbers don't have toUpperCase()
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Error Reference](#)

JavaScript Debugging

Table of Contents

- [JavaScript Debugging](#)
- [Using console.log\(\)](#)
- [Example 1](#)
- [The debugger Statement](#)
- [Example 2](#)
- [Try Catch for Debugging](#)
- [Example 3](#)
- [Reading Error Messages](#)
- [Example 4](#)
- [Common Beginner Mistakes](#)
- [Example 5](#)
- [A Simple Debugging Checklist](#)
- [Example 6](#)
- [Reference](#)

Using console.log()

The `console.log()` method is the simplest way to debug JavaScript code. It writes messages to the browser's console, allowing you to inspect variable values and track program flow without interrupting execution.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.log()</h4>
<p>Open the browser console (F12) to see the debug output.</p>
<p id="demo"></p>

<script>
let a = 5;
let b = 6;
let c = a + b;

console.log("Value of a: " + a);
console.log("Value of b: " + b);
console.log("Value of c: " + c);

document.getElementById("demo").innerHTML = "Result: " + c;
</script>

</body>
</html>
```


Example 1

Result [View Example](#)

The debugger Statement

The **debugger** statement stops the execution of JavaScript and calls the debugging function (if available). It acts like a breakpoint in the code. If no debugging is available, the debugger statement has no effect.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The debugger Statement</h4>
<p id="demo"></p>

<script>
let x = 15 * 5;

// debugger; will pause execution here if DevTools is open
console.log("x = " + x);

document.getElementById("demo").innerHTML =
  "x = " + x + " (check console)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Try Catch for Debugging

Using **try...catch** is a powerful debugging technique. It allows you to catch errors gracefully, display meaningful messages, and continue execution instead of crashing. This is essential for identifying and fixing problems.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Try Catch for Debugging</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  // Intentionally calling an undefined function
  myFunction();
  text += "This line is skipped.";
}
catch(err) {
  text += "Debug Info:<br>";
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message + "<br>";
  text += "Check your code for the error above.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reading Error Messages

Error messages in the console are your best debugging tool. They tell you exactly what went wrong and where. Learning to read and understand error messages is an essential debugging skill.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reading Error Messages</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  let result = undefinedVariable + 5;
}
catch(err) {
  text += "Error Type: " + err.name + "<br>";
}

```

```

    text += "Description: " + err.message + "<br><br>";
    text += "Tip: The error message tells you what and where.<br>";
    text += "Look in the console (F12) for line numbers.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Common Beginner Mistakes

Some of the most common debugging challenges for beginners include misspelled variable names, using `=` instead of `==` or `===`, and forgetting to close brackets or quotes.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Common Beginner Mistakes</h4>
<p id="demo"></p>

<script>
let text = "";

// Mistake 1: Assignment (=) instead of comparison (==)
let x = 10;
if (x = 5) { // This assigns 5 to x, always true!
    text += "Mistake: '=' instead of '=='. ";
    text += "x is now: " + x + "<br><br>";
}

// Mistake 2: Misspelled variable name
let total = 100;
try {
    text += "total: " + total + "<br>";
    // text += total; // This would cause a ReferenceError
    text += "(Misspelled variables cause ReferenceErrors)<br><br>";
}
catch(err) {
    text += err.name + ": " + err.message + "<br>";
}

// Mistake 3: Case sensitivity

```

```

let myVar = "Hello";
text += "myVar: " + myVar + "<br>";
text += "(JavaScript is case-sensitive: myvar !== myVar)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

A Simple Debugging Checklist

When your code doesn't work, follow this simple debugging checklist:

9. **Check the console** for error messages
10. **Use `console.log()`** to inspect variable values
11. **Check spelling** of variable and function names
12. **Check data types** — is it a string or a number?
13. **Use `try...catch`** to handle unexpected errors

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Simple Debugging Checklist</h4>
<p id="demo"></p>

<script>
let text = "";

// Step 1: Check browser console (F12)
console.log("Debugging Checklist Demo");

// Step 2 & 3 & 4: Inspect variables, check spelling and types
let num1 = "5";
let num2 = 10;

console.log("num1 type:", typeof num1); // string
console.log("num2 type:", typeof num2); // number

text += "num1 = \"" + num1 + "\" (type: " + typeof num1 + ")<br>";
text += "num2 = " + num2 + " (type: " + typeof num2 + ")<br><br>";

// Step 5: Try-catch for safety

```

```
try {
  let result = Number(num1) + num2; // Explicit conversion
  text += "Number(num1) + num2 = " + result;
}
catch(err) {
  text += "Error: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Debugging](#)

JavaScript Debugging Console

Table of Contents

- [JavaScript Debugging Console](#)
- [Using console.log\(\)](#)
- [Example 1](#)
- [Using console.warn\(\)](#)
- [Example 2](#)
- [Using console.error\(\)](#)
- [Example 3](#)
- [Logging Multiple Values](#)
- [Example 4](#)
- [Inspecting Objects](#)
- [Example 5](#)
- [Using console.table\(\)](#)
- [Example 6](#)
- [Debugging Tip: Stop Guessing](#)
- [Example 7](#)
- [Reference](#)

Using console.log()

The `console.log()` method is the most common console method. Use it to print values and see what your program is doing. It prints messages and can print variable values to the browser's console.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.log()</h4>
<p>Open the browser console (F12) to see the debug output.</p>
<p id="demo"></p>

<script>
let name = "John";
let age = 25;

console.log("Hello from JavaScript!");
console.log(name);
console.log(age);

document.getElementById("demo").innerHTML =
  "name = " + name + ", age = " + age + " (check console)";
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Using console.warn()

Use `console.warn()` to display warnings. Warnings tell you something might be wrong, but your code can still run. Use warnings when you want to highlight something suspicious in your program.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.warn()</h4>
<p>Open the browser console (F12) to see warning messages.</p>
<p id="demo"></p>

<script>
let age = 15;

if (age < 18) {
  console.warn("This is a warning! User is under 18 years old.");
  console.warn("Age: " + age);
}

document.getElementById("demo").innerHTML =
  "Warning example: age = " + age + " (check console for warning)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using console.error()

Use `console.error()` to display error messages. Errors show that something has failed. The error message is displayed in red in most browsers' consoles, making it easy to spot.

```
<!DOCTYPE html>
<html>
```

```

<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.error()</h4>
<p>Open the browser console (F12) to see error messages.</p>
<p id="demo"></p>

<script>
let text = "";

try {
  // Intentionally cause an error
  undefinedFunction();
}
catch(err) {
  console.error("Something went wrong!");
  console.error("Error: " + err.message);
  text = "Error caught: " + err.message;
}

document.getElementById("demo").innerHTML = text + " (check console)";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Logging Multiple Values

You can log more than one value at once with `console.log()`. Simply separate values with commas. This is very useful for tracking related variable values together.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Logging Multiple Values</h4>
<p>Open the browser console (F12) to see the debug output.</p>
<p id="demo"></p>

<script>
let x = 10;
let y = 5;
let sum = x + y;

```



```

console.log("x =", x, "y =", y);
console.log("x =", x, "y =", y, "sum =", sum);

document.getElementById("demo").innerHTML =
    "x = " + x + ", y = " + y + ", sum = " + sum + " (check console)";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Inspecting Objects

The console is very useful for inspecting objects. You can log an object with `console.log()`, then in the console, you can click the object to expand it and see its properties.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Inspecting Objects</h4>
<p>Open the browser console (F12) and expand the object to see its properties.</p>
<p id="demo"></p>

<script>
let user = {
    name: "John",
    age: 25,
    email: "john@example.com",
    city: "New York"
};

console.log(user);
console.log("User name:", user.name);
console.log("User age:", user.age);

document.getElementById("demo").innerHTML =
    "User: " + user.name + ", Age: " + user.age + " (check console to inspect object)";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Using console.table()

Use `console.table()` to display data in a table format. Tables make it much easier to debug arrays of objects. The data is displayed as a formatted table in the browser console.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.table()</h4>
<p>Open the browser console (F12) to see the table output.</p>
<p id="demo"></p>

<script>
let users = [
  {name: "John", age: 25, city: "New York"},
  {name: "Anna", age: 30, city: "London"},
  {name: "Peter", age: 28, city: "Paris"}
];

console.table(users);
console.log("Users array logged as table (check console)");

let displayText = "";
for (let user of users) {
  displayText += user.name + " (" + user.age + ") - " + user.city + "<br>";
}
document.getElementById("demo").innerHTML =
  displayText + "(check console for table view)";
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Debugging Tip: Stop Guessing

Beginners often **guess** what the value is. Professionals **log the value** and confirm it. If your code is not working, **do not guess**. Use `console.log()`. Every time you assume a value, log it to the console and verify.

```
<!DOCTYPE html>
```

```

<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Debugging Tip: Stop Guessing</h4>
<p>Open the browser console (F12) to see the debug output.</p>
<p id="demo"></p>

<script>
let result = "";

// Don't guess what the value is – log it!
let a = 5;
let b = "10";

console.log("Value of a:", a, "| type:", typeof a);
console.log("Value of b:", b, "| type:", typeof b);

let total = a + b;
console.log("total = a + b =", total, "| type:", typeof total);

// Now fix it by converting
let fixedTotal = a + Number(b);
console.log("fixedTotal = a + Number(b) =", fixedTotal, "| type:", typeof fixedTotal);

result += "a = " + a + " (type: " + typeof a + ")<br>";
result += "b = \"" + b + "\" (type: " + typeof b + ")<br>";
result += "a + b = " + total + " (unexpected string concatenation!)<br>";
result += "a + Number(b) = " + fixedTotal + " (correct addition)";

document.getElementById("demo").innerHTML = result;
</script>

</body>
</html>

```

Example 7

Result [View Example](#)

Reference

- [W3Schools JavaScript Debugging Console](#)

JavaScript Debugging Breakpoints

Table of Contents

- [JavaScript Debugging Breakpoints](#)
- [Setting a Breakpoint](#)
- [Example 1](#)
- [The debugger Keyword](#)
- [Example 2](#)
- [Stepping Through Code](#)
- [Example 3](#)
- [The Scope Panel](#)
- [Example 4](#)
- [Watching Variables](#)
- [Example 5](#)
- [Common Beginner Mistakes](#)
- [Example 6](#)
- [When to Use Breakpoints](#)
- [Example 7](#)
- [Reference](#)

Setting a Breakpoint

A breakpoint pauses code execution on a specific line. When the code stops, you can inspect variables and step through the code line by line. Breakpoints are set inside the browser developer tools. You do not guess values when using breakpoints—you see them.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Setting a Breakpoint</h4>
<p>Open the browser developer tools (F12), go to Sources tab, and click a line number
to set a breakpoint.</p>
<p id="demo"></p>

<script>
function add(a, b) {
  let result = a + b;
  return result; // Set a breakpoint on this line
}

document.getElementById("demo").innerHTML = add(10, 5);
console.log("Result: " + add(10, 5));
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

The debugger Keyword

The **debugger** keyword stops the execution of JavaScript and calls (if available) the debugging function. This has the same function as setting a breakpoint in the debugger. If no debugging is available, the **debugger** statement has no effect.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The debugger Keyword</h4>
<p>Open the browser console (F12). If DevTools is open, execution will pause at the
debugger statement.</p>
<p id="demo"></p>

<script>
let x = 15 * 5;
debugger; // Execution pauses here if DevTools is open
document.getElementById("demo").innerHTML = x;

console.log("x = " + x);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Stepping Through Code

When execution is paused, you can control how the code runs. **Step Over** runs the next line. **Step Into** enters a function. **Step Out** exits the current function. Step slowly and watch how values change.

```
<!DOCTYPE html>
<html>
```

```

<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Stepping Through Code</h4>
<p>Open DevTools (F12), go to Sources tab. Set a breakpoint at the first line inside
multiply(), then reload.</p>
<p id="demo"></p>

<script>
function multiply(a, b) {
  let step1 = a * 2;    // Step Over: watch a*2
  let step2 = b * 3;    // Step Over: watch b*3
  let result = step1 + step2; // Step Over: watch final result
  return result;
}

let output = multiply(5, 4);
document.getElementById("demo").innerHTML = "Result: " + output;
console.log("multiply(5,4) = " + output);
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The Scope Panel

The Scope panel shows which variables are available at the current line. It helps you understand where variables live. Local variables exist inside a function. Global variables exist everywhere.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Scope Panel</h4>
<p>Open DevTools (F12). Set a breakpoint inside the test() function to see local vs
global scope.</p>
<p id="demo"></p>

<script>
let x = 10; // Global variable - visible in Scope panel everywhere

function test() {
  let y = 5; // Local variable - visible only inside this function

```

```

    console.log(x + y); // Set breakpoint here to see Scope panel
    return x + y;
}

document.getElementById("demo").innerHTML =
    "Global x = " + x + ", Function result = " + test();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Watching Variables

The Watch panel lets you track variable values live. This is useful when values change many times. The value updates automatically as you step through the code. Watch variables instead of adding many `console.log()` calls.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Watching Variables</h4>
<p>Open DevTools (F12), go to Sources tab. Set a breakpoint at the start of the loop
to watch counter and total.</p>
<p id="demo"></p>

<script>
function calculateSum(count) {
    let total = 0;
    for (let i = 1; i <= count; i++) {
        total += i; // Set breakpoint here and watch 'i' and 'total'
        console.log("i =", i, "total =", total);
    }
    return total;
}

let result = calculateSum(5);
document.getElementById("demo").innerHTML =
    "Sum of 1 to 5 = " + result + " (check console to see each step)";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Common Beginner Mistakes

Beginners often forget to reload the page after adding a breakpoint. Breakpoints inside loops can trigger many times. If execution stops repeatedly, disable the breakpoint temporarily.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Common Beginner Mistakes</h4>
<p>Open DevTools (F12). Try setting a breakpoint inside the loop to see it trigger
multiple times.</p>
<p id="demo"></p>

<script>
let text = "";

// Mistake: Breakpoint inside loop triggers many times
for (let i = 0; i < 5; i++) {
  text += "Iteration " + i + "<br>"; // Set breakpoint here - triggers 5 times!
}

// Mistake: Variables change unexpectedly
let value = 10;
value = value * 2;
value = value + 5;
value = value - 3; // What is the value now? Don't guess - check the console!

console.log("Final value:", value);
text += "<br>Final value: " + value + " (check console)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

When to Use Breakpoints

Use breakpoints when values change unexpectedly, when code runs but produces wrong results, or when debugging complex logic. Breakpoints let you see exactly what's happening at every step of your code.


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Breakpoints</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>When to Use Breakpoints</h4>
<p>Open DevTools (F12) and set a breakpoint to debug this calculation.</p>
<p id="demo"></p>

<script>
// This function has complex logic that could produce wrong results
function calculateDiscount(price, customerType) {
  let discount = 0;

  if (customerType === "VIP") {
    discount = 0.2;
  } else if (customerType === "Member") {
    discount = 0.1;
  } else {
    discount = 0.05;
  }

  let discountAmount = price * discount; // Set breakpoint to verify values
  let finalPrice = price - discountAmount;

  return finalPrice;
}

let price1 = calculateDiscount(1000, "VIP");
let price2 = calculateDiscount(500, "Member");
let price3 = calculateDiscount(300, "Guest");

console.log("VIP pays:", price1);
console.log("Member pays:", price2);
console.log("Guest pays:", price3);

document.getElementById("demo").innerHTML =
  "VIP: " + price1 + " (20% off 1000)<br>" +
  "Member: " + price2 + " (10% off 500)<br>" +
  "Guest: " + price3 + " (5% off 300)";
</script>

</body>
</html>

```

Example 7

Result [View Example](#)

Reference

- [W3Schools JavaScript Debugging Breakpoints](#)

JavaScript Debugging Errors

Table of Contents

- [JavaScript Debugging Errors](#)
- [Common Errors Explained](#)
- [How to Read an Error Message](#)
- [ReferenceError](#)
- [Example 1](#)
- [TypeError](#)
- [Example 2](#)
- [SyntaxError](#)
- [Example 3](#)
- [NaN Errors](#)
- [Example 4](#)
- [Cannot Read Property of Undefined](#)
- [Example 5](#)
- [Common Error Meanings](#)
- [Debugging Tip](#)
- [Reference](#)

Common Errors Explained

JavaScript error messages look scary, but most of them mean very simple things. This page explains the most common errors in beginner-friendly language.

Error messages are clues. They tell you where to look and what kind of mistake happened.

How to Read an Error Message

An error message usually has three important parts:

- The error type.
- A short explanation.
- A line number.

Click the line number in the console to jump to the exact line of code.

ReferenceError

A **ReferenceError** means a variable or function name does not exist. This is often caused by a misspelling or a missing declaration.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>ReferenceError</h4>
<p>Open the browser console (F12) to see the error. The variable is not declared.</p>
<p id="demo"></p>

<script>
try {
  console.log(myValue); // ReferenceError: myValue is not defined
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log(e.name + ": " + e.message);
}
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

TypeError

A **TypeError** means you used a value in an invalid way. This usually happens with **undefined** or **null**.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>TypeError</h4>
<p>Open the browser console (F12) to see the error.</p>
<p id="demo"></p>

<script>
try {
  let x;
  console.log(x.length); // TypeError: Cannot read properties of undefined
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log("Tip: The variable exists, but it has no value. Log the value before
using it.");
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

SyntaxError

A **SyntaxError** means JavaScript cannot understand your code. This is often caused by missing brackets or parentheses. Syntax errors stop the script from running.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>SyntaxError</h4>
<p>Open the browser console (F12) to see the error from the eval below.</p>
<p id="demo"></p>

<script>
try {
  eval("if (x == 5 { console.log('Hello'); });"); // Missing closing parenthesis
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log("Tip: Check for missing brackets or parentheses.");
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

NaN Errors

NaN means **Not a Number**. This happens when JavaScript cannot perform valid math.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>NaN Errors</h4>
```

```

<p>Open the browser console (F12) to see the result.</p>
<p id="demo"></p>

<script>
let result = "abc" * 5;
console.log("Result of 'abc' * 5:", result); // NaN

if (isNaN(result)) {
  document.getElementById("demo").innerHTML =
    "Error: Result is NaN (Not a Number).<br>Tip: Check that both values are numbers
before doing math.";
} else {
  document.getElementById("demo").innerHTML = "Result: " + result;
}
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Cannot Read Property of Undefined

This is one of the most common beginner errors. It means you are trying to use something that does not exist. The variable exists, but the object does not.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Errors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot Read Property of Undefined</h4>
<p>Open the browser console (F12) to see the error.</p>
<p id="demo"></p>

<script>
try {
  let user;
  console.log(user.name); // Cannot read properties of undefined
} catch (e) {
  document.getElementById("demo").innerHTML =
    "Error: " + e.name + " - " + e.message;
  console.log("Tip: The variable 'user' exists but is undefined. Initialize it before
accessing properties.");
}
</script>

</body>

```

```
</html>
```

Example 5

Result [View Example](#)

Common Error Meanings

- **ReferenceError** means a name is not defined.
- **TypeError** means a value is used incorrectly.
- **SyntaxError** means broken code structure.
- **NaN** means invalid math.

Debugging Tip

Do not ignore errors. Fix the first error before moving on. One error often causes many others.

Reference

- [W3Schools JavaScript Debugging Errors](#)

JavaScript localStorage Counter

Table of Contents

- [JavaScript localStorage Counter](#)
- [Project Overview](#)
- [1. Create the HTML](#)
- [Example 1](#)
- [2. Create a Script](#)
- [Example 2](#)
- [3. Add an Increase Counter Function](#)
- [Example 3](#)
- [4. Add a Decrease Counter Function](#)
- [Example 4](#)
- [5. Add a Reset Counter Function](#)
- [Example 5](#)
- [6. Create a Save Counter Function](#)
- [Example 6](#)
- [7. Create a Load Counter Function](#)
- [Example 7](#)
- [Exercises & Solutions](#)
- [Example 8](#)
- [Reference](#)

Project Overview

In this project, you will build a counter with buttons to:

- **Increase** the counter
- **Decrease** the counter
- **Reset** the counter
- **Save** the counter to local storage
- **Load** the counter from local storage

Project Skills Required:

- JavaScript Variables
- JavaScript Functions
- JavaScript HTML DOM
- JavaScript Events (**onClick** attributes)
- JavaScript **localStorage** (to Save and Restore the counter)

1. Create the HTML

Create the basic HTML layout containing the counter display and buttons:

- Add a `<h2>` header element.

- Add a `<p>` element displaying a number.
- Add 5 `<button>` elements.
- Add an `onclick` attribute to each button to trigger JavaScript functions.

```
<h2>Counter</h2>

<p id="count" style="font-size:40px;">0</p>

<button onclick="increaseCount()">+</button>
<button onclick="decreaseCount()">-</button>
<button onclick="resetCount()">Reset</button>
<button onclick="saveCount()">Save</button>
<button onclick="loadCount()">Load</button>
```

Example 1

Result [View Example](#)

2. Create a Script

Create a script that:

- Declares a counter variable: `let count = 0`
- Creates a function `updateCount()` to update the display
- Uses `document.getElementById().innerHTML` to display the counter value

```
<script>
// Declare a counter
let count = 0;

// Function to display the counter
function updateCount() {
  document.getElementById("count").innerHTML = count;
}
</script>
```

Example 2

Result [View Example](#)

3. Add an Increase Counter Function

- Create `increaseCount()` to handle incrementing.
- Use `count++` to increase the counter.
- Call `updateCount()` to refresh the display.

```
// Function to increase the counter
function increaseCount() {
```

```
count++;  
updateCount();  
}
```

Example 3

Result [View Example](#)

4. Add a Decrease Counter Function

- Create `decreaseCount()` to handle decrementing.
- Use `count--` to decrease the counter.
- Call `updateCount()` to refresh the display.

```
// Function to decrease the counter  
function decreaseCount() {  
    count--;  
    updateCount();  
}
```

Example 4

Result [View Example](#)

5. Add a Reset Counter Function

- Create `resetCount()` to reset.
- Set `count = 0` to reset the counter value.
- Call `updateCount()` to refresh the display.

```
// Function to reset the counter  
function resetCount() {  
    count = 0;  
    updateCount();  
}
```

Example 5

Result [View Example](#)

6. Create a Save Counter Function

- Create `saveCount()` to save.
- Use `localStorage.setItem()` to store the current count value as text.

```
// Function to save the counter
function saveCount() {
  localStorage.setItem("count", count);
}
```

Example 6

Result [View Example](#)

7. Create a Load Counter Function

- Create `loadCount()` to load.
- Use `localStorage.getItem()` to get the saved value.
- Use `Number(saved)` to convert the saved text back into a number.
- Call `updateCount()` to refresh the display.

```
// Function to load the counter
function loadCount() {
  let saved = localStorage.getItem("count");
  if (saved !== null) {
    count = Number(saved);
  }
  updateCount();
}
```

> [!NOTE]

> `localStorage` stores values as text, so we use `Number()` to convert the value back into a number.

Example 7

Result [View Example](#)

Exercises & Solutions

Exercise 1

Make the counter start at 10 instead of 0.

Exercise 2

Prevent the counter from going below 0.

Exercise 3

Automatically load the saved counter value when the page opens.

Solutions Code

```
<h2>Counter</h2>

<p id="count" style="font-size:40px;">0</p>

<button onclick="increaseCount()">+</button>
<button onclick="decreaseCount()">-</button>
<button onclick="resetCount()">Reset</button>
<button onclick="saveCount()">Save</button>
<button onclick="loadCount()">Load</button>

<script>
// Declare the counter (starts at 10)
let count = 10;

// Load the counter when the page opens
loadCount();

// Function to display the counter
function updateCount() {
    document.getElementById("count").innerHTML = count;
}

// Function to increase the counter
function increaseCount() {
    count++;
    updateCount();
}

// Function to decrease the counter (prevent going below 0)
function decreaseCount() {
    if (count > 0) {
        count--;
        updateCount();
    }
}

// Function to reset the counter (resets to 10)
function resetCount() {
    count = 10;
    updateCount();
}

// Function to save the counter
function saveCount() {
    localStorage.setItem("count", count);
}

// Function to load the counter
function loadCount() {
    let saved = localStorage.getItem("count");
    if (saved !== null) {
        count = Number(saved);
    }
    updateCount();
}
```

```
}  
</script>
```

Example 8

Result [View Example](#)

Reference

- [W3Schools JavaScript Counter Project](#)

JavaScript Project - Event Listener

Table of Contents

- [JavaScript Project - Event Listener](#)
- [Using Event Listeners](#)
- [Example 1](#)
- [Improvements](#)
- [Example 2](#)
- [JavaScript in External File](#)
- [DOMContentLoaded](#)
- [Reference](#)

Using Event Listeners

Using `addEventListener()` makes it easier to:

- Keep HTML and JavaScript separated
- Add multiple events to the same element
- Write cleaner code in bigger projects

Here is the basic implementation using event listeners:

HTML Code

```
<h2>Counter</h2>

<p id="count" style="font-size:40px;">0</p>

<button id="btnPlus">+</button>
<button id="btnMinus">-</button>
<button id="btnReset">Reset</button>
<button id="btnSave">Save</button>
<button id="btnLoad">Load</button>

<script>
// Add event listeners
document.getElementById("btnPlus").addEventListener("click", increaseCount);
document.getElementById("btnMinus").addEventListener("click", decreaseCount);
document.getElementById("btnReset").addEventListener("click", resetCount);
document.getElementById("btnSave").addEventListener("click", saveCount);
document.getElementById("btnLoad").addEventListener("click", loadCount);

// Declare the counter
let count = 0;

// Load counter when the page opens
loadCount();

// Function to display the counter
function updateCount() {
```

```

    document.getElementById("count").innerHTML = count;
}

// Function to increase the counter
function increaseCount() {
    count++;
    updateCount();
}

// Function to decrease the counter
function decreaseCount() {
    if (count > 0) {
        count--;
        updateCount();
    }
}

// Function to reset the counter
function resetCount() {
    count = 0;
    updateCount();
}

// Function to save the counter
function saveCount() {
    localStorage.setItem("count", count);
}

// Function to load the counter
function loadCount() {
    let saved = localStorage.getItem("count");
    if (saved !== null) {
        count = Number(saved);
    }
    updateCount();
}
</script>

```

Example 1

Result [View Example](#)

Improvements

Here are the key improvements:

- JavaScript is moved to an external file (**counter.js**)
- Uses **DOMContentLoaded** so JavaScript runs only after the HTML is ready
- Cleaner structure using element variables
- Shows a temporary **Saved!** or **Loaded!** message

HTML Code

```

<!DOCTYPE html>
<html>
<body>

<h2>Counter</h2>

<p id="count" style="font-size:40px;">0</p>

<button id="btnPlus">+</button>
<button id="btnMinus">-</button>
<button id="btnReset">Reset</button>
<button id="btnSave">Save</button>
<button id="btnLoad">Load</button>

<!-- Message display paragraph -->
<p id="message"></p>

<script src="counter.js"></script>

</body>
</html>

```

counter.js

```

document.addEventListener("DOMContentLoaded", function () {

// Declare the counter
let count = 0;

// Use element variables
const countEl = document.getElementById("count");
const msgEl = document.getElementById("message");
const btnPlus = document.getElementById("btnPlus");
const btnMinus = document.getElementById("btnMinus");
const btnReset = document.getElementById("btnReset");
const btnSave = document.getElementById("btnSave");
const btnLoad = document.getElementById("btnLoad");

// Add event listeners
btnPlus.addEventListener("click", increaseCount);
btnMinus.addEventListener("click", decreaseCount);
btnReset.addEventListener("click", resetCount);
btnSave.addEventListener("click", saveCount);
btnLoad.addEventListener("click", loadCount);

// Function to display the counter
function updateCount() {
    countEl.innerHTML = count;
}

// Function to display message

```



```

function showMessage(text) {
    msgEl.innerHTML = text;
    setTimeout(function () {
        msgEl.innerHTML = "";
    }, 3000);
}

// Function to increase the counter
function increaseCount() {
    count++;
    updateCount();
}

// Function to decrease the counter
function decreaseCount() {
    count--;
    updateCount();
}

// Function to reset the counter
function resetCount() {
    count = 0;
    updateCount();
}

// Function to save the counter
function saveCount() {
    localStorage.setItem("count", count);
    showMessage("Saved!");
}

// Function to load the counter
function loadCount() {
    let saved = localStorage.getItem("count");
    if (saved !== null) {
        count = Number(saved);
        showMessage("Loaded!");
    }
    updateCount();
}

});

```

Example 2

Result [View Example](#)

JavaScript in External File

Using `counter.js` makes your project:

- More Organized
- Keeps HTML and JavaScript separated

- Easier to read and to reuse
- More like real-world projects

DOMContentLoaded

Why use `DOMContentLoaded`?

When you use an external JavaScript file, the browser may load the script before the HTML is ready. So we wrap the initialization code inside:

```
document.addEventListener("DOMContentLoaded", function() {  
    // JavaScript code  
});
```

This guarantees that the JavaScript code runs only after the page has finished parsing the HTML.

Reference

- [W3Schools JavaScript Event Listener Project](#)

JavaScript Project - To-Do List

Table of Contents

- [JavaScript Project - To-Do List](#)
- [To-Do List Overview](#)
- [You Will Learn](#)
- [First: Create the HTML](#)
- [Example 1](#)
- [Add a Display Function](#)
- [Example 2](#)
- [Add an Add Task Function](#)
- [Add a Remove Task Function](#)
- [Add a Clear All Function](#)
- [Function to Save a Task](#)
- [Function to Load the Tasks](#)
- [Final Project](#)
- [Example 3](#)
- [Exercises & Solutions](#)
- [Example 4](#)
- [Bonus Challenges \(Level Up\)](#)
- [Example 5](#)
- [Reference](#)

To-Do List Overview

In this project, you will build a functional **To-Do List**.

- You will be able to **add**, **remove**, and **clear all** tasks.
- The tasks will be kept synchronized within an **Array** saved in **localStorage**.

You Will Learn

- How to store items in an array structure
- How to add and remove items dynamically from an array
- How to generate and update HTML lists based on array values
- How to serialize and load JSON data using localStorage

First: Create the HTML

Start by creating an HTML file with:

- An `<input>` field with `id="task"`.
- An "Add" button calling `addTask()`.
- An unordered list `<ul>` with `id="list"`.
- A "Clear All" button calling `clearAll()`.
- A script tag initializing an empty `tasks` array.

```

<!DOCTYPE html>
<html>
<body>

<h2>To-Do List</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<input id="task" placeholder="New task">
<button onclick="addTask()">Add</button>
<ul id="list"></ul>
<button onclick="clearAll()">Clear All</button>

<script>
// Create a task Array
let tasks = [];
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Add a Display Function

The `displayTasks()` function loops through the array items, wraps each item in `<li>` elements alongside remove buttons, and prints the result inside the list element:

```

// Function to display the list
function displayTasks() {
  let html = "";
  for (let i = 0; i < tasks.length; i++) {
    html += "<li>" + tasks[i] +
      " <button onclick='removeTask(" + i + ")'>x</button></li>";
  }
  document.getElementById("list").innerHTML = html;
}

```

Example 2

Result [View Example](#)

Add an Add Task Function

Retrieves the input value, pushes it to the tasks array, resets input text, and updates list views:

```

// Function to Add a task

```

```
function addTask() {
  let taskInput = document.getElementById("task");
  let text = taskInput.value;
  if (text === "") {
    return;
  }
  tasks.push(text);
  taskInput.value = "";
  saveTasks();
  displayTasks();
}
```

Add a Remove Task Function

Removes tasks by index using the JavaScript array `splice()` method:

```
// Function to Remove a task
function removeTask(i) {
  tasks.splice(i, 1);
  saveTasks();
  displayTasks();
}
```

> [!NOTE]

> `tasks.splice(i, 1)` deletes exactly 1 item at index position `i`.

Add a Clear All Function

Empties the array and refreshes the displays:

```
// Function to Clear all tasks
function clearAll() {
  tasks = [];
  saveTasks();
  displayTasks();
}
```

Function to Save a Task

Converts the tasks array into text format and saves it inside `localStorage`:

```
// Function to Save tasks
function saveTasks() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
}
```

> [!IMPORTANT]

> `localStorage` only handles text keys and values. We must use `JSON.stringify(tasks)` to convert the tasks array into a string before saving.

Function to Load the Tasks

Retrieves string data and parses it back into an array structure:

```
function loadTasks() {
  let saved = localStorage.getItem("tasks");
  if (saved !== null) {
    tasks = JSON.parse(saved);
  }
}
```

Final Project

Combines HTML code structure and state persistence logics together:

```
<!DOCTYPE html>
<html>
<body>

<h2>To-Do List</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<input id="task" placeholder="New task">
<button onclick="addTask()">Add</button>
<ul id="list"></ul>
<button onclick="clearAll()">Clear All</button>

<script>
let tasks = [];

function displayTasks() {
  let html = "";
  for (let i = 0; i < tasks.length; i++) {
    html += "<li>" + tasks[i] +
      " <button onclick='removeTask(\" + i + \")'>x</button></li>";
  }
  document.getElementById("list").innerHTML = html;
}

function addTask() {
  let taskInput = document.getElementById("task");
  let text = taskInput.value;
  if (text === "") {
    return;
  }
  tasks.push(text);
}
```

```

    taskInput.value = "";
    saveTasks();
    displayTasks();
}

function removeTask(i) {
    tasks.splice(i, 1);
    saveTasks();
    displayTasks();
}

function clearAll() {
    tasks = [];
    saveTasks();
    displayTasks();
}

function saveTasks() {
    localStorage.setItem("tasks", JSON.stringify(tasks));
}

function loadTasks() {
    let saved = localStorage.getItem("tasks");
    if (saved !== null) {
        tasks = JSON.parse(saved);
    }
}

loadTasks();
displayTasks();
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Exercises & Solutions

Exercise 1

Show an alert if the user tries to add an empty task.

Exercise 2

Allow the user to press Enter to add a task.

Exercise 3

Add a "Saved!" message to show when saving.

Solutions Code

```
<input id="task" placeholder="New task" onkeydown="handleKey(event)">
<button onclick="addTask()">Add</button>
<ul id="list"></ul>
<button onclick="clearAll()">Clear All</button>
<p id="status" style="color: green; font-weight: bold;"></p>

<script>
// (Inside script)
function addTask() {
  let taskInput = document.getElementById("task");
  let text = taskInput.value.trim();
  if (text === "") {
    alert("Please enter a task! Task cannot be empty.");
    return;
  }
  tasks.push(text);
  taskInput.value = "";
  saveTasks();
  displayTasks();
}

function handleKey(event) {
  if (event.key === "Enter") {
    addTask();
  }
}

function saveTasks() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
  showStatus("Saved!");
}

function showStatus(msg) {
  let statusEl = document.getElementById("status");
  statusEl.innerHTML = msg;
  setTimeout(function () {
    statusEl.innerHTML = "";
  }, 2000);
}
</script>
```

Example 4

Result [View Example](#)

Bonus Challenges (Level Up)

Improve task workflows by adding premium styling, status checks, and filtering controls:

- **Checkbox markers:** Check off tasks as done (triggers text strikethrough).
- **Filters:** Toggle list views by active state categories (All / Active / Completed).

- **Inline Editing:** Modify list names and save task changes dynamically.
- **Sorting:** Sort list items alphabetically in one click.

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript To-Do List Project](#)

JavaScript Project - Modal Popup

Table of Contents

- [JavaScript Project - Modal Popup](#)
- [Modal Popup Overview](#)
- [What You Will Learn](#)
- [First: Create the HTML](#)
- [Example 1](#)
- [Then: Create the CSS](#)
- [Example 2](#)
- [CSS Explained](#)
- [Then: Create the JavaScript](#)
- [Example 3](#)
- [JavaScript Explained](#)
- [Connect Functions to Buttons](#)
- [Connect Function to Clicking Outside](#)
- [Connect Function to Escape Key](#)
- [Common Mistakes](#)
- [Bonus Challenges \(Level Up\)](#)
- [Example 4](#)
- [Reference](#)

Modal Popup Overview

A **modal** is a **popup window** that appears on top of the main webpage overlaying the current view.

In this project, we will build a modal that can be closed in three ways:

14. Clicking the close button (✕)
15. Clicking outside the modal container
16. Pressing the Escape key

What You Will Learn

- How to dynamically show and hide HTML elements
- How to add and remove CSS classes using JavaScript
- How to listen to click and keydown event targets
- How to detect clicks outside a bounding element box

First: Create the HTML

The modal layout consists of two main parts:

- **The overlay background:** Dims the page context behind.
- **The modal content box:** Displays the pop-up text and controls.

```

<!DOCTYPE html>
<html>
<body>

<h2>Modal Popup</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<button id="openBtn">Open Modal</button>

<div id="modal" class="modal-overlay">
  <div class="modal-box">
    <button id="closeBtn" class="modal-close">&times;</button>
    <h3>Hello!</h3>
    <p>This is a modal popup.</p>
  </div>
</div>

</body>
</html>

```

Example 1

Result [View Example](#)

Then: Create the CSS

Define structural position attributes to overlay the window and control visual overlays:

```

<style>
/* The overlay (background) */
.modal-overlay {
  display: none;
  position: fixed;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5);
}

/* Show the modal */
.modal-overlay.show {
  display: block;
}

/* The modal box */
.modal-box {
  background: white;
  width: 90%;
  max-width: 400px;
  margin: 100px auto;
  padding: 20px;
}

```

```

    border-radius: 10px;
    position: relative;
}

/* Close button */
.modal-close {
    position: absolute;
    right: 12px;
    top: 8px;
    font-size: 24px;
    border: none;
    background: none;
    cursor: pointer;
}
</style>

```

Example 2

Result [View Example](#)

CSS Explained

- **Hide modal by default:** We use `display: none` on `.modal-overlay`.
- **Toggle visibility:** We define a `.modal-overlay.show` selector specifying `display: block`. JavaScript will add and remove this helper class dynamically.

Then: Create the JavaScript

Select elements and implement toggle helpers mapping handlers to events:

```

<script>
const modal = document.getElementById("modal");
const openBtn = document.getElementById("openBtn");
const closeBtn = document.getElementById("closeBtn");

function openModal() {
    modal.classList.add("show");
}

function closeModal() {
    modal.classList.remove("show");
}

openBtn.addEventListener("click", openModal);
closeBtn.addEventListener("click", closeModal);

/* Close when clicking outside the modal box */
modal.addEventListener("click", function (event) {
    if (event.target === modal) {
        closeModal();
    }
}

```

```
});

/* Close when pressing Escape */
document.addEventListener("keydown", function (event) {
  if (event.key === "Escape") {
    closeModal();
  }
});
</script>
```

Example 3

Result [View Example](#)

JavaScript Explained

We use `classList.add("show")` and `classList.remove("show")` to adjust display states on triggers.

Connect Functions to Buttons

Register listeners to handle standard interaction click paths:

```
openBtn.addEventListener("click", openModal);
closeBtn.addEventListener("click", closeModal);
```

Connect Function to Clicking Outside

Check whether the target element triggering click events is the overlay container node itself:

```
modal.addEventListener("click", function (event) {
  if (event.target === modal) {
    closeModal();
  }
});
```

> [!NOTE]

> `event.target === modal` checks if click bubbles up/originates on the overlay background surface (outside the white `.modal-box`).

Connect Function to Escape Key

Listen globally for Escape keystrokes to allow quick exits:

```
document.addEventListener("keydown", function (event) {
  if (event.key === "Escape") {
```

```
    closeModal();
  }
});
```

Common Mistakes

- **Modal overlay does not close when clicked:** Check that you compare `event.target === modal` strictly.
- **Escape closes invisible modals:** You can optionally check `modal.classList.contains("show")` before hiding.
- **Display none overrides animation:** Animations require opacity transitions or fade keyframe steps rather than display swaps.

Bonus Challenges (Level Up)

Elevate the design with premium UX transitions and accessibility support:

- Add a fade-in and scale-up effect using CSS opacity and transform transition metrics.
- Move document focus inside the modal overlay dialog upon open.
- Lock/Trap tab key navigation focus cycles within active boundaries.
- Restore focus tracking state to previous active elements on exit.

```
<!-- Bonus Challenge Implementation -->
<script>
const modal = document.getElementById("modal");
const modalBox = modal.querySelector(".modal-box");
const openBtn = document.getElementById("openBtn");
const closeBtn = document.getElementById("closeBtn");

let previousActiveElement = null;

function openModal() {
  previousActiveElement = document.activeElement;
  modal.classList.add("show");
  modalBox.focus();
}

function closeModal() {
  modal.classList.remove("show");
  if (previousActiveElement) {
    previousActiveElement.focus();
  }
}

// Focus Trap inside the modal
modalBox.addEventListener("keydown", function (event) {
  const focusable = modalBox.querySelectorAll('button, [href], input,
[tabindex]:not([tabindex="-1"])');
  const first = focusable[0];
  const last = focusable[focusable.length - 1];
```

```
if (event.key === "Tab") {  
  if (event.shiftKey) {  
    if (document.activeElement === first) {  
      last.focus();  
      event.preventDefault();  
    }  
  } else {  
    if (document.activeElement === last) {  
      first.focus();  
      event.preventDefault();  
    }  
  }  
}  
});  
</script>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Modal Popup Project](#)

JavaScript Project - Form Validation

Table of Contents

- [JavaScript Project - Form Validation](#)
- [Form Validation Overview](#)
- [What You Will Learn](#)
- [Step 1 - Create the HTML](#)
- [Example 1](#)
- [Step 2 - Add the CSS](#)
- [Example 2](#)
- [Step 3 - Add JavaScript](#)
- [Example 3](#)
- [Step 4 - Validate Fields](#)
- [Example 4](#)
- [Exercises & Solutions](#)
- [Example 5](#)
- [Bonus Challenges \(Level Up\)](#)
- [Example 6](#)
- [Reference](#)

Form Validation Overview

In this project, you will build a sign-up form with input validation.

The form shows inline error messages under each field and prevents submission until all fields are filled and valid.

What You Will Learn

- How to validate user input against custom rules using JavaScript
- How to prevent forms from reloading the page during submit events
- How to write validation check helper functions
- How to check email formats using Regular Expressions (regex)

Step 1 - Create the HTML

Start by creating the basic structure containing four text fields (Name, Email, Password, Confirm Password), associated error message paragraphs, and a submit button.

```
<!DOCTYPE html>
<html>
<body>

<h2>Sign Up</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```



```

<form id="signupForm">

<div class="field">
  <label>Name:</label><br>
  <input id="name" type="text" placeholder="Your name">
  <p id="nameError" class="error"></p>
</div>

<div class="field">
  <label>Email:</label><br>
  <input id="email" type="text" placeholder="name@example.com">
  <p id="emailError" class="error"></p>
</div>

<div class="field">
  <label>Password:</label><br>
  <input id="password" type="password" placeholder="Min 8 characters">
  <p id="passwordError" class="error"></p>
</div>

<div class="field">
  <label>Confirm Password:</label><br>
  <input id="confirm" type="password" placeholder="Repeat password">
  <p id="confirmError" class="error"></p>
</div>

<p><button type="submit">Create Account</button></p>

</form>

<p id="result"></p>

<script>
// Create a task Array
let tasks = [];
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Step 2 - Add the CSS

Add styling to distinguish input boxes, error layout styles, and valid confirmation elements:

```

<style>
input {
  padding: 8px;
  width: 260px;

```

```

    margin-bottom: 4px;
  }
  .error {
    color: red;
    margin: 0;
  }
  .ok {
    color: green;
    margin: 0;
  }
  .field {
    margin-bottom: 12px;
  }
</style>

```

Example 2

Result [View Example](#)

Step 3 - Add JavaScript

Cache inputs and build common error layout handlers:

- **Create variables for each element:**

```

const form = document.getElementById("signupForm");
const nameInput = document.getElementById("name");
const emailInput = document.getElementById("email");
const passInput = document.getElementById("password");
const confirmInput = document.getElementById("confirm");
const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
const passError = document.getElementById("passwordError");
const confirmError = document.getElementById("confirmError");
const result = document.getElementById("result");

```

- **Define message display and clear helpers:**

```

function showError(el, message) {
  el.innerHTML = message;
}

function clearError(el) {
  el.innerHTML = "";
}

```

- **Intercept submits to prevent page reloading:**

```

form.addEventListener("submit", function (event) {

```

```

event.preventDefault();
result.innerHTML = "";

if (validateForm()) {
  result.innerHTML = "Form is valid!";
  result.className = "ok";
} else {
  result.innerHTML = "Please fix the errors.";
  result.className = "error";
}
});

```

Example 3

Result [View Example](#)

Step 4 - Validate Fields

Implement individual validation checks:

- **Name field:** Must contain at least 2 characters.

```

function validateName() {
  let value = nameInput.value.trim();
  if (value.length < 2) {
    showError(nameError, "Name must be at least 2 characters.");
    return false;
  }
  clearError(nameError);
  return true;
}

```

- **Email field:** Verified using a simple regex pattern.

```

function validateEmail() {
  let value = emailInput.value.trim();
  if (!(/^[\^s@]+@[^\s@]+\.[^\s@]+$/).test(value)) {
    showError(emailError, "Enter a valid email address.");
    return false;
  }
  clearError(emailError);
  return true;
}

```

- **Password field:** Must contain at least 8 characters.

```

function validatePassword() {
  let value = passInput.value;
  if (value.length < 8) {

```

```

    showError(passError, "Password must be at least 8 characters.");
    return false;
  }
  clearError(passError);
  return true;
}

```

- **Confirm Password:** Matches the password input.

```

function validateConfirm() {
  let pass = passInput.value;
  let confirm = confirmInput.value;
  if (confirm === "") {
    showError(confirmError, "Please confirm your password.");
    return false;
  }
  if (confirm !== pass) {
    showError(confirmError, "Passwords do not match.");
    return false;
  }
  clearError(confirmError);
  return true;
}

```

- **Final Form validator:**

```

function validateForm() {
  let okName = validateName();
  let okEmail = validateEmail();
  let okPass = validatePassword();
  let okConfirm = validateConfirm();
  return okName && okEmail && okPass && okConfirm;
}

```

Example 4

Result [View Example](#)

Exercises & Solutions

Exercise 1

Show an alert or custom styling if something is wrong.

Exercise 2

Check email format using regex (already covered in Step 4).

Exercise 3

Show a green success message next to fields that validate successfully.

Solutions Code

Update display functions to show success styles when parameters match successfully:

```
function showError(el, message) {
  el.innerHTML = message;
  el.className = "error";
}

function showSuccess(el, message) {
  el.innerHTML = message;
  el.className = "ok";
}

function validateName() {
  let value = nameInput.value.trim();
  if (value.length < 2) {
    showError(nameError, "Name must be at least 2 characters.");
    return false;
  }
  showSuccess(nameError, "Name is valid.");
  return true;
}
```

Example 5

Result [View Example](#)

Bonus Challenges (Level Up)

Elevate the interface with advanced real-world features:

- **Real-time inline validation:** Bind validators directly to `input` and `blur` events so users receive instant feedback.
- **Complex password rules checklist:** Test and display active checks for length, numbers, and uppercase letters.
- **Password visibility toggler:** Show or hide password inputs dynamically.

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Form Validation Project](#)

JavaScript HTML DOM

Table of Contents

- [JavaScript HTML DOM](#)
- [HTML Document Object Model](#)
- [Example 1](#)
- [The DOM Tree](#)
- [Accessing HTML Elements](#)
- [Example 2](#)
- [Reference](#)

HTML Document Object Model

The **HTML DOM** is a standard object model and programming interface for HTML. It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

In other words: The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTML DOM</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The DOM Tree

The HTML DOM model is constructed as a tree of objects:

- **Document:** The root of the tree
- **HTML element:** `<html>`

- **Head element:** `<head>`
- **Body element:** `<body>`
- **Text nodes:** Content within elements

With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

Accessing HTML Elements

JavaScript can access HTML elements using different methods:

- `document.getElementById(id)` - Find an element by id
- `document.getElementsByTagName(name)` - Find elements by tag name
- `document.getElementsByClassName(name)` - Find elements by class name

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Accessing HTML Elements</h4>
<p id="intro">Finding HTML Elements by Id</p>
<p id="demo"></p>

<script>
const element = document.getElementById("intro");
document.getElementById("demo").innerHTML =
  "Text from intro paragraph: " + element.innerHTML;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML DOM](#)

JavaScript HTML DOM Methods

Table of Contents

- [JavaScript HTML DOM Methods](#)
- [Application Programming Interface](#)
- [Example 1](#)
- [Selecting HTML Elements](#)
- [Example 2](#)
- [Accessing Element Content](#)
- [Example 3](#)
- [Changing Element Attributes](#)
- [Example 4](#)
- [Reference](#)

Application Programming Interface

The HTML DOM can be accessed with JavaScript (and with other programming languages).

In the DOM, all HTML elements are defined as **objects**.

The programming interface is the properties and methods of each object.

A **property** is a value that you can get or set (like changing the content of an HTML element).

A **method** is an action you can do (like add or deleting an HTML element).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>DOM API</h4>
<p id="demo">Original content</p>

<script>
document.getElementById("demo").innerHTML = "Content changed by getElementById!";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Selecting HTML Elements

Common methods for selecting HTML elements:

- `document.getElementById(id)` - Get element by id
- `document.getElementsByTagName(name)` - Get elements by tag name
- `document.getElementsByClassName(name)` - Get elements by class name
- `document.querySelector(selector)` - Get first element matching CSS selector
- `document.querySelectorAll(selector)` - Get all elements matching CSS selector

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Selecting HTML Elements</h4>
<p class="myClass">First paragraph with class="myClass"</p>
<p class="myClass">Second paragraph with class="myClass"</p>
<p id="demo"></p>

<script>
const elements = document.getElementsByClassName("myClass");
document.getElementById("demo").innerHTML =
    "Number of elements with class 'myClass': " + elements.length;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Accessing Element Content

The `innerHTML` property is the easiest way to get or change the content of an element.

Other ways to access content include:

- `element.innerHTML` - Get or set HTML content
- `element.textContent` - Get or set text content
- `element.innerText` - Get or set visible text content

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Accessing Element Content</h4>
```

```

<p id="intro">Hello <b>World</b>!</p>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const element = document.getElementById("intro");
document.getElementById("demo1").innerHTML = "innerHTML: " + element.innerHTML;
document.getElementById("demo2").innerHTML = "textContent: " + element.textContent;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Changing Element Attributes

You can change HTML attributes using the DOM:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Changing Element Attributes</h4>
<a id="myLink" href="https://www.w3schools.com">Visit W3Schools</a>
<p id="demo"></p>

<script>
const link = document.getElementById("myLink");
document.getElementById("demo").innerHTML = "Original href: " +
link.getAttribute("href");
// Change the href attribute
link.setAttribute("href", "https://www.google.com");
document.getElementById("demo").innerHTML += "<br>New href: " +
link.getAttribute("href");
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML DOM Methods](#)

JavaScript HTML DOM Elements

Table of Contents

- [JavaScript HTML DOM Elements](#)
- [Finding HTML Element by Id](#)
- [Example 1](#)
- [Finding HTML Elements by Tag Name](#)
- [Example 2](#)
- [Finding HTML Elements by Class Name](#)
- [Example 3](#)
- [The querySelector\(\) Method](#)
- [Example 4](#)
- [The querySelectorAll\(\) Method](#)
- [Example 5](#)
- [Finding HTML Elements by HTML Object Collections](#)
- [Example 6](#)
- [Reference](#)

Finding HTML Element by Id

The easiest way to find an HTML element in the DOM is by using the element id.

The `document.getElementById(id)` method is one of the most common methods in the HTML DOM:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Elements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Find Element by Id</h4>
<p id="intro">Hello World!</p>
<p id="demo"></p>

<script>
const element = document.getElementById("intro");
document.getElementById("demo").innerHTML =
  "Text: " + element.innerHTML;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Finding HTML Elements by Tag Name

The `document.getElementsByTagName(name)` method returns all elements with a specified tag name as an `HTMLCollection`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Elements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Find Elements by Tag Name</h4>
<p>First paragraph</p>
<p>Second paragraph</p>
<p>Third paragraph</p>
<p id="demo"></p>

<script>
const elements = document.getElementsByTagName("p");
document.getElementById("demo").innerHTML =
  "Number of <p> elements: " + elements.length;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Finding HTML Elements by Class Name

The `document.getElementsByClassName(name)` method returns all elements with a specified class name as an `HTMLCollection`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Elements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Find Elements by Class Name</h4>
<p class="intro">First paragraph with class 'intro'</p>
<p>Second paragraph (no class)</p>
<p class="intro">Third paragraph with class 'intro'</p>
<p id="demo"></p>

<script>
const elements = document.getElementsByClassName("intro");
```

```
document.getElementById("demo").innerHTML =
  "Elements with class 'intro': " + elements.length +
  "<br>First: " + elements[0].innerHTML;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The querySelector() Method

The `document.querySelector(selector)` method returns the **first** HTML element that matches a CSS selector:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Elements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The querySelector() Method</h4>
<p class="example">First paragraph with class 'example'</p>
<p class="example">Second paragraph with class 'example'</p>
<p id="demo"></p>

<script>
const element = document.querySelector(".example");
document.getElementById("demo").innerHTML =
  "First element with class 'example': " + element.innerHTML;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The querySelectorAll() Method

The `document.querySelectorAll(selector)` method returns **all** HTML elements that match a CSS selector as a NodeList:

```
<!DOCTYPE html>
<html>
```

```

<body>

<h2>JavaScript HTML DOM Elements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The querySelectorAll() Method</h4>
<p class="example">First paragraph with class 'example'</p>
<p class="example">Second paragraph with class 'example'</p>
<p id="demo"></p>

<script>
const elements = document.querySelectorAll(".example");
document.getElementById("demo").innerHTML =
  "Number of elements: " + elements.length +
  "<br>First: " + elements[0].innerHTML +
  "<br>Second: " + elements[1].innerHTML;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Finding HTML Elements by HTML Object Collections

HTML object collections are built-in collections like `document.forms`, `document.images`, `document.links`, etc:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Elements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTML Object Collections</h4>
<form id="frm1">
  First name: <input type="text" name="fname" value="John"><br>
  Last name: <input type="text" name="lname" value="Doe">
</form>
<p id="demo"></p>

<script>
const forms = document.forms;
const form = forms[0];
let text = "";
for (let i = 0; i < form.length; i++) {
  text += form.elements[i].name + ": " + form.elements[i].value + "<br>";
}
document.getElementById("demo").innerHTML = "Form fields:<br>" + text;

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML DOM Elements](#)

JavaScript HTML DOM Content

Table of Contents

- [JavaScript HTML DOM Content](#)
- [Changing HTML Content](#)
- [Example 1](#)
- [The innerHTML Property](#)
- [Example 2](#)
- [Changing an Attribute](#)
- [Example 3](#)
- [Dynamic HTML content](#)
- [Example 4](#)
- [document.write\(\)](#)
- [Example 5](#)
- [Reference](#)

Changing HTML Content

The easiest way to change the content of an HTML element is by using the `innerHTML` property.

To change the content of an HTML element, use this syntax:

```
document.getElementById(id).innerHTML = new text
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Content</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Changing HTML Content</h4>
<p id="demo">Original text</p>
<button type="button" onclick="document.getElementById('demo').innerHTML = 'Hello
JavaScript!'">Click Me!</button>

</body>
</html>
```

Example 1

Result [View Example](#)

The innerHTML Property

The `innerHTML` property can be used to get or change any HTML element, including `<html>` and `<body>`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Content</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The innerHTML Property</h4>
<p id="demo1">First paragraph</p>
<p id="demo2">Second paragraph</p>

<script>
document.getElementById("demo1").innerHTML = "Content changed!";
document.getElementById("demo2").innerHTML = "<b>Bold text</b> with HTML tags";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Changing an Attribute

To change an HTML attribute, use this syntax:

```
document.getElementById(id).attribute = new value
```

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Content</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Changing an Attribute</h4>

<p>
<button type="button"
onclick="document.getElementById('image').src='https://www.w3schools.com/js/pic_bulbon
.gif'">Turn on the light</button>
<button type="button"
onclick="document.getElementById('image').src='https://www.w3schools.com/js/pic_bulbof
f.gif'">Turn off the light</button>
</p>

</body>
</html>

```

Example 3

Result [View Example](#)

Dynamic HTML content

JavaScript can create dynamic HTML content:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Content</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Dynamic HTML content</h4>
<p id="demo"></p>

<script>
const date = new Date();
document.getElementById("demo").innerHTML = "Today's date: " + date;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

document.write()

The `document.write()` method can be used to write directly to the HTML output stream:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Content</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>document.write()</h4>
<p>Note: document.write() clears the document if used after the document has
loaded.</p>

<script>
document.write("Hello from document.write!");
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML DOM Content](#)

JavaScript HTML DOM CSS

Table of Contents

- [JavaScript HTML DOM CSS](#)
- [Changing HTML Style](#)
- [Example 1](#)
- [Using Events](#)
- [Example 2](#)
- [More Examples](#)
- [Example 3](#)
- [Reference](#)

Changing HTML Style

To change the style of an HTML element, use this syntax:

```
document.getElementById(id).style.property = new style
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM CSS</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Changing HTML Style</h4>
<p id="demo">This text will change color!</p>

<script>
document.getElementById("demo").style.color = "red";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using Events

The HTML DOM allows you to execute code when an event occurs. Events are generated by the browser when "things happen" to HTML elements.

You can change the style of an element when a user clicks a button:

```
<!DOCTYPE html>
<html>
```

```
<body>

<h2>JavaScript HTML DOM CSS</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Events</h4>
<p id="demo">Click the button to change style!</p>
<button type="button"
onclick="document.getElementById('demo').style.fontSize='35px'">Click Me!</button>

</body>
</html>
```

Example 2

Result [View Example](#)

More Examples

You can change multiple style properties and make elements appear/disappear:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM CSS</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>More Examples</h4>
<p id="demo" style="display:block">This paragraph will disappear</p>

<button type="button"
onclick="document.getElementById('demo').style.display='none'">Hide text</button>
<button type="button"
onclick="document.getElementById('demo').style.display='block'">Show text</button>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML DOM CSS](#)

JavaScript Validation

Table of Contents

- [JavaScript Validation](#)
- [JavaScript Form Validation](#)
- [Example 1](#)
- [JavaScript Can Validate Numeric Input](#)
- [Example 2](#)
- [Automatic HTML Form Validation](#)
- [Example 3](#)
- [Data Validation](#)
- [HTML Constraint Validation](#)
- [Reference](#)

JavaScript Form Validation

HTML form validation can be done by JavaScript.

If a form field (fname) is empty, the **required** attribute prevents form submission:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Form Validation</h4>
<form action="/action_page.php" method="post">
  <label for="fname">First name:</label>
  <input type="text" id="fname" name="fname" required>
  <input type="submit" value="Submit">
</form>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Can Validate Numeric Input

JavaScript is often used to validate numeric input:

```
<!DOCTYPE html>
<html>
```

```

<body>

<h2>JavaScript Validation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Validate Numeric Input</h4>
<p>Please input a number between 1 and 10:</p>
<input id="numb">
<button type="button" onclick="myFunction()">Submit</button>
<p id="demo"></p>

<script>
function myFunction() {
  let x = document.getElementById("numb").value;
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Automatic HTML Form Validation

HTML form validation can be performed automatically by the browser using the **required** attribute:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Automatic HTML Form Validation</h4>
<form action="/action_page.php" method="post">
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>
  <input type="submit" value="Submit">
</form>
<p>The required attribute prevents submission if the field is empty.</p>

</body>
</html>

```


Example 3

Result [View Example](#)

Data Validation

Data validation is the process of ensuring that user input is clean, correct, and useful.

Typical validation tasks:

- Has the user filled in all required fields?
- Has the user entered a valid date?
- Has the user entered text in a numeric field?
- Has the user entered a valid email address?

Constraint Validation HTML Input Attributes

Attribute	Description
<code>required</code>	Specifies that an input field must be filled in
<code>min</code> / <code>max</code>	Specifies min/max value of an input field
<code>minlength</code> / <code>maxlength</code>	Specifies min/max length of text input
<code>type</code>	Specifies the type of input (email, number, etc.)
<code>pattern</code>	Specifies a regex pattern for input validation

HTML Constraint Validation

Constraint validation in HTML is based on:

- **Constraint validation HTML Input Attributes** - described above
- **Constraint validation CSS Pseudo Selectors** - style form elements based on validity:
- `:valid` - Selects form elements with a valid value
- `:invalid` - Selects form elements with an invalid value

Example: Input fields with `type="email"` will automatically be validated when the form is submitted. The `:valid` and `:invalid` CSS pseudo-classes allow you to style valid/invalid fields.

Reference

- [W3Schools JavaScript Validation](#)

JavaScript Events

Table of Contents

- [JavaScript Events](#)
- [HTML Events](#)
- [Example 1](#)
- [Calling a JavaScript Function](#)
- [Example 2](#)
- [Common HTML Events](#)
- [Using an Event Listener](#)
- [Example 3](#)
- [Reference](#)

HTML Events

An HTML event can be something the browser does, or something a user does.

Here are some examples of HTML events:

- An HTML web page has finished loading
- An HTML input field was changed
- An HTML button was clicked

Often, when events occur, you may want to do something.

JavaScript lets you execute code when events are detected.

HTML allows event handler attributes, with JavaScript code, to be added to HTML elements.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTML Events</h4>
<p>Click the button to display the date.</p>
<button onclick="document.getElementById('demo').innerHTML = Date()">The time
is?</button>
<p id="demo"></p>

</body>
</html>
```

Example 1

Result [View Example](#)

Calling a JavaScript Function

You can call a JavaScript function when an event occurs, such as when a user clicks on an element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Calling a JavaScript Function</h4>
<p id="demo">Click the button to call a function.</p>
<button onclick="myFunction()">Click me</button>

<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Function called!";
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Common HTML Events

Here is a list of some common HTML events:

Event	Description
<code>onchange</code>	An HTML element has been changed
<code>onclick</code>	The user clicks an HTML element
<code>onmouseover</code>	The user moves the mouse over an HTML element
<code>onmouseout</code>	The user moves the mouse away from an HTML element
<code>onkeydown</code>	The user pushes a keyboard key
<code>onload</code>	The browser has finished loading the page

Using an Event Listener

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element. You can also add many event handlers of the same type to one element.

You can add event listeners to any DOM object not only HTML elements. i.e. the window object.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using an Event Listener</h4>
<button id="myBtn">Try it</button>
<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    document.getElementById("demo").innerHTML = "Button clicked using
addEventListener!";
});
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Events](#)

JavaScript Mouse Events

Table of Contents

- [JavaScript Mouse Events](#)
- [Mouseover and Mouseout](#)
- [Example 1](#)
- [Common Mouse Events](#)
- [Mouse Position](#)
- [Example 2](#)
- [Reference](#)

Mouseover and Mouseout

The **onmouseover** and **onmouseout** events can be used to trigger a function when the user hovers over or leaves an HTML element.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Mouse Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mouseover and Mouseout</h4>
<div onmouseover="mOver(this)" onmouseout="mOut(this)"
style="background-color:#D94A38;width:200px;height:50px;padding:40px;color:white;text-
align:center;font-size:20px">
Mouse Over Me
</div>

<script>
function mOver(obj) {
    obj.innerHTML = "Thank You"
}
function mOut(obj) {
    obj.innerHTML = "Mouse Over Me"
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Common Mouse Events

Here is a list of common mouse events:

Event	Description
<code>onclick</code>	The user clicks an element
<code>ondblclick</code>	The user double-clicks an element
<code>onmousedown</code>	A mouse button is pressed over an element
<code>onmouseup</code>	A mouse button is released over an element
<code>onmouseover</code>	The mouse pointer moves onto an element
<code>onmouseout</code>	The mouse pointer moves out of an element
<code>onmousemove</code>	The mouse pointer moves over an element
<code>onmouseenter</code>	Similar to <code>mouseover</code> but does not bubble

Mouse Position

You can get the mouse cursor position using `clientX` and `clientY` properties from the event object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Mouse Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mouse Position</h4>
<div onmousemove="getPosition(event)"
style="background-color:#D94A38;width:300px;height:150px;padding:20px;color:white;text-align:center;font-size:18px">
Move mouse over this box
</div>
<p id="demo"></p>

<script>
function getPosition(event) {
  let x = event.clientX;
  let y = event.clientY;
  document.getElementById("demo").innerHTML = "Mouse position: X=" + x + ", Y=" + y;
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Reference

- [W3Schools JavaScript Mouse Events](#)

JavaScript Keyboard Events

Table of Contents

- [JavaScript Keyboard Events](#)
- [The keydown Event](#)
- [Example 1](#)
- [Key Properties](#)
- [Detect Enter](#)
- [Example 2](#)
- [Reference](#)

The keydown Event

The **onkeydown** event occurs when the user presses a key on the keyboard.

You can use **event.key** to get the value of the key that was pressed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Keyboard Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The keydown Event</h4>
<p>Press a key in the input field:</p>
<input type="text" id="myInput" placeholder="Type something...">
<p id="demo"></p>

<script>
document.getElementById("myInput").addEventListener("keydown", function(event) {
    document.getElementById("demo").innerHTML = "Key pressed: " + event.key;
});
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Key Properties

Keyboard events have the following important properties:

Property	Description
----------	-------------

<code>event.key</code>	Returns the key value (character) of the key pressed
<code>event.code</code>	Returns the physical key code (e.g. "Enter", "KeyA")
<code>event.altKey</code>	Returns true if the ALT key was pressed
<code>event.ctrlKey</code>	Returns true if the CTRL key was pressed
<code>event.shiftKey</code>	Returns true if the SHIFT key was pressed
<code>event.metaKey</code>	Returns true if the META key (Windows key / Cmd) was pressed

Detect Enter

You can detect if the Enter key was pressed by checking `event.code === "Enter"`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Keyboard Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Detect Enter</h4>
<input type="text" id="in01" placeholder="Press Enter">
<p id="demo"></p>

<script>
const in01 = document.getElementById("in01");
in01.addEventListener("keydown", function(event) {
  if (event.code === "Enter") {
    document.getElementById("demo").innerHTML = "Enter was pressed!";
  }
});
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Reference

- [W3Schools JavaScript Keyboard Events](#)

JavaScript Load Events

Table of Contents

- [JavaScript Load Events](#)
- [DOMContentLoaded](#)
- [Example 1](#)
- [Window Load](#)
- [Example 2](#)
- [Other Load Events](#)
- [Image Load](#)
- [Example 3](#)
- [Reference](#)

DOMContentLoaded

The **DOMContentLoaded** event fires when the initial HTML document has been completely loaded and parsed, without waiting for stylesheets, images, and subframes to finish loading.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Load Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>DOMContentLoaded</h4>
<p id="demo"></p>

<script>
document.addEventListener("DOMContentLoaded", function() {
    document.getElementById("demo").innerHTML = "DOM fully loaded and parsed!";
});
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Window Load

The **window.onload** event fires when the entire page loads including all content (images, CSS, scripts, etc.).

```
<!DOCTYPE html>
<html>
```

```
<body>

<h2>JavaScript Load Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Load</h4>
<p id="demo"></p>

<script>
window.onload = function() {
    document.getElementById("demo").innerHTML = "Window fully loaded (including
images)!"
};
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Other Load Events

Here are other load-related events:

Event	Description
<code>DOMContentLoaded</code>	Fires when the HTML document is loaded and parsed
<code>load</code>	Fires when the entire page (including resources) has loaded
<code>beforeunload</code>	Fires when the document is about to be unloaded
<code>unload</code>	Fires once a page has unloaded (or the browser window has been closed)
<code>error</code>	Fires when an error occurs while loading an external resource

Image Load

The `load` event can be used on images to know when an image has finished loading:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Load Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Image Load</h4>

<p id="demo"></p>

<script>
document.getElementById("myImg").onload = function() {
    document.getElementById("demo").innerHTML = "Image loaded successfully!";
};
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Load Events](#)

JavaScript Event Management

Table of Contents

- [JavaScript Event Management](#)
- [Adding Events](#)
- [Example 1](#)
- [Removing Events](#)
- [Example 2](#)
- [Blocking Events](#)
- [Example 3](#)
- [Reference](#)

Adding Events

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element. You can also add many event handlers of the same type to one element (e.g., two "click" events):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Events</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    document.getElementById("demo").innerHTML += "Button clicked!<br>";
});
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Removing Events

The `removeEventListener()` method removes event handlers that have been attached with the `addEventListener()` method.

To remove an event handler, the function used to add it must be a named (referenced) function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Removing Events</h4>
<button id="myBtn">Click me</button>
<button id="removeBtn">Remove event</button>
<p id="demo"></p>

<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Button was clicked!";
}

document.getElementById("myBtn").addEventListener("click", myFunction);

document.getElementById("removeBtn").addEventListener("click", function() {
    document.getElementById("myBtn").removeEventListener("click", myFunction);
    document.getElementById("demo").innerHTML = "Event handler removed!";
});
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Blocking Events

The `preventDefault()` method cancels the event if it is cancelable, meaning that the default action that belongs to the event will not occur.

For example, this can prevent a link from opening the URL, or prevent a form from submitting:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Blocking Events</h4>
<a id="myLink" href="https://www.w3schools.com">Go to W3Schools</a>
<p id="demo"></p>
```

```
<script>
document.getElementById("myLink").addEventListener("click", function(event) {
    event.preventDefault();
    document.getElementById("demo").innerHTML = "Link click was prevented!";
});
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Event Management](#)

JavaScript HTML DOM Events

Table of Contents

- [JavaScript HTML DOM Events](#)
- [Reacting to Events](#)
- [Example 1](#)
- [HTML Event Attributes](#)
- [Example 2](#)
- [Assign Events Using the HTML DOM](#)
- [Example 3](#)
- [The onload Event](#)
- [Example 4](#)
- [The oninput Event](#)
- [Example 5](#)
- [The onchange Event](#)
- [Example 6](#)
- [Reference](#)

Reacting to Events

A JavaScript can be executed when an event occurs, like when a user clicks on an HTML element.

To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reacting to Events</h4>
<button onclick="document.getElementById('demo').innerHTML=Date()">The time
is?</button>
<p id="demo"></p>

</body>
</html>
```

Example 1

Result [View Example](#)

HTML Event Attributes

To assign events to HTML elements you can use event attributes like **onclick**, **onmouseover**, etc.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTML Event Attributes</h4>
<p>Click the button to display a message.</p>
<button onclick="displayDate()">Click me</button>
<p id="demo"></p>

<script>
function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Assign Events Using the HTML DOM

The HTML DOM allows you to assign events to HTML elements using JavaScript:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Assign Events Using the HTML DOM</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
document.getElementById("myBtn").onclick = function() {
    document.getElementById("demo").innerHTML = "Button clicked!";
};
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The onload Event

The **onload** event is often used to check the visitor's browser type and cookie version.

The **onload** event can be used to deal with cookies:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The onload Event</h4>
<p id="demo"></p>

<script>
window.onload = function() {
    document.getElementById("demo").innerHTML = "Page fully loaded!";
};
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The oninput Event

The **oninput** event fires when a user writes something in an input field:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM Events</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The oninput Event</h4>
<p>Write something in the input field:</p>
<input type="text" id="myInput" oninput="myFunction()">
<p id="demo"></p>

<script>
function myFunction() {
    let x = document.getElementById("myInput").value;
    document.getElementById("demo").innerHTML = "You wrote: " + x;
}
```

```
}  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

The onchange Event

The **onchange** event fires when a user changes the value of an input element:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript HTML DOM Events</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>The onchange Event</h4>  
<p>Select a car:</p>  
<select id="mySelect" onchange="myFunction()">  
  <option value="Volvo">Volvo</option>  
  <option value="BMW">BMW</option>  
  <option value="Audi">Audi</option>  
</select>  
<p id="demo"></p>  
  
<script>  
function myFunction() {  
  let x = document.getElementById("mySelect").value;  
  document.getElementById("demo").innerHTML = "You selected: " + x;  
}  
</script>  
  
</body>  
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML DOM Events](#)

JavaScript Event Listener

Table of Contents

- [JavaScript Event Listener](#)
- [The addEventListener\(\) Method](#)
- [Example 1](#)
- [Add an Event Handler to an Element](#)
- [Example 2](#)
- [Add Many Event Handlers to the Same Element](#)
- [Example 3](#)
- [Add an Event Handler to the window Object](#)
- [Example 4](#)
- [Passing Parameters](#)
- [Example 5](#)
- [Event Bubbling or Event Capturing](#)
- [Example 6](#)
- [The removeEventListener\(\) Method](#)
- [Example 7](#)
- [Reference](#)

The addEventListener() Method

The `addEventListener()` method attaches an event handler to the specified element.

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element.

You can add many event handlers of the same type to one element, i.e two "click" events.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The addEventListener() Method</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    document.getElementById("demo").innerHTML = "Hello World!";
});
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Add an Event Handler to an Element

You can add an event handler to any DOM element using `addEventListener()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add an Event Handler to an Element</h4>
<p id="demo">Click the button to change text color.</p>
<button id="myBtn">Click me</button>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    document.getElementById("demo").style.color = "red";
});
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Add Many Event Handlers to the Same Element

The `addEventListener()` method allows you to add many events to the same element, without overwriting existing events:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add Many Event Handlers</h4>
<button id="myBtn">Try it</button>
<p id="demo1"></p>
```

```

<p id="demo2"></p>

<script>
let x = document.getElementById("myBtn");
x.addEventListener("click", function1);
x.addEventListener("click", function2);

function function1() {
  document.getElementById("demo1").innerHTML = "First function executed!";
}

function function2() {
  document.getElementById("demo2").innerHTML = "Second function executed!";
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Add an Event Handler to the window Object

The `addEventListener()` method allows you to add event listeners to any HTML DOM object, such as HTML elements, the HTML document, the window object, or other objects that support events, like the `xmlHttpRequest` object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add an Event Handler to the window Object</h4>
<p id="demo"></p>

<script>
window.addEventListener("resize", function() {
  document.getElementById("demo").innerHTML = "Window resized! Width: " +
window.innerWidth;
});
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Passing Parameters

When passing parameter values, use an anonymous function that calls the specified function with the parameters:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Passing Parameters</h4>
<button id="myBtn">Try it</button>
<p id="demo"></p>

<script>
let p1 = 5;
let p2 = 7;
document.getElementById("myBtn").addEventListener("click", function() {
  myFunction(p1, p2);
});

function myFunction(a, b) {
  document.getElementById("demo").innerHTML = "Result: " + (a * b);
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Event Bubbling or Event Capturing

There are two ways of event propagation in the HTML DOM: **bubbling** and **capturing**.

In **bubbling**, the inner most element's event is handled first and then the outer.

In **capturing**, the outer most element's event is handled first and then the inner.

With `addEventListener()`, you can specify the propagation type by using the `useCapture` parameter:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
```



```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Event Capturing</h4>
<div id="myDiv" style="background-color: coral; padding: 40px;">
  <button id="myBtn">Click me</button>
</div>
<p id="demo"></p>

<script>
document.getElementById("myDiv").addEventListener("click", function() {
  document.getElementById("demo").innerHTML += "DIV clicked (capturing)<br>";
}, true);

document.getElementById("myBtn").addEventListener("click", function() {
  document.getElementById("demo").innerHTML += "BUTTON clicked<br>";
}, true);
</script>

</body>
</html>

```

Example 6

Result [View Example](#)

The removeEventListener() Method

The `removeEventListener()` method removes event handlers that have been attached with the `addEventListener()` method:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The removeEventListener() Method</h4>
<button id="myBtn">Click me</button>
<button id="removeBtn">Remove event</button>
<p id="demo"></p>

<script>
function myFunction() {
  document.getElementById("demo").innerHTML += "Button clicked!<br>";
}

document.getElementById("myBtn").addEventListener("click", myFunction);

document.getElementById("removeBtn").addEventListener("click", function() {
  document.getElementById("myBtn").removeEventListener("click", myFunction);
  document.getElementById("demo").innerHTML = "Event handler removed!";
});

```

```
});  
</script>  
  
</body>  
</html>
```

Example 7

Result [View Example](#)

Reference

- [W3Schools JavaScript Event Listener](#)

JavaScript HTML-First

Table of Contents

- [JavaScript HTML-First](#)
- [What Is HTML-First?](#)
- [Example 1](#)
- [Progressive Enhancement](#)
- [Example 2](#)
- [Browsers Are Already Powerful](#)
- [Example 3](#)
- [Semantic HTML Improves Accessibility](#)
- [Example 4](#)
- [Reference](#)

What Is HTML-First?

HTML-First is an approach to web development where you start with HTML to handle basic functionality before adding JavaScript.

This means:

- Use HTML form validation before adding JavaScript validation
- Use CSS for styling before using JavaScript for animations
- Write semantic HTML that works without JavaScript
- Add JavaScript only when necessary to enhance the user experience

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML-First</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What Is HTML-First?</h4>
<p>HTML-first means starting with HTML before adding JavaScript.</p>

<!-- HTML handles the form validation -->
<form action="/action_page.php" method="post">
  <label for="fname">First name (required):</label>
  <input type="text" id="fname" name="fname" required>
  <input type="submit" value="Submit">
</form>
<p>The required attribute is pure HTML validation - no JavaScript needed.</p>

</body>
</html>
```

Example 1

Result [View Example](#)

Progressive Enhancement

Progressive enhancement means starting with a basic HTML version that works everywhere, then adding JavaScript to enhance the experience for browsers that support it:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML-First</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Progressive Enhancement</h4>

<!-- Basic HTML structure works without JavaScript -->
<details>
  <summary>Click to toggle content (HTML only)</summary>
  <p>This content can be shown/hidden without JavaScript using the HTML <details>
element.</p>
</details>

<p id="demo"></p>

<script>
// JavaScript enhancement: Show a message if JavaScript is enabled
document.getElementById("demo").innerHTML = "JavaScript is enabled - page enhanced!";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Browsers Are Already Powerful

Modern browsers have built-in HTML features that reduce the need for JavaScript:

- `<input type="date">` - Date picker without JavaScript
- `<input type="range">` - Slider without JavaScript
- `<input type="email">` - Email validation without JavaScript
- `<details>` - Expandable content without JavaScript

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript HTML-First</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Browsers Are Already Powerful</h4>

<p>HTML date picker (no JavaScript needed):</p>
<input type="date">

<p>HTML range slider (no JavaScript needed):</p>
<input type="range" min="0" max="100">

<p>HTML color picker (no JavaScript needed):</p>
<input type="color">

</body>
</html>

```

Example 3

Result [View Example](#)

Semantic HTML Improves Accessibility

Semantic HTML elements help screen readers and assistive technologies understand the structure of your page. Using proper HTML elements is better than relying on JavaScript for accessibility:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML-First</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Semantic HTML Improves Accessibility</h4>

<!-- Using semantic HTML elements -->
<nav>
  <a href="#">Home</a> |
  <a href="#">About</a> |
  <a href="#">Contact</a>
</nav>

<main>
  <article>
    <h5>Article Title</h5>
    <p>This article uses semantic HTML elements like <article>, <nav>, and <main>.</p>
  </article>
</main>

<p id="demo">Semantic HTML works without JavaScript and improves accessibility.</p>

```

```
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML-First](#)

JavaScript Progressive Enhancement

Table of Contents

- [JavaScript Progressive Enhancement](#)
- [Start With HTML](#)
- [Example 1](#)
- [Add CSS for Better Design](#)
- [Example 2](#)
- [Add JavaScript as an Enhancement](#)
- [Example 3](#)
- [Graceful Degradation vs Progressive Enhancement](#)
- [Modern HTML Helps](#)
- [Reference](#)

Start With HTML

Progressive enhancement starts with a solid HTML foundation. Your web page should work and be accessible with HTML alone, before adding any CSS or JavaScript.

This ensures that:

- Content is accessible to all users and devices
- Search engines can index your content
- The page works in older browsers
- Screen readers can navigate the content

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Progressive Enhancement</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Start With HTML</h4>
<p>This page works with HTML only.</p>

<!-- HTML-only navigation -->
<nav>
  <a href="#">Home</a> |
  <a href="#">Products</a> |
  <a href="#">About</a> |
  <a href="#">Contact</a>
</nav>

<!-- HTML-only form -->
<form action="/action_page.php">
  <label for="email">Subscribe to newsletter:</label>
  <input type="email" id="email" name="email" required>
  <input type="submit" value="Subscribe">
</form>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Add CSS for Better Design

After the HTML foundation is solid, add CSS to improve the visual design and layout of the page. CSS enhances the appearance without affecting functionality:

```
<!DOCTYPE html>
<html>
<head>
<style>
.card {
  background-color: #f1f1f1;
  border-radius: 8px;
  padding: 20px;
  margin: 10px 0;
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}
.btn {
  background-color: #04AA6D;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
</style>
</head>
<body>

<h2>JavaScript Progressive Enhancement</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add CSS for Better Design</h4>

<div class="card">
  <h5>Card Title</h5>
  <p>This card is styled with CSS, but the HTML content is still accessible without it.</p>
  <button class="btn" onclick="alert('Button clicked!')">Click me</button>
</div>

</body>
</html>
```


Example 2

Result [View Example](#)

Add JavaScript as an Enhancement

After HTML and CSS are in place, add JavaScript to enhance the user experience. JavaScript should add functionality on top of what already works:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Progressive Enhancement</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add JavaScript as an Enhancement</h4>

<!-- HTML foundation: basic form validation -->
<form id="myForm" action="/action_page.php">
  <label for="fname">Name:</label>
  <input type="text" id="fname" name="fname" required>
  <br><br>
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>
  <br><br>
  <input type="submit" value="Submit">
</form>

<p id="demo"></p>

<script>
// JavaScript enhancement: add live validation feedback
let form = document.getElementById("myForm");
let fname = document.getElementById("fname");
let email = document.getElementById("email");

fname.addEventListener("input", function() {
  if (fname.value.length > 0) {
    fname.style.borderColor = "green";
  } else {
    fname.style.borderColor = "red";
  }
});

email.addEventListener("input", function() {
  if (email.value.includes("@")) {
    email.style.borderColor = "green";
    document.getElementById("demo").innerHTML = "Email looks valid!";
  } else if (email.value.length > 0) {
    email.style.borderColor = "orange";
    document.getElementById("demo").innerHTML = "Email needs @ symbol";
  } else {
    email.style.borderColor = "";
  }
});
```

```
        document.getElementById("demo").innerHTML = "";
    }
});
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Graceful Degradation vs Progressive Enhancement

Approach	Description
Progressive Enhancement	Start with a basic HTML version that works everywhere. Add CSS and JavaScript to enhance the experience for capable browsers. Content is always accessible.
Graceful Degradation	Build the full-featured version first (with JavaScript). Then try to make it work in older browsers. The user may get a broken experience if JavaScript fails.

Progressive enhancement is generally preferred because it ensures your content is always accessible, regardless of the user's browser or device.

Modern HTML Helps

Modern HTML5 provides many built-in features that reduce the need for JavaScript:

- `<input type="email">` - Built-in email validation
- `<input type="date">` - Native date picker
- `<input type="range">` - Native slider
- `<details>` / `<summary>` - Native expandable sections
- `<dialog>` - Native modal dialogs
- `<progress>` - Native progress bars

Using these HTML features means your page works without JavaScript, following the progressive enhancement approach.

Reference

- [W3Schools JavaScript Progressive Enhancement](#)

JavaScript HTML Features

Table of Contents

- [JavaScript HTML Features](#)
- [The details Element](#)
- [Example 1](#)
- [HTML Form Validation](#)
- [Example 2](#)
- [Input Types](#)
- [Example 3](#)
- [The datalist Element](#)
- [Example 4](#)
- [The dialog Element](#)
- [Example 5](#)
- [Lazy Loading Images](#)
- [Example 6](#)
- [Reference](#)

The details Element

The `<details>` element creates a disclosure widget that can be opened and closed without JavaScript:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The details Element</h4>

<details>
  <summary>Click to open (HTML only)</summary>
  <p>This content is shown/hidden using the HTML <details> element. No JavaScript
required!</p>
  <ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
  </ul>
</details>

</body>
</html>
```

Example 1

Result [View Example](#)

HTML Form Validation

HTML5 provides built-in form validation using attributes like **required**, **min**, **max**, **pattern**, etc.:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTML Form Validation</h4>

<form action="/action_page.php">
  <label for="fname">Name (required):</label>
  <input type="text" id="fname" name="fname" required>
  <br><br>

  <label for="age">Age (min=18, max=99):</label>
  <input type="number" id="age" name="age" min="18" max="99">
  <br><br>

  <label for="phone">Phone (pattern 10 digits):</label>
  <input type="tel" id="phone" name="phone" pattern="[0-9]{10}">
  <br><br>

  <input type="submit" value="Submit">
</form>

</body>
</html>
```

Example 2

Result [View Example](#)

Input Types

HTML5 introduced many new input types that provide native UI controls and validation:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Input Types</h4>

<form>
  <label>Color: <input type="color"></label><br><br>
```

```

<label>Date: <input type="date"></label><br><br>
<label>Range: <input type="range"></label><br><br>
<label>Email: <input type="email"></label><br><br>
<label>URL: <input type="url"></label><br><br>
<label>Search: <input type="search"></label><br><br>
<label>Tel: <input type="tel"></label><br><br>
<label>Number: <input type="number"></label>
</form>

</body>
</html>

```

Example 3

Result [View Example](#)

The datalist Element

The `<datalist>` element provides an autocomplete feature for input fields:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The datalist Element</h4>

<form>
  <label for="browser">Choose a browser:</label>
  <input list="browsers" id="browser" name="browser">
  <datalist id="browsers">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Edge">
    <option value="Safari">
    <option value="Opera">
  </datalist>
  <input type="submit">
</form>

</body>
</html>

```

Example 4

Result [View Example](#)

The dialog Element

The `<dialog>` element creates a native modal dialog without JavaScript:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The dialog Element</h4>

<dialog id="myDialog">
  <p>This is a native HTML dialog!</p>
  <button onclick="document.getElementById('myDialog').close()">Close</button>
</dialog>

<button onclick="document.getElementById('myDialog').showModal()">Open Dialog</button>

<p>The <dialog> element is a modern HTML feature for modal dialogs.</p>

</body>
</html>
```

Example 5

Result [View Example](#)

Lazy Loading Images

The `loading="lazy"` attribute tells the browser to defer loading of images until the user scrolls near them:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Lazy Loading Images</h4>

<p>The loading="lazy" attribute defers image loading until the user scrolls near the image.</p>



<p>Scroll down to see the lazy-loaded image below (if viewing on a small screen):</p>

<br><br><br><br><br><br><br><br><br>
```

```

```

```
</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript HTML Features](#)

JavaScript CSS Features

Table of Contents

- [JavaScript CSS Features](#)
- [Hover Effects](#)
- [Example 1](#)
- [Transitions](#)
- [Example 2](#)
- [Show and Hide Content](#)
- [Example 3](#)
- [Responsive Layouts](#)
- [Example 4](#)
- [CSS Animations](#)
- [Example 5](#)
- [Reference](#)

Hover Effects

Many interactive effects that previously required JavaScript can now be achieved with CSS alone using the `:hover` pseudo-class:

```
<!DOCTYPE html>
<html>
<head>
<style>
.btn {
  background-color: #04AA6D;
  color: white;
  padding: 15px 32px;
  text-align: center;
  font-size: 16px;
  border: none;
  cursor: pointer;
  transition: background-color 0.3s;
}
.btn:hover {
  background-color: #028a56;
}
.card {
  width: 200px;
  height: 100px;
  background-color: #f1f1f1;
  padding: 20px;
  transition: transform 0.3s;
}
.card:hover {
  transform: scale(1.1);
  box-shadow: 0 8px 16px rgba(0,0,0,0.2);
}
</style>
```



```
</head>
<body>

<h2>JavaScript CSS Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hover Effects (CSS only)</h4>
<button class="btn">Hover over me</button>
<br><br>
<div class="card">Hover over this card</div>

</body>
</html>
```

Example 1

Result [View Example](#)

Transitions

CSS transitions allow you to change property values smoothly over a given duration, without JavaScript:

```
<!DOCTYPE html>
<html>
<head>
<style>
.box {
  width: 100px;
  height: 100px;
  background-color: #04AA6D;
  transition: width 2s, height 2s, transform 2s;
}
.box:hover {
  width: 200px;
  height: 200px;
  transform: rotate(180deg);
}
</style>
</head>
<body>

<h2>JavaScript CSS Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Transitions (CSS only)</h4>
<div class="box"></div>
<p>Hover over the box to see the transition effect.</p>

</body>
</html>
```

Example 2

Result [View Example](#)

Show and Hide Content

CSS can show and hide content using pseudo-classes like `:hover`, `:focus`, or `:target`, without JavaScript:

```
<!DOCTYPE html>
<html>
<head>
<style>
.tooltip {
  position: relative;
  display: inline-block;
  border-bottom: 1px dotted black;
}
.tooltip .tooltip-text {
  visibility: hidden;
  width: 200px;
  background-color: #333;
  color: #fff;
  text-align: center;
  padding: 5px;
  border-radius: 6px;
  position: absolute;
  z-index: 1;
  bottom: 125%;
  left: 50%;
  margin-left: -100px;
  opacity: 0;
  transition: opacity 0.3s;
}
.tooltip:hover .tooltip-text {
  visibility: visible;
  opacity: 1;
}
.show-hide {
  display: none;
}
.toggle-btn:focus + .show-hide {
  display: block;
}
</style>
</head>
<body>

<h2>JavaScript CSS Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Show and Hide Content (CSS only)</h4>

<p>Tooltip example (CSS only):</p>
<div class="tooltip">Hover over me
  <span class="tooltip-text">This is a CSS tooltip! No JavaScript needed.</span>
```

```

</div>

<br><br>
<p>Click the button to show content (CSS only):</p>
<button class="toggle-btn">Click to toggle</button>
<div class="show-hide">This content is shown using CSS :focus pseudo-class!</div>

</body>
</html>

```

Example 3

Result [View Example](#)

Responsive Layouts

CSS provides powerful layout tools like Flexbox and Grid that create responsive designs without JavaScript:

```

<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-wrap: wrap;
  background-color: #f1f1f1;
  gap: 10px;
  padding: 10px;
}
.flex-container > div {
  background-color: #04AA6D;
  color: white;
  width: 150px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
  flex: 1 1 auto;
}
@media (max-width: 600px) {
  .flex-container > div {
    width: 100%;
  }
}
</style>
</head>
<body>

<h2>JavaScript CSS Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Responsive Layouts (CSS only)</h4>
<p>Resize the browser window to see the responsive effect.</p>

```

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>

</body>
</html>
```

Example 4

Result [View Example](#)

CSS Animations

CSS animations allow you to animate HTML elements without JavaScript:

```
<!DOCTYPE html>
<html>
<head>
<style>
@keyframes example {
  0% {background-color: red; left: 0px; top: 0px;}
  25% {background-color: yellow; left: 200px; top: 0px;}
  50% {background-color: blue; left: 200px; top: 200px;}
  75% {background-color: green; left: 0px; top: 200px;}
  100% {background-color: red; left: 0px; top: 0px;}
}
.animated-box {
  width: 100px;
  height: 100px;
  background-color: red;
  position: relative;
  animation-name: example;
  animation-duration: 4s;
  animation-iteration-count: infinite;
}
.bouncing-ball {
  width: 50px;
  height: 50px;
  background-color: #04AA6D;
  border-radius: 50%;
  position: relative;
  animation: bounce 2s infinite;
}
@keyframes bounce {
  0%, 100% { top: 0; }
  50% { top: 100px; }
}
</style>
</head>
<body>
```

```
<h2>JavaScript CSS Features</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>CSS Animations</h4>
<p>Moving box animation:</p>
<div class="animated-box"></div>
<br>
<p>Bouncing ball animation:</p>
<div class="bouncing-ball"></div>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript CSS Features](#)

JavaScript Function Definitions

Table of Contents

- [JavaScript Function Definitions](#)
- [Function Declarations](#)
- [Example 1](#)
- [Function Expressions](#)
- [Example 2](#)
- [Hoisting](#)
- [Example 3](#)
- [The Function\(\) Constructor](#)
- [Example 4](#)
- [Functions are Objects](#)
- [Example 5](#)
- [Reference](#)

Function Declarations

A JavaScript function is defined with the **function** keyword, followed by a name, followed by parentheses **()**.

Function declarations are hoisted - they can be called before they are defined:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Declaration</h4>
<p id="demo"></p>

<script>
// Function declaration can be called before it's defined
let result = myFunction(5);
document.getElementById("demo").innerHTML = "Result: " + result;

function myFunction(x) {
  return x * x;
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Function Expressions

A function expression can be stored in a variable. The function can be anonymous (without a name):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Expression</h4>
<p id="demo"></p>

<script>
const x = function(a, b) {
    return a * b;
};
document.getElementById("demo").innerHTML = "4 * 3 = " + x(4, 3);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Hoisting

Hoisting is JavaScript's default behavior of moving declarations to the top of the current scope.

Function declarations are hoisted to the top. Function expressions are NOT hoisted:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hoisting</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Function declaration is hoisted
document.getElementById("demo1").innerHTML = "Declaration result: " +
myDeclaration(5);

function myDeclaration(x) {
    return x * 2;
}
```

```

}

// Function expression is NOT hoisted
try {
  let result = myExpression(5);
  document.getElementById("demo2").innerHTML = "Expression result: " + result;
} catch(err) {
  document.getElementById("demo2").innerHTML = "Expression error: " + err.message;
}

const myExpression = function(x) {
  return x * 2;
};
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The Function() Constructor

Functions can also be defined using the built-in JavaScript function constructor `new Function()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Function() Constructor</h4>
<p id="demo"></p>

<script>
const myFunction = new Function("a", "b", "return a * b");
document.getElementById("demo").innerHTML = "4 * 3 = " + myFunction(4, 3);
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Functions are Objects

The **typeof** operator returns "function" for functions. But functions can be treated as objects:

- Functions have properties (like **length**, **name**)
- Functions have methods (like **toString()**, **call()**, **apply()**)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions are Objects</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
function myFunction(a, b) {
  return arguments.length;
}

document.getElementById("demo1").innerHTML = "typeof: " + typeof myFunction;
document.getElementById("demo2").innerHTML = "Function length: " + myFunction.length;
document.getElementById("demo3").innerHTML = "Function toString: " +
myFunction.toString();
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Function Definitions](#)

JavaScript Callbacks

Table of Contents

- [JavaScript Callbacks](#)
- [What is a Callback Function?](#)
- [Example 1](#)
- [Event Handling](#)
- [Example 2](#)
- [Asynchronous Operations](#)
- [Example 3](#)
- [Array Methods](#)
- [Example 4](#)
- [Sequence Control](#)
- [Example 5](#)
- [Reference](#)

What is a Callback Function?

A **callback** is a function passed as an argument to another function.

This technique allows a function to call another function after a task has been completed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is a Callback Function?</h4>
<p id="demo"></p>

<script>
function myCalculator(num, myCallback) {
  let result = num * 2;
  myCallback(result);
}

function displayResult(result) {
  document.getElementById("demo").innerHTML = "Result: " + result;
}

myCalculator(5, displayResult);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Event Handling

Callbacks are commonly used in event handlers. The `addEventListener()` method takes a callback function as its second parameter:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Event Handling</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
function handleClick() {
    document.getElementById("demo").innerHTML = "Button was clicked!";
}

document.getElementById("myBtn").addEventListener("click", handleClick);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Asynchronous Operations

Callbacks are essential for asynchronous operations. The `setTimeout()` function runs a callback after a delay:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Asynchronous Operations</h4>
<p id="demo"></p>

<script>
function showMessage() {
```

```

    document.getElementById("demo").innerHTML = "This message appears after 2 seconds!";
  }

  setTimeout(showMessage, 2000);
  document.getElementById("demo").innerHTML = "Waiting...";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Array Methods

Array methods like `forEach()` use callbacks to execute a function on each element:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Array Methods (forEach)</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5];
let text = "";

numbers.forEach(function(value, index) {
  text += "Index " + index + ": " + value + "<br>";
});

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Sequence Control

Callbacks can be used to control the sequence of function execution, ensuring one task completes before the next begins:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sequence Control</h4>
<p id="demo"></p>

<script>
function step1(callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 1 complete<br>";
    callback();
  }, 1000);
}

function step2(callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 2 complete<br>";
    callback();
  }, 1000);
}

function step3() {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 3 complete<br>";
    document.getElementById("demo").innerHTML += "All steps finished!";
  }, 1000);
}

// Chain the callbacks
step1(function() {
  step2(function() {
    step3();
  });
});
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Callbacks](#)

JavaScript this

Table of Contents

- [JavaScript this](#)
- [What is this?](#)
- [this in an Object Method](#)
- [Example 1](#)
- [this in a Function \(Default\)](#)
- [Example 2](#)
- [this Alone](#)
- [Example 3](#)
- [this in Event Handlers](#)
- [Example 4](#)
- [this in Arrow Functions](#)
- [Example 5](#)
- [Reference](#)

What is this?

In JavaScript, the **this** keyword refers to an **object**.

Which object depends on how **this** is being invoked (used or called).

The **this** keyword refers to different objects depending on how it is used:

- In an object method, **this** refers to the **object**
- Alone, **this** refers to the **global object**
- In a function, **this** refers to the **global object**
- In a function (in strict mode), **this** is **undefined**
- In an event, **this** refers to the **element** that received the event
- Methods like **call()**, **apply()**, and **bind()** can refer **this** to **any object**

this in an Object Method

When used in an object method, **this** refers to the object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in an Object Method</h4>
<p id="demo"></p>

<script>
const person = {
```

```

    firstName: "John",
    lastName: "Doe",
    id: 5566,
    fullName: function() {
        return this.firstName + " " + this.lastName;
    }
};

document.getElementById("demo").innerHTML = person.fullName();
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

this in a Function (Default)

In a regular function, **this** refers to the global object (window in a browser):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in a Function (Default)</h4>
<p id="demo"></p>

<script>
function myFunction() {
    return this;
}

document.getElementById("demo").innerHTML = myFunction();
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

this Alone

When used alone, **this** refers to the global object (window in a browser):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this Alone</h4>
<p id="demo"></p>

<script>
let x = this;
document.getElementById("demo").innerHTML = "this alone: " + x;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

this in Event Handlers

In HTML event handlers, **this** refers to the HTML element that received the event:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in Event Handlers</h4>
<button onclick="this.style.display='none'">Click to remove me!</button>
<br><br>
<button id="myBtn">Click to hide (using JS)</button>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    this.style.display = 'none';
});
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

this in Arrow Functions

Arrow functions do NOT have their own **this**. Instead, they inherit **this** from the surrounding scope.

This makes arrow functions useful in callbacks and methods where you want to preserve the context:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in Arrow Functions</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Regular function - has its own this
const obj1 = {
  name: "Object 1",
  greet: function() {
    return "Hello from " + this.name;
  }
};

// Arrow function - inherits this from surrounding scope
const obj2 = {
  name: "Object 2",
  greet: () => {
    return "Hello from " + this.name; // this is NOT obj2
  }
};

document.getElementById("demo1").innerHTML = obj1.greet();
document.getElementById("demo2").innerHTML = obj2.greet() + " (arrow function - this
is window)";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript this](#)

JavaScript Function call()

Table of Contents

- [JavaScript Function call\(\)](#)
- [Using call\(\) to Set this](#)
- [Example 1](#)
- [Borrowing a Method from Another Object](#)
- [Example 2](#)
- [The call\(\) Method with Arguments](#)
- [Example 3](#)
- [call\(\) vs Normal Function Call](#)
- [Example 4](#)
- [Reference](#)

Using call() to Set this

The `call()` method calls a function with a given `this` value and arguments provided individually.

With `call()`, you can write a method once and then inherit it in another object, without rewriting the method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using call() to Set this</h4>
<p id="demo"></p>

<script>
const person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const person1 = {
  firstName: "John",
  lastName: "Doe"
};

const person2 = {
  firstName: "Mary",
  lastName: "Smith"
};

document.getElementById("demo").innerHTML =
  person.fullName.call(person1) + "<br>" +
```

```
    person.fullName.call(person2);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Borrowing a Method from Another Object

You can use `call()` to borrow a method from one object and use it for another object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Borrowing a Method from Another Object</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const member = {
  firstName: "Jane",
  lastName: "Smith"
};

// Borrow the fullName method from person
document.getElementById("demo").innerHTML = person.fullName.call(member);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The call() Method with Arguments

The `call()` method can accept arguments:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The call() Method with Arguments</h4>
<p id="demo"></p>

<script>
const person = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName + ", " + city + ", " + country;
  }
};

const person1 = {
  firstName: "John",
  lastName: "Doe"
};

document.getElementById("demo").innerHTML = person.fullName.call(person1, "Oslo",
"Norway");
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

call() vs Normal Function Call

The difference between a normal function call and `call()` is that `call()` allows you to explicitly set what `this` refers to:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function call()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>call() vs Normal Function Call</h4>
<p id="demo1"></p>
<p id="demo2"></p>
```

```
<script>
function greet(greeting) {
  return greeting + ", " + this.name;
}

const user = {
  name: "Alice"
};

// Normal function call - "this" refers to window/global
// document.getElementById("demo1").innerHTML = greet("Hello"); // this.name would be
// undefined

// Using call() - "this" refers to user
document.getElementById("demo1").innerHTML = greet.call(user, "Hello");

const user2 = {
  name: "Bob"
};

document.getElementById("demo2").innerHTML = greet.call(user2, "Hi");
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Function call\(\)](#)

JavaScript Function apply()

Table of Contents

- [JavaScript Function apply\(\)](#)
- [Using apply\(\) to Set this](#)
- [Example 1](#)
- [The Difference Between call\(\) and apply\(\)](#)
- [Example 2](#)
- [The apply\(\) Method with Arguments](#)
- [Example 3](#)
- [Simulate Array Methods with apply\(\)](#)
- [Example 4](#)
- [Reference](#)

Using apply() to Set this

The `apply()` method is similar to `call()`. The difference is:

- `call()` expects arguments passed **separately**
- `apply()` expects arguments passed as an **array**

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function apply()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using apply() to Set this</h4>
<p id="demo"></p>

<script>
const person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const person1 = {
  firstName: "John",
  lastName: "Doe"
};

const person2 = {
  firstName: "Mary",
  lastName: "Smith"
};

document.getElementById("demo").innerHTML =
  person.fullName.apply(person1) + "<br>" +
```

```
    person.fullName.apply(person2);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Difference Between call() and apply()

The difference between `call()` and `apply()` is how arguments are passed:

- `call()`: arguments are passed **individually** (comma separated)
- `apply()`: arguments are passed as an **array**

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function apply()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Difference Between call() and apply()</h4>
<p id="demo"></p>

<script>
const person = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName + ", " + city + ", " + country;
  }
};

const person1 = {
  firstName: "John",
  lastName: "Doe"
};

// call() - arguments passed individually
let result1 = person.fullName.call(person1, "Oslo", "Norway");

// apply() - arguments passed as an array
let result2 = person.fullName.apply(person1, ["Oslo", "Norway"]);

document.getElementById("demo").innerHTML =
  "call(): " + result1 + "<br>" +
  "apply(): " + result2;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The apply() Method with Arguments

The `apply()` method accepts an array of arguments. This is useful when you have an array of data and want to pass it to a function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function apply()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The apply() Method with Arguments</h4>
<p id="demo"></p>

<script>
function greet(greeting, punctuation) {
  return greeting + ", " + this.name + punctuation;
}

const person = {
  name: "Alice"
};

const args = ["Hello", "!"];

document.getElementById("demo").innerHTML = greet.apply(person, args);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Simulate Array Methods with apply()

You can use `apply()` to simulate array methods like `Math.max()` and `Math.min()` on arrays:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function apply()</h2>
```



```
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Simulate Array Methods with apply()</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
const numbers = [4, 7, 1, 9, 3, 5];

// Math.max normally cannot accept an array directly
let max = Math.max.apply(null, numbers);

// Math.min similarly
let min = Math.min.apply(null, numbers);

document.getElementById("demo1").innerHTML = "Numbers: " + numbers;
document.getElementById("demo2").innerHTML = "Max value: " + max;
document.getElementById("demo3").innerHTML = "Min value: " + min;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Function apply\(\)](#)

JavaScript Function bind()

Table of Contents

- [JavaScript Function bind\(\)](#)
- [Using bind\(\) to Fix this](#)
- [Example 1](#)
- [bind\(\) for Functions Called Later](#)
- [Example 2](#)
- [Using bind\(\) for Preserving this](#)
- [Example 3](#)
- [bind\(\) with Arguments](#)
- [Example 4](#)
- [Reference](#)

Using bind() to Fix this

The `bind()` method creates a new function where `this` refers to the object passed as an argument.

With `bind()`, an object can borrow a method from another object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using bind() to Fix this</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const member = {
  firstName: "Jane",
  lastName: "Smith"
};

// Bind person's method to member object
let fullName = person.fullName.bind(member);
document.getElementById("demo").innerHTML = fullName();
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

bind() for Functions Called Later

The **bind()** method is useful when a function is called later (like in a callback) and you need **this** to refer to the correct object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>bind() for Functions Called Later</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  display: function() {
    document.getElementById("demo").innerHTML = this.firstName + " " + this.lastName;
  }
};

// Without bind - setTimeout would lose "this" context
// setTimeout(person.display, 3000); // this would be window

// With bind - "this" is properly preserved
setTimeout(person.display.bind(person), 3000);
document.getElementById("demo").innerHTML = "Waiting 3 seconds...";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using bind() for Preserving this

When you pass an object method as a callback, **this** is lost. **bind()** preserves it:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using bind() for Preserving this</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  display: function() {
    document.getElementById("demo").innerHTML = this.firstName + " " + this.lastName;
  }
};

// Without bind, "this" in the callback would refer to the button element
// document.getElementById("myBtn").addEventListener("click", person.display); //
Would show "undefined undefined"

// With bind, "this" correctly refers to person
document.getElementById("myBtn").addEventListener("click",
person.display.bind(person));
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

bind() with Arguments

bind() can also accept arguments, creating a new function with preset parameters (partial application):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>bind() with Arguments</h4>
<p id="demo"></p>

<script>

```

```
function multiply(a, b) {  
    return a * b;  
}  
  
// Create a new function by binding the first argument  
let multiplyBy2 = multiply.bind(this, 2);  
let multiplyBy10 = multiply.bind(this, 10);  
  
document.getElementById("demo").innerHTML =  
    "multiplyBy2(5): " + multiplyBy2(5) + "<br>" +  
    "multiplyBy10(5): " + multiplyBy10(5);  
</script>  
  
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Function bind\(\)](#)

JavaScript IIFE

Table of Contents

- [JavaScript IIFE](#)
- [What Is an IIFE?](#)
- [Example 1](#)
- [Avoid Polluting the Global Scope](#)
- [Example 2](#)
- [IIFE with Parameters](#)
- [Example 3](#)
- [IIFE with Return Value](#)
- [Example 4](#)
- [Arrow Function IIFE](#)
- [Example 5](#)
- [IIFE as a Module \(Private Variables\)](#)
- [Example 6](#)
- [Reference](#)

What Is an IIFE?

An **IIFE** (Immediately Invoked Function Expression) is a JavaScript function that runs as soon as it is defined.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript IIFE</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What Is an IIFE?</h4>
<p id="demo"></p>

<script>
(function() {
  document.getElementById("demo").innerHTML = "IIFE executed!";
})();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Avoid Polluting the Global Scope

IIFEs are commonly used to avoid polluting the global namespace. Variables declared inside an IIFE are not accessible outside:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript IIFE</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Avoid Polluting the Global Scope</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
(function() {
  let privateVar = "This is private";
  document.getElementById("demo1").innerHTML = "Inside IIFE: " + privateVar;
})();

// privateVar is NOT accessible here (would cause error)
document.getElementById("demo2").innerHTML = "Outside IIFE: privateVar is not
accessible";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

IIFE with Parameters

IIFEs can accept arguments, just like regular functions:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript IIFE</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>IIFE with Parameters</h4>
<p id="demo"></p>

<script>
(function(name, age) {
  document.getElementById("demo").innerHTML = "Hello " + name + ", you are " + age + "
years old.";
})("John", 30);
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 3

Result [View Example](#)

IIFE with Return Value

An IIFE can return a value that is assigned to a variable:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript IIFE</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>IIFE with Return Value</h4>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let result = (function() {
```

```
  let x = 5;
```

```
  let y = 10;
```

```
  return x * y;
```

```
})();
```

```
document.getElementById("demo").innerHTML = "Result: " + result;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 4

Result [View Example](#)

Arrow Function IIFE

IIFEs can also be written using arrow function syntax:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript IIFE</h2>
```



```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arrow Function IIFE</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Arrow function IIFE
(() => {
  document.getElementById("demo1").innerHTML = "Arrow IIFE executed!";
})();

// Arrow function IIFE with parameter
((message) => {
  document.getElementById("demo2").innerHTML = "Message: " + message;
})("Hello from arrow IIFE!");
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

IIFE as a Module (Private Variables)

IIFEs can be used to create modules with private variables and public methods:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript IIFE</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>IIFE as a Module</h4>
<p id="demo"></p>

<script>
const counterModule = (function() {
  let count = 0; // private variable

  return {
    increment: function() {
      count++;
      return count;
    },
    decrement: function() {
      count--;
      return count;
    },
  };
})();

```

```
        getCount: function() {
            return count;
        }
    };
})();

document.getElementById("demo").innerHTML =
    "Initial: " + counterModule.getCount() + "<br>" +
    "After increment: " + counterModule.increment() + "<br>" +
    "After increment: " + counterModule.increment() + "<br>" +
    "After decrement: " + counterModule.decrement();
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript IIFE](#)

JavaScript Closures

Table of Contents

- [JavaScript Closures](#)
- [Global and Local Variables](#)
- [Example 1](#)
- [A Counter Dilemma](#)
- [Example 2](#)
- [JavaScript Nested Functions](#)
- [Example 3](#)
- [JavaScript Closures](#)
- [Example 4](#)
- [Reference](#)

Global and Local Variables

A function can access all variables defined inside it (local), and all variables defined outside it (global).

A closure is a function that has access to the parent scope, even after the parent function has closed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Closures</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Global and Local Variables</h4>
<p id="demo"></p>

<script>
let a = 4;  // global variable

function myFunction() {
  let b = 2;  // local variable
  return a * b;
}

document.getElementById("demo").innerHTML = "a * b = " + myFunction();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

A Counter Dilemma

Suppose we want to use a variable to count something. We want this counter to be accessible to all functions, but we don't want it to be modified by other code.

Using a global variable won't work because it can be changed from anywhere:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Closures</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Counter Dilemma</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let counter = 0; // global counter - can be modified by anyone

function add() {
  counter += 1;
}

// Calling add() 3 times
add();
add();
add();

document.getElementById("demo1").innerHTML = "Counter: " + counter;

// Global variable can be accidentally changed
counter = 100;
document.getElementById("demo2").innerHTML = "Counter after modification: " + counter;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Nested Functions

A function can have nested functions. The inner function has access to the outer function's variables:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript Closures</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Nested Functions</h4>
<p id="demo"></p>

<script>
function outerFunction() {
  let outerVar = "I am from outer function";

  function innerFunction() {
    document.getElementById("demo").innerHTML = outerVar;
  }

  innerFunction();
}

outerFunction();
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JavaScript Closures

A closure is a function that remembers its outer variables and can access them.

In JavaScript, all functions are closures because they remember where they were created:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Closures</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Closures</h4>
<p id="demo"></p>

<script>
function createCounter() {
  let counter = 0; // private variable

  function increment() {
    counter += 1;
    return counter;
  }
}

```

```
    return increment; // return the inner function (closure)
}

const myCounter = createCounter();

document.getElementById("demo").innerHTML =
    myCounter() + "<br>" +
    myCounter() + "<br>" +
    myCounter();
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Closures](#)

JavaScript Object Definitions

Table of Contents

- [JavaScript Object Definitions](#)
- [Using an Object Literal](#)
- [Example 1](#)
- [Using the new Keyword](#)
- [Example 2](#)
- [JavaScript Object.create\(\)](#)
- [Example 3](#)
- [JavaScript Object.assign\(\)](#)
- [Example 4](#)
- [JavaScript Objects are Mutable](#)
- [Example 5](#)
- [Reference](#)

Using an Object Literal

The easiest way to create a JavaScript object is using an **object literal**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using an Object Literal</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  eyeColor: "blue"
};

document.getElementById("demo").innerHTML =
  person.firstName + " " + person.lastName + ", age " + person.age;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using the new Keyword

You can create an object using the `new Object()` constructor:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using the new Keyword</h4>
<p id="demo"></p>

<script>
const person = new Object();
person.firstName = "John";
person.lastName = "Doe";
person.age = 30;

document.getElementById("demo").innerHTML =
  person.firstName + " " + person.lastName + ", age " + person.age;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Object.create()

The `Object.create()` method creates a new object using an existing object as a prototype:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Object.create()</h4>
<p id="demo"></p>

<script>
// Define prototype object
const personProto = {
  greet: function() {
    return "Hello, I'm " + this.firstName;
  }
};
```



```
// Create new object based on prototype
const person = Object.create(personProto);
person.firstName = "John";
person.lastName = "Doe";

document.getElementById("demo").innerHTML = person.greet();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

JavaScript Object.assign()

The **Object.assign()** method copies properties from one or more source objects to a target object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Object.assign()</h4>
<p id="demo"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };
const address = { city: "New York", country: "USA" };
const contact = { email: "john@example.com" };

// Merge objects into a new object
const merged = Object.assign({}, person, address, contact);

document.getElementById("demo").innerHTML =
  merged.firstName + " " + merged.lastName +
  ", " + merged.city + ", " + merged.country +
  "<br>Email: " + merged.email;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript Objects are Mutable

Objects are mutable: they are accessed by reference, not by value.

If you change a property of a copied object, the original object is also changed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Objects are Mutable</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };

// Copy by reference (not by value)
const employee = person;
employee.firstName = "Jane";

document.getElementById("demo1").innerHTML = "person: " + person.firstName;
document.getElementById("demo2").innerHTML = "employee: " + employee.firstName;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Definitions](#)

JavaScript Object Definition

Table of Contents

- [JavaScript Object Definition](#)
- [Using an Object Literal](#)
- [Example 1](#)
- [Using the new Keyword](#)
- [Example 2](#)
- [JavaScript Object.create\(\)](#)
- [Example 3](#)
- [JavaScript Object.assign\(\)](#)
- [Example 4](#)
- [JavaScript Objects are Mutable](#)
- [Example 5](#)
- [Reference](#)

Using an Object Literal

The easiest way to create a JavaScript object is using an **object literal**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using an Object Literal</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  eyeColor: "blue"
};

document.getElementById("demo").innerHTML =
  person.firstName + " " + person.lastName + ", age " + person.age;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using the new Keyword

You can create an object using the `new Object()` constructor:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using the new Keyword</h4>
<p id="demo"></p>

<script>
const person = new Object();
person.firstName = "John";
person.lastName = "Doe";
person.age = 30;

document.getElementById("demo").innerHTML =
  person.firstName + " " + person.lastName + ", age " + person.age;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Object.create()

The `Object.create()` method creates a new object using an existing object as a prototype:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Object.create()</h4>
<p id="demo"></p>

<script>
// Define prototype object
const personProto = {
  greet: function() {
    return "Hello, I'm " + this.firstName;
  }
};
```

```
// Create new object based on prototype
const person = Object.create(personProto);
person.firstName = "John";
person.lastName = "Doe";

document.getElementById("demo").innerHTML = person.greet();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

JavaScript Object.assign()

The **Object.assign()** method copies properties from one or more source objects to a target object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Object.assign()</h4>
<p id="demo"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };
const address = { city: "New York", country: "USA" };
const contact = { email: "john@example.com" };

// Merge objects into a new object
const merged = Object.assign({}, person, address, contact);

document.getElementById("demo").innerHTML =
  merged.firstName + " " + merged.lastName +
  ", " + merged.city + ", " + merged.country +
  "<br>Email: " + merged.email;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript Objects are Mutable

Objects are mutable: they are accessed by reference, not by value.

If you change a property of a copied object, the original object is also changed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Objects are Mutable</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };

// Copy by reference (not by value)
const employee = person;
employee.firstName = "Jane";

document.getElementById("demo1").innerHTML = "person: " + person.firstName;
document.getElementById("demo2").innerHTML = "employee: " + employee.firstName;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Definition](#)

JavaScript this Keyword

Table of Contents

- [JavaScript this Keyword](#)
- [this in an Object Method](#)
- [Example 1](#)
- [Object Method Binding](#)
- [Example 2](#)
- [Explicit Function Binding](#)
- [Example 3](#)
- [Function Borrowing](#)
- [Example 4](#)
- [Reference](#)

What is this?

In JavaScript, the **this** keyword refers to an **object**.

The **this** keyword refers to different objects depending on how it is used:

- In an object method, **this** refers to the **object**
- Alone, **this** refers to the **global object**
- In a function, **this** refers to the **global object**
- In a function (strict mode), **this** is **undefined**
- In an event, **this** refers to the **element** that received the event
- Methods like **call()**, **apply()**, and **bind()** can refer **this** to **any object**

this in an Object Method

When used in an object method, **this** refers to the object itself:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this Keyword</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in an Object Method</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
```

```
};

document.getElementById("demo").innerHTML = person.fullName();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Object Method Binding

Object method binding refers to how **this** is bound when calling a method on an object.

When a function is called as a method of an object, **this** is bound to that object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this Keyword</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object Method Binding</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

// Method called on the object - "this" refers to person
document.getElementById("demo1").innerHTML = person.fullName();

// When method is extracted, "this" is lost
const myMethod = person.fullName;
document.getElementById("demo2").innerHTML = "Lost this: " + myMethod();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Explicit Function Binding

The `call()`, `apply()`, and `bind()` methods can explicitly set what `this` refers to:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this Keyword</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Explicit Function Binding</h4>
<p id="demo"></p>

<script>
const person1 = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const person2 = {
  firstName: "Jane",
  lastName: "Smith"
};

const person3 = {
  firstName: "Bob",
  lastName: "Johnson"
};

// Using call() to explicitly set "this"
document.getElementById("demo").innerHTML =
  "call() person2: " + person1.fullName.call(person2) + "<br>" +
  "call() person3: " + person1.fullName.call(person3);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Function Borrowing

Function borrowing allows an object to use a method from another object:

```
<!DOCTYPE html>
```

```

<html>
<body>

<h2>JavaScript this Keyword</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Borrowing</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const member = {
  firstName: "Jane",
  lastName: "Smith"
};

const member2 = {
  firstName: "Bob",
  lastName: "Johnson"
};

// Borrow the fullName method from person using bind()
const memberFullName = person.fullName.bind(member);
const member2FullName = person.fullName.bind(member2);

document.getElementById("demo").innerHTML =
  "Member 1: " + memberFullName() + "<br>" +
  "Member 2: " + member2FullName();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript this Keyword](#)

JavaScript Object Iterations

Table of Contents

- [JavaScript Object Iterations](#)
- [JavaScript Object.entries\(\)](#)
- [Example 1](#)
- [JavaScript Object.values\(\)](#)
- [Example 2](#)
- [JavaScript Object.keys\(\)](#)
- [Example 3](#)
- [JavaScript for...in Loop](#)
- [Example 4](#)
- [JavaScript Object.groupBy\(\)](#)
- [Example 5](#)
- [Reference](#)

JavaScript Object.entries()

`Object.entries()` returns an array of the key/value pairs of an object (as arrays):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Iterations</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.entries()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

const entries = Object.entries(person);
let text = "";
for (let entry of entries) {
  text += entry[0] + ": " + entry[1] + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Object.values()

Object.values() returns an array of the property values of an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Iterations</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.values()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

const values = Object.values(person);
document.getElementById("demo").innerHTML = "Values: " + values;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Object.keys()

Object.keys() returns an array of the property keys (names) of an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Iterations</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.keys()</h4>
<p id="demo"></p>

<script>
```

```

const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

const keys = Object.keys(person);
let text = "";
for (let key of keys) {
  text += key + "<br>";
}
document.getElementById("demo").innerHTML = "Keys:<br>" + text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JavaScript for...in Loop

The **for...in** loop iterates over the properties of an object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Iterations</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>for...in Loop</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

let text = "";
for (let key in person) {
  text += key + ": " + person[key] + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

JavaScript Object.groupBy()

`Object.groupBy()` groups elements of an array by a string value returned from a callback function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Iterations</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.groupBy()</h4>
<p id="demo"></p>

<script>
const fruits = [
  { name: "apple", color: "red" },
  { name: "banana", color: "yellow" },
  { name: "grape", color: "purple" },
  { name: "strawberry", color: "red" }
];

// Group fruits by color
const result = Object.groupBy(fruits, item => item.color);

let text = "";
for (let color in result) {
  text += color + ": " + result[color].map(f => f.name).join(", ") + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Iterations](#)

JavaScript Object Accessors

Table of Contents

- [JavaScript Object Accessors](#)
- [JavaScript Getter \(The get Keyword\)](#)
- [Example 1](#)
- [JavaScript Setter \(The set Keyword\)](#)
- [Example 2](#)
- [JavaScript Function or Getter?](#)
- [Example 3](#)
- [Data Quality](#)
- [Example 4](#)
- [Object.defineProperty\(\)](#)
- [Example 5](#)
- [Reference](#)

JavaScript Getter (The get Keyword)

JavaScript accessors (getters and setters) allow you to define object properties as functions.

The **get** keyword binds an object property to a function that will be called when that property is looked up:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Accessors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Getter</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  get fullName() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML = person.fullName;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Setter (The set Keyword)

The **set** keyword binds an object property to a function that will be called when an attempt is made to set that property:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Accessors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Setter</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  language: "",
  set lang(lang) {
    this.language = lang;
  }
};

person.lang = "English";
document.getElementById("demo").innerHTML = person.language;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Function or Getter?

Getters give you a simpler syntax. Compare a getter vs a regular function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Accessors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function vs Getter</h4>
<p id="demo1"></p>
```



```

<p id="demo2"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  },
  get name() {
    return this.firstName + " " + this.lastName;
  }
};

// Using function - needs parentheses
document.getElementById("demo1").innerHTML = "Function: " + person.fullName();

// Using getter - no parentheses needed
document.getElementById("demo2").innerHTML = "Getter: " + person.name;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Data Quality

Setters can be used to execute validation before setting a value:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Accessors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Data Quality</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 0,
  set setAge(a) {
    if (a > 0 && a < 120) {
      this.age = a;
    } else {
      console.log("Invalid age");
    }
  }
};

```

```

    }
  }
};

person.setAge = 25;
document.getElementById("demo").innerHTML = "Age: " + person.age;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Object.defineProperty()

The **Object.defineProperty()** method can also be used to define getters and setters:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Accessors</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.defineProperty()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe"
};

Object.defineProperty(person, "fullName", {
  get: function() {
    return this.firstName + " " + this.lastName;
  },
  set: function(value) {
    const parts = value.split(" ");
    this.firstName = parts[0];
    this.lastName = parts[1];
  }
});

// Use the getter
document.getElementById("demo").innerHTML = "Before: " + person.fullName + "<br>";

// Use the setter
person.fullName = "Jane Smith";
document.getElementById("demo").innerHTML += "After: " + person.fullName;

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Accessors](#)

JavaScript Object Management

Table of Contents

- [JavaScript Object Management](#)
- [Adding a New Property](#)
- [Example 1](#)
- [Changing a Property Value](#)
- [Example 2](#)
- [Property Attributes](#)
- [List All Object Properties](#)
- [Example 3](#)
- [Adding Getters and Setters](#)
- [Example 4](#)
- [A Counter Example](#)
- [Example 5](#)
- [Reference](#)

Adding a New Property

You can add new properties to an existing object using `Object.defineProperty()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding a New Property</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe"
};

// Add a new property
Object.defineProperty(person, "fullName", {
  value: "John Doe",
  writable: true
});

document.getElementById("demo").innerHTML = person.fullName;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Changing a Property Value

You can change a property value using `Object.defineProperty()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Changing a Property Value</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Change the age property
Object.defineProperty(person, "age", {
  value: 35
});

document.getElementById("demo").innerHTML = "Age: " + person.age;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Property Attributes

Property attributes control how properties behave:

- ``writable`` - If `true`, the value can be changed
- ``enumerable`` - If `true`, the property shows up in enumeration (`for...in`)
- ``configurable`` - If `true`, the property can be deleted or changed

List All Object Properties

`Object.getOwnPropertyNames()` returns all properties (including non-enumerable):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>List All Object Properties</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  eyeColor: "blue"
};

// Get all property names
const properties = Object.getOwnPropertyNames(person);
let text = "";
for (let prop of properties) {
  text += prop + ": " + person[prop] + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Adding Getters and Setters

`Object.defineProperty()` can also add getters and setters:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Getters and Setters</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",

```

```

    lastName: "Doe"
  };

  Object.defineProperty(person, "fullName", {
    get: function() {
      return this.firstName + " " + this.lastName;
    },
    set: function(value) {
      const parts = value.split(" ");
      this.firstName = parts[0];
      this.lastName = parts[1];
    },
    enumerable: true
  });

  document.getElementById("demo").innerHTML = "Before: " + person.fullName + "<br>";
  person.fullName = "Jane Smith";
  document.getElementById("demo").innerHTML += "After: " + person.fullName;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

A Counter Example

Using `Object.defineProperty()` to create a counter with controlled access:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>A Counter Example</h4>
<p id="demo"></p>

<script>
const obj = { counter: 0 };

Object.defineProperty(obj, "increment", {
  get: function() {
    this.counter++;
    return this.counter;
  }
});

Object.defineProperty(obj, "decrement", {

```

```
    get: function() {
        this.counter--;
        return this.counter;
    }
});

Object.defineProperty(obj, "reset", {
    get: function() {
        this.counter = 0;
        return this.counter;
    }
});

document.getElementById("demo").innerHTML =
    "Initial: " + obj.reset + "<br>" +
    "After increment: " + obj.increment + "<br>" +
    "After increment: " + obj.increment + "<br>" +
    "After decrement: " + obj.decrement + "<br>" +
    "After reset: " + obj.reset;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Management](#)

JavaScript Object Protection

Table of Contents

- [JavaScript Object Protection](#)
- [JavaScript Object.preventExtensions\(\)](#)
- [Example 1](#)
- [JavaScript Object.isExtensible\(\)](#)
- [Example 2](#)
- [JavaScript Object.seal\(\)](#)
- [Example 3](#)
- [JavaScript Object.isSealed\(\)](#)
- [Example 4](#)
- [JavaScript Object.freeze\(\)](#)
- [Example 5](#)
- [JavaScript Object.isFrozen\(\)](#)
- [Example 6](#)
- [Reference](#)

JavaScript Object.preventExtensions()

`Object.preventExtensions()` prevents new properties from being added to an object. Existing properties can still be changed or deleted:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Protection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.preventExtensions()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Prevent adding new properties
Object.preventExtensions(person);

// Try to add a new property
person.nationality = "American";

document.getElementById("demo").innerHTML =
  "firstName: " + person.firstName + "<br>" +
```

```
"nationality: " + person.nationality + " (undefined - cannot add)";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Object.isExtensible()

`Object.isExtensible()` checks if new properties can be added to an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Protection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.isExtensible()</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person1 = { firstName: "John" };
const person2 = { firstName: "Jane" };

Object.preventExtensions(person2);

document.getElementById("demo1").innerHTML =
  "person1 is extensible: " + Object.isExtensible(person1);
document.getElementById("demo2").innerHTML =
  "person2 is extensible: " + Object.isExtensible(person2);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Object.seal()

`Object.seal()` prevents adding or deleting properties. Existing properties can still be changed:

```
<!DOCTYPE html>
```

```

<html>
<body>

<h2>JavaScript Object Protection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.seal()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Seal the object
Object.seal(person);

// Try to add a new property (will be ignored in non-strict mode)
person.nationality = "American";

// Try to delete a property (will be ignored in non-strict mode)
delete person.age;

// Change existing property (allowed)
person.firstName = "Jane";

document.getElementById("demo").innerHTML =
  "firstName: " + person.firstName + "<br>" +
  "age: " + person.age + " (still exists, cannot delete)" + "<br>" +
  "nationality: " + person.nationality + " (cannot add)";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JavaScript Object.isSealed()

Object.isSealed() checks if an object is sealed:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Protection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

```

```

<h4>Object.isSealed()</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person1 = { firstName: "John" };
const person2 = { firstName: "Jane" };

Object.seal(person2);

document.getElementById("demo1").innerHTML =
  "person1 is sealed: " + Object.isSealed(person1);
document.getElementById("demo2").innerHTML =
  "person2 is sealed: " + Object.isSealed(person2);
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

JavaScript Object.freeze()

`Object.freeze()` prevents any changes to an object. No properties can be added, deleted, or changed:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Protection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.freeze()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Freeze the object
Object.freeze(person);

// Try to change a property (will be ignored in non-strict mode)
person.firstName = "Jane";

// Try to add a new property
person.nationality = "American";

```

```
// Try to delete a property
delete person.age;

document.getElementById("demo").innerHTML =
  "firstName: " + person.firstName + " (cannot change)" + "<br>" +
  "age: " + person.age + " (cannot delete)" + "<br>" +
  "nationality: " + person.nationality + " (cannot add)";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

JavaScript Object.isFrozen()

`Object.isFrozen()` checks if an object is frozen:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Protection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Object.isFrozen()</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person1 = { firstName: "John" };
const person2 = { firstName: "Jane" };

Object.freeze(person2);

document.getElementById("demo1").innerHTML =
  "person1 is frozen: " + Object.isFrozen(person1);
document.getElementById("demo2").innerHTML =
  "person2 is frozen: " + Object.isFrozen(person2);
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Protection](#)

JavaScript Object Prototypes

Table of Contents

- [JavaScript Object Prototypes](#)
- [Prototype Inheritance](#)
- [Example 1](#)
- [Adding Properties to Objects](#)
- [Example 2](#)
- [Adding Methods to Objects](#)
- [Example 3](#)
- [Using the prototype Property](#)
- [Example 4](#)
- [Reference](#)

Prototype Inheritance

All JavaScript objects inherit properties and methods from a **prototype**.

- **Date** objects inherit from **Date.prototype**
- **Array** objects inherit from **Array.prototype**
- **Person** objects inherit from **Person.prototype**

The **Object.prototype** is on the top of the prototype chain:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Prototype Inheritance</h4>
<p id="demo"></p>

<script>
function Person(first, last, age) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
}

const myFather = new Person("John", "Doe", 50);
const myMother = new Person("Jane", "Doe", 48);

document.getElementById("demo").innerHTML =
  "My father is " + myFather.firstName + " " + myFather.lastName + "<br>" +
  "My mother is " + myMother.firstName + " " + myMother.lastName;
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Adding Properties to Objects

You can add new properties to an existing object by simply giving it a value. However, this only adds it to that specific object, not to the prototype:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Properties to Objects</h4>
<p id="demo"></p>

<script>
function Person(first, last) {
  this.firstName = first;
  this.lastName = last;
}

const person1 = new Person("John", "Doe");
const person2 = new Person("Jane", "Smith");

// Add a property to person1 only
person1.nationality = "American";

document.getElementById("demo").innerHTML =
  "person1 nationality: " + person1.nationality + "<br>" +
  "person2 nationality: " + person2.nationality + " (undefined)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Adding Methods to Objects

You can add a method to an existing object. But again, this only adds it to that specific object:


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Methods to Objects</h4>
<p id="demo"></p>

<script>
function Person(first, last) {
  this.firstName = first;
  this.lastName = last;
}

const person1 = new Person("John", "Doe");
const person2 = new Person("Jane", "Smith");

// Add a method to person1 only
person1.fullName = function() {
  return this.firstName + " " + this.lastName;
};

document.getElementById("demo").innerHTML =
  "person1: " + person1.fullName() + "<br>" +
  "person2 would have an error if we called fullName()";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Using the prototype Property

The JavaScript **prototype** property allows you to add new properties and methods to **all** objects of a given type:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using the prototype Property</h4>
<p id="demo"></p>

```

```
<script>
function Person(first, last) {
  this.firstName = first;
  this.lastName = last;
}

// Add a property to ALL Person objects using prototype
Person.prototype.nationality = "American";

// Add a method to ALL Person objects using prototype
Person.prototype.fullName = function() {
  return this.firstName + " " + this.lastName;
};

const person1 = new Person("John", "Doe");
const person2 = new Person("Jane", "Smith");

document.getElementById("demo").innerHTML =
  "person1: " + person1.fullName() + ", " + person1.nationality + "<br>" +
  "person2: " + person2.fullName() + ", " + person2.nationality;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Object Prototypes](#)

JavaScript Classes

Table of Contents

- [JavaScript Classes](#)
- [JavaScript Class Syntax](#)
- [Example 1](#)
- [Using a Class](#)
- [Example 2](#)
- [The Constructor Method](#)
- [Example 3](#)
- [Class Methods](#)
- [Example 4](#)
- [Reference](#)

JavaScript Class Syntax

ECMAScript 2015 (ES6) introduced JavaScript **Classes**.

JavaScript classes are templates for JavaScript objects.

Use the keyword **class** to create a class, and always add the **constructor()** method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Classes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Class Syntax</h4>
<p id="demo"></p>

<script>
class Car {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
}

const myCar1 = new Car("Ford", 2014);
const myCar2 = new Car("Audi", 2019);

document.getElementById("demo").innerHTML =
  myCar1.name + " " + myCar1.year + "<br>" +
  myCar2.name + " " + myCar2.year;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using a Class

When you have a class, you can use the class to create objects:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Classes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using a Class</h4>
<p id="demo"></p>

<script>
class Person {
  constructor(name) {
    this.name = name;
  }
}

const person1 = new Person("John");
const person2 = new Person("Jane");

document.getElementById("demo").innerHTML =
  "Person 1: " + person1.name + "<br>" +
  "Person 2: " + person2.name;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The Constructor Method

The constructor method is a special method:

- It has to have the exact name "constructor"
- It is executed automatically when a new object is created
- It is used to initialize object properties
- If you do not define a constructor method, JavaScript will add an empty constructor method

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Classes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Constructor Method</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
class Student {
  constructor(name, grade) {
    this.name = name;
    this.grade = grade;
    document.getElementById("demo1").innerHTML = "Constructor called for " + name;
  }
}

const student1 = new Student("Alice", "A");
const student2 = new Student("Bob", "B");

document.getElementById("demo2").innerHTML =
  student1.name + " got " + student1.grade + "<br>" +
  student2.name + " got " + student2.grade;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Class Methods

Class methods are created with the same syntax as object methods.

Use the keyword **class** to create a class, then add methods inside the class:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Classes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Class Methods</h4>
<p id="demo"></p>

<script>

```

```
class Car {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
  age() {
    const date = new Date();
    return date.getFullYear() - this.year;
  }
}

const myCar = new Car("Ford", 2014);
document.getElementById("demo").innerHTML =
  "My car is " + myCar.age() + " years old.";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Classes](#)

JavaScript Class Inheritance

Table of Contents

- [JavaScript Class Inheritance](#)
- [Class Inheritance](#)
- [Example 1](#)
- [Getters and Setters](#)
- [Example 2](#)
- [Hoisting](#)
- [Example 3](#)
- [Reference](#)

Class Inheritance

To create a class inheritance, use the **extends** keyword.

A class created with a class inheritance inherits all the methods from another class:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class Inheritance</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Class Inheritance</h4>
<p id="demo"></p>

<script>
class Car {
  constructor(brand) {
    this.carname = brand;
  }
  present() {
    return 'I have a ' + this.carname;
  }
}

class Model extends Car {
  constructor(brand, mod) {
    super(brand);
    this.model = mod;
  }
  show() {
    return this.present() + ', it is a ' + this.model;
  }
}

const myCar = new Model("Ford", "Mustang");
document.getElementById("demo").innerHTML = myCar.show();
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Getters and Setters

Classes also allow you to use getters and setters.

Use the **get** and **set** keywords to define getters and setters:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class Inheritance</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Getters and Setters</h4>
<p id="demo"></p>

<script>
class Car {
  constructor(brand) {
    this.carname = brand;
  }
  get carname() {
    return this._carname;
  }
  set carname(name) {
    this._carname = name;
  }
}

const myCar = new Car("Ford");
document.getElementById("demo").innerHTML = "Car name: " + myCar.carname;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Hoisting

Unlike functions, class declarations are **not hoisted**.

You must first declare a class before using it:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class Inheritance</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hoisting</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Trying to use the class before declaration would cause an error
try {
  const myCar = new Car("Ford");
} catch(err) {
  document.getElementById("demo1").innerHTML = "Error: " + err.message;
}

// Class declaration
class Car {
  constructor(brand) {
    this.carname = brand;
  }
}

// This works - class declared before use
const myCar = new Car("Toyota");
document.getElementById("demo2").innerHTML = "Success: " + myCar.carname;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Class Inheritance](#)

JavaScript Class Static

Table of Contents

- [JavaScript Class Static](#)
- [Static Class Methods](#)
- [Example 1](#)
- [Calling Static Methods](#)
- [Example 2](#)
- [Static vs Instance Methods](#)
- [Example 3](#)
- [Reference](#)

Static Class Methods

Static class methods are defined with the **static** keyword.

Static methods are called directly on the class (without creating an instance of the class).

Static methods are often used for utility functions that are not specific to a particular object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class Static</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Static Class Methods</h4>
<p id="demo"></p>

<script>
class Car {
  constructor(name) {
    this.name = name;
  }
  static hello() {
    return "Hello!";
  }
}

document.getElementById("demo").innerHTML = Car.hello();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Calling Static Methods

Static methods are called on the class itself, not on instances of the class:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class Static</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Calling Static Methods</h4>
<p id="demo"></p>

<script>
class Calculator {
  static add(a, b) {
    return a + b;
  }
  static multiply(a, b) {
    return a * b;
  }
}

// Call static methods on the class directly
let sum = Calculator.add(5, 3);
let product = Calculator.multiply(5, 3);

document.getElementById("demo").innerHTML =
  "5 + 3 = " + sum + "<br>" +
  "5 * 3 = " + product;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Static vs Instance Methods

Static methods are defined on the class level. Instance methods are defined on the prototype and are available on object instances:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class Static</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Static vs Instance Methods</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
class Car {
  constructor(name) {
    this.name = name;
  }
  // Instance method
  start() {
    return this.name + " engine started!";
  }
  // Static method
  static description() {
    return "A car is a wheeled motor vehicle used for transportation.";
  }
}

const myCar = new Car("Ford");

// Call instance method on the object
document.getElementById("demo1").innerHTML = myCar.start();

// Call static method on the class
document.getElementById("demo2").innerHTML = Car.description();

// Trying to call static method on an instance would cause an error
try {
  myCar.description();
} catch(err) {
  document.getElementById("demo3").innerHTML = "Error: " + err.message;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Class Static](#)

JavaScript JSON

Table of Contents

- [JavaScript JSON](#)
- [What Is JSON?](#)
- [Example 1](#)
- [Converting JSON to JavaScript](#)
- [Example 2](#)
- [Converting JavaScript to JSON](#)
- [Example 3](#)
- [JSON Round Trip](#)
- [Example 4](#)
- [Storing Data](#)
- [Example 5](#)
- [Reference](#)

What Is JSON?

JSON stands for **J**avaScript **O**bject **N**otation.

JSON is a lightweight data interchange format.

JSON is language independent.

JSON is "self-describing" and easy to understand.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What Is JSON?</h4>
<p id="demo"></p>

<script>
// A JSON string
const jsonString = '{"name":"John","age":30,"city":"New York"}';

// JSON syntax is similar to JavaScript object syntax
const obj = {name:"John", age:30, city:"New York"};

document.getElementById("demo").innerHTML = "JSON string: " + jsonString;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Converting JSON to JavaScript

When receiving data from a web server, the data is always a string.

Parse the data with `JSON.parse()`, and the data becomes a JavaScript object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Converting JSON to JavaScript</h4>
<p id="demo"></p>

<script>
// JSON text received from a server
const jsonText = '{"name":"John","age":30,"city":"New York"}';

// Convert JSON string to JavaScript object
const obj = JSON.parse(jsonText);

document.getElementById("demo").innerHTML =
  "Name: " + obj.name + "<br>" +
  "Age: " + obj.age + "<br>" +
  "City: " + obj.city;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Converting JavaScript to JSON

When sending data to a web server, the data has to be a string.

Convert a JavaScript object into a string with `JSON.stringify()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Converting JavaScript to JSON</h4>
<p id="demo"></p>

<script>
// JavaScript object
const obj = {name: "John", age: 30, city: "New York"};

// Convert JavaScript object to JSON string
const jsonString = JSON.stringify(obj);

document.getElementById("demo").innerHTML = jsonString;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JSON Round Trip

A common use of JSON is to exchange data to/from a web server.

When receiving data from a web server, the data is always a string. You can use `JSON.parse()` to convert it to a JavaScript object.

When sending data to a web server, the data has to be a string. You can use `JSON.stringify()` to convert it to JSON.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Round Trip</h4>
<p id="demo"></p>

<script>
// Start with a JavaScript object
const obj1 = {name: "John", age: 30, city: "New York"};

// Convert to JSON string (sending to server)
const jsonString = JSON.stringify(obj1);

// Convert back to JavaScript object (receiving from server)
const obj2 = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =

```

```

    "Original: " + obj1.name + "<br>" +
    "JSON: " + jsonString + "<br>" +
    "Back to object: " + obj2.name;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Storing Data

JSON can be used to store data in localStorage:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Storing Data</h4>
<p id="demo"></p>

<script>
// Create an object
const person = {
  name: "John",
  age: 30,
  city: "New York"
};

// Store as JSON in localStorage
localStorage.setItem("person", JSON.stringify(person));

// Retrieve and parse from localStorage
const storedPerson = JSON.parse(localStorage.getItem("person"));

document.getElementById("demo").innerHTML =
  "Stored name: " + storedPerson.name + "<br>" +
  "Stored age: " + storedPerson.age + "<br>" +
  "Stored city: " + storedPerson.city;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON](#)

JavaScript JSON Syntax

Table of Contents

- [JavaScript JSON Syntax](#)
- [JSON Syntax Rules](#)
- [Example 1](#)
- [JSON Property Names](#)
- [Example 2](#)
- [JSON Strings](#)
- [Example 3](#)
- [JSON Array Literals](#)
- [Example 4](#)
- [Reference](#)

JSON Syntax Rules

JSON syntax is derived from JavaScript object notation syntax:

- Data is in name/value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Syntax Rules</h4>
<p id="demo"></p>

<script>
// JSON data - name/value pairs
const jsonString = '{"firstName":"John","lastName":"Doe","age":30}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "firstName: " + obj.firstName + "<br>" +
  "lastName: " + obj.lastName + "<br>" +
  "age: " + obj.age;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JSON Property Names

JSON property names must be in double quotes. JavaScript property names can be in single or double quotes, but JSON requires double quotes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Property Names</h4>
<p id="demo"></p>

<script>
// Valid JSON - property names in double quotes
const jsonString = '{"name":"John","age":30,"city":"New York"}';

const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "name: " + obj.name + "<br>" +
  "age: " + obj.age + "<br>" +
  "city: " + obj.city;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JSON Strings

JSON strings must be written in double quotes. Single quotes are not valid in JSON:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Strings</h4>
<p id="demo1"></p>
<p id="demo2"></p>
```

```

<script>
// Valid JSON - strings in double quotes
const validJSON = '{"message":"Hello World"}';

// Invalid JSON would be: {'message':'Hello World'} (single quotes)
try {
  const obj = JSON.parse(validJSON);
  document.getElementById("demo1").innerHTML = "Valid JSON: " + obj.message;
} catch(err) {
  document.getElementById("demo1").innerHTML = "Error: " + err.message;
}

// Invalid JSON - single quotes not allowed
const invalidJSON = '{"message':'Hello World'}';
try {
  JSON.parse(invalidJSON);
} catch(err) {
  document.getElementById("demo2").innerHTML = "Invalid JSON: " + err.message;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

JSON Array Literals

JSON arrays are written inside square brackets and can hold multiple values:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Syntax</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Array Literals</h4>
<p id="demo"></p>

<script>
// JSON object with an array
const jsonString = '{"employees":["John","Jane","Bob"]}';
const obj = JSON.parse(jsonString);

let text = "";
for (let emp of obj.employees) {
  text += emp + "<br>";
}
document.getElementById("demo").innerHTML = text;

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON Syntax](#)

JavaScript JSON Data Types

Table of Contents

- [JavaScript JSON Data Types](#)
- [JSON Value Types](#)
- [Example 1](#)
- [JSON String Values](#)
- [Example 2](#)
- [JSON Number Values](#)
- [Example 3](#)
- [JSON Boolean Values](#)
- [Example 4](#)
- [JSON Null](#)
- [Example 5](#)
- [Reference](#)

JSON Value Types

In JSON, values must be one of the following data types:

- a **string**
- a **number**
- an **object** (JSON object)
- an **array**
- a **boolean**
- **null**

JSON values **cannot** be one of the following data types:

- a **function**
- a **date**
- **undefined**

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Value Types</h4>
<p id="demo"></p>

<script>
// JSON with different value types
const jsonString =
'{"name":"John","age":30,"isStudent":false,"car":null,"hobbies":["reading","music"]}';
const obj = JSON.parse(jsonString);
```

```

document.getElementById("demo").innerHTML =
  "String: " + obj.name + "<br>" +
  "Number: " + obj.age + "<br>" +
  "Boolean: " + obj.isStudent + "<br>" +
  "Null: " + obj.car + "<br>" +
  "Array: " + obj.hobbies;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

JSON String Values

In JSON, string values must be written with double quotes:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON String Values</h4>
<p id="demo"></p>

<script>
// JSON with string values
const jsonString = '{"firstName":"John","lastName":"Doe","city":"New York"}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "First name: " + obj.firstName + "<br>" +
  "Last name: " + obj.lastName + "<br>" +
  "City: " + obj.city;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

JSON Number Values

In JSON, number values must be an integer or a floating point. They must not be written in quotes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Number Values</h4>
<p id="demo"></p>

<script>
// JSON with number values (integers and floating point)
const jsonString = '{"age":30,"weight":75.5,"height":180}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "Age: " + obj.age + " (integer)<br>" +
  "Weight: " + obj.weight + " (float)<br>" +
  "Height: " + obj.height + " cm";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

JSON Boolean Values

In JSON, boolean values must be either **true** or **false** (without quotes):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Boolean Values</h4>
<p id="demo"></p>

<script>
// JSON with boolean values
const jsonString = '{"isStudent":false,"hasGraduated":true,"isEmployed":true}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "Is student: " + obj.isStudent + "<br>" +
  "Has graduated: " + obj.hasGraduated + "<br>" +
```



```
"Is employed: " + obj.isEmployed;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JSON Null

In JSON, **null** represents an empty or non-existent value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Data Types</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Null</h4>
<p id="demo"></p>

<script>
// JSON with null values
const jsonString = '{"firstName":"John","middleName":null,"lastName":"Doe"}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "First name: " + obj.firstName + "<br>" +
  "Middle name: " + obj.middleName + " (null)<br>" +
  "Last name: " + obj.lastName;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON Data Types](#)

JavaScript JSON parse()

Table of Contents

- [JavaScript JSON parse\(\)](#)
- [Parsing a JSON Object](#)
- [Example 1](#)
- [Parsing a JSON Array](#)
- [Example 2](#)
- [Parsing Other JSON Values](#)
- [Example 3](#)
- [The Reviver Function](#)
- [Example 4](#)
- [Handling Parse Errors](#)
- [Example 5](#)
- [Reference](#)

The JSON.parse() Method

When receiving data from a web server, the data is always a string.

Parse the data with `JSON.parse()`, and the data becomes a JavaScript object.

Parsing a JSON Object

`JSON.parse()` converts a JSON string into a JavaScript object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON parse()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Parsing a JSON Object</h4>
<p id="demo"></p>

<script>
const jsonString = '{"name":"John","age":30,"city":"New York"}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "Name: " + obj.name + "<br>" +
  "Age: " + obj.age + "<br>" +
  "City: " + obj.city;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Parsing a JSON Array

`JSON.parse()` can also parse a JSON array into a JavaScript array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON parse()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Parsing a JSON Array</h4>
<p id="demo"></p>

<script>
const jsonArray = ['Ford','BMW','Audi','Fiat'];
const cars = JSON.parse(jsonArray);

let text = "";
for (let car of cars) {
  text += car + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Parsing Other JSON Values

`JSON.parse()` can parse all valid JSON value types:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON parse()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Parsing Other JSON Values</h4>
<p id="demo"></p>

<script>
```

```

const jsonString =
'{"string":"Hello","number":42,"float":3.14,"boolean":true,"nullValue":null}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "String: " + obj.string + "<br>" +
  "Number: " + obj.number + "<br>" +
  "Float: " + obj.float + "<br>" +
  "Boolean: " + obj.boolean + "<br>" +
  "Null: " + obj.nullValue;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The Reviver Function

The `JSON.parse()` method can accept a second parameter called a **reviver** function.

The reviver function can be used to transform the parsed values:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON parse()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Reviver Function</h4>
<p id="demo"></p>

<script>
const jsonString = '{"name":"John","birth":"1990-05-15","city":"New York"}';
const obj = JSON.parse(jsonString, function(key, value) {
  if (key === "birth") {
    return new Date(value);
  }
  return value;
});

document.getElementById("demo").innerHTML =
  "Name: " + obj.name + "<br>" +
  "Birth: " + obj.birth + "<br>" +
  "Birth (Date object): " + obj.birth.getFullYear();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Handling Parse Errors

Invalid JSON will cause a parse error. Use try/catch to handle errors:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON parse()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Handling Parse Errors</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Valid JSON
const validJSON = '{"name":"John","age":30}';
try {
  const obj = JSON.parse(validJSON);
  document.getElementById("demo1").innerHTML = "Valid: " + obj.name;
} catch(err) {
  document.getElementById("demo1").innerHTML = "Error: " + err.message;
}

// Invalid JSON (single quotes)
const invalidJSON = '{"name':'John','age':30}";
try {
  const obj = JSON.parse(invalidJSON);
  document.getElementById("demo2").innerHTML = "Valid: " + obj.name;
} catch(err) {
  document.getElementById("demo2").innerHTML = "Invalid JSON error: " + err.message;
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON parse\(\)](#)

JavaScript JSON stringify()

Table of Contents

- [JavaScript JSON stringify\(\)](#)
- [Converting an Object](#)
- [Example 1](#)
- [Converting an Array](#)
- [Example 2](#)
- [Selecting Properties](#)
- [Example 3](#)
- [Formatting JSON](#)
- [Example 4](#)
- [Stringifying Dates](#)
- [Example 5](#)
- [Reference](#)

The JSON.stringify() Method

When sending data to a web server, the data has to be a string.

Convert a JavaScript object into a string with `JSON.stringify()`.

Converting an Object

`JSON.stringify()` converts a JavaScript object into a JSON string:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON stringify()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Converting an Object</h4>
<p id="demo"></p>

<script>
const obj = {name: "John", age: 30, city: "New York"};
const jsonString = JSON.stringify(obj);

document.getElementById("demo").innerHTML = jsonString;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Converting an Array

`JSON.stringify()` can also convert a JavaScript array into a JSON string:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON stringify()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Converting an Array</h4>
<p id="demo"></p>

<script>
const arr = ["Ford", "BMW", "Audi", "Fiat"];
const jsonString = JSON.stringify(arr);

document.getElementById("demo").innerHTML = jsonString;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Selecting Properties

The `JSON.stringify()` method can accept a **replacer** array to select which properties to include:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON stringify()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Selecting Properties</h4>
<p id="demo"></p>

<script>
const obj = {name: "John", age: 30, city: "New York", country: "USA"};
const jsonString = JSON.stringify(obj, ["name", "city"]);

document.getElementById("demo").innerHTML = jsonString;
</script>
```

```
</body>
</html>
```

Example 3

Result [View Example](#)

Formatting JSON

The `JSON.stringify()` method can accept a **space** parameter to format the output with spaces:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON stringify()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Formatting JSON</h4>
<pre id="demo"></pre>

<script>
const obj = {name: "John", age: 30, city: "New York", hobbies: ["reading", "music"]};
const jsonString = JSON.stringify(obj, null, 2);

document.getElementById("demo").innerHTML = jsonString;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Stringifying Dates

Dates are converted to strings when using `JSON.stringify()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON stringify()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Stringifying Dates</h4>
<p id="demo"></p>
```



```
<script>
const obj = {name: "John", today: new Date(), city: "New York"};
const jsonString = JSON.stringify(obj);

document.getElementById("demo").innerHTML = jsonString;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON stringify\(\)](#)

JavaScript JSON Server

Table of Contents

- [JavaScript JSON Server](#)
- [Loading JSON from a Server](#)
- [Example 1](#)
- [Loading a JSON Array](#)
- [Example 2](#)
- [Handling Errors](#)
- [Example 3](#)
- [Sending JSON](#)
- [Example 4](#)
- [Reference](#)

The fetch() Method

The `fetch()` method is used to send and receive data from a server.

It returns a Promise that resolves to a Response object.

The `response.json()` method parses the response body as JSON.

Loading JSON from a Server

Use `fetch()` to load a JSON file from a server, then use `response.json()` to parse it:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Server</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loading JSON from a Server</h4>
<p id="demo"></p>

<script>
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(response => response.json())
  .then(data => {
    document.getElementById("demo").innerHTML =
      "Name: " + data.name + "<br>" +
      "Email: " + data.email + "<br>" +
      "City: " + data.address.city;
  })
  .catch(error => {
    document.getElementById("demo").innerHTML = "Error: " + error.message;
  });
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Loading a JSON Array

You can also load a JSON array from a server:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Server</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loading a JSON Array</h4>
<p id="demo"></p>

<script>
fetch('https://jsonplaceholder.typicode.com/users')
  .then(response => response.json())
  .then(data => {
    let text = "<ul>";
    for (let user of data) {
      text += "<li>" + user.name + " - " + user.email + "</li>";
    }
    text += "</ul>";
    document.getElementById("demo").innerHTML = text;
  })
  .catch(error => {
    document.getElementById("demo").innerHTML = "Error: " + error.message;
  });
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Handling Errors

Always handle errors when loading JSON from a server. Check the response status:

```
<!DOCTYPE html>
<html>
```

```

<body>

<h2>JavaScript JSON Server</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Handling Errors</h4>
<p id="demo"></p>

<script>
fetch('https://jsonplaceholder.typicode.com/users/nonexistent')
  .then(response => {
    if (!response.ok) {
      throw new Error("HTTP error! Status: " + response.status);
    }
    return response.json();
  })
  .then(data => {
    document.getElementById("demo").innerHTML = "Name: " + data.name;
  })
  .catch(error => {
    document.getElementById("demo").innerHTML = "Error: " + error.message;
  });
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Sending JSON

You can use `fetch()` with the `POST` method to send JSON data to a server:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON Server</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sending JSON</h4>
<p id="demo"></p>

<script>
const userData = {
  name: "John Doe",
  email: "john@example.com",
  username: "johndoe"
};

```

```
fetch('https://jsonplaceholder.typicode.com/users', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(userData)
})
.then(response => response.json())
.then(data => {
  document.getElementById("demo").innerHTML =
    "User created!<br>" +
    "ID: " + data.id + "<br>" +
    "Name: " + data.name + "<br>" +
    "Email: " + data.email;
})
.catch(error => {
  document.getElementById("demo").innerHTML = "Error: " + error.message;
});
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON Server](#)

JavaScript JSON HTML

Table of Contents

- [JavaScript JSON HTML](#)
- [Displaying a Property](#)
- [Example 1](#)
- [Displaying Multiple Properties](#)
- [Example 2](#)
- [Displaying an Object](#)
- [Example 3](#)
- [Displaying a JSON Array](#)
- [Example 4](#)
- [Displaying JSON in a Table](#)
- [Example 5](#)
- [Reference](#)

Displaying a Property

You can display a specific property from a JSON object in HTML:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON HTML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Displaying a Property</h4>
<p id="demo"></p>

<script>
const jsonString = '{"name":"John","age":30,"city":"New York"}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML = "Name: " + obj.name;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Displaying Multiple Properties

You can display multiple properties from a JSON object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON HTML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Displaying Multiple Properties</h4>
<p id="demo"></p>

<script>
const jsonString = '{"name":"John","age":30,"city":"New York"}';
const obj = JSON.parse(jsonString);

document.getElementById("demo").innerHTML =
  "Name: " + obj.name + "<br>" +
  "Age: " + obj.age + "<br>" +
  "City: " + obj.city;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Displaying an Object

You can display an entire JSON object using `JSON.stringify()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON HTML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Displaying an Object</h4>
<pre id="demo"></pre>

<script>
const jsonString = '{"name":"John","age":30,"city":"New York","hobbies":["reading","music"]}';
const obj = JSON.parse(jsonString);

// Display formatted JSON
document.getElementById("demo").innerHTML = JSON.stringify(obj, null, 2);
</script>

</body>

```

```
</html>
```

Example 3

Result [View Example](#)

Displaying a JSON Array

You can loop through a JSON array and display all values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON HTML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Displaying a JSON Array</h4>
<p id="demo"></p>

<script>
const jsonArray = ['Ford','BMW','Audi','Fiat'];
const cars = JSON.parse(jsonArray);

let text = "<ul>";
for (let car of cars) {
  text += "<li>" + car + "</li>";
}
text += "</ul>";
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Displaying JSON in a Table

You can display an array of JSON objects in an HTML table:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON HTML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```



```
<h4>Displaying JSON in a Table</h4>
<table id="demo" border="1"></table>

<script>
// JSON array of objects
const jsonString = '[{"name":"John","age":30,"city":"New
York"}, {"name":"Jane","age":25,"city":"London"}, {"name":"Bob", "age":35, "city":"Paris"}
]';
const data = JSON.parse(jsonString);

let table = "<tr><th>Name</th><th>Age</th><th>City</th></tr>";
for (let person of data) {
  table += "<tr><td>" + person.name + "</td><td>" + person.age + "</td><td>" +
person.city + "</td></tr>";
}
document.getElementById("demo").innerHTML = table;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON HTML](#)

JavaScript JSON vs XML

Table of Contents

- [JavaScript JSON vs XML](#)
- [JSON Example](#)
- [Example 1](#)
- [XML Example](#)
- [Example 2](#)
- [JSON Uses Objects and Arrays](#)
- [Example 3](#)
- [Working with JSON in JavaScript](#)
- [Example 4](#)
- [Reference](#)

JSON Example

JSON is a lightweight data interchange format. Here is a JSON example:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON vs XML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Example</h4>
<p>JSON data:</p>
<pre id="demo"></pre>

<script>
// JSON object representing a person
const person = {
  "firstName": "John",
  "lastName": "Doe",
  "age": 30,
  "address": {
    "street": "123 Main St",
    "city": "New York"
  },
  "phoneNumbers": ["555-1234", "555-5678"]
};

document.getElementById("demo").innerHTML = JSON.stringify(person, null, 2);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

XML Example

XML (Extensible Markup Language) is another format for storing and transporting data:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON vs XML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>XML Example</h4>
<p>XML data displayed in HTML:</p>
<pre id="demo" style="color:blue"></pre>

<script>
// Simulating XML-like data structure
const xmlData =
  "<person>\n" +
  "  <firstName>John</firstName>\n" +
  "  <lastName>Doe</lastName>\n" +
  "  <age>30</age>\n" +
  "  <address>\n" +
  "    <street>123 Main St</street>\n" +
  "    <city>New York</city>\n" +
  "  </address>\n" +
  "  <phoneNumbers>\n" +
  "    <number>555-1234</number>\n" +
  "    <number>555-5678</number>\n" +
  "  </phoneNumbers>\n" +
  "</person>";

// Use textContent to display raw XML without parsing HTML tags
document.getElementById("demo").textContent = xmlData;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JSON Uses Objects and Arrays

JSON uses objects (key/value pairs) and arrays. XML uses elements with opening and closing tags:

```
<!DOCTYPE html>
<html>
```

```

<body>

<h2>JavaScript JSON vs XML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JSON Uses Objects and Arrays</h4>
<p id="demo"></p>

<script>
// JSON: employees array with objects
const employees = {
  "employees": [
    { "name": "John", "department": "Sales" },
    { "name": "Jane", "department": "Marketing" },
    { "name": "Bob", "department": "IT" }
  ]
};

let text = "JSON Employees:<br>";
for (let emp of employees.employees) {
  text += emp.name + " - " + emp.department + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Working with JSON in JavaScript

JSON integrates naturally with JavaScript since it shares the same syntax:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript JSON vs XML</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Working with JSON in JavaScript</h4>
<p id="demo"></p>

<script>
// JSON data as string
const jsonString = '{"name":"John","age":30,"city":"New York"}';

// Parse JSON to JavaScript object
const obj = JSON.parse(jsonString);

```

```
// Easily access properties
document.getElementById("demo").innerHTML =
  "Name: " + obj.name + "<br>" +
  "Age: " + obj.age + "<br>" +
  "City: " + obj.city;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript JSON vs XML](#)

JavaScript Async

Table of Contents

- [JavaScript Async](#)
- [Synchronous JavaScript](#)
- [Example 1](#)
- [Function Sequence](#)
- [Example 2](#)
- [Blocking Code](#)
- [Example 3](#)
- [Common Asynchronous Operations](#)
- [Example 4](#)
- [Reference](#)

Synchronous JavaScript

Synchronous code runs in sequence. Each operation waits for the previous one to complete before executing:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Synchronous JavaScript</h4>
<p id="demo"></p>

<script>
function myFunction() {
  return "Hello from synchronous function!";
}

let result = myFunction();
document.getElementById("demo").innerHTML = result;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Function Sequence

JavaScript functions are executed in the sequence they are called, not in the sequence they are defined:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Sequence</h4>
<p id="demo"></p>

<script>
function displayResult(result) {
    document.getElementById("demo").innerHTML = "Result: " + result;
}

function calculate(a, b) {
    return a * b;
}

// Functions execute in sequence
let product = calculate(5, 3);
displayResult(product);
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Blocking Code

Long-running synchronous operations can block code execution. This is why asynchronous programming is important:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Blocking Code</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
document.getElementById("demo1").innerHTML = "Start of script";

// Simulate a blocking operation

```

```
function blockingOperation() {
  let start = Date.now();
  let end = start;
  while (end < start + 2000) {
    end = Date.now();
  }
  return "Blocking operation finished after 2 seconds";
}

let result = blockingOperation();
document.getElementById("demo2").innerHTML = result;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Common Asynchronous Operations

Common async operations include `setTimeout()` and `fetch()`. These do not block the execution flow:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Common Asynchronous Operations</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
// This runs first
document.getElementById("demo1").innerHTML = "1. Start of script";

// setTimeout is asynchronous - it doesn't block
setTimeout(function() {
  document.getElementById("demo2").innerHTML = "3. Timeout finished (after 1 second)";
}, 1000);

// This runs second (not blocked by setTimeout)
document.getElementById("demo3").innerHTML = "2. End of script (not blocked)";
</script>

</body>
</html>
```


Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Async](#)

JavaScript Async Callbacks

> **Note:** This topic builds on the Callbacks fundamentals covered in [05.2012-js-Callbacks](#).

Table of Contents

- [JavaScript Async Callbacks](#)
- [Synchronous Callbacks](#)
- [Example 1](#)
- [Asynchronous Callbacks](#)
- [Example 2](#)
- [Callback Chains](#)
- [Example 3](#)
- [Reference](#)

Synchronous Callbacks

A synchronous callback is executed during the execution of the higher-order function. Array methods like `forEach()` use synchronous callbacks:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Synchronous Callbacks</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5];
let text = "";

// forEach uses a synchronous callback
numbers.forEach(function(value) {
  text += value * 2 + " ";
});

document.getElementById("demo").innerHTML = "Doubled values: " + text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Asynchronous Callbacks

An asynchronous callback is executed after the higher-order function has completed. `setTimeout()` is a classic example:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Asynchronous Callbacks</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function myDisplayer(something) {
    document.getElementById("demo2").innerHTML = "Result: " + something;
}

function myCalculator(num1, num2, myCallback) {
    let sum = num1 + num2;
    myCallback(sum);
}

document.getElementById("demo1").innerHTML = "Calculating...";
myCalculator(5, 5, myDisplayer);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Callback Chains

When one callback depends on the result of another, you get callback chains (sometimes called "callback hell"):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Callback Chains</h4>
<p id="demo"></p>
```

```

<script>
function step1(callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 1: Data loaded<br>";
    callback("Data from step 1");
  }, 1000);
}

function step2(data, callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 2: Processing " + data +
    "<br>";
    callback("Processed data");
  }, 1000);
}

function step3(data) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 3: Displaying " + data +
    "<br>";
    document.getElementById("demo").innerHTML += "All steps complete!";
  }, 1000);
}

// Callback chain
step1(function(result1) {
  step2(result1, function(result2) {
    step3(result2);
  });
});
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Async Callbacks](#)

JavaScript Promises

Table of Contents

- [JavaScript Promises](#)
- [Creating a Promise](#)
- [Example 1](#)
- [The then\(\) Method](#)
- [Example 2](#)
- [The catch\(\) Method](#)
- [Example 3](#)
- [The finally\(\) Method](#)
- [Example 4](#)
- [Promise Chaining](#)
- [Example 5](#)
- [Reference](#)

Why Promises?

Promises provide a cleaner way to handle asynchronous operations compared to callbacks. A Promise is an object representing the eventual completion or failure of an asynchronous operation.

Promise states:

- **pending**: Initial state, neither fulfilled nor rejected
- **fulfilled**: Operation completed successfully
- **rejected**: Operation failed

Creating a Promise

A Promise is created using the `new Promise()` constructor which takes a function with `resolve` and `reject` parameters:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Promises</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating a Promise</h4>
<p id="demo"></p>

<script>
const myPromise = new Promise(function(resolve, reject) {
  let x = 0;
  if (x === 0) {
    resolve("OK");
  } else {
```

```

        reject("Error");
    }
});

myPromise.then(function(value) {
    document.getElementById("demo").innerHTML = "Promise resolved: " + value;
});
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

The then() Method

The **then()** method takes two arguments: a callback for success and another for failure. It is the primary way to handle resolved promises:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Promises</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The then() Method</h4>
<p id="demo"></p>

<script>
function fetchData() {
    return new Promise(function(resolve) {
        setTimeout(function() {
            resolve("Data loaded successfully!");
        }, 2000);
    });
}

fetchData().then(function(value) {
    document.getElementById("demo").innerHTML = value;
});

document.getElementById("demo").innerHTML = "Loading...";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

The catch() Method

The `catch()` method provides a callback for when a promise is rejected:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Promises</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The catch() Method</h4>
<p id="demo"></p>

<script>
const myPromise = new Promise(function(resolve, reject) {
  let success = false;
  if (success) {
    resolve("Operation succeeded");
  } else {
    reject("Operation failed");
  }
});

myPromise
  .then(function(value) {
    document.getElementById("demo").innerHTML = value;
  })
  .catch(function(error) {
    document.getElementById("demo").innerHTML = "Error: " + error;
  });
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The finally() Method

The `finally()` method executes regardless of whether the promise was fulfilled or rejected:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript Promises</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The finally() Method</h4>
<p id="demo"></p>

<script>
const myPromise = new Promise(function(resolve) {
  setTimeout(function() {
    resolve("Promise completed");
  }, 1500);
});

myPromise
  .then(function(value) {
    document.getElementById("demo").innerHTML = value + "<br>";
  })
  .finally(function() {
    document.getElementById("demo").innerHTML += "Finally: cleanup done!";
  });
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Promise Chaining

Promises can be chained to handle sequential asynchronous operations:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Promises</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Promise Chaining</h4>
<p id="demo"></p>

<script>
function step1() {
  return new Promise(function(resolve) {
    setTimeout(function() {
      document.getElementById("demo").innerHTML += "Step 1 complete<br>";
      resolve(10);
    }, 1000);
  });
}

```



```

function step2(number) {
  return new Promise(function(resolve) {
    setTimeout(function() {
      let result = number * 2;
      document.getElementById("demo").innerHTML += "Step 2: " + number + " * 2 = " +
result + "<br>";
      resolve(result);
    }, 1000);
  });
}

function step3(number) {
  return new Promise(function(resolve) {
    setTimeout(function() {
      let result = number + 5;
      document.getElementById("demo").innerHTML += "Step 3: " + number + " + 5 = " +
result + "<br>";
      resolve(result);
    }, 1000);
  });
}

// Chain the promises
step1()
  .then(function(result1) { return step2(result1); })
  .then(function(result2) { return step3(result2); })
  .then(function(result3) {
    document.getElementById("demo").innerHTML += "Final result: " + result3;
  });
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Promises](#)

JavaScript Async/Await

Table of Contents

- [JavaScript Async/Await](#)
- [The async Keyword](#)
- [Example 1](#)
- [The await Keyword](#)
- [Example 2](#)
- [Compare Promise and async/await](#)
- [Example 3](#)
- [Handling Errors](#)
- [Example 4](#)
- [Multiple await](#)
- [Example 5](#)
- [Reference](#)

The async Keyword

The **async** keyword makes a function return a Promise.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The async Keyword</h4>
<p id="demo"></p>

<script>
async function myFunction() {
  return "Hello";
}

myFunction().then(function(value) {
  document.getElementById("demo").innerHTML = value;
});
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The await Keyword

The **await** keyword makes a function wait for a Promise. It can only be used inside an **async** function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The await Keyword</h4>
<p id="demo"></p>

<script>
async function myDisplay() {
  let myPromise = new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Hello after 2 seconds!");
    }, 2000);
  });
  document.getElementById("demo").innerHTML = await myPromise;
}

myDisplay();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Compare Promise and async/await

Both approaches work with Promises, but async/await provides cleaner syntax:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Compare Promise and async/await</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function getData() {
  return new Promise(function(resolve) {
```

```

        setTimeout(function() {
            resolve("Data loaded");
        }, 1000);
    });
}

// Promise version
getData().then(function(value) {
    document.getElementById("demo1").innerHTML = "Promise: " + value;
});

// Async/await version
async function loadData() {
    let value = await getData();
    document.getElementById("demo2").innerHTML = "Async/await: " + value;
}
loadData();
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Handling Errors

Use **try/catch** to handle errors in async/await:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Handling Errors</h4>
<p id="demo"></p>

<script>
async function fetchData() {
    try {
        let result = await new Promise(function(resolve, reject) {
            let success = false;
            if (success) {
                resolve("Success!");
            } else {
                reject("Something went wrong");
            }
        });
    } catch {
        // Handle error
    }
    document.getElementById("demo").innerHTML = result;
}
fetchData();
</script>

```

```

    } catch(error) {
      document.getElementById("demo").innerHTML = "Error caught: " + error;
    }
  }

  fetchData();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Multiple await

You can use multiple **await** statements in sequence:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Multiple await</h4>
<p id="demo"></p>

<script>
async function processSteps() {
  let step1 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Step 1 done");
    }, 1000);
  });
  document.getElementById("demo").innerHTML += step1 + "<br>";

  let step2 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Step 2 done");
    }, 1000);
  });
  document.getElementById("demo").innerHTML += step2 + "<br>";

  let step3 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Step 3 done");
    }, 1000);
  });
  document.getElementById("demo").innerHTML += step3 + "<br>";
}

```

```
    document.getElementById("demo").innerHTML += "All steps complete!";  
  }  
  
  processSteps();  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Async/Await](#)

JavaScript Async Parallel

Table of Contents

- [JavaScript Async Parallel](#)
- [Sequential Operations](#)
- [Example 1](#)
- [Independent Operations](#)
- [Example 2](#)
- [Promise.all\(\)](#)
- [Example 3](#)
- [Promise.allSettled\(\)](#)
- [Example 4](#)
- [Promise.race\(\)](#)
- [Example 5](#)
- [Reference](#)

Sequential Operations

Sequential operations run one after another. Each operation waits for the previous one to complete:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sequential Operations</h4>
<p id="demo"></p>

<script>
async function sequential() {
  let result1 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("First result");
    }, 1000);
  });

  let result2 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Second result");
    }, 1000);
  });

  document.getElementById("demo").innerHTML = result1 + "<br>" + result2 + "<br>Total:
~2 seconds";
}

sequential();
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Independent Operations

Independent operations do not depend on each other. They can run in parallel to save time:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Async Parallel</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>Independent Operations</h4>
```

```
<p id="demo"></p>
```

```
<script>
```

```
async function independent() {  
  let promise1 = new Promise(function(resolve) {  
    setTimeout(function() {  
      resolve("Task 1 done");  
    }, 1000);  
  }, 1000);  
});
```

```
let promise2 = new Promise(function(resolve) {  
  setTimeout(function() {  
    resolve("Task 2 done");  
  }, 1000);  
}, 1000);  
});
```

```
// Start both promises, then await both  
let result1 = await promise1;  
let result2 = await promise2;
```

```
document.getElementById("demo").innerHTML = result1 + "<br>" + result2 + "<br>Total:  
~1 second (parallel)";  
}
```

```
independent();
```

```
</script>
```

```
</body>
```

```
</html>
```


Example 2

Result [View Example](#)

Promise.all()

`Promise.all()` runs multiple promises in parallel and waits for ALL of them to complete:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Promise.all()</h4>
<p id="demo"></p>

<script>
const promise1 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Promise 1 done"); }, 1000);
});

const promise2 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Promise 2 done"); }, 2000);
});

const promise3 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Promise 3 done"); }, 1500);
});

Promise.all([promise1, promise2, promise3]).then(function(values) {
  document.getElementById("demo").innerHTML =
    "All promises resolved in parallel:<br>" +
    values[0] + "<br>" +
    values[1] + "<br>" +
    values[2] + "<br>" +
    "Total time: ~2 seconds (not 4.5)";
});
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Promise.allSettled()

`Promise.allSettled()` waits for all promises to complete (regardless of resolve or reject):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Promise.allSettled()</h4>
<p id="demo"></p>

<script>
const promise1 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Success!"); }, 1000);
});

const promise2 = new Promise(function(resolve, reject) {
  setTimeout(function() { reject("Failed!"); }, 500);
});

const promise3 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Done!"); }, 1500);
});

Promise.allSettled([promise1, promise2, promise3]).then(function(results) {
  let text = "";
  for (let result of results) {
    text += "Status: " + result.status + ", ";
    if (result.status === "fulfilled") {
      text += "Value: " + result.value;
    } else {
      text += "Reason: " + result.reason;
    }
    text += "<br>";
  }
  document.getElementById("demo").innerHTML = text;
});
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Promise.race()

Promise.race() returns as soon as the FIRST promise settles (resolves or rejects):

```

<!DOCTYPE html>
<html>
<body>

```

```

<h2>JavaScript Async Parallel</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Promise.race()</h4>
<p id="demo"></p>

<script>
const promise1 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Fast Promise won!"); }, 1000);
});

const promise2 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Slow Promise"); }, 3000);
});

Promise.race([promise1, promise2]).then(function(value) {
  document.getElementById("demo").innerHTML =
    "Winner: " + value + "<br>" +
    "Promise.race() returns the fastest result";
});
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Async Parallel](#)

JavaScript Event Loop

Table of Contents

- [JavaScript Event Loop](#)
- [JavaScript Executes One Task at a Time](#)
- [Example 1](#)
- [Example: setTimeout\(\)](#)
- [Example 2](#)
- [Example: Promise](#)
- [Example 3](#)
- [Task Queue and Microtask Queue](#)
- [Example 4](#)
- [Long Tasks Block JavaScript](#)
- [Example 5](#)
- [Reference](#)

JavaScript Executes One Task at a Time

JavaScript is single-threaded. It can only execute one task at a time.

However, the browser has Web APIs (like `setTimeout`, `fetch`, DOM events) that run in the background.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Executes One Task at a Time</h4>
<p id="demo"></p>

<script>
function first() {
  document.getElementById("demo").innerHTML += "1. First function<br>";
}

function second() {
  document.getElementById("demo").innerHTML += "2. Second function<br>";
}

function third() {
  document.getElementById("demo").innerHTML += "3. Third function<br>";
}

// Functions execute in order, one at a time
first();
second();
third();
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Example: setTimeout()

When `setTimeout()` is called, the browser handles the timer. When the timer expires, the callback is moved to the **Task Queue**. The Event Loop picks it up when the call stack is empty:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Example: setTimeout()</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML += "1. Start<br>";

setTimeout(function() {
  document.getElementById("demo").innerHTML += "3. setTimeout callback<br>";
}, 0);

document.getElementById("demo").innerHTML += "2. End<br>";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Example: Promise

Promises use the **Microtask Queue** which has higher priority than the Task Queue. Microtasks execute before the next task:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript Event Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Example: Promise</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML += "1. Start<br>";

// Promise goes to Microtask Queue
Promise.resolve().then(function() {
  document.getElementById("demo").innerHTML += "3. Promise callback<br>";
});

// setTimeout goes to Task Queue
setTimeout(function() {
  document.getElementById("demo").innerHTML += "4. setTimeout callback<br>";
}, 0);

document.getElementById("demo").innerHTML += "2. End<br>";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Task Queue and Microtask Queue

The Event Loop processes all microtasks before moving to the next task. This means Promise callbacks execute before setTimeout callbacks:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Task Queue and Microtask Queue</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML += "1. Synchronous code<br>";

// Promise (Microtask Queue)
Promise.resolve().then(function() {
  document.getElementById("demo").innerHTML += "3. Promise (microtask)<br>";
});

setTimeout(function() {
  document.getElementById("demo").innerHTML += "2. setTimeout callback<br>";
}, 0);
</script>

```

```
});

// Promise (Microtask Queue)
Promise.resolve().then(function() {
  document.getElementById("demo").innerHTML += "4. Another promise (microtask)<br>";
});

// setTimeout (Task Queue)
setTimeout(function() {
  document.getElementById("demo").innerHTML += "5. setTimeout (task)<br>";
}, 0);

document.getElementById("demo").innerHTML += "2. End of synchronous code<br>";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Long Tasks Block JavaScript

Long-running synchronous operations block the Event Loop. Async operations can't run until the call stack is empty:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Loop</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Long Tasks Block JavaScript</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML += "1. Start<br>";

// This timeout should fire after 0ms
setTimeout(function() {
  document.getElementById("demo").innerHTML += "4. setTimeout (delayed by blocking
code)<br>";
}, 0);

// Blocking operation - simulates 3 seconds of work
let start = Date.now();
while (Date.now() < start + 3000) {
  // Blocking loop
}
```

```
document.getElementById("demo").innerHTML += "2. After blocking loop (3 seconds  
later)<br>";  
  
document.getElementById("demo").innerHTML += "3. End of synchronous code<br>";  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Event Loop](#)

JavaScript Async Fetch

Table of Contents

- [JavaScript Async Fetch](#)
- [Fetching a File](#)
- [Example 1](#)
- [The Response Object](#)
- [Example 2](#)
- [Fetching JSON](#)
- [Example 3](#)
- [Loading Multiple Files](#)
- [Example 4](#)
- [Handling Errors](#)
- [Example 5](#)
- [Reference](#)

Fetching a File

The `fetch()` method loads a file from a web server asynchronously. It returns a Promise that resolves to a Response object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Fetching a File</h4>
<p id="demo"></p>

<script>
async function getData() {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts/1');
  const data = await response.json();
  document.getElementById("demo").innerHTML =
    "Title: " + data.title + "<br>" +
    "Body: " + data.body;
}

getData();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Response Object

The Response object has useful properties like `ok`, `status`, and `url`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Response Object</h4>
<p id="demo"></p>

<script>
async function getResponse() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users/1');

  document.getElementById("demo").innerHTML =
    "ok: " + response.ok + "<br>" +
    "status: " + response.status + "<br>" +
    "statusText: " + response.statusText + "<br>" +
    "url: " + response.url;

  const data = await response.json();
  document.getElementById("demo").innerHTML += "<br>Name: " + data.name;
}

getResponse();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Fetching JSON

The `response.json()` method parses the response body as JSON:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Fetching JSON</h4>
<p id="demo"></p>

<script>
async function fetchJSON() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users');
  const users = await response.json();

  let text = "";
  for (let user of users) {
    text += user.name + " - " + user.email + "<br>";
  }
  document.getElementById("demo").innerHTML = text;
}

fetchJSON();
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Loading Multiple Files

You can load multiple files in parallel using `Promise.all()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loading Multiple Files</h4>
<p id="demo"></p>

<script>
async function loadMultiple() {
  const [user, posts] = await Promise.all([
    fetch('https://jsonplaceholder.typicode.com/users/1'),
    fetch('https://jsonplaceholder.typicode.com/posts?userId=1')
  ]);

  const userData = await user.json();
  const postsData = await posts.json();

  document.getElementById("demo").innerHTML =
    "User: " + userData.name + "<br>" +
    "Email: " + userData.email + "<br><br>" +

```

```

    "Posts (" + postsData.length + "):<br>";

    for (let post of postsData.slice(0, 3)) {
        document.getElementById("demo").innerHTML += "- " + post.title + "<br>";
    }
}

loadMultiple();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Handling Errors

Always handle errors when using `fetch()`. Use `try/catch` to catch network errors and check `response.ok` for HTTP errors:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Handling Errors</h4>
<p id="demo"></p>

<script>
async function fetchWithErrorHandling() {
    try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/users/invalid');

        if (!response.ok) {
            throw new Error("HTTP error! Status: " + response.status);
        }

        const data = await response.json();
        document.getElementById("demo").innerHTML = "Name: " + data.name;
    } catch(error) {
        document.getElementById("demo").innerHTML = "Error caught: " + error.message;
    }
}

fetchWithErrorHandling();
</script>

```

```
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Async Fetch](#)

JavaScript Async Common Mistakes

Table of Contents

- [JavaScript Async Common Mistakes](#)
- [Mistake 1: Forgetting await](#)
- [Example 1](#)
- [Mistake 2: Using await Outside an Async Function](#)
- [Example 2](#)
- [Mistake 3: Assuming fetch\(\) Fails on HTTP Errors](#)
- [Example 3](#)
- [Mistake 4: Blocking the Event Loop](#)
- [Example 4](#)
- [Mistake 5: Running Independent Tasks One After Another](#)
- [Example 5](#)
- [Reference](#)

Mistake 1: Forgetting await

Forgetting `await` returns a Promise object instead of the actual value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 1: Forgetting await</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
async function getValue() {
  return "Hello World";
}

// WRONG: missing await - returns a Promise object
let wrong = getValue();
document.getElementById("demo1").innerHTML = "Without await: " + wrong;

// CORRECT: with await - returns the actual value
async function correctVersion() {
  let correct = await getValue();
  document.getElementById("demo2").innerHTML = "With await: " + correct;
}
correctVersion();
</script>

</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Mistake 2: Using await Outside an Async Function

`await` can only be used inside `async` functions:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 2: await Outside Async Function</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// WRONG: await cannot be used at the top level (in older JS)
// let result = await getValue(); // SyntaxError

// CORRECT: wrap in async function
async function getData() {
  let promise = new Promise(function(resolve) {
    setTimeout(function() { resolve("Data loaded"); }, 1000);
  });
  let result = await promise;
  document.getElementById("demo1").innerHTML = "await works inside async: " + result;
}

getData();

// Alternative: using .then() outside async function
let promise2 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Data via .then()"); }, 1500);
});
promise2.then(function(value) {
  document.getElementById("demo2").innerHTML = "Using .then() outside async: " +
value;
});
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Mistake 3: Assuming fetch() Fails on HTTP Errors

`fetch()` only rejects on network errors, not HTTP errors (like 404 or 500). You must check `response.ok`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 3: fetch() and HTTP Errors</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
async function fetchWithoutCheck() {
  // fetch() does NOT throw on HTTP errors like 404
  const response = await fetch('https://jsonplaceholder.typicode.com/users/invalid');
  document.getElementById("demo1").innerHTML =
    "fetch() did NOT throw! Status: " + response.status + " " + response.statusText;
}
fetchWithoutCheck();

async function fetchWithCheck() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users/invalid');
  if (!response.ok) {
    document.getElementById("demo2").innerHTML =
      "Error caught: HTTP " + response.status + " (" + response.statusText + ")";
    return;
  }
  const data = await response.json();
  document.getElementById("demo2").innerHTML = "Name: " + data.name;
}
fetchWithCheck();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Mistake 4: Blocking the Event Loop

Long-running synchronous operations block async code from executing:

```
<!DOCTYPE html>
```



```

<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 4: Blocking the Event Loop</h4>
<p id="demo"></p>

<script>
// This timeout should run after 0ms
setTimeout(function() {
  document.getElementById("demo").innerHTML += "2. setTimeout (delayed!)<br>";
}, 0);

// Blocking operation for 2 seconds
let start = Date.now();
while (Date.now() < start + 2000) {
  // Blocks the event loop
}

document.getElementById("demo").innerHTML += "1. After blocking loop (2 seconds)<br>";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Mistake 5: Running Independent Tasks One After Another

Independent tasks should run in parallel, not sequentially:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 5: Sequential Independent Tasks</h4>
<p id="demo"></p>

<script>
async function demonstrate() {
  // WRONG: running independent tasks sequentially (2 seconds total)
  let task1 = await new Promise(function(resolve) {
    setTimeout(function() { resolve("Task 1 done"); }, 1000);
  });
  let task2 = await new Promise(function(resolve) {

```

```

    setTimeout(function() { resolve("Task 2 done"); }, 1000);
  });

  document.getElementById("demo").innerHTML += "WRONG (sequential): ~2 seconds<br>";
  document.getElementById("demo").innerHTML += task1 + "<br>" + task2 + "<br><br>";

  // CORRECT: running independent tasks in parallel (~1 second)
  let start = Date.now();
  let [result1, result2] = await Promise.all([
    new Promise(function(resolve) {
      setTimeout(function() { resolve("Task 1 done"); }, 1000);
    }),
    new Promise(function(resolve) {
      setTimeout(function() { resolve("Task 2 done"); }, 1000);
    })
  ]);

  document.getElementById("demo").innerHTML += "CORRECT (parallel): ~1 second<br>";
  document.getElementById("demo").innerHTML += result1 + "<br>" + result2;
}

demonstrate();
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Async Common Mistakes](#)

JavaScript Async Debugging

Table of Contents

- [JavaScript Async Debugging](#)
- [Log Events](#)
- [Example 1](#)
- [Rule 1: Always Handle Errors](#)
- [Example 2](#)
- [Rule 2: Check response.ok](#)
- [Example 3](#)
- [Rule 3: Log the Right Things](#)
- [Example 4](#)
- [Understand Execution Order](#)
- [Example 5](#)
- [Reference](#)

Log Events

Use `console.log()` to track the flow of async operations. This helps you understand the order of execution:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Log Events</h4>
<p id="demo"></p>

<script>
console.log("1. Script starts");

async function fetchData() {
  console.log("2. Inside async function");
  let result = await new Promise(function(resolve) {
    setTimeout(function() {
      console.log("3. Promise resolved");
      resolve("Data loaded");
    }, 1000);
  });
  console.log("4. After await");
  document.getElementById("demo").innerHTML = result;
}

fetchData();
console.log("5. After calling fetchData (not blocked)");
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Rule 1: Always Handle Errors

Always use **try/catch** around async operations to catch errors:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Rule 1: Always Handle Errors</h4>
<p id="demo"></p>

<script>
async function fetchWithCatch() {
  try {
    let response = await fetch('https://jsonplaceholder.typicode.com/users/1');
    if (!response.ok) {
      throw new Error("HTTP " + response.status);
    }
    let data = await response.json();
    document.getElementById("demo").innerHTML =
      "Success: " + data.name + "<br>" +
      "Email: " + data.email;
  } catch(error) {
    document.getElementById("demo").innerHTML =
      "Error caught: " + error.message + "<br>" +
      "Tip: Check the URL or network connection";
  }
}

fetchWithCatch();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Rule 2: Check response.ok

`fetch()` does NOT reject on HTTP errors (like 404). Always check `response.ok`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Rule 2: Check response.ok</h4>
<p id="demo"></p>

<script>
async function checkResponse() {
  // Try with an invalid URL
  const response = await fetch('https://jsonplaceholder.typicode.com/users/invalid');

  console.log("Status:", response.status);
  console.log("ok:", response.ok);

  if (!response.ok) {
    document.getElementById("demo").innerHTML =
      "response.ok is " + response.ok + "<br>" +
      "Status: " + response.status + " " + response.statusText + "<br>" +
      "Always check response.ok after fetch!";
    return;
  }

  const data = await response.json();
  document.getElementById("demo").innerHTML = "Name: " + data.name;
}

checkResponse();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Rule 3: Log the Right Things

Log meaningful information to understand what's happening:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Rule 3: Log the Right Things</h4>
<p id="demo"></p>

<script>
async function logCorrectly() {
  const url = 'https://jsonplaceholder.typicode.com/users/1';

  // Log the URL being fetched
  console.log("Fetching:", url);

  try {
    const response = await fetch(url);

    // Log response status
    console.log("Response status:", response.status);

    if (!response.ok) {
      throw new Error("HTTP error " + response.status);
    }

    const data = await response.json();

    // Log the received data structure
    console.log("Data received:", data);
    console.log("User name:", data.name);

    document.getElementById("demo").innerHTML =
      "Check the browser console (F12) for logs<br>" +
      "User: " + data.name + "<br>" +
      "Email: " + data.email;

  } catch(error) {
    console.error("Error:", error.message);
    document.getElementById("demo").innerHTML = "Error: " + error.message;
  }
}

logCorrectly();
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Understand Execution Order

Async code does not execute in the order it appears. Use logs to understand the flow:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Understand Execution Order</h4>
<p id="demo"></p>

<script>
console.log("A: Start of script");

setTimeout(function() {
  console.log("B: setTimeout callback");
}, 0);

Promise.resolve().then(function() {
  console.log("C: Promise microtask");
});

async function test() {
  console.log("D: Inside async function (before await)");
  await Promise.resolve();
  console.log("E: After await");
}
test();

console.log("F: End of script (sync code)");

// Expected console order: A, D, F, C, E, B
document.getElementById("demo").innerHTML =
  "Check browser console (F12) for execution order:<br>" +
  "Expected: A → D → F → C → E → B<br><br>" +
  "A: Start<br>" +
  "D: Inside async (before await)<br>" +
  "F: End of sync code<br>" +
  "C: Promise microtask<br>" +
  "E: After await<br>" +
  "B: setTimeout (Task queue - last)";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Async Debugging](#)

JavaScript AbortController

Table of Contents

- [JavaScript AbortController](#)
- [Creating an AbortController](#)
- [Example 1](#)
- [Canceling a Fetch Request](#)
- [Example 2](#)
- [Cancel with a Button](#)
- [Example 3](#)
- [Cancel After a Timeout](#)
- [Example 4](#)
- [Cancel the Previous Request](#)
- [Example 5](#)
- [Reference](#)

Why Cancel a Request?

Sometimes you need to cancel an ongoing async operation. For example:

- The user clicked a "Cancel" button
- The request is taking too long
- A newer request makes the previous one irrelevant

The `AbortController` API provides a way to cancel fetch requests (and other async operations).

Creating an AbortController

An `AbortController` is created with `new AbortController()`. The controller has a `signal` property that is passed to the fetch request:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript AbortController</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating an AbortController</h4>
<p id="demo"></p>

<script>
const controller = new AbortController();
const signal = controller.signal;

document.getElementById("demo").innerHTML =
  "AbortController created<br>" +
  "signal.aborted: " + signal.aborted;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Canceling a Fetch Request

Pass the **signal** to **fetch()** and call **controller.abort()** to cancel:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript AbortController</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>Canceling a Fetch Request</h4>
```

```
<p id="demo"></p>
```

```
<script>
```

```
async function fetchData() {
```

```
  const controller = new AbortController();
```

```
  const signal = controller.signal;
```

```
  // Cancel after 1 second
```

```
  setTimeout(function() {
```

```
    controller.abort();
```

```
  }, 1000);
```

```
  try {
```

```
    const response = await fetch('https://jsonplaceholder.typicode.com/users/1', {  
signal });
```

```
    const data = await response.json();
```

```
    document.getElementById("demo").innerHTML = "Success: " + data.name;
```

```
  } catch(error) {
```

```
    if (error.name === "AbortError") {
```

```
      document.getElementById("demo").innerHTML = "Fetch was aborted!";
```

```
    } else {
```

```
      document.getElementById("demo").innerHTML = "Error: " + error.message;
```

```
    }
```

```
  }
```

```
}
```

```
fetchData();
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 2

Result [View Example](#)

Cancel with a Button

Allow users to cancel a request with a button click:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript AbortController</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cancel with a Button</h4>
<button id="startBtn">Start Fetch</button>
<button id="cancelBtn">Cancel</button>
<p id="demo"></p>

<script>
let controller;

document.getElementById("startBtn").addEventListener("click", function() {
  controller = new AbortController();
  const signal = controller.signal;

  document.getElementById("demo").innerHTML = "Fetching...";

  fetch('https://jsonplaceholder.typicode.com/users', { signal })
    .then(function(response) { return response.json(); })
    .then(function(data) {
      document.getElementById("demo").innerHTML = "Loaded " + data.length + " users";
    })
    .catch(function(error) {
      if (error.name === "AbortError") {
        document.getElementById("demo").innerHTML = "Request was canceled by user";
      } else {
        document.getElementById("demo").innerHTML = "Error: " + error.message;
      }
    });
});

document.getElementById("cancelBtn").addEventListener("click", function() {
  if (controller) {
    controller.abort();
  }
});
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Cancel After a Timeout

Automatically cancel a request if it takes too long (timeout):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript AbortController</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cancel After a Timeout</h4>
<p id="demo"></p>

<script>
async function fetchWithTimeout(url, timeoutMs) {
  const controller = new AbortController();
  const signal = controller.signal;

  // Set timeout to abort
  const timeoutId = setTimeout(function() {
    controller.abort();
  }, timeoutMs);

  try {
    const response = await fetch(url, { signal });
    clearTimeout(timeoutId); // Cancel the timeout if fetch completed
    const data = await response.json();
    document.getElementById("demo").innerHTML = "Data loaded in time!<br>User: " +
data.name;
  } catch(error) {
    if (error.name === "AbortError") {
      document.getElementById("demo").innerHTML = "Timeout: Request took too long!";
    } else {
      document.getElementById("demo").innerHTML = "Error: " + error.message;
    }
  }
}

// Try with a 3-second timeout (should complete quickly)
fetchWithTimeout('https://jsonplaceholder.typicode.com/users/1', 3000);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Cancel the Previous Request

Cancel an old request when a new one starts (useful for search-as-you-type):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript AbortController</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cancel the Previous Request</h4>
<p>Type in the input field:</p>
<input type="text" id="searchInput" placeholder="Search users...">
<p id="demo"></p>

<script>
let currentController = null;

document.getElementById("searchInput").addEventListener("input", function() {
  const query = this.value;
  if (query.length < 2) {
    document.getElementById("demo").innerHTML = "";
    return;
  }

  // Cancel the previous request
  if (currentController) {
    currentController.abort();
  }

  currentController = new AbortController();
  const signal = currentController.signal;

  document.getElementById("demo").innerHTML = "Searching for: " + query + "...";

  fetch('https://jsonplaceholder.typicode.com/users', { signal })
    .then(function(response) { return response.json(); })
    .then(function(users) {
      const results = users.filter(function(u) {
        return u.name.toLowerCase().includes(query.toLowerCase());
      });
      let text = "Found " + results.length + " users:<br>";
      for (let user of results) {
        text += user.name + "<br>";
      }
      document.getElementById("demo").innerHTML = text;
    })
    .catch(function(error) {
      if (error.name !== "AbortError") {
        document.getElementById("demo").innerHTML = "Error: " + error.message;
      }
    })
  });
});
```

```
    });  
  });  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript AbortController](#)

JavaScript Web Workers

Table of Contents

- [JavaScript Web Workers](#)
- [Creating a Worker](#)
- [Example 1](#)
- [Sending and Receiving Messages](#)
- [Example 2](#)
- [Starting and Stopping a Worker](#)
- [Example 3](#)
- [Worker Errors](#)
- [Example 4](#)
- [Using Timers Inside a Worker](#)
- [Example 5](#)
- [Reference](#)

Why Web Workers?

JavaScript is single-threaded. Long-running tasks block the UI. Web Workers run JavaScript in the background, on a separate thread, without interfering with the user interface.

A Web Worker is a JavaScript file that runs in the background, independently of other scripts.

Creating a Worker

To create a Web Worker, write a separate JavaScript file and create a **Worker** object:

worker.js - The worker file that runs in the background:

```
onmessage = function(e) {  
  let result = e.data * 2;  
  postMessage("Result: " + result);  
};
```

Main page:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Web Workers</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>Creating a Worker</h4>  
<button onclick="startWorker()">Start Worker</button>  
<p id="demo"></p>
```

```

<script>
let worker;

function startWorker() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(worker) == "undefined") {
      worker = new Worker("ex1-create_worker/worker.js");
    }
    worker.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
  } else {
    document.getElementById("demo").innerHTML = "Sorry, Web Workers are not supported.";
  }
}
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Sending and Receiving Messages

Communication between the main page and a worker uses `postMessage()` and `onmessage`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sending and Receiving Messages</h4>
<button onclick="sendMessage()">Send Message to Worker</button>
<p id="demo"></p>

<script>
let worker2;

function sendMessage() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(worker2) == "undefined") {
      worker2 = new Worker("ex2-messages/worker.js");
    }
    worker2.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
    worker2.postMessage("Hello Worker!");
  }
}

```



```

    } else {
        document.getElementById("demo").innerHTML = "Web Workers not supported.";
    }
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Starting and Stopping a Worker

You can stop a worker using `worker.terminate()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Start and Stop a Worker</h4>
<button onclick="startWorker3()">Start Worker</button>
<button onclick="stopWorker3()">Stop Worker</button>
<p id="demo"></p>

<script>
let worker3;

function startWorker3() {
    if (typeof(Worker) !== "undefined") {
        if (typeof(worker3) == "undefined") {
            worker3 = new Worker("ex3-start_stop/worker.js");
        }
        worker3.onmessage = function(event) {
            document.getElementById("demo").innerHTML = event.data;
        };
    } else {
        document.getElementById("demo").innerHTML = "Web Workers not supported.";
    }
}

function stopWorker3() {
    if (worker3) {
        worker3.terminate();
        worker3 = undefined;
        document.getElementById("demo").innerHTML = "Worker stopped";
    }
}

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 3

Result [View Example](#)

Worker Errors

You can catch errors in workers using **onerror**:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Web Workers</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>Worker Errors</h4>
```

```
<button onclick="startWorker4()">Start Worker (with error)</button>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let worker4;
```

```
function startWorker4() {
```

```
  if (typeof(Worker) !== "undefined") {
```

```
    worker4 = new Worker("ex4-errors/worker.js");
```

```
    worker4.onmessage = function(event) {
```

```
      document.getElementById("demo").innerHTML = event.data;
```

```
    };
```

```
    worker4.onerror = function(error) {
```

```
      document.getElementById("demo").innerHTML =
```

```
        "Error in worker: " + error.message + "<br>" +
```

```
        "Line: " + error.lineno + "<br>" +
```

```
        "File: " + error.filename;
```

```
    };
```

```
  } else {
```

```
    document.getElementById("demo").innerHTML = "Web Workers not supported.";
```

```
  }
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 4

Result [View Example](#)

Using Timers Inside a Worker

Workers can use `setTimeout()` and `setInterval()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Timers Inside a Worker</h4>
<button onclick="startWorker5()">Start Timer Worker</button>
<button onclick="stopWorker5()">Stop Timer Worker</button>
<p id="demo"></p>

<script>
let worker5;

function startWorker5() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(worker5) == "undefined") {
      worker5 = new Worker("ex5-timers/worker.js");
    }
    worker5.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
  } else {
    document.getElementById("demo").innerHTML = "Web Workers not supported.";
  }
}

function stopWorker5() {
  if (worker5) {
    worker5.terminate();
    worker5 = undefined;
    document.getElementById("demo").innerHTML = "Timer worker stopped";
  }
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Web Workers](#)

JavaScript Modules

Table of Contents

- [JavaScript Modules](#)
- [Module Files](#)
- [Example 1](#)
- [Export and Import](#)
- [Example 2](#)
- [Export Objects](#)
- [Example 3](#)
- [Reference](#)

What are Modules?

JavaScript modules allow you to break up your code into separate files.

This makes it easier to maintain the code-base.

Modules are imported from external files with the `import` statement.

Modules also rely on `type="module"` in the `<script>` tag.

Module Files

A module file exports functions or objects using the `export` keyword. The main file imports them using `import`:

math.js - Module file:

```
export const add = (a, b) => a + b;
export const multiply = (a, b) => a * b;
export const subtract = (a, b) => a - b;
export const divide = (a, b) => a / b;
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Modules</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Module Files</h4>
<p id="demo"></p>

<script type="module">
import { add, multiply } from './math.js';
```

```
document.getElementById("demo").innerHTML =  
  "5 + 3 = " + add(5, 3) + "<br>" +  
  "5 * 3 = " + multiply(5, 3);  
</script>  
  
</body>  
</html>
```

Example 1

Result [View Example](#)

Export and Import

You can export individual functions and import them by name:

message.js - Module file:

```
export const greeting = "Hello";  
export function welcome(name) {  
  return greeting + ", " + name + "!";  
}  
export const farewell = (name) => "Goodbye, " + name + "!";
```

Main HTML file:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Modules</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>Export and Import</h4>  
<p id="demo"></p>  
  
<script type="module">  
import { greeting, welcome, farewell } from './message.js';  
  
document.getElementById("demo").innerHTML =  
  greeting + "<br>" +  
  welcome("John") + "<br>" +  
  farewell("Jane");  
</script>  
  
</body>  
</html>
```

Example 2

Result [View Example](#)

Export Objects

You can export entire objects from a module:

person.js - Module file:

```
export const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

export const cars = ["BMW", "Volvo", "Saab", "Ford"];
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Modules</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Export Objects</h4>
<p id="demo"></p>

<script type="module">
import { person, cars } from './person.js';

document.getElementById("demo").innerHTML =
  "Person: " + person.fullName() + "<br>" +
  "Age: " + person.age + "<br><br>" +
  "Cars: " + cars.join(", ");
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Modules](#)

JavaScript Module Export

Table of Contents

- [JavaScript Module Export](#)
- [Named Exports](#)
- [Example 1](#)
- [Default Exports](#)
- [Example 2](#)
- [Combining Default + Named](#)
- [Example 3](#)
- [Reference](#)

The Export Keyword

The **export** keyword is used to export functions, objects, or primitive values from a module file so they can be used in other files with the **import** keyword.

There are two types of exports: **Named Exports** and **Default Exports**.

Named Exports

Named exports allow you to export multiple items from a module. The import must use the exact same name:

math.js - Module file with named exports:

```
export const square = (x) => x * x;  
export const cube = (x) => x * x * x;  
export const double = (x) => x * 2;
```

Main HTML file:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Module Export</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>Named Exports</h4>  
<p id="demo"></p>  
  
<script type="module">  
import { square, cube, double } from './math.js';  
  
document.getElementById("demo").innerHTML =  
  "square(5) = " + square(5) + "<br>" +  
  "cube(3) = " + cube(3) + "<br>" +
```



```
"double(10) = " + double(10);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Default Exports

A module can have one default export. Default exports don't need curly braces when importing, and can be imported with any name:

message.js - Module file with default export:

```
const greet = (name) => "Hello, " + name + "!";
export default greet;
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Export</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Default Exports</h4>
<p id="demo"></p>

<script type="module">
import myGreeting from './message.js';

document.getElementById("demo").innerHTML =
  myGreeting("John") + "<br>" +
  myGreeting("Jane");
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Combining Default + Named

You can combine default and named exports in the same module:

toolbox.js - Module file with both default and named exports:

```
export default function(a, b) {  
  return a + b;  
}  
  
export const subtract = (a, b) => a - b;  
export const multiply = (a, b) => a * b;
```

Main HTML file:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Module Export</h2>  
<p>From w3schools.com, Experiment by Teeratus_R</p>  
  
<h4>Combining Default + Named</h4>  
<p id="demo"></p>  
  
<script type="module">  
import myAdd, { subtract, multiply } from './toolbox.js';  
  
document.getElementById("demo").innerHTML =  
  "Default (add): 10 + 5 = " + myAdd(10, 5) + "<br>" +  
  "Named (subtract): 10 - 5 = " + subtract(10, 5) + "<br>" +  
  "Named (multiply): 10 * 5 = " + multiply(10, 5);  
</script>  
  
</body>  
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Module Export](#)

JavaScript Module Import

Table of Contents

- [JavaScript Module Import](#)
- [Named Imports](#)
- [Example 1](#)
- [Default Import](#)
- [Example 2](#)
- [Default + Named Import](#)
- [Example 3](#)
- [Importing All](#)
- [Example 4](#)
- [Reference](#)

Named Imports

Named imports use curly braces `{}` and must match the exported name exactly. You can import multiple items:

utils.js - Module file:

```
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const multiply = (a, b) => a * b;
export const divide = (a, b) => a / b;
export const PI = 3.14159;
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Import</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Named Imports</h4>
<p id="demo"></p>

<script type="module">
import { add, multiply, PI } from './utils.js';

document.getElementById("demo").innerHTML =
  "add(10, 5) = " + add(10, 5) + "<br>" +
  "multiply(10, 5) = " + multiply(10, 5) + "<br>" +
  "PI = " + PI;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Default Import

Default imports don't need curly braces. You can name the import anything you want:

greet.js - Module file:

```
export default function(name) {
  return "Hello " + name + "!";
}
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Import</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Default Import</h4>
<p id="demo"></p>

<script type="module">
import sayHello from './greet.js';

document.getElementById("demo").innerHTML =
  sayHello("John") + "<br>" +
  sayHello("Jane");
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Default + Named Import

You can import both default and named exports in a single statement:

calc.js - Module file:

```
export default function(a, b) {
  return a + b;
}

export const square = (x) => x * x;
export const cube = (x) => x * x * x;
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Import</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Default + Named Import</h4>
<p id="demo"></p>

<script type="module">
import myAdd, { square, cube } from './calc.js';

document.getElementById("demo").innerHTML =
  "Default (add): 10 + 5 = " + myAdd(10, 5) + "<br>" +
  "square(5) = " + square(5) + "<br>" +
  "cube(3) = " + cube(3);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Importing All

Use ***** to import everything from a module as a namespace object:

colors.js - Module file:

```
export const red = "#FF0000";
export const green = "#00FF00";
export const blue = "#0000FF";
export const white = "#FFFFFF";
export const black = "#000000";
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Import</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Importing All</h4>
<p id="demo"></p>

<script type="module">
import * as Colors from './colors.js';

document.getElementById("demo").innerHTML =
  "Red: " + Colors.red + "<br>" +
  "Green: " + Colors.green + "<br>" +
  "Blue: " + Colors.blue + "<br>" +
  "White: " + Colors.white + "<br>" +
  "Black: " + Colors.black;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Module Import](#)

JavaScript Module Namespace

Table of Contents

- [JavaScript Module Namespace](#)
- [The Module Namespace Object](#)
- [Example 1](#)
- [Module Namespace Export](#)
- [Example 2](#)
- [Aggregator Scripts](#)
- [Example 3](#)
- [Reference](#)

The Module Namespace Object

A module namespace object is created when you import everything from a module using `import * as name`.

The namespace object contains all the exports from the module as properties:

shapes.js - Module file:

```
export const circle = (r) => Math.PI * r * r;
export const square = (s) => s * s;
export const rectangle = (w, h) => w * h;
export const triangle = (b, h) => (b * h) / 2;
export const PI = Math.PI;
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Namespace</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Module Namespace Object</h4>
<p id="demo"></p>

<script type="module">
import * as Shapes from './shapes.js';

document.getElementById("demo").innerHTML =
  "Circle area (r=5): " + Shapes.circle(5).toFixed(2) + "<br>" +
  "Square area (s=4): " + Shapes.square(4) + "<br>" +
  "Rectangle area (4x3): " + Shapes.rectangle(4, 3) + "<br>" +
  "Triangle area (6x2): " + Shapes.triangle(6, 2) + "<br>" +
  "PI: " + Shapes.PI.toFixed(4);
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Module Namespace Export

Namespace exports allow you to re-export everything from another module as a namespace:

math_ops.js - Individual math functions:

```
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const multiply = (a, b) => a * b;
export const divide = (a, b) => a / b;
```

utilities.js - Aggregator that re-exports with namespace:

```
export * as MathOps from './math_ops.js';
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Namespace</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Module Namespace Export</h4>
<p id="demo"></p>

<script type="module">
import { MathOps } from './utilities.js';

document.getElementById("demo").innerHTML =
  "MathOps.add(10, 5): " + MathOps.add(10, 5) + "<br>" +
  "MathOps.subtract(10, 5): " + MathOps.subtract(10, 5) + "<br>" +
  "MathOps.multiply(10, 5): " + MathOps.multiply(10, 5) + "<br>" +
  "MathOps.divide(10, 5): " + MathOps.divide(10, 5);
</script>

</body>
</html>
```


Example 2

Result [View Example](#)

Aggregator Scripts

Aggregator scripts combine multiple modules into a single import point using re-exports:

string_utils.js - String utility functions:

```
export const toUpper = (s) => s.toUpperCase();
export const toLower = (s) => s.toLowerCase();
export const reverse = (s) => s.split("").reverse().join("");
```

number_utils.js - Number utility functions:

```
export const isEven = (n) => n % 2 === 0;
export const isOdd = (n) => n % 2 !== 0;
export const factorial = (n) => n <= 1 ? 1 : n * factorial(n - 1);
```

index.js - Aggregator script that re-exports everything:

```
export * from './string_utils.js';
export * from './number_utils.js';
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Module Namespace</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Aggregator Scripts</h4>
<p id="demo"></p>

<script type="module">
import { toUpper, reverse, isEven, factorial } from './index.js';

document.getElementById("demo").innerHTML =
  "toUpper('hello'): " + toUpper("hello") + "<br>" +
  "reverse('world'): " + reverse("world") + "<br>" +
  "isEven(7): " + isEven(7) + "<br>" +
  "factorial(5): " + factorial(5);
</script>

</body>
```

```
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Module Namespace](#)

JavaScript Dynamic Import

Table of Contents

- [JavaScript Dynamic Import](#)
- [How Dynamic Import Works](#)
- [Example 1](#)
- [Another Example](#)
- [Example 2](#)
- [Conditional Loading](#)
- [Example 3](#)
- [Reference](#)

JavaScript Dynamic Import

Dynamic import allows you to import modules on demand, at runtime, using the `import()` function.

Unlike static imports (`import ... from ...`), dynamic imports:

- Can be used inside regular scripts (not just `<script type="module">`)
- Can be called conditionally
- Return a Promise that resolves to the module namespace object
- Can improve performance by loading code only when needed

How Dynamic Import Works

The `import()` function returns a Promise. The module is loaded when the `import()` call is executed:

math.js - Module file:

```
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const multiply = (a, b) => a * b;
export const divide = (a, b) => a / b;
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dynamic Import</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How Dynamic Import Works</h4>
<button onclick="loadMath()">Load Math Module</button>
<p id="demo"></p>

<script>
```

```
function loadMath() {
  import('./math.js')
  .then(function(module) {
    document.getElementById("demo").innerHTML =
      "Module loaded dynamically!<br>" +
      "add(10, 5) = " + module.add(10, 5) + "<br>" +
      "subtract(10, 5) = " + module.subtract(10, 5) + "<br>" +
      "multiply(10, 5) = " + module.multiply(10, 5) + "<br>" +
      "divide(10, 5) = " + module.divide(10, 5);
  })
  .catch(function(err) {
    document.getElementById("demo").innerHTML = "Error: " + err.message;
  });
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Another Example

Dynamic import can load modules that contain data conversion functions:

temperatures.js - Module file:

```
export const toCelsius = (f) => (f - 32) * 5 / 9;
export const toFahrenheit = (c) => (c * 9 / 5) + 32;
export const toKelvin = (c) => c + 273.15;
```

Main HTML file:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dynamic Import</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Another Example</h4>
<button onclick="loadTemp()">Load Temperature Module</button>
<p id="demo"></p>

<script>
function loadTemp() {
  import('./temperatures.js')
  .then(function(module) {
    document.getElementById("demo").innerHTML =
```

```

        "Temperature conversions:<br>" +
        "32°F = " + module.toCelsius(32).toFixed(1) + "°C<br>" +
        "100°F = " + module.toCelsius(100).toFixed(1) + "°C<br>" +
        "0°C = " + module.toFahrenheit(0).toFixed(1) + "°F<br>" +
        "100°C = " + module.toKelvin(100).toFixed(1) + "K";
    })
    .catch(function(err) {
        document.getElementById("demo").innerHTML = "Error: " + err.message;
    });
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Conditional Loading

Dynamic import is useful for conditionally loading modules based on user interaction or other runtime conditions:

greetings.js - Module file:

```

export const sayHello = (name) => "Hello, " + name + "!";
export const sayGoodbye = (name) => "Goodbye, " + name + "!";
export const sayThanks = (name) => "Thank you, " + name + "!";

```

Main HTML file:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Dynamic Import</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Conditional Loading</h4>
<p>Select a greeting:</p>
<select id="greetingSelect">
  <option value="hello">Hello</option>
  <option value="goodbye">Goodbye</option>
  <option value="thanks">Thanks</option>
</select>
<br><br>
<label>Name: <input type="text" id="nameInput" value="John"></label>
<br><br>
<button onclick="showGreeting()">Show Greeting</button>
<p id="demo"></p>

```

```

<script>
function showGreeting() {
  const type = document.getElementById("greetingSelect").value;
  const name = document.getElementById("nameInput").value || "Guest";

  import('./greetings.js')
    .then(function(module) {
      let message = "";
      if (type === "hello") {
        message = module.sayHello(name);
      } else if (type === "goodbye") {
        message = module.sayGoodbye(name);
      } else if (type === "thanks") {
        message = module.sayThanks(name);
      }
      document.getElementById("demo").innerHTML = message;
    })
    .catch(function(err) {
      document.getElementById("demo").innerHTML = "Error: " + err.message;
    });
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Dynamic Import](#)

JavaScript Metaprogramming

Table of Contents

- [JavaScript Metaprogramming](#)
- [Inspecting Objects](#)
- [Example 1](#)
- [Modify Objects](#)
- [Example 2](#)
- [Generate Dynamic Code](#)
- [Example 3](#)
- [Proxy Metaprogramming](#)
- [Example 4](#)
- [Proxy with Reflect](#)
- [Example 5](#)
- [Reference](#)

Metaprogramming

Metaprogramming is a programming technique where programs can treat other programs as their data. In JavaScript, metaprogramming allows you to:

- **Inspect** objects (get properties, check types)
- **Modify** objects (add/change properties)
- **Generate** dynamic code at runtime
- **Intercept** operations using Proxy and Reflect

Inspecting Objects

You can inspect objects using methods like `Object.keys()`, `Object.getOwnPropertyNames()`, and `typeof`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Inspecting Objects</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

let text = "Properties:<br>"
```

```

const keys = Object.keys(person);
for (let key of keys) {
  text += key + ": " + person[key] + " (" + typeof person[key] + ")<br>";
}
text += "<br>Number of properties: " + keys.length;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Modify Objects

You can modify objects at runtime by adding, changing, or deleting properties:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Modify Objects</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe"
};

// Add a new property
Object.defineProperty(person, "age", {
  value: 30,
  writable: true,
  enumerable: true
});

// Modify an existing property
Object.defineProperty(person, "firstName", {
  value: "Jane"
});

let text = "Modified object:<br>";
const keys = Object.keys(person);
for (let key of keys) {
  text += key + ": " + person[key] + "<br>";
}

```



```

}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Generate Dynamic Code

The `Function()` constructor can create functions dynamically from strings:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Generate Dynamic Code</h4>
<p id="demo"></p>

<script>
// Create a function dynamically using the Function constructor
const operation = "multiply";
let dynamicFunction;

if (operation === "add") {
  dynamicFunction = new Function("a", "b", "return a + b");
} else if (operation === "multiply") {
  dynamicFunction = new Function("a", "b", "return a * b");
} else if (operation === "power") {
  dynamicFunction = new Function("a", "b", "return Math.pow(a, b)");
}

document.getElementById("demo").innerHTML =
  "Dynamic function created for: " + operation + "<br>" +
  "5 * 3 = " + dynamicFunction(5, 3);
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Proxy Metaprogramming

A **Proxy** object wraps another object and intercepts operations like property access, assignment, and deletion:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy Metaprogramming</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

const handler = {
  get: function(target, property) {
    if (property === "fullName") {
      return target.firstName + " " + target.lastName;
    }
    return target[property];
  },
  set: function(target, property, value) {
    if (property === "age" && (value < 0 || value > 120)) {
      console.log("Invalid age: " + value);
      return false;
    }
    target[property] = value;
    return true;
  }
};

const proxy = new Proxy(person, handler);

// Use the proxy
proxy.age = 25; // Valid
proxy.age = 200; // Invalid - will be rejected

document.getElementById("demo").innerHTML =
  "firstName: " + proxy.firstName + "<br>" +
  "fullName: " + proxy.fullName + "<br>" +
  "age: " + proxy.age;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Proxy with Reflect

The **Reflect** object provides methods for interceptable JavaScript operations. It's often used with Proxy:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy with Reflect</h4>
<p id="demo"></p>

<script>
const logger = {
  get: function(target, property) {
    console.log("Accessing property: " + property);
    return Reflect.get(target, property);
  },
  set: function(target, property, value) {
    console.log("Setting property: " + property + " = " + value);
    return Reflect.set(target, property, value);
  }
};

const user = {
  name: "Alice",
  role: "admin"
};

const proxy = new Proxy(user, logger);

// Operations are logged via the proxy
proxy.name = "Bob";
let role = proxy.role;

document.getElementById("demo").innerHTML =
  "name: " + proxy.name + "<br>" +
  "role: " + proxy.role + "<br>" +
  "(Check the browser console for logged operations)";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Metaprogramming](#)

JavaScript Reflect

Table of Contents

- [JavaScript Reflect](#)
- [Reflect.has\(\)](#)
- [Example 1](#)
- [Reflect.get\(\) and Reflect.set\(\)](#)
- [Example 2](#)
- [Reflect.deleteProperty\(\)](#)
- [Example 3](#)
- [Reflect.apply\(\)](#)
- [Example 4](#)
- [Reflect.construct\(\)](#)
- [Example 5](#)
- [Reference](#)

The Reflect Object

The **Reflect** object provides methods for interceptable JavaScript operations. It makes metaprogramming easier by providing a unified way to perform operations like getting, setting, and deleting properties.

Before **Reflect**, developers had to use different syntax for these operations. With **Reflect**, you get a consistent API.

Reflect.has()

Reflect.has() checks if a property exists in an object. It's similar to the **in** operator:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.has()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Check if properties exist using Reflect.has()
let text = "Checking properties using Reflect.has():<br>";
text += "Has 'firstName': " + Reflect.has(person, "firstName") + "<br>";
```

```

text += "Has 'email': " + Reflect.has(person, "email") + "<br>";
text += "Has 'age': " + Reflect.has(person, "age");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Reflect.get() and Reflect.set()

Reflect.get() gets a property value. **Reflect.set()** sets a property value:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.get() and Reflect.set()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Get a property using Reflect.get()
let name = Reflect.get(person, "firstName");

// Set a property using Reflect.set()
Reflect.set(person, "age", 35);
Reflect.set(person, "email", "john@example.com");

document.getElementById("demo").innerHTML =
  "Reflect.get(person, 'firstName'): " + name + "<br>" +
  "Reflect.set(person, 'age', 35): " + Reflect.get(person, "age") + "<br>" +
  "Reflect.set(person, 'email', 'john@example.com'): " + Reflect.get(person, "email")
+ "<br>" +
  "All properties: " + Object.keys(person).join(", ");
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Reflect.deleteProperty()

`Reflect.deleteProperty()` deletes a property from an object. It's similar to the `delete` operator:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.deleteProperty()</h4>
<p id="demo"></p>

<script>
const car = {
  make: "Toyota",
  model: "Corolla",
  year: 2020,
  color: "blue"
};

let text = "Before delete:<br>";
text += Object.keys(car).join(", ") + "<br><br>";

// Delete a property using Reflect.deleteProperty()
Reflect.deleteProperty(car, "color");

text += "After deleteProperty(car, 'color'):<br>";
text += Object.keys(car).join(", ") + "<br><br>";

// Check if property still exists
text += "Has 'color': " + Reflect.has(car, "color");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reflect.apply()

`Reflect.apply()` calls a function with a given `this` value and arguments. It's similar to `Function.prototype.apply()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.apply()</h4>
<p id="demo"></p>

<script>
function greet(greeting, punctuation) {
  return greeting + ", " + this.name + punctuation;
}

const person = { name: "John" };
const member = { name: "Jane" };

// Using Reflect.apply() with different 'this' values
let result1 = Reflect.apply(greet, person, ["Hello", "!"]);
let result2 = Reflect.apply(greet, member, ["Hi", "!!"]);

document.getElementById("demo").innerHTML =
  "Reflect.apply(greet, person, ['Hello', '!']): " + result1 + "<br>" +
  "Reflect.apply(greet, member, ['Hi', '!!']): " + result2;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reflect.construct()

Reflect.construct() creates a new instance of a constructor function. It's similar to the **new** keyword:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.construct()</h4>
<p id="demo"></p>

<script>
class Car {
  constructor(make, model, year) {

```



```

        this.make = make;
        this.model = model;
        this.year = year;
    }
    getInfo() {
        return this.year + " " + this.make + " " + this.model;
    }
}

// Using Reflect.construct() instead of 'new Car()'
const myCar = Reflect.construct(Car, ["Toyota", "Corolla", 2020]);
const yourCar = Reflect.construct(Car, ["Honda", "Civic", 2022]);

document.getElementById("demo").innerHTML =
    "Reflect.construct(Car, [...]) :<br>" +
    "My car: " + myCar.getInfo() + "<br>" +
    "Your car: " + yourCar.getInfo() + "<br><br>" +
    "Instanceof check: " + (myCar instanceof Car);
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Reflect](#)

JavaScript Proxy

Table of Contents

- [JavaScript Proxy](#)
- [What is a Proxy?](#)
- [Example 1](#)
- [Proxy Logging](#)
- [Example 2](#)
- [Proxy Validation](#)
- [Example 3](#)
- [Virtual Properties](#)
- [Example 4](#)
- [Dynamic Functions](#)
- [Example 5](#)
- [Reference](#)

What is a Proxy?

A Proxy object wraps another object and intercepts fundamental operations like property lookup, assignment, enumeration, and function invocation.

The Proxy is created with two parameters:

- **target:** the original object that the proxy wraps
- **handler:** an object that defines which operations will be intercepted (traps)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is a Proxy?</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe"
};

const handler = {
  get: function(target, property) {
    return target[property];
  }
};

const proxy = new Proxy(person, handler);
```

```

document.getElementById("demo").innerHTML =
  "proxy.firstName: " + proxy.firstName + "<br>" +
  "proxy.lastName: " + proxy.lastName + "<br>" +
  "proxy is a Proxy: " + (proxy !== person);
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Proxy Logging

A Proxy can log all property accesses without modifying the original object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy Logging</h4>
<p id="demo"></p>

<script>
const user = {
  name: "Alice",
  age: 30,
  role: "admin"
};

const loggingHandler = {
  get: function(target, property) {
    console.log("LOG: Accessing property '" + property + "'");
    console.log("LOG: Current value is:", target[property]);
    return target[property];
  },
  set: function(target, property, value) {
    console.log("LOG: Setting property '" + property + "' to", value);
    console.log("LOG: Previous value was:", target[property]);
    target[property] = value;
    return true;
  }
};

const proxy = new Proxy(user, loggingHandler);

// Access and modify through the proxy

```

```

proxy.name = "Bob";
let userRole = proxy.role;
proxy.age = 31;

document.getElementById("demo").innerHTML =
  "name: " + proxy.name + "<br>" +
  "age: " + proxy.age + "<br>" +
  "role: " + proxy.role + "<br><br>" +
  "(Check browser console to see all log messages)";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Proxy Validation

A Proxy can validate data before setting it on the target object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy Validation</h4>
<p id="demo"></p>

<script>
const person = {
  name: "John",
  age: 25
};

const validator = {
  set: function(target, property, value) {
    if (property === "age") {
      if (typeof value !== "number") {
        throw new Error("Age must be a number");
      }
      if (value < 0 || value > 120) {
        throw new Error("Age must be between 0 and 120");
      }
    }
    if (property === "name") {
      if (typeof value !== "string") {
        throw new Error("Name must be a string");
      }
    }
  }
}

```

```

        if (value.length < 2) {
            throw new Error("Name must be at least 2 characters");
        }
    }
    target[property] = value;
    return true;
}
};

const proxy = new Proxy(person, validator);

try {
    proxy.age = 30;    // Valid
    proxy.name = "Jane"; // Valid
    // proxy.age = 200; // Would throw error
    // proxy.name = "J"; // Would throw error

    document.getElementById("demo").innerHTML =
        "Validated proxy:<br>" +
        "name: " + proxy.name + "<br>" +
        "age: " + proxy.age + "<br><br>" +
        "Age and name validation is active - invalid values are rejected!";
} catch(err) {
    document.getElementById("demo").innerHTML = "Error: " + err.message;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Virtual Properties

A Proxy can create virtual properties that don't exist on the target object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Virtual Properties</h4>
<p id="demo"></p>

<script>
const person = {
    firstName: "John",
    lastName: "Doe",

```

```

    birthYear: 1990
  };

  const handler = {
    get: function(target, property) {
      if (property === "fullName") {
        return target.firstName + " " + target.lastName;
      }
      if (property === "age") {
        const year = new Date().getFullYear();
        return year - target.birthYear;
      }
      return target[property];
    }
  };

  const proxy = new Proxy(person, handler);

  document.getElementById("demo").innerHTML =
    "firstName: " + proxy.firstName + "<br>" +
    "lastName: " + proxy.lastName + "<br>" +
    "fullName (virtual): " + proxy.fullName + "<br>" +
    "age (virtual): " + proxy.age;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Dynamic Functions

A Proxy can intercept function calls using the **apply** trap:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Dynamic Functions</h4>
<p id="demo"></p>

<script>
function greet(name) {
  return "Hello, " + name + "!";
}

const handler = {

```

```

    apply: function(target, thisArg, args) {
        console.log("Function called with arguments:", args);
        console.log("Call count: " + (++callCount));
        return target.apply(thisArg, args);
    }
};

let callCount = 0;
const proxy = new Proxy(greet, handler);

// Call through proxy
let result1 = proxy("Alice");
let result2 = proxy("Bob");
let result3 = proxy("Charlie");

document.getElementById("demo").innerHTML =
    result1 + "<br>" +
    result2 + "<br>" +
    result3 + "<br><br>" +
    "Total function calls: " + callCount + "<br>" +
    "(Check browser console for call logs)";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Proxy](#)

JavaScript Typed Arrays

Table of Contents

- [JavaScript Typed Arrays](#)
- [Typed Array Basics](#)
- [Example 1](#)
- [8 Bit Integers](#)
- [Example 2](#)
- [16-Bit and 32-Bit Integers](#)
- [Example 3](#)
- [Floating Point Numbers](#)
- [Example 4](#)
- [Uint8Array vs Uint8ClampedArray](#)
- [Example 5](#)
- [Reference](#)

Typed Array Basics

Typed arrays are array-like objects that provide a mechanism for reading and writing raw binary data in memory buffers.

Unlike regular arrays, typed arrays always hold the same type of data and have a fixed length.

Common typed array types:

Type	Description	Size
<code>Int8Array</code>	8-bit signed integer	1 byte
<code>Uint8Array</code>	8-bit unsigned integer	1 byte
<code>Uint8ClampedArray</code>	8-bit unsigned integer (clamped)	1 byte
<code>Int16Array</code>	16-bit signed integer	2 bytes
<code>Uint16Array</code>	16-bit unsigned integer	2 bytes
<code>Int32Array</code>	32-bit signed integer	4 bytes
<code>Uint32Array</code>	32-bit unsigned integer	4 bytes
<code>Float32Array</code>	32-bit floating point	4 bytes
<code>Float64Array</code>	64-bit floating point	8 bytes


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Typed Array Basics</h4>
<p id="demo"></p>

<script>
// Create a typed array with initial values
const numbers = new Int8Array([10, 20, 30, 40, 50]);

let text = "Int8Array values:<br>";
for (let num of numbers) {
    text += num + " ";
}
text += "<br><br>";
text += "Length: " + numbers.length + "<br>";
text += "BYTES_PER_ELEMENT: " + numbers.BYTES_PER_ELEMENT + "<br>";
text += "Constructor: " + numbers.constructor.name;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

8 Bit Integers

Int8Array holds 8-bit signed integers (-128 to 127). **Uint8Array** holds 8-bit unsigned integers (0 to 255):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>8 Bit Integers</h4>
<p id="demo"></p>

<script>
// Int8Array - signed (-128 to 127)
const signed = new Int8Array([-10, 0, 50, 127, -128]);

```

```
// Uint8Array - unsigned (0 to 255)
const unsigned = new Uint8Array([0, 100, 200, 255]);

let text = "Int8Array (signed -128 to 127):<br>";
for (let num of signed) {
    text += num + " ";
}
text += "<br><br>";

text += "Uint8Array (unsigned 0 to 255):<br>";
for (let num of unsigned) {
    text += num + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

16-Bit and 32-Bit Integers

Int16Array holds 16-bit signed integers. **Int32Array** holds 32-bit signed integers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>16-Bit and 32-Bit Integers</h4>
<p id="demo"></p>

<script>
// Int16Array - larger range
const int16 = new Int16Array([1000, 20000, 30000]);

// Int32Array - even larger range
const int32 = new Int32Array([100000, 2000000]);

let text = "Int16Array (16-bit):<br>";
for (let num of int16) {
    text += num + " ";
}
text += "<br>BYTES_PER_ELEMENT: " + int16.BYTES_PER_ELEMENT + "<br><br>";

text += "Int32Array (32-bit):<br>";
```

```

for (let num of int32) {
    text += num + " ";
}
text += "<br>BYTES_PER_ELEMENT: " + int32.BYTES_PER_ELEMENT;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Floating Point Numbers

Float32Array and **Float64Array** hold floating point numbers:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Floating Point Numbers</h4>
<p id="demo"></p>

<script>
// Float32Array - 32-bit floating point
const float32 = new Float32Array([1.5, 3.14, 9.99, 100.05]);

// Float64Array - 64-bit floating point (double precision)
const float64 = new Float64Array([0.123456789, 123456.789]);

let text = "Float32Array (32-bit float):<br>";
for (let num of float32) {
    text += num + " ";
}
text += "<br><br>";

text += "Float64Array (64-bit double):<br>";
for (let num of float64) {
    text += num + " ";
}
text += "<br><br>";

text += "Float32 BYTES_PER_ELEMENT: " + float32.BYTES_PER_ELEMENT + "<br>";
text += "Float64 BYTES_PER_ELEMENT: " + float64.BYTES_PER_ELEMENT;

document.getElementById("demo").innerHTML = text;

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 4

Result [View Example](#)

Uint8Array vs Uint8ClampedArray

The difference between `Uint8Array` and `Uint8ClampedArray` is how they handle values outside the 0-255 range:

- `Uint8Array`: wraps around (modulo 256)
- `Uint8ClampedArray`: clamps to 0 or 255

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Typed Arrays</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>Uint8Array vs Uint8ClampedArray</h4>
```

```
<p id="demo"></p>
```

```
<script>
```

```
// Uint8Array - wraps around (modulo 256)
```

```
const uint8 = new Uint8Array([-10, 0, 100, 255, 300]);
```

```
// Uint8ClampedArray - clamps to 0 or 255
```

```
const clamped = new Uint8ClampedArray([-10, 0, 100, 255, 300]);
```

```
let text = "Uint8Array (wraps around):<br>";
```

```
for (let num of uint8) {
```

```
  text += num + " ";
```

```
}
```

```
text += "<br>(-10 wraps to " + uint8[0] + ", 300 wraps to " + uint8[4] + ")";
```

```
text += "<br><br>";
```

```
text += "Uint8ClampedArray (clamps):<br>";
```

```
for (let num of clamped) {
```

```
  text += num + " ";
```

```
}
```

```
text += "<br>(-10 clamped to " + clamped[0] + ", 300 clamped to " + clamped[4] + ")";
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Typed Arrays](#)

JavaScript Typed Array Methods

Table of Contents

- [JavaScript Typed Array Methods](#)
- [The from\(\) Method](#)
- [Example 1](#)
- [The of\(\) Method](#)
- [Example 2](#)
- [The fill\(\) Method](#)
- [Example 3](#)
- [The find\(\) Method](#)
- [Example 4](#)
- [The some\(\) Method](#)
- [Example 5](#)
- [Reference](#)

The from() Method

`TypedArray.from()` creates a new typed array from an array-like or iterable object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The from() Method</h4>
<p id="demo"></p>

<script>
// Create Int8Array from a regular array
const regularArray = [10, 20, 30, 40, 50];
const typedArray = Int8Array.from(regularArray);

let text = "Original array: " + regularArray.join(", ") + "<br>";
text += "Typed array: ";
for (let num of typedArray) {
  text += num + " ";
}
text += "<br>Constructor: " + typedArray.constructor.name;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The of() Method

`TypedArray.of()` creates a new typed array with a variable number of arguments:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The of() Method</h4>
<p id="demo"></p>

<script>
// Create typed array from individual values
const values = Int16Array.of(100, 200, 300, 400, 500);

let text = "Int16Array.of(100, 200, 300, 400, 500):<br>";
for (let num of values) {
  text += num + " ";
}
text += "<br><br>";
text += "Length: " + values.length + "<br>";
text += "Constructor: " + values.constructor.name;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The fill() Method

`fill()` fills all elements of a typed array with a static value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>The fill() Method</h4>
<p id="demo"></p>

<script>
// Create an array and fill it
const arr = new Int8Array(8);
arr.fill(42);

let text = "Int8Array(8) filled with 42:<br>";
for (let num of arr) {
  text += num + " ";
}
text += "<br><br>";

// Fill with a different value from index 3 to 6
arr.fill(99, 3, 6);
text += "After arr.fill(99, 3, 6):<br>";
for (let num of arr) {
  text += num + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The find() Method

find() returns the first element that satisfies a testing function:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The find() Method</h4>
<p id="demo"></p>

<script>
const numbers = new Int8Array([10, 25, 30, 45, 50, 65, 70]);

// Find first element > 40
const found = numbers.find(function(value) {
  return value > 40;
});

```



```
// Find first element > 10 and < 30
const found2 = numbers.find(function(value) {
  return value > 10 && value < 30;
});

document.getElementById("demo").innerHTML =
  "Numbers: 10, 25, 30, 45, 50, 65, 70<br><br>" +
  "First > 40: " + found + "<br>" +
  "First > 10 and < 30: " + found2;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The some() Method

some() checks if any element satisfies a testing function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Methods</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The some() Method</h4>
<p id="demo"></p>

<script>
const numbers = new Int8Array([5, 10, 15, 20, 25]);

// Check if any number > 20
const hasLarge = numbers.some(function(value) {
  return value > 20;
});

// Check if any number < 0
const hasNegative = numbers.some(function(value) {
  return value < 0;
});

// Check if any number == 15
const hasFifteen = numbers.some(function(value) {
  return value === 15;
});

document.getElementById("demo").innerHTML =
```

```
"Numbers: 5, 10, 15, 20, 25<br><br>" +  
"Any > 20? " + hasLarge + "<br>" +  
"Any < 0? " + hasNegative + "<br>" +  
"Any == 15? " + hasFifteen;  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Typed Array Methods](#)

JavaScript Typed Array Reference

Table of Contents

- [JavaScript Typed Array Reference](#)
- [Typed Array Types Overview](#)
- [Example 1](#)
- [Common Methods Reference](#)
- [Example 2](#)
- [Working with Buffers](#)
- [Example 3](#)
- [Reference](#)

Typed Array Types Overview

JavaScript provides the following typed array types:

Constructor	Type	Range	Size	Description
<code>Int8Array</code>	8-bit signed integer	-128 to 127	1 byte	
<code>Uint8Array</code>	8-bit unsigned integer	0 to 255	1 byte	
<code>Uint8ClampedArray</code>	8-bit unsigned integer (clamped)	0 to 255	1 byte	Canvas data
<code>Int16Array</code>	16-bit signed integer	-32768 to 32767	2 bytes	
<code>Uint16Array</code>	16-bit unsigned integer	0 to 65535	2 bytes	
<code>Int32Array</code>	32-bit signed integer	-2^{31} to $2^{31}-1$	4 bytes	
<code>Uint32Array</code>	32-bit unsigned integer	0 to $2^{32}-1$	4 bytes	
<code>BigInt64Array</code>	64-bit signed integer	$-(2^{63})$ to $2^{63}-1$	8 bytes	
<code>BigUint64Array</code>	64-bit unsigned integer	0 to $2^{64}-1$	8 bytes	

Float32Array	32-bit floating point	~1.2E-38 to 3.4E38	4 bytes
--------------	-----------------------	--------------------	---------

Float64Array	64-bit floating point	~5.0E-324 to 1.8E308	8 bytes
--------------	-----------------------	----------------------	---------

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Typed Array Types Overview</h4>
<p id="demo"></p>

<script>
// Demonstrate different typed array types
const int8 = new Int8Array([-10, 0, 50, 100]);
const uint8 = new Uint8Array([0, 100, 200, 255]);
const int16 = new Int16Array([-1000, 0, 1000, 30000]);
const int32 = new Int32Array([-100000, 0, 100000]);
const float32 = new Float32Array([1.5, 3.14, 9.99]);
const float64 = new Float64Array([0.123456789, 123456.789]);

let text = "";

text += "Int8Array: " + Array.from(int8).join(", ") + "<br>";
text += "Uint8Array: " + Array.from(uint8).join(", ") + "<br>";
text += "Int16Array: " + Array.from(int16).join(", ") + "<br>";
text += "Int32Array: " + Array.from(int32).join(", ") + "<br>";
text += "Float32Array: " + Array.from(float32).join(", ") + "<br>";
text += "Float64Array: " + Array.from(float64).join(", ");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Common Methods Reference

Typed arrays support many standard array methods. Here's a quick reference of commonly used methods:

Method	Description
<code>from()</code>	Creates a typed array from an array-like object
<code>of()</code>	Creates a typed array from variable arguments
<code>fill()</code>	Fills elements with a static value
<code>find()</code>	Returns first element that passes a test
<code>some()</code>	Checks if any element passes a test
<code>every()</code>	Checks if all elements pass a test
<code>filter()</code>	Creates a new array with elements that pass a test
<code>map()</code>	Creates a new array by applying a function
<code>reduce()</code>	Reduces the array to a single value
<code>sort()</code>	Sorts the elements
<code>slice()</code>	Extracts a section of the array
<code>includes()</code>	Checks if an element is in the array
<code>indexOf()</code>	Returns the index of a specified element
<code>join()</code>	Joins all elements into a string
<code>reverse()</code>	Reverses the order of elements
<code>forEach()</code>	Executes a function for each element

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Common Methods Reference</h4>
<p id="demo"></p>

```

```

<script>
const numbers = new Int8Array([10, 5, 20, 15, 25, 30]);

// Using various array methods on typed arrays
let text = "Original: " + Array.from(numbers).join(", ") + "<br><br>";

// filter
const filtered = numbers.filter(function(v) { return v > 15; });
text += "filter(v > 15): " + Array.from(filtered).join(", ") + "<br>";

// map
const doubled = numbers.map(function(v) { return v * 2; });
text += "map(v * 2): " + Array.from(doubled).join(", ") + "<br>";

// includes
text += "includes(20): " + numbers.includes(20) + "<br>";

// indexOf
text += "indexOf(15): " + numbers.indexOf(15) + "<br>";

// every
const allPositive = numbers.every(function(v) { return v > 0; });
text += "every(v > 0): " + allPositive + "<br>";

// reduce
const sum = numbers.reduce(function(acc, v) { return acc + v; }, 0);
text += "reduce(sum): " + sum;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Working with Buffers

Typed arrays are built on top of **ArrayBuffer**. Multiple typed arrays can share the same buffer:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Array Reference</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Working with Buffers</h4>
<p id="demo"></p>

```

```

<script>
// Create a buffer of 16 bytes
const buffer = new ArrayBuffer(16);

// Create different views of the same buffer
const int8View = new Int8Array(buffer);
const uint16View = new Uint16Array(buffer);
const int32View = new Int32Array(buffer);

// Write data using one view
int8View[0] = 10;
int8View[1] = 20;
int32View[2] = 100000;

let text = "Same buffer, different views:<br><br>";
text += "Int8Array view: ";
for (let i = 0; i < int8View.length; i++) text += int8View[i] + " ";
text += "<br><br>";

text += "Buffer byteLength: " + buffer.byteLength + " bytes<br><br>";

text += "Int8Array length: " + int8View.length + " (1 byte each)<br>";
text += "Uint16Array length: " + uint16View.length + " (2 bytes each)<br>";
text += "Int32Array length: " + int32View.length + " (4 bytes each)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Typed Array Reference](#)

JavaScript ArrayBuffer

Table of Contents

- [JavaScript ArrayBuffer](#)
- [Creating an ArrayBuffer](#)
- [Example 1](#)
- [Using an Uint8Array with ArrayBuffer](#)
- [Example 2](#)
- [Using a DataView](#)
- [Example 3](#)
- [ArrayBuffer.slice\(\)](#)
- [Example 4](#)
- [SharedArrayBuffer](#)
- [Example 5](#)
- [Reference](#)

ArrayBuffer

An **ArrayBuffer** is a fixed-length, raw binary data buffer in memory. It cannot be read or written directly. Instead, you create a **typed array view** or a **DataView** to read/write the buffer.

Creating an ArrayBuffer

You create an **ArrayBuffer** by specifying its size in bytes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating an ArrayBuffer</h4>
<p id="demo"></p>

<script>
// Create an ArrayBuffer with 16 bytes
const buffer = new ArrayBuffer(16);

let text = "ArrayBuffer created with 16 bytes<br>";
text += "byteLength: " + buffer.byteLength + "<br>";
text += "Max byteLength: " + ArrayBuffer.maxByteLength + "<br>";
text += "byteLength is read-only? " + (buffer.byteLength === 16);

document.getElementById("demo").innerHTML = text;
</script>

</body>
```



```
</html>
```

Example 1

Result [View Example](#)

Using an Uint8Array with ArrayBuffer

To access an **ArrayBuffer**, you need a typed array view. **Uint8Array** treats each byte as an unsigned 8-bit integer:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using an Uint8Array with ArrayBuffer</h4>
<p id="demo"></p>

<script>
// Create buffer and view
const buffer = new ArrayBuffer(8);
const view = new Uint8Array(buffer);

// Write values
view[0] = 65;   // 'A'
view[1] = 66;   // 'B'
view[2] = 67;   // 'C'
view[3] = 97;   // 'a'
view[4] = 98;   // 'b'

// Read values
let text = "Uint8Array view of ArrayBuffer(8):<br>";
for (let i = 0; i < view.length; i++) {
  text += "view[" + i + "] = " + view[i];
  if (view[i] >= 65 && view[i] <= 90) {
    text += " (char: " + String.fromCharCode(view[i]) + ")";
  }
  text += "<br>";
}
text += "<br>byteLength: " + buffer.byteLength;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using a DataView

A **DataView** provides a flexible interface to read/write different data types at any byte offset:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using a DataView</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(16);
const dataView = new DataView(buffer);

// Write different data types
dataView.setInt32(0, 42);           // 4 bytes at offset 0
dataView.setFloat32(4, 3.14);      // 4 bytes at offset 4
dataView.setInt16(8, 255);         // 2 bytes at offset 8

// Read back
let text = "DataView on ArrayBuffer(16):<br>";
text += "getInt32(0): " + dataView.getInt32(0) + "<br>";
text += "getFloat32(4): " + dataView.getFloat32(4) + "<br>";
text += "getInt16(8): " + dataView.getInt16(8) + "<br><br>";

// Check byte order (endianness)
const testBuffer = new ArrayBuffer(2);
const testView = new DataView(testBuffer);
testView.setInt16(0, 256);
text += "Endianness test: ";
if (testView.getUint8(0) === 1) {
  text += "Big-endian";
} else {
  text += "Little-endian";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

ArrayBuffer.slice()

You can create a new `ArrayBuffer` from a portion of an existing buffer using `slice()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>ArrayBuffer.slice()</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(16);
const view = new Uint8Array(buffer);

// Fill with values
for (let i = 0; i < view.length; i++) {
  view[i] = i * 10;
}

// Slice a portion
const sliced = buffer.slice(4, 12);
const slicedView = new Uint8Array(sliced);

let text = "Original buffer: ";
for (let i = 0; i < view.length; i++) text += view[i] + " ";
text += "<br>";

text += "Sliced buffer (bytes 4-12): ";
for (let i = 0; i < slicedView.length; i++) text += slicedView[i] + " ";
text += "<br><br>";

text += "Original byteLength: " + buffer.byteLength + "<br>";
text += "Sliced byteLength: " + sliced.byteLength;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

SharedArrayBuffer

A `SharedArrayBuffer` is similar to `ArrayBuffer` but can be shared between workers (threads):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>SharedArrayBuffer</h4>
<p id="demo"></p>

<script>
// Create a SharedArrayBuffer
const sharedBuffer = new SharedArrayBuffer(16);
const sharedView = new Int32Array(sharedBuffer);

// Write values
for (let i = 0; i < sharedView.length; i++) {
    sharedView[i] = i * 100;
}

let text = "SharedArrayBuffer created with 16 bytes<br>";
text += "byteLength: " + sharedBuffer.byteLength + "<br>";
text += "Int32Array length: " + sharedView.length + " (4 bytes each)<br><br>";

text += "Values: ";
for (let i = 0; i < sharedView.length; i++) {
    text += sharedView[i] + " ";
}
text += "<br><br>";

text += "SharedArrayBuffer can be used with:<br>";
text += "- Atomics.add()<br>";
text += "- Atomics.load()<br>";
text += "- Atomics.store()<br>";
text += "- Web Workers for multi-threading";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript ArrayBuffer](#)

JavaScript DataView

Table of Contents

- [JavaScript DataView](#)
- [Creating a DataView](#)
- [Example 1](#)
- [Reading and Writing Values](#)
- [Example 2](#)
- [Endianness \(Byte Order\)](#)
- [Example 3](#)
- [Mixed Types](#)
- [Example 4](#)
- [Encoding and Decoding a Record](#)
- [Example 5](#)
- [Reference](#)

What Is a DataView?

A **DataView** provides a low-level interface for reading and writing multiple number types in an **ArrayBuffer**.

Unlike typed arrays (like **Int32Array**), a **DataView**:

- Can read/write different data types in the same buffer
- Can start at any byte offset (not aligned to element size)
- Supports both big-endian and little-endian byte order

Creating a DataView

Create a **DataView** from an existing **ArrayBuffer**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DataView</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating a DataView</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(16);
const dataView = new DataView(buffer);

let text = "DataView created from ArrayBuffer(16)<br>";
text += "buffer: " + dataView.buffer.byteLength + " bytes<br>";
text += "byteOffset: " + dataView.byteOffset + "<br>";
text += "byteLength: " + dataView.byteLength;
```

```
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Reading and Writing Values

DataView provides methods like `setInt32()`, `getFloat32()`, etc. to read/write values at specific byte offsets:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DataView</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reading and Writing Values</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(16);
const dv = new DataView(buffer);

// Write values at different offsets
dv.setInt32(0, 42);           // 4 bytes at offset 0
dv.setFloat64(4, 3.14159);    // 8 bytes at offset 4
dv.setInt16(12, 255);         // 2 bytes at offset 12

// Read values back
let text = "Values written to DataView:<br><br>";
text += "getInt32(0): " + dv.getInt32(0) + " (4 bytes)<br>";
text += "getFloat64(4): " + dv.getFloat64(4) + " (8 bytes)<br>";
text += "getInt16(12): " + dv.getInt16(12) + " (2 bytes)<br><br>";

// Total bytes used: 4 + 8 + 2 = 14 bytes
text += "Total bytes used: 14 of " + buffer.byteLength;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Endianness (Byte Order)

Endianness refers to the byte order of multi-byte values. The `DataView` methods accept an optional `littleEndian` parameter:

- **Big-endian:** Most significant byte first (default)
- **Little-endian:** Least significant byte first

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DataView</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Endianness (Byte Order)</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(4);
const dv = new DataView(buffer);

// Write the same value with different endianness
dv.setInt32(0, 0x12345678, false); // Big-endian (default)
const bigEndian = dv.getInt32(0, false);

dv.setInt32(0, 0x12345678, true); // Little-endian
const littleEndian = dv.getInt32(0, true);

// Inspect individual bytes
const bytes = new Uint8Array(buffer);

let text = "Value: 0x12345678 (305419896)<br><br>";

text += "Big-endian bytes: ";
for (let b of bytes) text += "0x" + b.toString(16).toUpperCase() + " ";
text += "<br>";

dv.setInt32(0, 0x12345678, true);
text += "Little-endian bytes: ";
for (let b of bytes) text += "0x" + b.toString(16).toUpperCase() + " ";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Mixed Types

DataView allows mixing different data types in a single buffer:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DataView</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mixed Types</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(20);
const dv = new DataView(buffer);

// Write mixed data types
dv.setInt32(0, 2024);           // Year (4 bytes)
dv.setInt16(4, 12);            // Month (2 bytes)
dv.setInt16(6, 25);            // Day (2 bytes)
dv.setFloat32(8, 36.5);        // Temperature (4 bytes)
dv.setInt8(12, 72);            // Humidity % (1 byte)
dv.setInt8(13, 1);            // Active flag (1 byte)

// Read back
let text = "Mixed Data Types in DataView:<br><br>";
text += "Year: " + dv.getInt32(0) + "<br>";
text += "Month: " + dv.getInt16(4) + "<br>";
text += "Day: " + dv.getInt16(6) + "<br>";
text += "Temperature: " + dv.getFloat32(8) + "°C<br>";
text += "Humidity: " + dv.getInt8(12) + "%<br>";
text += "Active: " + (dv.getInt8(13) === 1);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Encoding and Decoding a Record

DataView is ideal for binary data formats like file headers or network packets:


```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DataView</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Encoding and Decoding a Record</h4>
<p id="demo"></p>

<script>
// Define a record structure
function encodeRecord(id, score, name) {
  const buf = new ArrayBuffer(16);
  const dv = new DataView(buf);
  dv.setInt32(0, id);           // ID (4 bytes)
  dv.setFloat64(4, score);      // Score (8 bytes)
  for (let i = 0; i < name.length; i++) {
    dv.setUint8(12 + i, name.charCodeAt(i)); // Name chars
  }
  dv.setUint8(15, 0);           // Null terminator
  return buf;
}

function decodeRecord(buf) {
  const dv = new DataView(buf);
  const id = dv.getInt32(0);
  const score = dv.getFloat64(4);
  let name = "";
  for (let i = 12; i < 16; i++) {
    const ch = dv.getUint8(i);
    if (ch === 0) break;
    name += String.fromCharCode(ch);
  }
  return { id, score, name };
}

// Encode a record
const buffer = encodeRecord(101, 95.5, "Bob");
const decoded = decodeRecord(buffer);

let text = "Binary Record (16 bytes):<br><br>";
text += "Encoded:<br>";
const view = new Uint8Array(buffer);
for (let b of view) text += b.toString(16).padStart(2, "0") + " ";
text += "<br><br>";

text += "Decoded:<br>";
text += "ID: " + decoded.id + "<br>";
text += "Score: " + decoded.score + "<br>";
text += "Name: " + decoded.name;

document.getElementById("demo").innerHTML = text;
</script>

```

```
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript DataView](#)

JavaScript Atomics

Table of Contents

- [JavaScript Atomics](#)
- [Creating a Shared Typed Array](#)
- [Example 1](#)
- [Atomics.load\(\) and Atomics.store\(\)](#)
- [Example 2](#)
- [Atomics.add\(\) and Atomics.sub\(\)](#)
- [Example 3](#)
- [Atomics.exchange\(\)](#)
- [Example 4](#)
- [Atomics.compareExchange\(\)](#)
- [Example 5](#)
- [Reference](#)

The Atomics Object

The **Atomics** object provides atomic operations as static methods. They are used with **SharedArrayBuffer** objects to ensure that operations complete without interruption, which is critical for multi-threaded environments (Web Workers).

Atomics operations help prevent race conditions when multiple threads access shared memory.

Creating a Shared Typed Array

To use Atomics, you first need a **SharedArrayBuffer** and a typed array view:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Atomics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating a Shared Typed Array</h4>
<p id="demo"></p>

<script>
// Create a SharedArrayBuffer with 4 x 32-bit integers = 16 bytes
const sharedBuffer = new SharedArrayBuffer(16);
const sharedArray = new Int32Array(sharedBuffer);

// Initialize values
sharedArray[0] = 10;
sharedArray[1] = 20;
sharedArray[2] = 30;
sharedArray[3] = 40;
```

```

let text = "SharedArrayBuffer created (16 bytes)<br>";
text += "Int32Array length: " + sharedArray.length + "<br><br>";
text += "Initial values: ";
for (let i = 0; i < sharedArray.length; i++) {
  text += sharedArray[i] + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Atoms.load() and Atoms.store()

`Atoms.load()` reads a value from a shared memory location. `Atoms.store()` writes a value to a shared memory location atomically:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Atomics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Atoms.load() and Atoms.store()</h4>
<p id="demo"></p>

<script>
const sharedBuffer = new SharedArrayBuffer(8);
const sharedArray = new Int32Array(sharedBuffer);

// Store values atomically
Atoms.store(sharedArray, 0, 42);
Atoms.store(sharedArray, 1, 100);

// Load values atomically
const val0 = Atoms.load(sharedArray, 0);
const val1 = Atoms.load(sharedArray, 1);

let text = "Atoms.store() and Atoms.load():<br><br>";
text += "After Atoms.store(sharedArray, 0, 42): " + val0 + "<br>";
text += "After Atoms.store(sharedArray, 1, 100): " + val1 + "<br><br>";

text += "All values: ";
for (let i = 0; i < sharedArray.length; i++) {
  text += Atoms.load(sharedArray, i) + " ";
}

```

```

}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Atoms.add() and Atoms.sub()

Atoms.add() adds a value at a given position and returns the old value. **Atoms.sub()** subtracts a value and returns the old value:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Atomics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Atoms.add() and Atoms.sub()</h4>
<p id="demo"></p>

<script>
const sharedBuffer = new SharedArrayBuffer(8);
const sharedArray = new Int32Array(sharedBuffer);

sharedArray[0] = 50;
sharedArray[1] = 100;

// Add and subtract atomically
let old1 = Atomics.add(sharedArray, 0, 25); // 50 + 25 = 75
let old2 = Atomics.sub(sharedArray, 1, 30); // 100 - 30 = 70

let text = "Atoms.add() and Atoms.sub():<br><br>";
text += "Atoms.add(sharedArray, 0, 25):<br>";
text += "  Old value: " + old1 + "<br>";
text += "  New value: " + Atomics.load(sharedArray, 0) + "<br><br>";

text += "Atoms.sub(sharedArray, 1, 30):<br>";
text += "  Old value: " + old2 + "<br>";
text += "  New value: " + Atomics.load(sharedArray, 1) + "<br><br>";

text += "Final values: " + Atomics.load(sharedArray, 0) + ", " +
Atoms.load(sharedArray, 1);

document.getElementById("demo").innerHTML = text;

```

```
</script>
```

```
</body>
```

```
</html>
```

Example 3

Result [View Example](#)

Atoms.exchange()

`Atoms.exchange()` exchanges a value at a given position and returns the old value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Atomics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Atoms.exchange()</h4>
<p id="demo"></p>

<script>
const sharedBuffer = new SharedArrayBuffer(8);
const sharedArray = new Int32Array(sharedBuffer);

sharedArray[0] = 99;

// Exchange the value atomically
let oldValue = Atomics.exchange(sharedArray, 0, 200);

let text = "Atoms.exchange(sharedArray, 0, 200):<br><br>";
text += "Old value: " + oldValue + "<br>";
text += "New value: " + Atomics.load(sharedArray, 0) + "<br><br>";

text += "This is useful for safely swapping values in shared memory.";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Atoms.compareExchange()

`Atomsics.compareExchange()` exchanges a value only if the current value equals a specified expected value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Atomics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Atomsics.compareExchange()</h4>
<p id="demo"></p>

<script>
const sharedBuffer = new SharedArrayBuffer(8);
const sharedArray = new Int32Array(sharedBuffer);

sharedArray[0] = 50;

// Try to exchange - only succeeds if current value is 50
let result1 = Atomics.compareExchange(sharedArray, 0, 50, 100);
let result2 = Atomics.compareExchange(sharedArray, 0, 50, 200); // Fails (value is now 100)

let text = "Atomsics.compareExchange():<br><br>";

text += "Initial value: 50<br><br>";

text += "Attempt 1: compareExchange(0, 50, 100)<br>";
text += "  Expected 50, actual was " + result1 + " (matched!)<br>";
text += "  New value: " + Atomics.load(sharedArray, 0) + "<br><br>";

text += "Attempt 2: compareExchange(0, 50, 200)<br>";
text += "  Expected 50, actual was " + result2 + " (did NOT match!)<br>";
text += "  New value: " + Atomics.load(sharedArray, 0) + " (unchanged)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Atomics](#)

JavaScript DOM Navigation

Table of Contents

- [JavaScript DOM Navigation](#)
- [DOM Nodes](#)
- [Child Nodes and Node Values](#)
- [Example 1](#)
- [DOM Root Nodes](#)
- [Example 2](#)
- [The nodeName Property](#)
- [Example 3](#)
- [The nodeType Property](#)
- [Example 4](#)
- [Navigating Between Nodes](#)
- [Example 5](#)
- [Reference](#)

DOM Nodes

According to the W3C HTML DOM standard, everything in an HTML document is a node:

- The entire document is a **document node**
- Every HTML element is an **element node**
- The text inside HTML elements are **text nodes**
- Every HTML attribute is an **attribute node**
- Comments are **comment nodes**

Child Nodes and Node Values

You can access node values using `nodeValue` and `childNodes[0].nodeValue`. The `innerHTML` property returns the text content of an element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Navigation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Child Nodes and Node Values</h4>

<p id="demo1">Hello World!</p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
const element = document.getElementById("demo1");
```



```
// Access node value via childNodes
document.getElementById("demo2").innerHTML =
  "Text via childNodes[0].nodeValue: " + element.childNodes[0].nodeValue;

// Access node value via innerHTML
document.getElementById("demo3").innerHTML =
  "Text via innerHTML: " + element.innerHTML;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

DOM Root Nodes

There are two root nodes in the DOM tree:

- `document.documentElement` - the full `<html>` element
- `document.body` - the `<body>` element

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Navigation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>DOM Root Nodes</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
document.getElementById("demo1").innerHTML =
  "document.documentElement: " + document.documentElement.tagName;

document.getElementById("demo2").innerHTML =
  "document.body: " + document.body.tagName;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The nodeName Property

The **nodeName** property returns the name of a node:

- For element nodes: the tag name (in uppercase)
- For text nodes: **#text**
- For attribute nodes: the attribute name
- For document nodes: **#document**

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Navigation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The nodeName Property</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
document.getElementById("demo1").innerHTML =
    "document.nodeName: " + document.nodeName;

document.getElementById("demo2").innerHTML =
    "document.body.nodeName: " + document.body.nodeName;

const element = document.getElementById("demo1");
document.getElementById("demo3").innerHTML =
    "First child nodeName: " + element.childNodes[0].nodeName;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The nodeType Property

The **nodeType** property returns an integer that specifies the type of the node:

Value	Type
1	Element
2	Attribute

3	Text
8	Comment
9	Document

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Navigation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The nodeType Property</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>
<p id="demo4"></p>

<script>
document.getElementById("demo1").innerHTML =
    "document.nodeType: " + document.nodeType + " (Document)";

document.getElementById("demo2").innerHTML =
    "document.body.nodeType: " + document.body.nodeType + " (Element)";

const element = document.getElementById("demo1");
document.getElementById("demo3").innerHTML =
    "First child nodeType: " + element.childNodes[0].nodeType + " (Text)";

const comments = [];
for (let node of document.body.childNodes) {
    if (node.nodeType === 8) {
        comments.push(node);
    }
}
document.getElementById("demo4").innerHTML =
    "Number of comment nodes: " + comments.length + " (Comment)";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Navigating Between Nodes

You can navigate the DOM tree using these properties:

- `parentNode` - the parent node
- `childNodes[nodenum]` - child nodes (including text nodes)
- `firstChild` - the first child node
- `lastChild` - the last child node
- `nextSibling` - the next sibling node
- `previousSibling` - the previous sibling node

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Navigation</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Navigating Between Nodes</h4>

<div id="container">
  <p id="first">First paragraph</p>
  <p id="second">Second paragraph</p>
  <p id="third">Third paragraph</p>
</div>

<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>
<p id="demo4"></p>
<p id="demo5"></p>

<script>
const second = document.getElementById("second");

document.getElementById("demo1").innerHTML =
  "parentNode: " + second.parentNode.id;

document.getElementById("demo2").innerHTML =
  "previousSibling (element): " + second.previousElementSibling.textContent;

document.getElementById("demo3").innerHTML =
  "nextSibling (element): " + second.nextElementSibling.textContent;

const parent = document.getElementById("container");
document.getElementById("demo4").innerHTML =
  "firstElementChild: " + parent.firstElementChild.textContent;

document.getElementById("demo5").innerHTML =
  "lastElementChild: " + parent.lastElementChild.textContent;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript DOM Navigation](#)

JavaScript DOM Nodes

Table of Contents

- [JavaScript DOM Nodes](#)
- [Creating New HTML Elements](#)
- [Example 1](#)
- [insertBefore\(\)](#)
- [Example 2](#)
- [Removing Existing HTML Elements](#)
- [Example 3](#)
- [Removing a Child Node](#)
- [Example 4](#)
- [Replacing HTML Elements](#)
- [Example 5](#)
- [Reference](#)

Creating New HTML Elements

To create new HTML elements, use `document.createElement()` and `appendChild()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Nodes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating New HTML Elements</h4>

<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<button onclick="createElement()">Create New Paragraph</button>

<script>
function createElement() {
  const para = document.createElement("p");
  const node = document.createTextNode("This is new.");
  para.appendChild(node);
  document.getElementById("div1").appendChild(para);
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

insertBefore()

You can insert a new element before an existing element using `insertBefore()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Nodes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>insertBefore()</h4>

<div id="div2">
  <p id="p3">First paragraph</p>
  <p id="p4">Second paragraph</p>
</div>

<button onclick="insertElement()">Insert Before First</button>

<script>
function insertElement() {
  const para = document.createElement("p");
  const node = document.createTextNode("Inserted paragraph.");
  para.appendChild(node);

  const parent = document.getElementById("div2");
  const child = document.getElementById("p3");
  parent.insertBefore(para, child);
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Removing Existing HTML Elements

Use `remove()` to delete an HTML element:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript DOM Nodes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Removing Existing HTML Elements</h4>

<div id="div3">
  <p id="p5">Paragraph 1</p>
  <p id="p6">Paragraph 2 - will be removed</p>
  <p id="p7">Paragraph 3</p>
</div>

<button onclick="removeElement()">Remove Paragraph 2</button>

<script>
function removeElement() {
  const element = document.getElementById("p6");
  element.remove();
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Removing a Child Node

For older browsers that do not support `remove()`, you can use `removeChild()`:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Nodes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Removing a Child Node</h4>

<div id="div4">
  <p id="p8">Item A</p>
  <p id="p9">Item B - will be removed via parent</p>
  <p id="p10">Item C</p>
</div>

<button onclick="removeChildElement()">Remove Item B</button>

<script>
function removeChildElement() {
  const parent = document.getElementById("div4");
  const child = document.getElementById("p9");

```



```
    parent.removeChild(child);
  }
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Replacing HTML Elements

Use `replaceChild()` to replace an element with another:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript DOM Nodes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Replacing HTML Elements</h4>

<div id="div5">
  <p id="p11">Original paragraph</p>
</div>

<button onclick="replaceElement()">Replace Paragraph</button>

<script>
function replaceElement() {
  const para = document.createElement("p");
  const node = document.createTextNode("Replaced paragraph!");
  para.appendChild(node);

  const parent = document.getElementById("div5");
  const child = document.getElementById("p11");
  parent.replaceChild(para, child);
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript DOM Nodes](#)

JavaScript HTMLCollection

Table of Contents

- [JavaScript HTMLCollection](#)
- [The HTMLCollection Object](#)
- [Example 1](#)
- [HTMLCollection Length](#)
- [Example 2](#)
- [Looping Through an HTMLCollection](#)
- [Example 3](#)
- [Reference](#)

The HTMLCollection Object

An **HTMLCollection** is an array-like list of HTML elements.

Methods like **getElementsByTagName()** return an **HTMLCollection**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTMLCollection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The HTMLCollection Object</h4>

<p>First paragraph</p>
<p>Second paragraph</p>
<p>Third paragraph</p>

<p id="demo"></p>

<script>
const myCollection = document.getElementsByTagName("p");

document.getElementById("demo").innerHTML =
  "Number of <p> elements: " + myCollection.length + "<br>" +
  "First paragraph text: " + myCollection[0].innerHTML + "<br>" +
  "Second paragraph text: " + myCollection[1].innerHTML;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

HTMLCollection Length

The **length** property defines the number of elements in the HTMLCollection. You can use it to loop through the collection:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTMLCollection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTMLCollection Length</h4>

<div id="list">
  <p>Apple</p>
  <p>Banana</p>
  <p>Orange</p>
  <p>Grape</p>
  <p>Mango</p>
</div>

<p id="demo"></p>

<script>
const collection = document.getElementById("list").getElementsByTagName("p");
let text = "Items (" + collection.length + "):<br>";
for (let i = 0; i < collection.length; i++) {
  text += (i + 1) + ". " + collection[i].innerHTML + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Looping Through an HTMLCollection

You cannot use **forEach()** on an HTMLCollection directly. You must convert it to an array first, or use a **for** loop:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTMLCollection</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Looping Through an HTMLCollection</h4>

<div id="colors">
  <p class="item">Red</p>
  <p class="item">Green</p>
  <p class="item">Blue</p>
  <p class="item">Yellow</p>
</div>

<button onclick="changeColors()">Change Colors</button>
<p id="demo"></p>

<script>
function changeColors() {
  const items = document.getElementById("colors").getElementsByClassName("item");

  // Convert HTMLCollection to Array for using forEach
  const arr = Array.from(items);
  arr.forEach(function(item, index) {
    item.innerHTML += " (" + (index + 1) + ")";
  });

  document.getElementById("demo").innerHTML =
    "Modified " + items.length + " items using Array.from() + forEach()";
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript HTMLCollection](#)

JavaScript NodeList

Table of Contents

- [JavaScript NodeList](#)
- [The HTML DOM NodeList Object](#)
- [Example 1](#)
- [HTML DOM Node List Length](#)
- [Example 2](#)
- [The Difference Between HTMLCollection and NodeList](#)
- [Example 3](#)
- [Reference](#)

The HTML DOM NodeList Object

A **NodeList** is a list of nodes extracted from a document.

The **querySelectorAll()** method returns a **NodeList**.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NodeList</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The HTML DOM NodeList Object</h4>

<p>First paragraph</p>
<p>Second paragraph</p>
<p>Third paragraph</p>

<p id="demo"></p>

<script>
const myNodeList = document.querySelectorAll("p");

document.getElementById("demo").innerHTML =
  "Number of <p> elements: " + myNodeList.length + "<br>" +
  "First paragraph text: " + myNodeList[0].innerHTML + "<br>" +
  "Second paragraph text: " + myNodeList[1].innerHTML;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

HTML DOM Node List Length

The **length** property defines the number of elements in the NodeList. You can use it to loop through the list:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NodeList</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTML DOM Node List Length</h4>

<div id="list">
  <p>Apple</p>
  <p>Banana</p>
  <p>Orange</p>
</div>

<p id="demo"></p>

<script>
const nodeList = document.querySelectorAll("#list p");
let text = "Items (" + nodeList.length + "):<br>";
for (let i = 0; i < nodeList.length; i++) {
  text += (i + 1) + ". " + nodeList[i].innerHTML + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The Difference Between HTMLCollection and NodeList

An **HTMLCollection** is live (updates when DOM changes). A **NodeList** from **querySelectorAll()** is static (does not update):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript NodeList</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>HTMLCollection vs NodeList</h4>
```

```

<div id="container">
  <p class="test">Item 1</p>
  <p class="test">Item 2</p>
</div>

<button onclick="addItem()">Add Item</button>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function addItem() {
  const para = document.createElement("p");
  para.className = "test";
  para.innerHTML = "Item " + (document.querySelectorAll(".test").length + 1);
  document.getElementById("container").appendChild(para);

  // HTMLCollection is LIVE - automatically updated
  const liveCollection = document.getElementsByClassName("test");
  document.getElementById("demo1").innerHTML =
    "HTMLCollection (live) count: " + liveCollection.length;

  // NodeList is STATIC - shows original count
  const staticList = document.querySelectorAll(".test");
  document.getElementById("demo2").innerHTML =
    "NodeList (static) count: " + staticList.length;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript NodeList](#)

JavaScript Window Object

Table of Contents

- [JavaScript Window Object](#)
- [The Browser Object Model \(BOM\)](#)
- [The Window Object](#)
- [Example 1](#)
- [Window Size](#)
- [Example 2](#)
- [Window Methods](#)
- [Example 3](#)
- [Other Window Properties](#)
- [Example 4](#)
- [Reference](#)

The Browser Object Model (BOM)

The Browser Object Model (BOM) allows JavaScript to "talk to" the browser. The `window` object is the global object in the browser and represents the browser window.

All global JavaScript objects, functions, and variables automatically become members of the `window` object.

The Window Object

The `window` object is the browser's global object. It contains properties and methods for interacting with the browser window:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Object</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Window Object</h4>
<p id="demo"></p>

<script>
let text = "Window properties:<br><br>";
text += "window.innerWidth: " + window.innerWidth + "px<br>";
text += "window.innerHeight: " + window.innerHeight + "px<br>";
text += "window.outerWidth: " + window.outerWidth + "px<br>";
text += "window.outerHeight: " + window.outerHeight + "px<br>";
text += "window.name: " + window.name + "<br>";
text += "window.closed: " + window.closed + "<br>";
text += "window.opener: " + window.opener;

document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Window Size

The `window.innerWidth` and `window.innerHeight` properties get the browser window's size (excluding toolbars and scrollbars):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Object</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Size</h4>

<button onclick="showSize()">Show Window Size</button>
<button onclick="resizeWindow()">Resize to 800x600</button>
<p id="demo"></p>

<script>
function showSize() {
  document.getElementById("demo").innerHTML =
    "Inner window size:<br>" +
    "Width: " + window.innerWidth + "px<br>" +
    "Height: " + window.innerHeight + "px<br><br>" +
    "Outer window size:<br>" +
    "Width: " + window.outerWidth + "px<br>" +
    "Height: " + window.outerHeight + "px";
}

function resizeWindow() {
  window.resizeTo(800, 600);
  showSize();
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Window Methods

The **window** object provides methods to open, close, and control browser windows:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Object</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Methods</h4>

<button onclick="openWindow()">Open New Window</button>
<button onclick="moveWindow()">Move Window</button>
<button onclick="closeWindow()">Close Window</button>
<p id="demo"></p>

<script>
let myWindow;

function openWindow() {
  myWindow = window.open("", "", "width=300,height=200");
  myWindow.document.write("<p>This is a new window.</p>");
  document.getElementById("demo").innerHTML = "New window opened!";
}

function moveWindow() {
  if (myWindow && !myWindow.closed) {
    myWindow.moveTo(500, 100);
    myWindow.resizeTo(400, 300);
    document.getElementById("demo").innerHTML = "Window moved and resized!";
  } else {
    document.getElementById("demo").innerHTML = "No open window to move!";
  }
}

function closeWindow() {
  if (myWindow && !myWindow.closed) {
    myWindow.close();
    document.getElementById("demo").innerHTML = "Window closed!";
  } else {
    document.getElementById("demo").innerHTML = "No open window to close!";
  }
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Other Window Properties

The `window` object also provides methods like `focus()`, `blur()`, `scrollBy()`, `scrollTo()`, and `print()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Object</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Other Window Properties</h4>

<button onclick="scrollDown()">Scroll Down</button>
<button onclick="scrollToTop()">Scroll to Top</button>
<button onclick="printPage()">Print</button>
<p id="demo"></p>

<div style="height: 1000px; background: linear-gradient(to bottom, #f0f0f0, #ddd);">
  <p>Scroll down to see the scroll effect.</p>
  <p style="margin-top: 800px;">You are at the bottom!</p>
</div>

<script>
function scrollDown() {
  window.scrollBy(0, 300);
  document.getElementById("demo").innerHTML =
    "Scrolled down! pageYOffset: " + window.pageYOffset;
}

function scrollToTop() {
  window.scrollTo(0, 0);
  document.getElementById("demo").innerHTML =
    "Scrolled to top! pageYOffset: " + window.pageYOffset;
}

function printPage() {
  window.print();
}
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Window Object](#)

JavaScript Window Screen

Table of Contents

- [JavaScript Window Screen](#)
- [Window Screen Width](#)
- [Example 1](#)
- [Window Screen Height](#)
- [Example 2](#)
- [Window Screen Available Width](#)
- [Example 3](#)
- [Window Screen Available Height](#)
- [Example 4](#)
- [Window Screen Color and Pixel Depth](#)
- [Example 5](#)
- [Reference](#)

Window Screen Width

The `screen.width` property returns the width of the visitor's screen in pixels:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Screen</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Screen Width</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Screen width: " + screen.width + "px";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Window Screen Height

The `screen.height` property returns the height of the visitor's screen in pixels:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Screen</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Screen Height</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Screen height: " + screen.height + "px";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Window Screen Available Width

The `screen.availWidth` property returns the width of the visitor's screen in pixels, minus interface features like the taskbar:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Screen</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Screen Available Width</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Available screen width: " + screen.availWidth + "px";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Window Screen Available Height

The `screen.availHeight` property returns the height of the visitor's screen in pixels, minus interface features like the taskbar:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Screen</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Screen Available Height</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Available screen height: " + screen.availHeight + "px";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Window Screen Color and Pixel Depth

The `screen.colorDepth` property returns the number of bits used to display one color. The `screen.pixelDepth` property returns the pixel depth of the screen:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Screen</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Screen Color and Pixel Depth</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Screen color depth: " + screen.colorDepth + " bits<br>" +
  "Screen pixel depth: " + screen.pixelDepth + " bits<br><br>" +
  "All screen properties:<br>" +
  "Total size: " + screen.width + " x " + screen.height + "<br>" +
  "Available: " + screen.availWidth + " x " + screen.availHeight + "<br>" +
  "Color depth: " + screen.colorDepth + " bits<br>" +
  "Pixel depth: " + screen.pixelDepth + " bits";
</script>
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Window Screen](#)

JavaScript Window Location

Table of Contents

- [JavaScript Window Location](#)
- [Window Location Href](#)
- [Example 1](#)
- [Window Location Hostname](#)
- [Example 2](#)
- [Window Location Pathname](#)
- [Example 3](#)
- [Window Location Protocol](#)
- [Example 4](#)
- [Window Location Assign](#)
- [Example 5](#)
- [Reference](#)

Window Location Href

The `window.location.href` property returns the URL of the current page:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Location</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Location Href</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Page URL (href): " + window.location.href;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Window Location Hostname

The `window.location.hostname` property returns the domain name of the web host:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Location</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Location Hostname</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Page hostname: " + window.location.hostname;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Window Location Pathname

The `window.location.pathname` property returns the pathname of the current page:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Location</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Location Pathname</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Page pathname: " + window.location.pathname;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Window Location Protocol

The `window.location.protocol` property returns the web protocol of the page (http: or https:):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Location</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Location Protocol</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "Page protocol: " + window.location.protocol;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Window Location Assign

The `window.location.assign()` method loads a new document (navigates to a new URL):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Location</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window Location Assign</h4>
<p id="demo"></p>

<button onclick="showAllProperties()">Show All Location Properties</button>
<p id="demo2"></p>

<script>
function showAllProperties() {
  const loc = window.location;
  document.getElementById("demo2").innerHTML =
    "href: " + loc.href + "<br>" +
    "hostname: " + loc.hostname + "<br>" +
    "pathname: " + loc.pathname + "<br>" +
    "protocol: " + loc.protocol + "<br>" +
    "port: " + loc.port + "<br>" +
    "host: " + loc.host + "<br>" +
    "origin: " + loc.origin;
}
```

```
}  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Window Location](#)

JavaScript Window History

Table of Contents

- [JavaScript Window History](#)
- [Window History Back](#)
- [Example 1](#)
- [Window History Forward](#)
- [Example 2](#)
- [Window History Go](#)
- [Example 3](#)
- [Reference](#)

Window History

The `window.history` object contains the browser's history.

For security reasons, it does not allow reading the URLs in the history, but it does allow navigating through the history.

Window History Back

The `history.back()` method loads the previous URL in the history list. It is the same as clicking the Back button in the browser:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window History</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window History Back</h4>
<p id="demo"></p>

<button onclick="goBack()">Go Back</button>
<button onclick="showInfo()">History Info</button>

<script>
function goBack() {
    window.history.back();
}

function showInfo() {
    document.getElementById("demo").innerHTML =
        "History length: " + window.history.length + "<br>" +
        "Click 'Go Back' to navigate to the previous page";
}
</script>
```

```
</body>
</html>
```

Example 1

Result [View Example](#)

Window History Forward

The `history.forward()` method loads the next URL in the history list. It is the same as clicking the Forward button in the browser:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window History</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window History Forward</h4>
<p id="demo"></p>

<button onclick="goBack()">Go Back</button>
<button onclick="goForward()">Go Forward</button>
<button onclick="showInfo()">History Info</button>

<script>
function goBack() {
    window.history.back();
}

function goForward() {
    window.history.forward();
}

function showInfo() {
    document.getElementById("demo").innerHTML =
        "History length: " + window.history.length + "<br>" +
        "Use Back/Forward buttons to navigate history";
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Window History Go

The `history.go()` method loads a specific URL from the history list. You can go backward or forward using positive or negative numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window History</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Window History Go</h4>
<p id="demo"></p>

<button onclick="goBack2()">Go Back 2 Pages</button>
<button onclick="goForward2()">Go Forward 2 Pages</button>
<button onclick="showInfo()">History Info</button>

<script>
function goBack2() {
    window.history.go(-2);
}

function goForward2() {
    window.history.go(2);
}

function showInfo() {
    document.getElementById("demo").innerHTML =
        "History length: " + window.history.length + "<br><br>" +
        "history.go(-2): Go back 2 pages<br>" +
        "history.go(2): Go forward 2 pages<br>" +
        "history.go(0): Reload current page<br>" +
        "history.go(-1): Go back 1 page (same as back())<br>" +
        "history.go(1): Go forward 1 page (same as forward())";
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Window History](#)

JavaScript Window Navigator

Table of Contents

- [JavaScript Window Navigator](#)
- [Browser Cookies](#)
- [Example 1](#)
- [The Browser Language](#)
- [Example 2](#)
- [Is The Browser Online?](#)
- [Example 3](#)
- [The Browser Agent](#)
- [Example 4](#)
- [The Browser Platform](#)
- [Example 5](#)
- [Reference](#)

The Navigator Object

The `window.navigator` object contains information about the visitor's browser. It can be used to detect browser properties like cookies enabled, language, platform, and user agent.

Browser Cookies

The `navigator.cookieEnabled` property returns true if cookies are enabled in the browser:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Navigator</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Browser Cookies</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
    "navigator.cookieEnabled: " + navigator.cookieEnabled;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Browser Language

The `navigator.language` property returns the browser's language:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Navigator</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Browser Language</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "navigator.language: " + navigator.language;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Is The Browser Online?

The `navigator.onLine` property returns true if the browser is online:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Navigator</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Is The Browser Online?</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "navigator.onLine: " + navigator.onLine;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The Browser Agent

The `navigator.userAgent` property returns the user-agent string sent by the browser:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Navigator</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Browser Agent</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "navigator.userAgent: " + navigator.userAgent;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The Browser Platform

The `navigator.platform` property returns the browser platform (operating system):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Window Navigator</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Browser Platform</h4>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
  "navigator.platform: " + navigator.platform + "<br><br>" +
  "All navigator properties:<br>" +
  "cookieEnabled: " + navigator.cookieEnabled + "<br>" +
  "language: " + navigator.language + "<br>" +
  "onLine: " + navigator.onLine + "<br>" +
  "platform: " + navigator.platform + "<br>" +
  "appName: " + navigator.appName + "<br>" +
```

```
"appVersion: " + navigator.appVersion + "<br>" +  
"userAgent: " + navigator.userAgent + "<br>" +  
"product: " + navigator.product + "<br>" +  
"appCodeName: " + navigator.appCodeName + "<br>" +  
"javaEnabled: " + navigator.javaEnabled();  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Window Navigator](#)

JavaScript Popup Boxes

Table of Contents

- [JavaScript Popup Boxes](#)
- [Alert Box](#)
- [Example 1](#)
- [Confirm Box](#)
- [Example 2](#)
- [Prompt Box](#)
- [Example 3](#)
- [Reference](#)

Alert Box

An alert box is often used to display information to the user. It has an OK button.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Popup Boxes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Alert Box</h4>
<p id="demo"></p>

<button onclick="showAlert()">Show Alert</button>

<script>
function showAlert() {
  alert("Hello! This is an alert box.");
  document.getElementById("demo").innerHTML = "Alert box was displayed!";
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Confirm Box

A confirm box asks the user to confirm something. It has OK and Cancel buttons.

- If the user clicks OK, the box returns **true**
- If the user clicks Cancel, the box returns **false**

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Popup Boxes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Confirm Box</h4>
<p id="demo"></p>

<button onclick="showConfirm()">Show Confirm</button>

<script>
function showConfirm() {
    let text;
    if (confirm("Do you want to continue?")) {
        text = "You clicked OK!";
    } else {
        text = "You clicked Cancel!";
    }
    document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Prompt Box

A prompt box asks the user to input a value. It has OK and Cancel buttons.

- If the user clicks OK, the box returns the input value
- If the user clicks Cancel, the box returns **null**

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Popup Boxes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Prompt Box</h4>
<p id="demo"></p>

<button onclick="showPrompt()">Show Prompt</button>

<script>

```

```
function showPrompt() {  
  let person = prompt("Please enter your name:", "John Doe");  
  if (person != null && person != "") {  
    document.getElementById("demo").innerHTML =  
      "Hello " + person + "! How are you today?";  
  }  
}  
</script>  
  
</body>  
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Popup Boxes](#)

JavaScript Cookies

Table of Contents

- [JavaScript Cookies](#)
- [Create a Cookie](#)
- [Example 1](#)
- [Read a Cookie](#)
- [Example 2](#)
- [Change a Cookie](#)
- [Example 3](#)
- [Delete a Cookie](#)
- [Example 4](#)
- [Cookie Functions](#)
- [Example 5](#)
- [Reference](#)

What are Cookies?

Cookies are data, stored in small text files on your computer. When a web server sends a web page to a browser, the connection is closed. The server forgets everything about the user. Cookies were invented to solve the problem of "remembering" the user.

Create a Cookie

A cookie is created with `document.cookie`. You can add an expiry date (in UTC time) and a path:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Cookies</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Create a Cookie</h4>
<p id="demo"></p>

<button onclick="createCookie()">Create Cookie</button>

<script>
function createCookie() {
    document.cookie = "username=John Doe; expires=Thu, 31 Dec 2027 23:59:59 UTC;
    path="/";
    document.getElementById("demo").innerHTML = "Cookie 'username' created!";
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Read a Cookie

You can read all cookies with `document.cookie`. It returns all cookies in one string:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Cookies</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Read a Cookie</h4>
<p id="demo"></p>

<button onclick="createCookie()">Create Cookie</button>
<button onclick="readCookie()">Read Cookie</button>

<script>
function createCookie() {
  document.cookie = "username=John Doe; expires=Thu, 31 Dec 2027 23:59:59 UTC;
path="/;
  document.getElementById("demo").innerHTML = "Cookie 'username' created!";
}

function readCookie() {
  let cookies = document.cookie;
  document.getElementById("demo").innerHTML = "All cookies: " + cookies;
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Change a Cookie

To change a cookie, just set it again with a new value:

```
<!DOCTYPE html>
<html>
<body>
```



```

<h2>JavaScript Cookies</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Change a Cookie</h4>
<p id="demo"></p>

<button onclick="createCookie()">Create Cookie</button>
<button onclick="changeCookie()">Change Cookie</button>
<button onclick="readCookie()">Read Cookie</button>

<script>
function createCookie() {
    document.cookie = "username=John Doe; expires=Thu, 31 Dec 2027 23:59:59 UTC;
path=/";
    document.getElementById("demo").innerHTML = "Cookie 'username' created!";
}

function changeCookie() {
    document.cookie = "username=Jane Smith; expires=Thu, 31 Dec 2027 23:59:59 UTC;
path=/";
    document.getElementById("demo").innerHTML = "Cookie 'username' changed to Jane
Smith!";
}

function readCookie() {
    document.getElementById("demo").innerHTML = "All cookies: " + document.cookie;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Delete a Cookie

To delete a cookie, set the expires parameter to a past date:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Cookies</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Delete a Cookie</h4>
<p id="demo"></p>

<button onclick="createCookie()">Create Cookie</button>
<button onclick="deleteCookie()">Delete Cookie</button>

```

```

<button onclick="readCookie()">Check Cookies</button>

<script>
function createCookie() {
    document.cookie = "username=John Doe; expires=Thu, 31 Dec 2027 23:59:59 UTC;
path="/";
    document.getElementById("demo").innerHTML = "Cookie created!";
}

function deleteCookie() {
    document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path="/";
    document.getElementById("demo").innerHTML = "Cookie deleted!";
}

function readCookie() {
    document.getElementById("demo").innerHTML = "All cookies: " + document.cookie;
}
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Cookie Functions

Create a complete set of cookie functions for easier management:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Cookies</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cookie Functions</h4>

<input type="text" id="nameInput" placeholder="Enter name" value="John">
<button onclick="saveName()">Save Name</button>
<br><br>
<button onclick="loadName()">Load Name</button>
<button onclick="deleteName()">Delete Name</button>
<p id="demo"></p>

<script>
function setCookie(cname, cvalue, exdays) {
    const d = new Date();
    d.setTime(d.getTime() + (exdays * 24 * 60 * 60 * 1000));
    let expires = "expires=" + d.toUTCString();
    document.cookie = cname + "=" + cvalue + ";" + expires + ";path="/;
}

```

```

}

function getCookie(cname) {
  let name = cname + "=";
  let ca = document.cookie.split(";");
  for (let i = 0; i < ca.length; i++) {
    let c = ca[i].trim();
    if (c.indexOf(name) === 0) return c.substring(name.length, c.length);
  }
  return "";
}

function checkCookie() {
  let user = getCookie("username");
  if (user !== "") {
    document.getElementById("demo").innerHTML = "Welcome back " + user;
  }
}

function saveName() {
  let name = document.getElementById("nameInput").value;
  setCookie("username", name, 365);
  document.getElementById("demo").innerHTML = "Name '" + name + "' saved in cookie!";
}

function loadName() {
  let user = getCookie("username");
  if (user !== "") {
    document.getElementById("demo").innerHTML = "Loaded: " + user;
  } else {
    document.getElementById("demo").innerHTML = "No cookie found!";
  }
}

function deleteName() {
  setCookie("username", "", -1);
  document.getElementById("demo").innerHTML = "Cookie deleted!";
}

checkCookie();
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Cookies](#)

JavaScript Web API Intro

Table of Contents

- [JavaScript Web API Intro](#)
- [What is Web API?](#)
- [Example 1](#)
- [Browser APIs](#)
- [Example 2](#)
- [Web Storage API](#)
- [Example 3](#)
- [Third Party APIs](#)
- [Example 4](#)
- [Reference](#)

What is Web API?

API stands for **A**pplication **P**rogramming **I**nterface.

A Web API is an application programming interface for the web.

Browser APIs are built into your web browser. They extend the functionality of the browser.

Third Party APIs are not built into your browser. You need to download their code from the web.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web API Intro</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is Web API?</h4>
<p id="demo"></p>

<script>
// The Browser API is built into the browser
// It extends the browser's functionality

let text = "Browser APIs built into your browser:<br><br>";
text += "1. DOM API - Manipulate HTML elements<br>";
text += "2. Fetch API - Make HTTP requests<br>";
text += "3. Web Storage API - Store data locally<br>";
text += "4. History API - Navigate browser history<br>";
text += "5. Geolocation API - Get user location<br>";
text += "6. Canvas API - Draw graphics<br>";
text += "7. Console API - Debug code<br>";
text += "8. Timer API - setTimeout/setInterval<br>";
text += "9. Screen API - Get screen info<br>";
text += "10. Navigator API - Get browser info<br>";

document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example 1

Result [View Example](#)

Browser APIs

All browsers have a set of built-in Web APIs. Here are some of the most important ones:

API	Description	Topic
DOM API	Access and modify HTML elements	05.2101-05.2104
Fetch API	Make HTTP requests to servers	05.2061
Web Storage API	Store data in the browser (localStorage/sessionStorage)	This topic
History API	Navigate browser history	05.2114
Geolocation API	Get user's geographic position	This topic
Canvas API	Draw graphics on a web page	External
Console API	Debug and log information	Built-in
Screen API	Get screen information	05.2112
Navigator API	Get browser/device info	05.2115

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Web API Intro</h2>
```

```
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```
<h4>Browser APIs</h4>
```

```
<p id="demo"></p>
```

```

<script>
// Geolocation API example
let text = "Geolocation API - Checking browser support...<br><br>";

if (navigator.geolocation) {
  text += "✓ Geolocation is supported in this browser<br>";
  text += "✓ Fetch API is supported: " + ('fetch' in window) + "<br>";
  text += "✓ Web Storage is supported: " + ('localStorage' in window) + "<br>";
  text += "✓ History API is supported: " + ('history' in window) + "<br>";
  text += "✓ Console API is supported: " + ('console' in window) + "<br>";
} else {
  text += "✗ Geolocation is NOT supported";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Web Storage API

The Web Storage API provides mechanisms for browsers to store key/value pairs:

- **localStorage** - stores data with no expiration date
- **sessionStorage** - stores data for one session (data is lost when the browser tab is closed)

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web API Intro</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Web Storage API</h4>
<p id="demo"></p>

<script>
// Store data using localStorage
localStorage.setItem("name", "John Doe");
localStorage.setItem("age", "30");

// Retrieve data
let name = localStorage.getItem("name");
let age = localStorage.getItem("age");

let text = "Web Storage API (localStorage):<br><br>";
text += "Stored name: " + name + "<br>";

```

```

text += "Stored age: " + age + "<br><br>";

// Count stored items
text += "Number of items: " + localStorage.length + "<br><br>";

// Remove an item
localStorage.removeItem("age");
text += "After removing 'age': " + localStorage.getItem("age") + " (null)<br><br>";

// Clear all storage
localStorage.clear();
text += "After clearing all storage: " + localStorage.length + " items";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Third Party APIs

Third Party APIs require you to download code from the web. Examples include:

- **Google Maps API** - Display maps on web pages
- **YouTube API** - Embed YouTube videos
- **Twitter API** - Display tweets
- **Facebook API** - Login with Facebook
- **Weather API** - Get weather data

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web API Intro</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Third Party APIs</h4>
<p id="demo"></p>

<button onclick="fetchUsers()">Fetch Users (JSONPlaceholder API)</button>
<p id="result"></p>

<script>
function fetchUsers() {
  // Using the free JSONPlaceholder API (a third-party API)
  fetch('https://jsonplaceholder.typicode.com/users')
    .then(function(response) { return response.json(); })

```

```

.then(function(data) {
  let text = "Third Party API Result (JSONPlaceholder):<br><br>";
  for (let i = 0; i < 3; i++) {
    text += (i + 1) + ". " + data[i].name + " - " + data[i].email + "<br>";
  }
  text += "<br>... and " + (data.length - 3) + " more users";
  document.getElementById("result").innerHTML = text;
})
.catch(function(error) {
  document.getElementById("result").innerHTML = "Error: " + error.message;
});

document.getElementById("demo").innerHTML = "Fetching data from JSONPlaceholder
API...";
}
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Web API Intro](#)

JavaScript API Fetch

> **Note:** This topic covers the Fetch API from the Web API perspective. For async/await patterns with fetch, see [05.2061-js-AsyncFetch](#).

Table of Contents

- [JavaScript API Fetch](#)
- [The Response Object](#)
- [Example 1](#)
- [The ok Property](#)
- [Example 2](#)
- [The status Property](#)
- [Example 3](#)
- [The url Property](#)
- [Example 4](#)
- [Loading Multiple Files](#)
- [Example 5](#)
- [Reference](#)

The Response Object

The `fetch()` method returns a Promise that resolves to a **Response** object. The Response object contains information about the HTTP response:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Response Object</h4>
<p id="demo"></p>

<script>
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(function(response) {
    let text = "Response Object Properties:<br><br>";
    text += "type: " + response.type + "<br>";
    text += "ok: " + response.ok + "<br>";
    text += "status: " + response.status + "<br>";
    text += "statusText: " + response.statusText + "<br>";
    text += "url: " + response.url + "<br>";
    text += "headers: " + response.headers.get('content-type') + "<br>";
    text += "redirected: " + response.redirected;
    document.getElementById("demo").innerHTML = text;
    return response.json();
  })
  .then(function(data) {
```

```

        document.getElementById("demo").innerHTML += "<br><br>User: " + data.name;
    });
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

The ok Property

The **ok** property returns **true** if the HTTP status is in the range 200-299 (success range):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The ok Property</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Successful request (200 OK)
fetch('https://jsonplaceholder.typicode.com/users/1')
    .then(function(response) {
        document.getElementById("demo1").innerHTML =
            "Valid URL - status: " + response.status + "<br>" +
            "ok: " + response.ok + "<br>" +
            "Response ok means success!";
    });

// Invalid request (404 Not Found)
fetch('https://jsonplaceholder.typicode.com/users/nonexistent')
    .then(function(response) {
        document.getElementById("demo2").innerHTML =
            "Invalid URL - status: " + response.status + "<br>" +
            "ok: " + response.ok + "<br>" +
            "Response not ok - check response.ok!";
    });
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

The status Property

The **status** property returns the HTTP status code (200, 404, 500, etc.):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The status Property</h4>
<p id="demo"></p>

<script>
function checkStatus(url) {
  return fetch(url)
    .then(function(response) {
      let msg = "URL: " + url + "<br>";
      msg += "Status: " + response.status + " " + response.statusText + "<br>";

      if (response.status === 200) msg += "✓ Success (200 OK)";
      else if (response.status === 201) msg += "✓ Created (201)";
      else if (response.status === 404) msg += "✗ Not Found (404)";
      else if (response.status >= 400 && response.status < 500) msg += "✗ Client
Error (" + response.status + ")";
      else if (response.status >= 500) msg += "✗ Server Error (" + response.status +
      ")";

      msg += "<br><br>";
      return msg;
    });
}

// Check multiple URLs
Promise.all([
  checkStatus('https://jsonplaceholder.typicode.com/users/1'),
  checkStatus('https://jsonplaceholder.typicode.com/users/nonexistent'),
])
  .then(function(results) {
    document.getElementById("demo").innerHTML = results.join("");
  });
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The url Property

The `url` property returns the URL of the response. It can be different from the request URL if there were redirects:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The url Property</h4>
<p id="demo"></p>

<script>
const requestUrl = 'https://jsonplaceholder.typicode.com/users/1';

fetch(requestUrl)
  .then(function(response) {
    document.getElementById("demo").innerHTML =
      "Request URL: " + requestUrl + "<br>" +
      "Response URL: " + response.url + "<br><br>" +
      "Same URL? " + (requestUrl === response.url);
  });
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Loading Multiple Files

You can load multiple files using `Promise.all()` with multiple fetch requests:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loading Multiple Files</h4>
<p id="demo"></p>

<script>
// Fetch multiple resources in parallel
const urls = [
```

```

    'https://jsonplaceholder.typicode.com/users/1',
    'https://jsonplaceholder.typicode.com/users/2',
    'https://jsonplaceholder.typicode.com/users/3'
  ];

  Promise.all(urls.map(function(url) {
    return fetch(url).then(function(response) {
      if (!response.ok) throw new Error("HTTP " + response.status);
      return response.json();
    });
  })))
  .then(function(users) {
    let text = "Loaded " + users.length + " users:<br><br>";
    for (let user of users) {
      text += user.name + " - " + user.email + "<br>";
    }
    document.getElementById("demo").innerHTML = text;
  })
  .catch(function(error) {
    document.getElementById("demo").innerHTML = "Error: " + error.message;
  });
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript API Fetch](#)
- See also: [05.2061-js-AsyncFetch](#) for async/await patterns

JavaScript Geolocation API

Table of Contents

- [JavaScript Geolocation API](#)
- [Locate the User's Position](#)
- [Example 1](#)
- [Handling Errors and Rejections](#)
- [Example 2](#)
- [Geolocation Object - Return Data](#)
- [Example 3](#)
- [Watch Position](#)
- [Example 4](#)
- [Reference](#)

Locate the User's Position

The `geolocation.getCurrentPosition()` method is used to get the user's current position:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Geolocation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Locate the User's Position</h4>
<button onclick="getLocation()">Get My Position</button>
<p id="demo"></p>

<script>
function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition);
  } else {
    document.getElementById("demo").innerHTML = "Geolocation is not supported by this browser.";
  }
}

function showPosition(position) {
  document.getElementById("demo").innerHTML =
    "Latitude: " + position.coords.latitude + "<br>" +
    "Longitude: " + position.coords.longitude;
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Handling Errors and Rejections

The second parameter of `getCurrentPosition()` is a callback function that handles errors:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Geolocation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Handling Errors and Rejections</h4>
<button onclick="getLocation()">Get My Position</button>
<p id="demo"></p>

<script>
function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition, showError);
  } else {
    document.getElementById("demo").innerHTML = "Geolocation is not supported.";
  }
}

function showPosition(position) {
  document.getElementById("demo").innerHTML =
    "Latitude: " + position.coords.latitude + "<br>" +
    "Longitude: " + position.coords.longitude;
}

function showError(error) {
  let msg = "";
  switch(error.code) {
    case error.PERMISSION_DENIED:
      msg = "User denied the request for Geolocation.";
      break;
    case error.POSITION_UNAVAILABLE:
      msg = "Location information is unavailable.";
      break;
    case error.TIMEOUT:
      msg = "The request to get user location timed out.";
      break;
    case error.UNKNOWN_ERROR:
      msg = "An unknown error occurred.";
      break;
  }
  document.getElementById("demo").innerHTML = "Error: " + msg;
}
</script>

</body>
```

```
</html>
```

Example 2

Result [View Example](#)

Geolocation Object - Return Data

The `getCurrentPosition()` method returns an object with various properties:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Geolocation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Geolocation Object - Return Data</h4>
<button onclick="getLocation()">Get My Full Position</button>
<p id="demo"></p>

<script>
function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition);
  } else {
    document.getElementById("demo").innerHTML = "Geolocation is not supported.";
  }
}

function showPosition(position) {
  const coords = position.coords;
  let text = "Position Object:<br><br>";
  text += "latitude: " + coords.latitude + "<br>";
  text += "longitude: " + coords.longitude + "<br>";
  text += "accuracy: " + coords.accuracy + " meters<br>";
  text += "altitude: " + coords.altitude + "<br>";
  text += "altitudeAccuracy: " + coords.altitudeAccuracy + " meters<br>";
  text += "heading: " + coords.heading + " degrees<br>";
  text += "speed: " + coords.speed + " m/s<br><br>";
  text += "Timestamp: " + new Date(position.timestamp).toLocaleString();
  document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Watch Position

The `watchPosition()` method continuously monitors the user's position and calls a callback each time the position changes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Geolocation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Watch Position</h4>
<button onclick="startWatch()">Start Watching</button>
<button onclick="stopWatch()">Stop Watching</button>
<p id="demo"></p>
<p id="watchStatus"></p>

<script>
let watchId = null;

function startWatch() {
  if (navigator.geolocation) {
    document.getElementById("watchStatus").innerHTML = "Watching position...";
    watchId = navigator.geolocation.watchPosition(showPosition, showError);
  } else {
    document.getElementById("demo").innerHTML = "Geolocation is not supported.";
  }
}

function stopWatch() {
  if (watchId !== null) {
    navigator.geolocation.clearWatch(watchId);
    watchId = null;
    document.getElementById("watchStatus").innerHTML = "Watching stopped.";
  }
}

function showPosition(position) {
  document.getElementById("demo").innerHTML =
    "Updated position:<br>" +
    "Latitude: " + position.coords.latitude + "<br>" +
    "Longitude: " + position.coords.longitude + "<br>" +
    "Accuracy: " + position.coords.accuracy + " meters<br>" +
    "Time: " + new Date(position.timestamp).toLocaleTimeString();
}

function showError(error) {
  document.getElementById("demo").innerHTML = "Error: " + error.message;
}
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Geolocation API](#)

JavaScript API History

> **Note:** For the basic `window.history` object (back/forward/go), see [05.2114-js-WindowHistory](#). This topic covers the History API from the Web API perspective.

Table of Contents

- [JavaScript API History](#)
- [The History back\(\) Method](#)
- [Example 1](#)
- [The History go\(\) Method](#)
- [Example 2](#)
- [History Object Properties](#)
- [Example 3](#)
- [History Object Methods](#)
- [Example 4](#)
- [Reference](#)

The History back() Method

The `history.back()` method loads the previous URL in the history list:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API History</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The History back() Method</h4>
<button onclick="goBack()">Go Back</button>
<p id="demo"></p>

<script>
function goBack() {
    window.history.back();
}

// Show current history length
document.getElementById("demo").innerHTML =
    "History length: " + window.history.length;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The History go() Method

The `history.go()` method loads a specific URL from the history list:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API History</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The History go() Method</h4>
<button onclick="goBack()">Go Back 1 Page</button>
<button onclick="goForward()">Go Forward 1 Page</button>
<button onclick="reloadPage()">Reload (go 0)</button>
<p id="demo"></p>

<script>
function goBack() {
    window.history.go(-1);
}

function goForward() {
    window.history.go(1);
}

function reloadPage() {
    window.history.go(0);
}

document.getElementById("demo").innerHTML =
    "History API provides navigation control<br>" +
    "history.go(-1): back<br>" +
    "history.go(1): forward<br>" +
    "history.go(0): reload<br>" +
    "Current history length: " + window.history.length;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

History Object Properties

The History object has a `length` property that returns the number of URLs in the history list:

```
<!DOCTYPE html>
<html>
<body>
```

```

<h2>JavaScript API History</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>History Object Properties</h4>
<p id="demo"></p>

<script>
let text = "History Object Properties:<br><br>";
text += "history.length: " + window.history.length + "<br>";
text += "(Number of URLs in the history list)<br><br>";

text += "Note: For security reasons,<br>";
text += "history does not allow reading<br>";
text += "the actual URLs in the history.";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

History Object Methods

The History object provides methods for navigating and manipulating the browser history:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API History</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>History Object Methods</h4>
<p id="demo"></p>

<script>
let text = "History Object Methods:<br><br>";

text += "<b>history.back()</b><br>";
text += "- Loads the previous URL<br>";
text += "- Same as clicking the Back button<br><br>";

text += "<b>history.forward()</b><br>";
text += "- Loads the next URL<br>";
text += "- Same as clicking the Forward button<br><br>";

text += "<b>history.go(n)</b><br>";

```

```
text += "- Loads a specific page from history<br>";  
text += "- Negative numbers go back<br>";  
text += "- Positive numbers go forward<br>";  
text += "- go(0) reloads the current page<br><br>";  
  
text += "Current history length: " + window.history.length;  
  
document.getElementById("demo").innerHTML = text;  
</script>  
  
</body>  
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript API History](#)
- See also: [05.2114-js-WindowHistory](#) for Window History basics

JavaScript Pointer Events API

Table of Contents

- [JavaScript Pointer Events API](#)
- [Event Types](#)
- [Example 1](#)
- [Event Properties](#)
- [Example 2](#)
- [Pointer Events API Benefits](#)
- [Example 3](#)
- [CSS pointer-events Property](#)
- [Example 4](#)
- [Reference](#)

Event Types

Pointer events are DOM events that are fired for pointing devices. They handle input from mice, pens, touch screens, and other pointing devices with a single event model.

The pointer events API provides the following event types:

Event	Description
<code>pointerdown</code>	When a pointer becomes active
<code>pointerup</code>	When a pointer is deactivated
<code>pointermove</code>	When a pointer changes coordinates
<code>pointerover</code>	When a pointer enters an element
<code>pointerout</code>	When a pointer leaves an element
<code>pointerenter</code>	When a pointer enters the hit test boundaries
<code>pointerleave</code>	When a pointer leaves the hit test boundaries
<code>pointercancel</code>	When a pointer event is canceled

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Pointer Events API</h2>
```

```

<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Event Types</h4>

<div id="target" style="width:300px;height:150px;background:#f0f0f0;border:2px solid
#333;padding:10px;">
  Click, move, or press here
</div>

<p id="demo"></p>

<script>
const target = document.getElementById("target");

target.addEventListener("pointerdown", function(e) {
  document.getElementById("demo").innerHTML +=
    "pointerdown - button: " + e.button + "<br>";
});

target.addEventListener("pointerup", function(e) {
  document.getElementById("demo").innerHTML +=
    "pointerup<br>";
});

target.addEventListener("pointermove", function(e) {
  // Only update on every 10th move to reduce noise
  if (e.clientX % 10 === 0) {
    document.getElementById("demo").innerHTML =
      "pointermove - X: " + e.clientX + ", Y: " + e.clientY;
  }
});
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Event Properties

Pointer event objects have properties that provide information about the pointer device:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Pointer Events API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Event Properties</h4>

```



```

<div id="target2" style="width:300px;height:150px;background:#e8f0fe;border:2px solid
#333;padding:10px;">
  Click here to see pointer properties
</div>

<p id="demo2"></p>

<script>
document.getElementById("target2").addEventListener("pointerdown", function(e) {
  document.getElementById("demo2").innerHTML =
    "pointerType: " + e.pointerType + "<br>" +
    "pointerId: " + e.pointerId + "<br>" +
    "button: " + e.button + "<br>" +
    "buttons: " + e.buttons + "<br>" +
    "clientX: " + e.clientX + "<br>" +
    "clientY: " + e.clientY + "<br>" +
    "screenX: " + e.screenX + "<br>" +
    "screenY: " + e.screenY + "<br>" +
    "pressure: " + e.pressure + "<br>" +
    "tiltX: " + e.tiltX + "<br>" +
    "tiltY: " + e.tiltY + "<br>" +
    "twist: " + e.twist + "<br>" +
    "width: " + e.width + "<br>" +
    "height: " + e.height;
});
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Pointer Events API Benefits

Pointer events unify mouse, touch, and pen input into a single event model. Benefits include:

- **Cross-device compatibility:** Works with mouse, touch, and pen
- **Simplified code:** One event handler instead of separate mouse and touch handlers
- **Additional data:** Pressure, tilt, twist, and contact geometry for pen/stylus
- **Event capture:** The pointer can be captured to a specific element

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Pointer Events API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Pointer Events API Benefits</h4>

```

```

<p id="demo3"></p>

<script>
// Check browser support for pointer events
let text = "Pointer Events API Support Check:<br><br>";

if (window.PointerEvent) {
  text += "✓ PointerEvent is supported in this browser<br><br>";
  text += "This means you can use:<br>";
  text += "- pointerdown, pointerup, pointermove<br>";
  text += "- Instead of separate mousedown/mouseup/touchstart handlers<br><br>";
  text += "Examples where pointer events help:<br>";
  text += "- Drawing applications (pen + touch + mouse)<br>";
  text += "- Drag and drop (any input device)<br>";
  text += "- Signature capture (pen pressure)<br>";
  text += "- Games (multiple touch points)";
} else {
  text += "✗ PointerEvent is NOT supported in this browser<br>";
  text += "Support falls back to mouse + touch events";
}

document.getElementById("demo3").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

CSS pointer-events Property

The CSS **pointer-events** property specifies whether an element responds to pointer events:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Pointer Events API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>CSS pointer-events Property</h4>
<p id="demo4"></p>

<div id="parent" style="position:relative;width:300px;height:100px;background:#ddd;">
  <div id="blocked" style="width:100px;height:100px;background:red;opacity:0.5;">
    Clickable (default)
  </div>
  <div id="clickable"
style="position:absolute;top:0;left:150px;width:100px;height:100px;background:green;op
acity:0.5;">

```

```

    <span id="child" style="pointer-events:none;">Not clickable</span>
  </div>
</div>

<button onclick="togglePointerEvents()">Toggle pointer-events</button>
<p id="status"></p>

<script>
const child = document.getElementById("child");
let enabled = true;

function togglePointerEvents() {
  enabled = !enabled;
  child.style.pointerEvents = enabled ? "none" : "auto";
  document.getElementById("status").innerHTML =
    "pointer-events: " + (enabled ? "none" : "auto") +
    " (element " + (enabled ? "cannot" : "can") + " receive events)";
}

document.getElementById("parent").addEventListener("click", function() {
  document.getElementById("demo4").innerHTML = "Parent div clicked!";
});

// Show initial state
document.getElementById("status").innerHTML =
  "pointer-events: none (green area child cannot receive events)";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Pointer Events API](#)

JavaScript Web Storage API

Table of Contents

- [JavaScript Web Storage API](#)
- [The localStorage Object](#)
- [Example 1](#)
- [The setItem\(\) and getItem\(\) Methods](#)
- [Example 2](#)
- [The sessionStorage Object](#)
- [Example 3](#)
- [Storage Object Properties and Methods](#)
- [Example 4](#)
- [Practical Example - Visitor Counter](#)
- [Example 5](#)
- [Reference](#)

The localStorage Object

The **localStorage** object stores data with no expiration date. The data is NOT deleted when the browser is closed, and will be available the next day, week, or year.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Storage API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The localStorage Object</h4>
<p id="demo"></p>

<script>
// Store data
localStorage.setItem("name", "John Doe");
localStorage.setItem("age", "30");
localStorage.setItem("city", "New York");

// Retrieve data
document.getElementById("demo").innerHTML =
  "Stored in localStorage:<br><br>" +
  "name: " + localStorage.getItem("name") + "<br>" +
  "age: " + localStorage.getItem("age") + "<br>" +
  "city: " + localStorage.getItem("city");
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The setItem() and getItem() Methods

`setItem(key, value)` stores data. `getItem(key)` retrieves data. Values are stored as strings:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Storage API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The setItem() and getItem() Methods</h4>

<input type="text" id="keyInput" placeholder="Key" value="color">
<input type="text" id="valueInput" placeholder="Value" value="blue">
<br><br>
<button onclick="saveItem()">Save Item</button>
<button onclick="loadItem()">Load Item</button>
<button onclick="removeItem()">Remove Item</button>
<p id="demo2"></p>

<script>
function saveItem() {
  const key = document.getElementById("keyInput").value;
  const value = document.getElementById("valueInput").value;
  localStorage.setItem(key, value);
  document.getElementById("demo2").innerHTML =
    "Saved: " + key + " = " + value;
}

function loadItem() {
  const key = document.getElementById("keyInput").value;
  const value = localStorage.getItem(key);
  if (value !== null) {
    document.getElementById("demo2").innerHTML =
      "Loaded: " + key + " = " + value;
  } else {
    document.getElementById("demo2").innerHTML =
      "Key '" + key + "' not found!";
  }
}

function removeItem() {
  const key = document.getElementById("keyInput").value;
  localStorage.removeItem(key);
  document.getElementById("demo2").innerHTML =
    "Removed: " + key;
}
</script>

</body>
```

```
</html>
```

Example 2

Result [View Example](#)

The sessionStorage Object

The **sessionStorage** object stores data for one session only. The data is deleted when the browser tab is closed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Storage API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The sessionStorage Object</h4>
<p id="demo3"></p>

<script>
// Store session data
sessionStorage.setItem("sessionId", "abc123");
sessionStorage.setItem("startTime", new Date().toLocaleTimeString());

// Retrieve session data
document.getElementById("demo3").innerHTML =
  "sessionStorage data (will be deleted when tab closes):<br><br>" +
  "sessionId: " + sessionStorage.getItem("sessionId") + "<br>" +
  "startTime: " + sessionStorage.getItem("startTime");
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Storage Object Properties and Methods

The Storage object provides these properties and methods:

Method/Property	Description
<code>setItem(key, value)</code>	Stores a key/value pair

<code>getItem(key)</code>	Retrieves a value by key
<code>removeItem(key)</code>	Removes a key/value pair
<code>clear()</code>	Removes all key/value pairs
<code>key(index)</code>	Gets the key at a specific index
<code>length</code>	Returns the number of stored items

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Storage API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Storage Object Properties and Methods</h4>
<p id="demo4"></p>

<button onclick="showAll()">Show All Items</button>
<button onclick="clearAll()">Clear All</button>

<script>
function showAll() {
  let text = "Number of items: " + localStorage.length + "<br><br>";

  for (let i = 0; i < localStorage.length; i++) {
    const key = localStorage.key(i);
    const value = localStorage.getItem(key);
    text += "[" + i + "] " + key + ": " + value + "<br>";
  }

  if (localStorage.length === 0) {
    text += "(empty)";
  }

  document.getElementById("demo4").innerHTML = text;
}

function clearAll() {
  localStorage.clear();
  document.getElementById("demo4").innerHTML = "All items cleared! (" +
localStorage.length + " items)";
}

// Add some demo items if empty
if (localStorage.length === 0) {
  localStorage.setItem("demo1", "value1");
  localStorage.setItem("demo2", "value2");
}

```

```

    localStorage.setItem("demo3", "value3");
  }
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Practical Example - Visitor Counter

A practical example using localStorage to count and remember visitors:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Storage API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Practical Example - Visitor Counter</h4>
<p id="counter"></p>
<p id="firstVisit"></p>
<p id="lastVisit"></p>

<button onclick="resetCounter()">Reset Counter</button>

<script>
function updateCounter() {
  let count = localStorage.getItem("visitCount");

  if (count === null) {
    count = 0;
    localStorage.setItem("firstVisit", new Date().toLocaleString());
  } else {
    count = parseInt(count);
  }

  count++;
  localStorage.setItem("visitCount", count);
  localStorage.setItem("lastVisit", new Date().toLocaleString());

  document.getElementById("counter").innerHTML =
    "You have visited this page " + count + " time(s).";

  document.getElementById("firstVisit").innerHTML =
    "First visit: " + localStorage.getItem("firstVisit");

  document.getElementById("lastVisit").innerHTML =
    "Last visit: " + localStorage.getItem("lastVisit");
}

```



```
function resetCounter() {  
  localStorage.removeItem("visitCount");  
  localStorage.removeItem("firstVisit");  
  localStorage.removeItem("lastVisit");  
  updateCounter();  
}  
  
updateCounter();  
</script>  
  
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Web Storage API](#)

JavaScript Validation API

Table of Contents

- [JavaScript Validation API](#)
- [Constraint Validation DOM Methods](#)
- [Example 1](#)
- [Constraint Validation DOM Properties](#)
- [Example 2](#)
- [Validity Properties](#)
- [Example 3](#)
- [The rangeOverflow Property](#)
- [Example 4](#)
- [The rangeUnderflow Property](#)
- [Example 5](#)
- [Reference](#)

Constraint Validation DOM Methods

The Constraint Validation API provides methods for validating form elements:

Method	Description
<code>checkValidity()</code>	Returns true if an element contains valid data
<code>setCustomValidity()</code>	Sets a custom validation message

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constraint Validation DOM Methods</h4>

<input type="text" id="id1" required minlength="3" placeholder="Enter at least 3
chars">
<button onclick="validate()">Check Validity</button>
<p id="demo"></p>

<script>
function validate() {
  const input = document.getElementById("id1");

  if (input.checkValidity()) {
    document.getElementById("demo").innerHTML = "Input is valid!";
```

```

    } else {
      document.getElementById("demo").innerHTML = "Input is invalid. " +
input.validationMessage;
    }
  }
}
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Constraint Validation DOM Properties

Form elements have these validation properties:

Property	Description
<code>validity</code>	Returns a <code>ValidityState</code> object
<code>validationMessage</code>	Returns the validation message
<code>willValidate</code>	Returns true if element will be validated

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constraint Validation DOM Properties</h4>

<input type="email" id="email" placeholder="Enter email address" required>
<button onclick="checkEmail()">Check Email</button>
<p id="demo2"></p>

<script>
function checkEmail() {
  const input = document.getElementById("email");
  let text = "Validation Properties:<br><br>";

  text += "willValidate: " + input.willValidate + "<br>";
  text += "validationMessage: " + input.validationMessage + "<br>";

  if (input.validity) {
    text += "validity.valid: " + input.validity.valid + "<br>";
  }
}

```

```
    }

    if (input.value === "") {
        text += "<br>Please enter an email address.";
    }

    document.getElementById("demo2").innerHTML = text;
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Validity Properties

The **validity** object contains several boolean properties:

Property	Description
valueMissing	Element has no value (when required)
typeMismatch	Value doesn't match type (e.g., email)
patternMismatch	Value doesn't match pattern
tooLong	Value is too long
tooShort	Value is too short
rangeUnderflow	Value is below minimum
rangeOverflow	Value is above maximum
stepMismatch	Value doesn't match step
badInput	Value is invalid
customError	Custom validation error
valid	Element's value is valid

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Validity Properties</h4>

<form id="myForm">
  <input type="text" id="username" required minlength="3" maxlength="10"
placeholder="Username (3-10 chars)">
  <br><br>
  <input type="number" id="age" min="18" max="120" placeholder="Age (18-120)">
  <br><br>
  <input type="email" id="email2" placeholder="Email address">
  <br><br>
  <button type="button" onclick="showValidity()">Check All Fields</button>
</form>

<p id="demo3"></p>

<script>
function showValidity() {
  const fields = ["username", "age", "email2"];
  let text = "";

  for (let id of fields) {
    const input = document.getElementById(id);
    const v = input.validity;

    text += "<b>" + id + "</b>: ";
    if (v.valid) {
      text += "✓ Valid";
    } else {
      text += "✗ Invalid: ";
      if (v.valueMissing) text += "Required, ";
      if (v.typeMismatch) text += "Wrong type, ";
      if (v.patternMismatch) text += "No pattern match, ";
      if (v.tooShort) text += "Too short, ";
      if (v.tooLong) text += "Too long, ";
      if (v.rangeUnderflow) text += "Below minimum, ";
      if (v.rangeOverflow) text += "Above maximum, ";
      if (v.stepMismatch) text += "Wrong step, ";
      if (v.customError) text += "Custom error, ";
    }
    text += "<br>";
  }

  document.getElementById("demo3").innerHTML = text;
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The rangeOverflow Property

The `rangeOverflow` property is true when a value exceeds the `max` attribute:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The rangeOverflow Property</h4>

<input type="number" id="quantity" max="100" placeholder="Max: 100">
<button onclick="checkRangeOverflow()">Check</button>
<p id="demo4"></p>

<script>
function checkRangeOverflow() {
  const input = document.getElementById("quantity");
  const value = input.value;

  if (value === "") {
    document.getElementById("demo4").innerHTML = "Please enter a value.";
    return;
  }

  if (input.validity.rangeOverflow) {
    document.getElementById("demo4").innerHTML =
      "Value " + value + " is too high! Max is " + input.max;
  } else {
    document.getElementById("demo4").innerHTML =
      "Value " + value + " is within range. Valid!";
  }
}
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The rangeUnderflow Property

The `rangeUnderflow` property is true when a value is less than the `min` attribute:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation API</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The rangeUnderflow Property</h4>

<input type="number" id="score" min="50" placeholder="Min: 50">
<button onclick="checkRangeUnderflow()">Check</button>
<p id="demo5"></p>

<script>
function checkRangeUnderflow() {
  const input = document.getElementById("score");
  const value = input.value;

  if (value === "") {
    document.getElementById("demo5").innerHTML = "Please enter a value.";
    return;
  }

  if (input.validity.rangeUnderflow) {
    document.getElementById("demo5").innerHTML =
      "Value " + value + " is too low! Min is " + input.min;
  } else {
    document.getElementById("demo5").innerHTML =
      "Value " + value + " is within range. Valid!";
  }
}
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Validation API](#)

JavaScript API Web Workers

> **Note:** For a complete tutorial on creating and managing Web Workers, see [05.2065-js-WebWorkers](#). This topic covers the Web Workers API from the Web API perspective.

Table of Contents

- [JavaScript API Web Workers](#)
- [Web Workers Example](#)
- [Example 1](#)
- [Terminate a Web Worker](#)
- [Example 2](#)
- [Web Workers and the DOM](#)
- [Example 3](#)
- [Reference](#)

What is a Web Worker?

A Web Worker is a JavaScript running in the background, without affecting the performance of the page. While the worker runs in the background, you can continue doing whatever you want: clicking, selecting things, etc.

When a worker runs, the code runs independently in a separate thread.

Web Workers Example

The example below creates a web worker that counts numbers in the background:

worker.js - The worker file:

```
let i = 0;
function timedCount() {
  i++;
  postMessage(i);
  setTimeout("timedCount()", 500);
}
timedCount();
```

Main page:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Web Workers Example</h4>
```



```

<p>Count numbers: <output id="result"></output></p>
<button onclick="startWorker()">Start Worker</button>
<button onclick="stopWorker()">Stop Worker</button>

<p id="status"></p>

<script>
let w;

function startWorker() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(w) === "undefined") {
      w = new Worker("worker.js");
    }
    w.onmessage = function(event) {
      document.getElementById("result").innerHTML = event.data;
    };
    document.getElementById("status").innerHTML = "Worker running...";
  } else {
    document.getElementById("status").innerHTML = "Web Workers not supported.";
  }
}

function stopWorker() {
  if (w) {
    w.terminate();
    w = undefined;
    document.getElementById("status").innerHTML = "Worker stopped.";
  }
}
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Terminate a Web Worker

When a web worker is created, it continues to run (even after the page is closed) until it is terminated. Use `terminate()` to stop it:

worker.js - Worker that counts continuously:

```

let counter = 0;
function count() {
  counter++;
  postMessage(counter);
  setTimeout(count, 1000);
}
count();

```

Main page:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Terminate a Web Worker</h4>

<p>Timer: <span id="timer">0</span> seconds</p>
<button onclick="startTimer()">Start Timer</button>
<button onclick="stopTimer()">Stop Timer</button>
<p id="status2"></p>

<script>
let timerWorker;

function startTimer() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(timerWorker) === "undefined") {
      timerWorker = new Worker("timer_worker.js");
    }
    timerWorker.onmessage = function(event) {
      document.getElementById("timer").innerHTML = event.data;
    };
    document.getElementById("status2").innerHTML = "Timer running in worker...";
  } else {
    document.getElementById("status2").innerHTML = "Web Workers not supported.";
  }
}

function stopTimer() {
  if (timerWorker) {
    timerWorker.terminate();
    timerWorker = undefined;
    document.getElementById("status2").innerHTML = "Timer stopped with terminate().";
  }
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Web Workers and the DOM

Since web workers are in external files, they do NOT have access to:

- The DOM (document object)
- The window object
- The parent object

They CAN access:

- The **navigator** object
- The **location** object (read-only)
- **setTimeout()**, **setInterval()**, **clearTimeout()**, **clearInterval()**
- The **XMLHttpRequest** object (for AJAX calls)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Web Workers and the DOM</h4>
<p id="demo3"></p>

<script>
let text = "Web Worker Restrictions:<br><br>";

text += "Workers CANNOT access:<br>";
text += "X The DOM (document object)<br>";
text += "X The window object<br>";
text += "X The parent object<br><br>";

text += "Workers CAN access:<br>";
text += "✓ navigator object<br>";
text += "✓ location object (read-only)<br>";
text += "✓ setTimeout()/setInterval()<br>";
text += "✓ XMLHttpRequest (AJAX)<br><br>";

text += "Communication happens via:<br>";
text += "worker.postMessage() - send data to worker<br>";
text += "worker.onmessage - receive data from worker<br>";
text += "worker.terminate() - stop the worker";

document.getElementById("demo3").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript API Web Workers](#)
- See also: [05.2065-js-WebWorkers](#) for detailed examples

JavaScript Graphics

Table of Contents

- [JavaScript Graphics](#)
- [Graphic Libraries](#)
- [Example 1](#)
- [Plotly.js](#)
- [Example 2](#)
- [Chart.js](#)
- [Example 3](#)
- [Google Chart](#)
- [Example 4](#)
- [Reference](#)

Graphic Libraries

There are many JavaScript graphic libraries available for creating charts, graphs, and visualizations on web pages:

Library	Description	Type
Plotly.js	High-level charting library (scatter, bar, pie, 3D charts)	Open source
Chart.js	Simple yet flexible charting (responsive, canvas-based)	Open source
Google Chart	Google's charting library (wide variety of chart types)	Free
D3.js	Data-driven documents (low-level, highly customizable)	Open source
Highcharts	Feature-rich charting library	Commercial
Three.js	3D graphics and animations	Open source

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>
```

```

<h4>Graphic Libraries</h4>
<p id="demo"></p>

<script>
let text = "JavaScript Graphics Libraries:<br><br>";

text += "1. <b>Plotly.js</b> - Interactive charts (scatter, bar, 3D)<br>";
text += "2. <b>Chart.js</b> - Simple, responsive canvas charts<br>";
text += "3. <b>Google Chart</b> - Google's charting service<br>";
text += "4. <b>D3.js</b> - Data-driven document manipulation<br>";
text += "5. <b>Three.js</b> - 3D graphics in the browser<br>";
text += "6. <b>Canvas API</b> - Built-in 2D drawing<br>";
text += "7. <b>SVG</b> - Scalable Vector Graphics<br><br>";

text += "To use a library, include its CDN link:";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Plotly.js

Plotly.js is a high-level, declarative charting library. It supports over 40 chart types including scatter plots, bar charts, pie charts, and 3D charts:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Plotly.js</h4>

<div id="plotlyChart"></div>
<p id="demo2"></p>

<!-- Plotly.js CDN -->
<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
// Sample data
const xValues = [1, 2, 3, 4, 5, 6, 7];
const yValues = [10, 15, 13, 17, 14, 20, 18];

```

```
// Create a bar chart
const data = [{
  x: xValues,
  y: yValues,
  type: "bar",
  marker: {
    color: "#4CAF50"
  }
}];

const layout = {
  title: "Weekly Sales (Plotly.js)",
  xaxis: {title: "Day"},
  yaxis: {title: "Sales"}
};

Plotly.newPlot("plotlyChart", data, layout);
document.getElementById("demo2").innerHTML = "Plotly.js bar chart created!";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Chart.js

Chart.js is a simple yet flexible JavaScript charting library for designers and developers. It creates responsive, canvas-based charts:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Chart.js</h4>

<canvas id="chartjsChart" style="width:100%;max-width:600px;"></canvas>
<p id="demo3"></p>

<!-- Chart.js CDN -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.9.4/Chart.js"></script>

<script>
const xValues2 = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"];
const yValues2 = [55, 49, 80, 70, 65, 90];
const barColors = ["red", "green", "blue", "orange", "brown", "purple"];
```

```

new Chart("chartjsChart", {
  type: "bar",
  data: {
    labels: xValues2,
    datasets: [{
      backgroundColor: barColors,
      data: yValues2
    }]
  },
  options: {
    title: {
      display: true,
      text: "Monthly Revenue (Chart.js)"
    },
    legend: {display: false}
  }
});

document.getElementById("demo3").innerHTML = "Chart.js bar chart created!";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Google Chart

Google Chart provides a wide variety of chart types powered by Google. Charts are rendered using HTML5/SVG technology:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Google Chart</h4>

<div id="googleChart"></div>
<p id="demo4"></p>

<!-- Google Chart CDN -->
<script src="https://www.gstatic.com/charts/loader.js"></script>

<script>
google.charts.load("current", {packages: ["corechart"]});
google.charts.setOnLoadCallback(drawChart);

```



```
function drawChart() {
  const data = google.visualization.arrayToDataTable([
    ["Category", "Value"],
    ["Product A", 45],
    ["Product B", 25],
    ["Product C", 15],
    ["Product D", 10],
    ["Product E", 5]
  ]);

  const options = {
    title: "Market Share (Google Chart)",
    is3D: true,
    pieHole: 0.4
  };

  const chart = new
  google.visualization.PieChart(document.getElementById("googleChart"));
  chart.draw(data, options);

  document.getElementById("demo4").innerHTML = "Google Chart pie chart created!";
}
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Reference

- [W3Schools JavaScript Graphics](#)

JavaScript Canvas Graphics

Table of Contents

- [JavaScript Canvas Graphics](#)
- [Scatter Plots](#)
- [Example 1](#)
- [Line Graphs](#)
- [Example 2](#)
- [Combined Charts](#)
- [Example 3](#)
- [Reference](#)

Scatter Plots

A scatter plot displays values for two variables using dots. The position of each dot on the horizontal and vertical axis indicates values for an individual data point:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Canvas Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Scatter Plots</h4>

<div id="scatterPlot"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const trace1 = {
  x: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  y: [2, 4, 5, 7, 6, 8, 9, 11, 12, 10],
  mode: 'markers',
  type: 'scatter',
  marker: {
    size: 10,
    color: 'rgba(76, 175, 80, 0.7)'
  },
  name: 'Data Points'
};

const data = [trace1];

const layout = {
  title: 'Scatter Plot (Plotly.js)',
  xaxis: {title: 'X Values'},
  yaxis: {title: 'Y Values'},
  showlegend: true
```

```
};

Plotly.newPlot('scatterPlot', data, layout);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Line Graphs

A line graph displays information as a series of data points connected by straight line segments:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Canvas Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Line Graphs</h4>

<div id="lineGraph"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const trace1 = {
  x: [1, 2, 3, 4, 5, 6, 7, 8],
  y: [10, 15, 13, 17, 14, 20, 18, 22],
  mode: 'lines+markers',
  type: 'scatter',
  line: {color: '#2196F3', width: 3},
  marker: {size: 8, color: '#1565C0'},
  name: 'Series A'
};

const trace2 = {
  x: [1, 2, 3, 4, 5, 6, 7, 8],
  y: [5, 8, 7, 11, 9, 13, 12, 15],
  mode: 'lines+markers',
  type: 'scatter',
  line: {color: '#FF5722', width: 3, dash: 'dot'},
  marker: {size: 8, color: '#BF360C'},
  name: 'Series B'
};

const data = [trace1, trace2];
```

```

const layout = {
  title: 'Line Graph (Plotly.js)',
  xaxis: {title: 'Time'},
  yaxis: {title: 'Values'},
  showlegend: true
};

Plotly.newPlot('lineGraph', data, layout);
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Combined Charts

Combined charts overlay multiple chart types (bars + lines) in the same figure for comparison:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Canvas Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Combined Charts</h4>

<div id="combinedChart"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const trace1 = {
  x: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
  y: [30, 45, 38, 55, 48, 62],
  type: 'bar',
  name: 'Revenue',
  marker: {color: '#4CAF50'}
};

const trace2 = {
  x: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
  y: [20, 28, 24, 35, 30, 40],
  type: 'scatter',
  mode: 'lines+markers',
  name: 'Expenses',
  line: {color: '#F44336', width: 3},
  marker: {size: 8, color: '#D32F2F'},
  yaxis: 'y2'

```

```
};

const data = [trace1, trace2];

const layout = {
  title: 'Combined Chart (Bar + Line)',
  xaxis: {title: 'Month'},
  yaxis: {title: 'Revenue'},
  yaxis2: {
    title: 'Expenses',
    overlaying: 'y',
    side: 'right'
  },
  showlegend: true,
  barmode: 'group'
};

Plotly.newPlot('combinedChart', data, layout);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Reference

- [W3Schools JavaScript Canvas Graphics](#)

JavaScript Plotly Graphics

Table of Contents

- [JavaScript Plotly Graphics](#)
- [Bar Charts](#)
- [Example 1](#)
- [Pie Charts](#)
- [Example 2](#)
- [Scatter Plots](#)
- [Example 3](#)
- [Bubble Plots](#)
- [Example 4](#)
- [Multiple Lines](#)
- [Example 5](#)
- [Reference](#)

Bar Charts

Bar charts display data with rectangular bars with heights proportional to the values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Plotly Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Bar Charts</h4>

<div id="barChart"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const data = [{
  x: ['Product A', 'Product B', 'Product C', 'Product D', 'Product E'],
  y: [25, 40, 30, 55, 20],
  type: 'bar',
  marker: {
    color: ['#4CAF50', '#2196F3', '#FF9800', '#F44336', '#9C27B0']
  }
}];

const layout = {
  title: 'Product Sales (Bar Chart)',
  xaxis: {title: 'Product'},
  yaxis: {title: 'Sales ($)'}
};
```

```
Plotly.newPlot('barChart', data, layout);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Pie Charts

Pie charts show data as slices of a circle:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Plotly Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Pie Charts</h4>

<div id="pieChart"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const data = [{
  labels: ['Chrome', 'Firefox', 'Safari', 'Edge', 'Other'],
  values: [55, 20, 15, 8, 2],
  type: 'pie',
  marker: {
    colors: ['#4CAF50', '#2196F3', '#FF9800', '#F44336', '#9C27B0']
  },
  textinfo: 'label+percent'
}];

const layout = {
  title: 'Browser Market Share (Pie Chart)',
  height: 400
};

Plotly.newPlot('pieChart', data, layout);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Scatter Plots

Scatter plots display values for two variables using dots:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Plotly Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Scatter Plots</h4>

<div id="scatterChart"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const trace1 = {
  x: [1, 3, 4, 6, 7, 9],
  y: [2, 5, 4, 8, 7, 10],
  mode: 'markers',
  type: 'scatter',
  name: 'Dataset 1',
  marker: {color: '#4CAF50', size: 10}
};

const trace2 = {
  x: [2, 4, 5, 7, 8, 10],
  y: [3, 6, 5, 9, 8, 11],
  mode: 'markers',
  type: 'scatter',
  name: 'Dataset 2',
  marker: {color: '#F44336', size: 10, symbol: 'square'}
};

const data = [trace1, trace2];

const layout = {
  title: 'Two Datasets (Scatter Plot)',
  xaxis: {title: 'X Values'},
  yaxis: {title: 'Y Values'}
};

Plotly.newPlot('scatterChart', data, layout);
</script>

</body>
</html>
```


Example 3

Result [View Example](#)

Bubble Plots

Bubble plots are scatter plots where the size of markers represents a third dimension of data:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Plotly Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Bubble Plots</h4>

<div id="bubbleChart"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const trace = {
  x: [1, 2, 3, 4, 5, 6, 7],
  y: [10, 15, 13, 20, 18, 25, 22],
  mode: 'markers',
  type: 'scatter',
  marker: {
    size: [20, 30, 15, 45, 25, 50, 35],
    color: ['#4CAF50', '#2196F3', '#FF9800', '#F44336', '#9C27B0', '#00BCD4',
'#E91E63'],
    sizeref: 2
  },
  name: 'Bubbles'
};

const data = [trace];

const layout = {
  title: 'Bubble Plot (Size = Third Dimension)',
  xaxis: {title: 'X Values'},
  yaxis: {title: 'Y Values'},
  showlegend: true
};

Plotly.newPlot('bubbleChart', data, layout);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Multiple Lines

Multiple line graphs display several data series as connected lines:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Plotly Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Multiple Lines</h4>

<div id="multiLineChart"></div>

<script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>

<script>
const trace1 = {
  x: [1, 2, 3, 4, 5, 6, 7, 8],
  y: [10, 12, 15, 14, 18, 20, 22, 25],
  mode: 'lines+markers',
  name: 'Series 1',
  line: {color: '#4CAF50', width: 3}
};

const trace2 = {
  x: [1, 2, 3, 4, 5, 6, 7, 8],
  y: [5, 8, 7, 11, 9, 13, 12, 16],
  mode: 'lines+markers',
  name: 'Series 2',
  line: {color: '#F44336', width: 3, dash: 'dash'}
};

const trace3 = {
  x: [1, 2, 3, 4, 5, 6, 7, 8],
  y: [3, 5, 4, 7, 6, 9, 8, 11],
  mode: 'lines+markers',
  name: 'Series 3',
  line: {color: '#2196F3', width: 3, dash: 'dot'}
};

const data = [trace1, trace2, trace3];

const layout = {
  title: 'Multiple Line Graphs',
  xaxis: {title: 'Time'},
  yaxis: {title: 'Values'}
};

Plotly.newPlot('multiLineChart', data, layout);
</script>
```

```
</body>  
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Plotly Graphics](#)

JavaScript Chart.js Graphics

Table of Contents

- [JavaScript Chart.js Graphics](#)
- [Bar Charts](#)
- [Example 1](#)
- [Horizontal Bars](#)
- [Example 2](#)
- [Pie Charts](#)
- [Example 3](#)
- [Doughnut Charts](#)
- [Example 4](#)
- [Multiple Lines](#)
- [Example 5](#)
- [Reference](#)

Bar Charts

Bar charts display data with rectangular bars with heights proportional to the values. Chart.js creates responsive, canvas-based charts:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Chart.js Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Bar Charts</h4>

<canvas id="barChart" style="width:100%;max-width:600px;"></canvas>
<p id="demo1"></p>

<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.9.4/Chart.js"></script>

<script>
const xValues = ["Product A", "Product B", "Product C", "Product D", "Product E"];
const yValues = [25, 40, 30, 55, 20];
const barColors = ["#4CAF50", "#2196F3", "#FF9800", "#F44336", "#9C27B0"];

new Chart("barChart", {
  type: "bar",
  data: {
    labels: xValues,
    datasets: [{
      backgroundColor: barColors,
      data: yValues
    }]
  },
},
```

```

options: {
  title: {
    display: true,
    text: "Product Sales (Bar Chart)"
  },
  legend: {display: false},
  scales: {
    yAxes: [{ticks: {beginAtZero: true}}]
  }
}
});

document.getElementById("demo1").innerHTML = "Chart.js bar chart created!";
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Horizontal Bars

Horizontal bar charts display bars horizontally instead of vertically:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Chart.js Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Horizontal Bars</h4>

<canvas id="hBarChart" style="width:100%;max-width:600px;"></canvas>
<p id="demo2"></p>

<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.9.4/Chart.js"></script>

<script>
const xValues = ["Item A", "Item B", "Item C", "Item D", "Item E"];
const yValues = [30, 45, 25, 60, 35];

new Chart("hBarChart", {
  type: "horizontalBar",
  data: {
    labels: xValues,
    datasets: [{
      backgroundColor: ["#4CAF50", "#2196F3", "#FF9800", "#F44336", "#9C27B0"],
      data: yValues
    }]
  }
});

```

```

    },
    options: {
      title: {
        display: true,
        text: "Horizontal Bar Chart"
      },
      legend: {display: false},
      scales: {
        xAxes: [{ticks: {beginAtZero: true}}]
      }
    }
  });

document.getElementById("demo2").innerHTML = "Chart.js horizontal bar chart created!";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Pie Charts

Pie charts show data as slices of a circle:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Chart.js Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Pie Charts</h4>

<canvas id="pieChart" style="width:100%;max-width:500px;"></canvas>
<p id="demo3"></p>

<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.9.4/Chart.js"></script>

<script>
const xValues = ["Chrome", "Firefox", "Safari", "Edge", "Other"];
const yValues = [55, 20, 15, 8, 2];
const colors = ["#4CAF50", "#2196F3", "#FF9800", "#F44336", "#9C27B0"];

new Chart("pieChart", {
  type: "pie",
  data: {
    labels: xValues,
    datasets: [{
      backgroundColor: colors,

```

```

        data: yValues
    }]
},
options: {
    title: {
        display: true,
        text: "Browser Market Share (Pie Chart)"
    }
}
});

document.getElementById("demo3").innerHTML = "Chart.js pie chart created!";
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Doughnut Charts

Doughnut charts are similar to pie charts but with a hollow center:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Chart.js Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Doughnut Charts</h4>

<canvas id="doughnutChart" style="width:100%;max-width:500px;"></canvas>
<p id="demo4"></p>

<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.9.4/Chart.js"></script>

<script>
const xValues = ["North", "South", "East", "West"];
const yValues = [35, 25, 20, 20];
const colors = ["#4CAF50", "#2196F3", "#FF9800", "#F44336"];

new Chart("doughnutChart", {
    type: "doughnut",
    data: {
        labels: xValues,
        datasets: [{
            backgroundColor: colors,
            data: yValues
        }]
    }
});

```

```

    },
    options: {
      title: {
        display: true,
        text: "Regional Sales (Doughnut Chart)"
      }
    }
  });

document.getElementById("demo4").innerHTML = "Chart.js doughnut chart created!";
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Multiple Lines

Multiple line graphs display several data series as connected lines:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Chart.js Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Multiple Lines</h4>

<canvas id="multiLineChart" style="width:100%;max-width:600px;"></canvas>
<p id="demo5"></p>

<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.9.4/Chart.js"></script>

<script>
new Chart("multiLineChart", {
  type: "line",
  data: {
    labels: [1, 2, 3, 4, 5, 6, 7, 8],
    datasets: [{
      label: "Series 1",
      data: [10, 12, 15, 14, 18, 20, 22, 25],
      borderColor: "#4CAF50",
      fill: false
    }, {
      label: "Series 2",
      data: [5, 8, 7, 11, 9, 13, 12, 16],
      borderColor: "#F44336",
      fill: false,

```



```

        borderDash: [5, 5]
    }, {
        label: "Series 3",
        data: [3, 5, 4, 7, 6, 9, 8, 11],
        borderColor: "#2196F3",
        fill: false,
        borderDash: [2, 2]
    }]
},
options: {
    title: {
        display: true,
        text: "Multiple Line Graphs"
    },
    legend: {display: true}
}
});

document.getElementById("demo5").innerHTML = "Chart.js multiple line chart created!";
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Chart.js Graphics](#)

JavaScript Google Chart Graphics

Table of Contents

- [JavaScript Google Chart Graphics](#)
- [Bar Charts](#)
- [Example 1](#)
- [Pie Charts](#)
- [Example 2](#)
- [3D Pie](#)
- [Example 3](#)
- [Line Graph](#)
- [Example 4](#)
- [Scatter Plots](#)
- [Example 5](#)
- [Reference](#)

Bar Charts

Google Bar Chart displays data with rectangular bars:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Google Chart Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Bar Charts</h4>

<div id="barChart"></div>
<p id="demo1"></p>

<script src="https://www.gstatic.com/charts/loader.js"></script>

<script>
google.charts.load("current", {packages: ["corechart"]});
google.charts.setOnLoadCallback(drawBarChart);

function drawBarChart() {
  const data = google.visualization.arrayToDataTable([
    ["Product", "Sales"],
    ["Product A", 25],
    ["Product B", 40],
    ["Product C", 30],
    ["Product D", 55],
    ["Product E", 20]
  ]);

  const options = {
```

```

    title: "Product Sales (Bar Chart)",
    hAxis: {title: "Product"},
    vAxis: {title: "Sales ($)", minValue: 0},
    legend: "none"
  };

  const chart = new
  google.visualization.ColumnChart(document.getElementById("barChart"));
  chart.draw(data, options);
  document.getElementById("demo1").innerHTML = "Google Bar Chart created!";
}
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Pie Charts

Google Pie Chart shows data as slices of a circle:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Google Chart Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Pie Charts</h4>

<div id="pieChart"></div>
<p id="demo2"></p>

<script src="https://www.gstatic.com/charts/loader.js"></script>

<script>
google.charts.load("current", {packages: ["corechart"]});
google.charts.setOnLoadCallback(drawPieChart);

function drawPieChart() {
  const data = google.visualization.arrayToDataTable([
    ["Browser", "Share"],
    ["Chrome", 55],
    ["Firefox", 20],
    ["Safari", 15],
    ["Edge", 8],
    ["Other", 2]
  ]);
}

```

```

const options = {
  title: "Browser Market Share (Pie Chart)",
  colors: ["#4CAF50", "#2196F3", "#FF9800", "#F44336", "#9C27B0"]
};

const chart = new
google.visualization.PieChart(document.getElementById("pieChart"));
chart.draw(data, options);
document.getElementById("demo2").innerHTML = "Google Pie Chart created!";
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

3D Pie

Google 3D Pie Chart adds a three-dimensional effect:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Google Chart Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>3D Pie</h4>

<div id="pie3dChart"></div>
<p id="demo3"></p>

<script src="https://www.gstatic.com/charts/loader.js"></script>

<script>
google.charts.load("current", {packages: ["corechart"]});
google.charts.setOnLoadCallback(draw3DPieChart);

function draw3DPieChart() {
  const data = google.visualization.arrayToDataTable([
    ["Category", "Value"],
    ["North", 35],
    ["South", 25],
    ["East", 20],
    ["West", 20]
  ]);

  const options = {
    title: "Regional Sales (3D Pie Chart)",

```

```

        is3D: true,
        colors: ["#4CAF50", "#2196F3", "#FF9800", "#F44336"]
    };

    const chart = new
google.visualization.PieChart(document.getElementById("pie3dChart"));
    chart.draw(data, options);
    document.getElementById("demo3").innerHTML = "Google 3D Pie Chart created!";
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Line Graph

A line graph displays information as a series of data points connected by straight line segments:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Google Chart Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Line Graph</h4>

<div id="lineChart"></div>
<p id="demo4"></p>

<script src="https://www.gstatic.com/charts/loader.js"></script>

<script>
google.charts.load("current", {packages: ["corechart"]});
google.charts.setOnLoadCallback(drawLineChart);

function drawLineChart() {
    const data = google.visualization.arrayToDataTable([
        ["Time", "Series 1", "Series 2"],
        [1, 10, 5],
        [2, 15, 8],
        [3, 13, 7],
        [4, 17, 11],
        [5, 14, 9],
        [6, 20, 13],
        [7, 18, 12],
        [8, 22, 16]
    ]);
}

```

```

const options = {
  title: "Line Graph",
  hAxis: {title: "Time"},
  vAxis: {title: "Values", minValue: 0},
  colors: ["#4CAF50", "#F44336"],
  curveType: "function"
};

const chart = new
google.visualization.LineChart(document.getElementById("lineChart"));
chart.draw(data, options);
document.getElementById("demo4").innerHTML = "Google Line Chart created!";
}
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Scatter Plots

A scatter plot displays values for two variables using dots:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Google Chart Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Scatter Plots</h4>

<div id="scatterChart"></div>
<p id="demo5"></p>

<script src="https://www.gstatic.com/charts/loader.js"></script>

<script>
google.charts.load("current", {packages: ["corechart"]});
google.charts.setOnLoadCallback(drawScatterChart);

function drawScatterChart() {
  const data = google.visualization.arrayToDataTable([
    ["X", "Y"],
    [1, 2], [3, 5], [4, 4], [6, 8], [7, 7],
    [9, 10], [2, 3], [4, 6], [5, 5], [7, 9],
    [8, 8], [10, 11]
  ]);
}

```

```
const options = {
  title: "Scatter Plot",
  hAxis: {title: "X Values", minValue: 0, maxValue: 11},
  vAxis: {title: "Y Values", minValue: 0, maxValue: 12},
  legend: "none",
  pointSize: 8,
  colors: ["#4CAF50"]
};

const chart = new
google.visualization.ScatterChart(document.getElementById("scatterChart"));
chart.draw(data, options);
document.getElementById("demo5").innerHTML = "Google Scatter Chart created!";
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Reference

- [W3Schools JavaScript Google Chart Graphics](#)

JavaScript D3.js Graphics

Table of Contents

- [JavaScript D3.js Graphics](#)
- [How to Use D3.js?](#)
- [Example 1](#)
- [Reference](#)

How to Use D3.js?

D3.js (Data-Driven Documents) is a JavaScript library for manipulating documents based on data. D3 helps you bring data to life using HTML, SVG, and CSS.

To use D3.js, include the library via CDN:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript D3.js Graphics</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>How to Use D3.js?</h4>

<svg id="d3Chart" width="400" height="200"></svg>
<p id="demo"></p>

<script src="https://d3js.org/d3.v7.min.js"></script>

<script>
const data = [
  {x: 10, y: 20},
  {x: 40, y: 60},
  {x: 80, y: 40},
  {x: 120, y: 90},
  {x: 160, y: 50},
  {x: 200, y: 110},
  {x: 240, y: 70},
  {x: 280, y: 130},
  {x: 320, y: 85},
  {x: 360, y: 150}
];

const svg = d3.select("#d3Chart");

// Add circles for each data point
svg.selectAll("circle")
  .data(data)
  .enter()
  .append("circle")
  .attr("cx", function(d) { return d.x; })
  .attr("cy", function(d) { return d.y; })
```



```
.attr("r", 6)
.attr("fill", "#4CAF50");

// Add X and Y axis lines
svg.append("line")
  .attr("x1", 5).attr("y1", 5)
  .attr("x2", 5).attr("y2", 180)
  .attr("stroke", "black").attr("stroke-width", 1);

svg.append("line")
  .attr("x1", 5).attr("y1", 180)
  .attr("x2", 395).attr("y2", 180)
  .attr("stroke", "black").attr("stroke-width", 1);

document.getElementById("demo").innerHTML =
  "D3.js scatter plot with " + data.length + " data points";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Reference

- [W3Schools JavaScript D3.js Graphics](#)